# MBSniffer v3.2 Technical Code Documentation

Architecture, runtime flows, every callable, regression tests, build automation, and complete source-code screenshots

**Documented package**

`MBSniffer_v3.2_Package.zip`

**Source snapshot SHA-256**

`ffb7a24a3d15ebfb5057f059fb0ba5b01c1c9a26f63bd19726b10ff3df3c14c3`

**Documentation build date**

23 October 2026

**Verification**

86/86 regression tests passed before documentation generation.

This document describes the exact source snapshot above. It does not document proposed future Modbus TCP or Slave-Simulator features that are not present in this package.

The final appendix reproduces every Python and Windows batch source line as code screenshots. The explanatory chapters refer to the same line numbers.

# Contents


1. Documented source snapshot ..... 3
2. Version history and changes in this version ..... 6
3. Architecture ..... 8
4. Passive capture pipeline ..... 10
5. Frame model and parser behavior ..... 11
6. Session metrics and Bus Health ..... 13
7. Active Bus Slave Finder ..... 15
8. Master (poll) tab ..... 18
9. UI, language and theme design ..... 22
10. Persistence, logs and export ..... 23
11. Module-by-module reference ..... 24
12. Regression-test suite ..... 104
13. Build and launch automation ..... 112
14. Known boundaries and extension points ..... 113
15. Appendix A - Complete source screenshots ..... 114

# 1. Documented source snapshot

This reference was generated from the current MBSniffer v3.2 package, without modifying the application source.

| Python source lines | Documented Python + BAT lines |
|---|---|
| 14,526 | 14,932 |

## Package structure

```
MBSniffer_v3.2_Package.zip
├── tools/  (documentation and package builders)
├── MBSniffer_v3.2_Technical_Code_Documentation.pdf  (this document)
├── MBSniffer.bat
├── README.md
├── README.en.md
├── README.txt
└── MBSniffer v3.2/
    ├── MBSniffer.py
    ├── mb_config.py
    ├── mb_protocol.py
    ├── mb_capture.py
    ├── mb_diagnostics.py
    ├── mb_view.py
    ├── mb_slave_finder.py
    ├── mb_master.py
    ├── mb_master_ui.py
    ├── mb_i18n.py
    ├── mb_theme.py
    ├── mb_widgets.py
    ├── mb_gui.py
    ├── tests/test_regression.py
    ├── tests/test_master.py
    ├── run_tests.bat
    ├── build_exe.bat
    ├── MBSniffer.ico
    ├── DEVELOPMENT_NOTES.md
    └── CHANGELOG.md
```

## Source inventory

| Python module | Lines | Primary responsibility |
|---|---|---|
| `MBSniffer.py` | 29 | Application launcher and top-level entry point. |
| `mb_config.py` | 486 | Central configuration, constants, user settings validation and filesystem paths. |

| Python module | Lines | Primary responsibility |
|---|---|---|
| `mb_protocol.py` | 1007 | Protocol engine for passive Modbus RTU parsing and session metrics. |
| `mb_capture.py` | 889 | Serial-port management and passive acquisition pipeline. |
| `mb_diagnostics.py` | 286 | Presentation-oriented diagnostics derived from captured records. |
| `mb_view.py` | 1871 | Traffic view, filtering, sorting, Frame Inspector, Bus Health updates, CSV export and TXT logging. |
| `mb_slave_finder.py` | 1625 | Active Modbus RTU Slave discovery and Device Identification. |
| `mb_master.py` | 686 | Modbus RTU master engine, frames and data formats. |
| `mb_master_ui.py` | 1993 | Master (poll) tab user interface. |
| `mb_i18n.py` | 610 | Runtime Portuguese/English translation layer. |
| `mb_theme.py` | 692 | Fixed-geometry Light/Dark theme implementation. |
| `mb_widgets.py` | 418 | Custom reusable Tkinter widgets. |
| `mb_gui.py` | 2584 | Main application composition and global UI behavior. |
| `test_regression.py` | 787 | Automated regression suite. |
| `test_master.py` | 563 | Master regression tests. |

# Integrity hashes

| Source file | Lines | SHA-256 |
|---|---|---|
| `MBSniffer v3.2/MBSniffer.py` | 29 | `c5eae72b053b4d96a112c492ec200b5a046724ff0a0893b3e50f86ae1df564bb` |
| `MBSniffer v3.2/mb_config.py` | 486 | `98a2340445ccd04c8983fca1589878d9fc629a44a152ad7201f65958532d5a72` |
| `MBSniffer v3.2/mb_protocol.py` | 1007 | `8c4c41051e41f0c998692206ed0204d91f057cce5fce726b15ad2cce503c1ef2` |
| `MBSniffer v3.2/mb_capture.py` | 889 | `cc61c566dd04c4ec3ce2bd61b6fef50738ccce2f85d53b7ffe1c0650f652ddb2` |
| `MBSniffer v3.2/mb_diagnostics.py` | 286 | `d5edc15bf0259d130fdf9b42ffaf2caca770782101aeed1886f980e0fcf24bda` |
| `MBSniffer v3.2/mb_view.py` | 1871 | `0c65dd97ebf6d836431fa4dc0b729e3aab9f1351559de04f3dd2b713ce781601` |
| `MBSniffer v3.2/mb_slave_finder.py` | 1625 | `f192a6305486ea73411d920d9b6b9de19091bc612afef60ea3f5871fcd66938b` |
| `MBSniffer v3.2/mb_master.py` | 686 | `79ad42ac5c1f3a658b4252c2196ccb9829ad4cfdbcc5237052ef8a3a3423221e` |

| Source file | Lines | SHA-256 |
|---|---|---|
| `MBSniffer v3.2/mb_master_ui.py` | 1993 | `be93855e2ca6df91634d4dfa36609a4e5a15e7a28de330ac63b50d4c56f3ee98` |
| `MBSniffer v3.2/mb_i18n.py` | 610 | `c189b677eeb59213b98d3717d9187b99999c0f0853c8c5cb46b327d1d3544be8` |
| `MBSniffer v3.2/mb_theme.py` | 692 | `88c0c2592f4b6e0666aa42e4d163ae120845b154f7faa3b9f6a007fef7ed8265` |
| `MBSniffer v3.2/mb_widgets.py` | 418 | `fb45d18d621a734220afa46d92dfb882228a5cba2702e4201ccf02120b2bff97` |
| `MBSniffer v3.2/mb_gui.py` | 2584 | `f9489ca19586a7ede032d90a59d632e1c777da560f48895c9f0778b81c10e62d` |
| `MBSniffer v3.2/tests/test_regression.py` | 787 | `b5a3f7e75c0b385f8cb874fdf3b16c8e867108c59406078d966f333db4dba70c` |
| `MBSniffer v3.2/tests/test_master.py` | 563 | `23f733b881f14e4300d50ab5587347ea1e02eedcc933691a6d060d2c1c066c05` |
| `MBSniffer.bat` | 210 | `3c1a036c6ef094c26390c883d8f59aa5e226da5394100edad2e523410fdf5a1d` |
| `MBSniffer v3.2/build_exe.bat` | 151 | `fbe1caf24e1c48fe6a55f959a61ecc0d0f6a50b376f02297bb6c7d642b8b0775` |
| `MBSniffer v3.2/run_tests.bat` | 45 | `8066035cd8d62a7aded1f8314b457d438795a8c59319bf1290c47bc63d34372f` |

> **Regression status.** The supplied suite was executed directly from this source tree. All 86 tests passed.

# 2. Version history and changes in this version

Release history of the documented code base. The detailed list below applies to v3.2, the version described by every other chapter of this document.

| Version | Date | Summary |
|---|---|---|
| **v3.0** | 21 September 2026 | Bilingual PT-PT / English interface, FC43/14 Device Identification, regression suite and first Technical Code Documentation. |
| **v3.1** | 23 October 2026 | Broadcast/echo-write pairing fixes, USB split-frame reassembly, in-place timeout marking, Finder attempts and idle-bus check, address-mapped read data, table-driven CRC and centralised translations. |
| **v3.2** | 23 October 2026 | Master (poll) tab with one window per Slave / data area, cyclic reads, writes, 16/32-bit data formats and per-window counters; the release ZIP now also carries this PDF and the documentation tools. |

## Changes in v3.2

v3.2 adds the first phase of a Modbus RTU master in the style of Modbus Poll: a **Master** tab with several poll windows, one per Slave or data area. The feature list was taken from the public Modbus Poll documentation. Every other chapter of this document is regenerated from the v3.2 source, so the module reference, line numbers, hashes, tests and Appendix A describe v3.2.

## Master tab (phase 1)

- **New modules:** `mb_master.py` (frames, data formats and `MasterEngine`, no widgets) and `mb_master_ui.py` (`MasterMixin`, the tab and its windows). See chapter "Master (poll) tab".
- **Windows:** Slave ID, FC01–FC04, address, quantity, scan rate and default data format, with Apply (highlighted while fields are not applied), Pause, Read once and Duplicate; up to 32 windows arranged in a responsive grid or popped out to their own top-level window. Tables are 3–15 rows high, with alternating row shading and a thin 1 px frame, and values turn grey while the last read failed.
- **One bus, one request at a time:** all windows share one serial port; writes and "Read once" are served first, then the window whose scan time is due earliest, with the configured delay between polls (at least t3.5).
- **Data formats:** signed/unsigned 16-bit, hex, binary, signed/unsigned/float 32-bit in byte orders ABCD, CDAB, BADC and DCBA, and signed/unsigned/double 64-bit in ABCDEFGH, GHEFCDAB, BADCFEHG and HGFEDCBA, per window and per address (right-click menu, with scaling to engineering units, Copy and the address base).
- **Open in Master** from the Sniffer Traffic table and the Bus Slave Finder results, and **automatic reconnection** when the serial port fails.
- **Writes:** FC05/FC15 from coil windows and FC06/FC16 from holding-register windows, several values separated by ";", broadcast with Slave 0, automatic re-read after an accepted write.

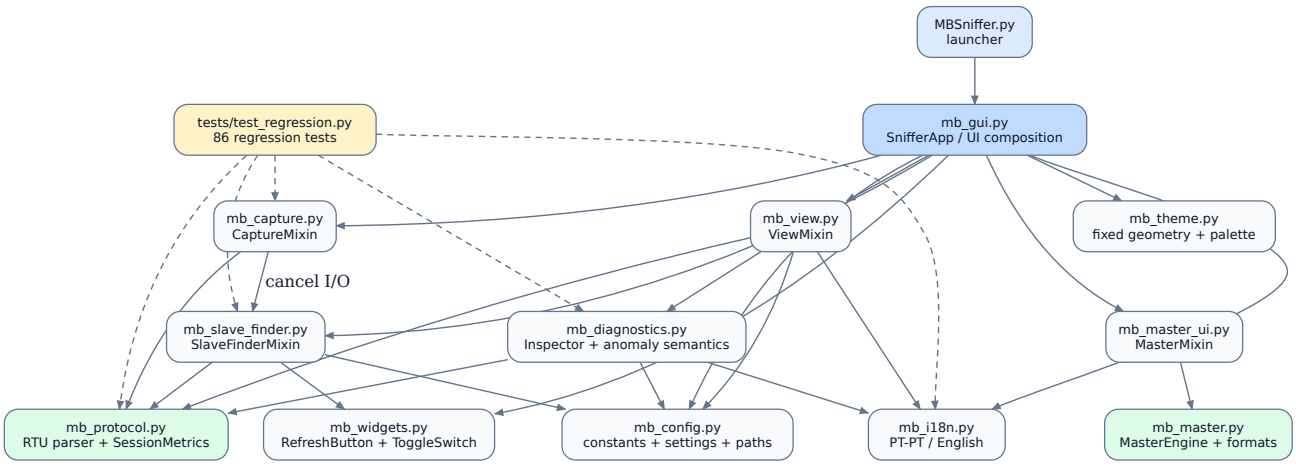
- **Diagnostics:** Tx, error and timeout counters, last response time and last error (Timeout, Exception, invalid response) per window.
- **Safety:** the same confirmation and idle-bus check as Bus Slave Finder; while connected, Sniffer, Simulation and Finder are locked through the global busy flag.
- **Window header:** each window has its own name (Rename), Pop out and a × close button.
- **Workspaces:** a Workspace menu saves the connection, windows, formats and per-address names under a chosen name in `%APPDATA%\MBSniffer\Workspaces` and lists the saved workspaces to open or delete. The last workspace reopens at start-up; closing with unsaved changes asks whether to save.

# Packaging and documentation

- The release ZIP holds the application files, the README files, this PDF and `tools/`, and nothing related to git or GitHub, which the user manages. `tools/build_package.py` builds it from the working folder.
- Because this PDF travels inside the ZIP, the cover quotes a **Source snapshot SHA-256** (one hash over every documented file) instead of the ZIP hash.
- The regression suite grew from 45 to 86 tests, including an in-memory Modbus slave used to test the Master engine.

# 3. Architecture

MBSniffer uses one Tkinter root object with four behavior mixins. Protocol parsing and metrics are separated from the GUI, while the active Finder and Master have their own worker logic. This is a pragmatic modular architecture rather than a strict MVC/MVVM framework.



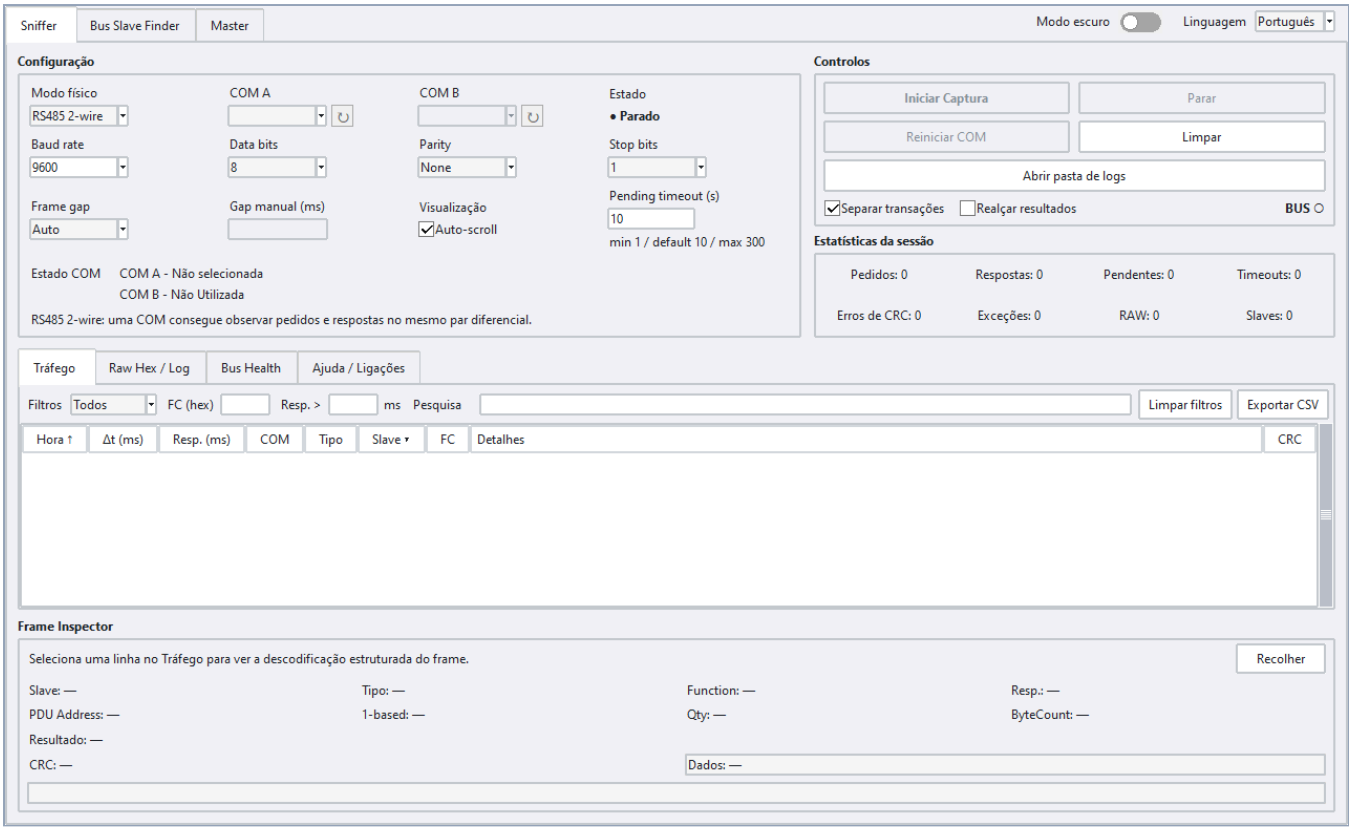

Module dependency and responsibility overview.

## Composition of SnifferApp

`SnifferApp(MasterMixin, SlaveFinderMixin, CaptureMixin, ViewMixin, tk.Tk)` composes the application. `mb_gui.py` constructs and coordinates the UI; the four mixins supply the Master tab, active discovery, passive capture, and presentation/logging behavior.

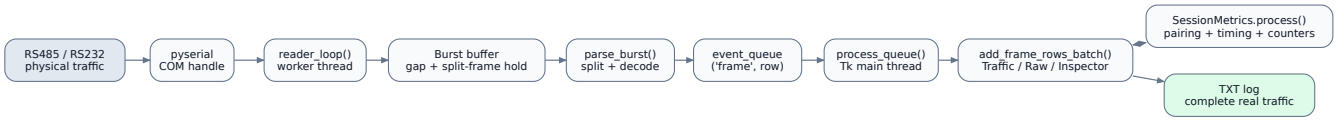
## Important architectural boundaries

- **Protocol vs UI:** CRC, frame recognition, parsing and SessionMetrics are in `mb_protocol.py` and have no Tkinter dependency.
- **Workers vs Tk:** serial reader/discovery threads communicate through `event_queue`; Tk widgets are updated from the main thread.
- **Source history vs presentation:** filters and sorting rebuild the view from `_frame_history`; they do not modify the raw session record or statistics.
- **Passive vs active:** the Sniffer only reads. Bus Slave Finder intentionally transmits requests and is protected by a safety confirmation/global busy state.
- **Appearance vs geometry:** Light/Dark changes colors; language changes text; both are designed not to rebuild widget geometry.




Current Sniffer UI generated from the documented source snapshot.

# 4. Passive capture pipeline




From physical bytes to UI records and disk logging.

1. **Port selection.** `selected_port_plan()` maps RS485/RS232 mode to one or two receive channels.
2. **Serial configuration.** `serial_config()` translates Baud/Data bits/Parity/Stop bits into pyserial constants.
3. **Gap calculation.** Automatic mode uses the shared Modbus t3.5 helper; manual mode validates the user-entered millisecond value.
4. **Reader workers.** One daemon thread per active receive channel reads bytes, records the arrival time of every read and closes a burst after the configured silence. If the burst ends with the start of an unfinished known frame, the reader waits up to `SNIFFER_SPLIT_FRAME_HOLD_MS` for the rest (USB latency-timer splits).
5. **Parsing.** Each burst is split/resynchronised and decoded by `parse_burst()`; each frame keeps the arrival time of the read holding its first byte.
6. **Thread boundary.** Parsed rows are placed in `event_queue`.
7. **Main-thread processing.** `process_queue()` drains events in batches and passes frame batches to `add_frame_rows_batch()`.
8. **Metrics/view/log.** The view runs SessionMetrics, updates Traffic/Raw/Inspector/Bus Health and writes complete real traffic to the lazy TXT log.

> **Threading rule.** The serial threads do not call Tkinter methods. The queue is the explicit synchronization boundary between workers and the GUI thread.

# 5. Frame model and parser behavior

## RTU CRC and frame timing

`crc16_modbus()` implements the Modbus CRC-16 algorithm with a 256-entry lookup table (`_CRC16_TABLE`). `append_crc()` appends low byte then high byte. `modbus_rtu_interframe_gap_seconds()` provides the common t3.5 timing used by Sniffer and Finder.

## Recognised frame shapes

`_candidate_prefixes_at()` recognises structural lengths for FC01/02/03/04, FC05/06/08, FC07, FC15/16, FC22, FC23 and standard Exception frames. Unknown Function Codes are deliberately handled conservatively.

## Resynchronisation

The parser first tries a recognised, CRC-valid frame at offset zero. If that fails, it searches later offsets but only using recognised frame shapes. This is the performance and false-positive safeguard introduced after the earlier brute-force CRC scan.

- Known-shape prefix before a recovered valid frame + bad CRC → `CRC ERROR`.
- Unstructured bytes before a recovered frame → `RAW/UNSYNC`.
- Known-shape complete remainder + bad CRC → `CRC ERROR`.
- Unsupported-FC complete remainder with valid CRC → `UNKNOWN / OK`.
- Otherwise → `RAW/UNPARSED`.

## Bursts split by USB adapters

`burst_may_continue()` re-runs the splitter on the burst and inspects only the final unvalidated segment: if it starts with a plausible Slave ID and a known Function Code but is shorter than every candidate length, more bytes are expected. The reader re-checks only when the buffer length changes, so a long noise burst is not re-parsed on every poll.

## Read-response bits

FC01/FC02 responses are decoded least-significant bit first by `read_bits_from_response()`. `read_bits_details()` groups the bits per data byte, shows at most 32 of them in Traffic and trims the padding of the last byte once the matching request quantity is known.

# Parser row schema

| Field | Meaning |
|---|---|
| `time` | Display timestamp string. |
| `channel` | BUS, A→B, B→A or SIM source. |
| `type` | REQUEST, RESPONSE, EXCEPTION, RAW/UNSYNC, RAW/UNPARSED, UNKNOWN or REQ/RESP (FC05/06/08/22, resolved to REQUEST/RESPONSE by SessionMetrics). |
| `slave`, `fc` | Decoded address and hexadecimal Function Code when available. |
| `details` | Human-readable protocol details. |
| `crc` | OK, ERROR or ?. |
| `raw` | Original frame bytes rendered as hexadecimal. |
| `pdu_address`, `one_based`, `qty` | Addressing/quantity metadata when decoded. |
| `_frame_len` | Number of observed bytes represented by the row. |
| `_epoch` | Floating timestamp used for timing/pairing: arrival time of the read holding the frame's first byte. |

# 6. Session metrics and Bus Health

`SessionMetrics` is stateful per capture/session. It records counters, pending requests, response samples and diagnostic aggregates before GUI filters are applied.

## Request/response pairing

A request key is `(Slave ID, base Function Code)`. Valid REQUEST rows are queued FIFO per key. RESPONSE/EXCEPTION rows remove the oldest matching pending request and compute response time from the two epoch timestamps.

> **Pairing limitation.** Modbus RTU has no transaction identifier. The implementation therefore uses FIFO matching by Slave and Function Code. If a non-standard master pipelines multiple same-Slave/same-FC requests without waiting for responses, pairing cannot be proven uniquely from RTU headers alone.

## Echo-shaped functions

FC05, FC06, FC08 and FC22 requests and their normal responses have the same length, so the parser emits `REQ/RESP`. `_resolve_echo_direction()` classifies the frame before accounting: RESPONSE when an older pending request with the same key exists (identical bytes for the exact-echo FCs 05/06/22; opposite channel in RS232 dual RX), otherwise REQUEST.

> **Echo limitation.** On a single RS485 channel, if a master repeats the same write to a Slave that never answers, the repetition is indistinguishable from an echo response and is paired as one.

## Broadcast

Requests to Slave 0 are counted as requests and in `broadcast_requests`, but are never pending: by definition they receive no response and must not become timeouts.

## Timeouts

Pending requests older than the configured Pending timeout are expired. The timeout is attached to the original REQUEST record through its sequence ID; MBSniffer does not invent a fake timeout frame. The view retags only the affected rows instead of rebuilding the table.

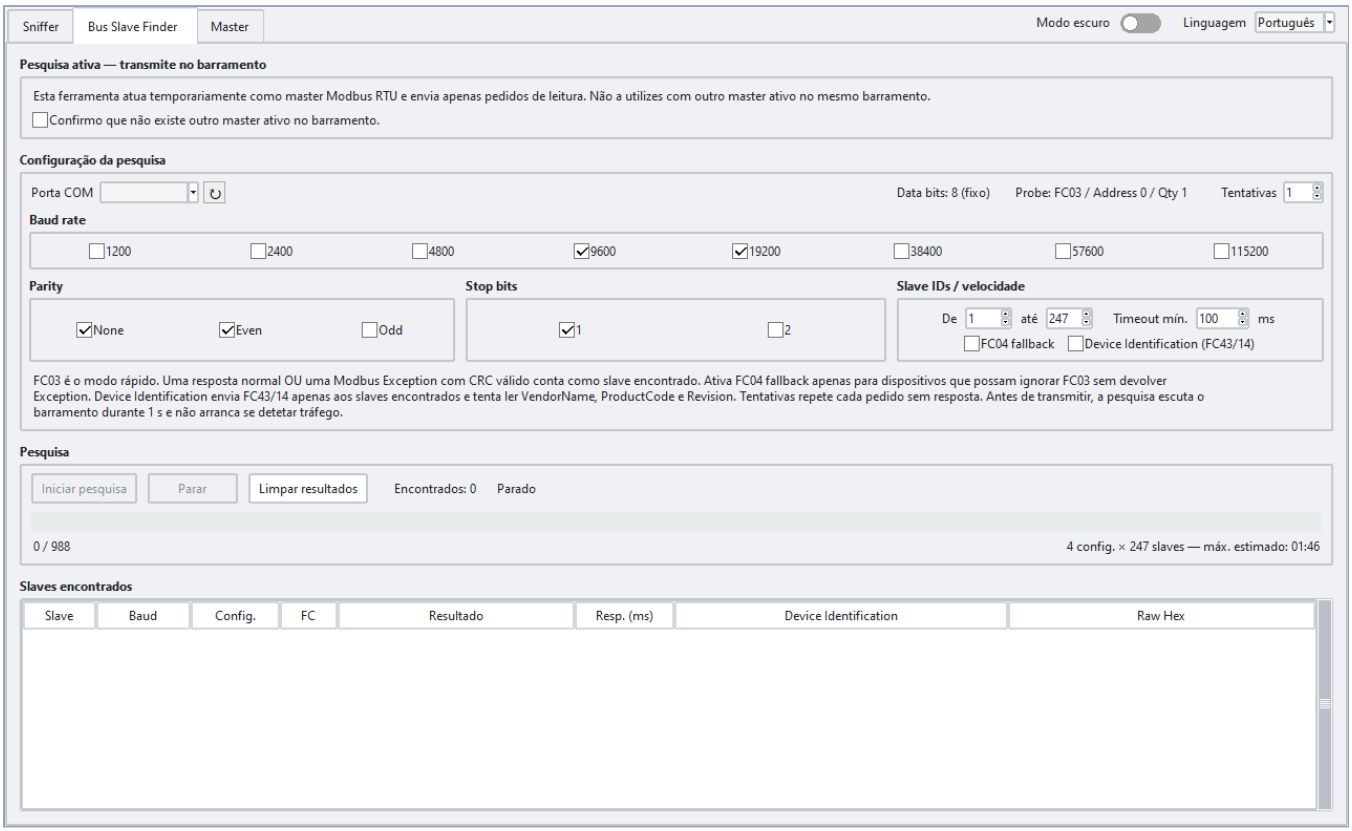
## Bus Health calculations

- **Request rate:** requests divided by observed elapsed session time.
- **Response time:** average, minimum, maximum and nearest-rank P95 from valid paired responses.

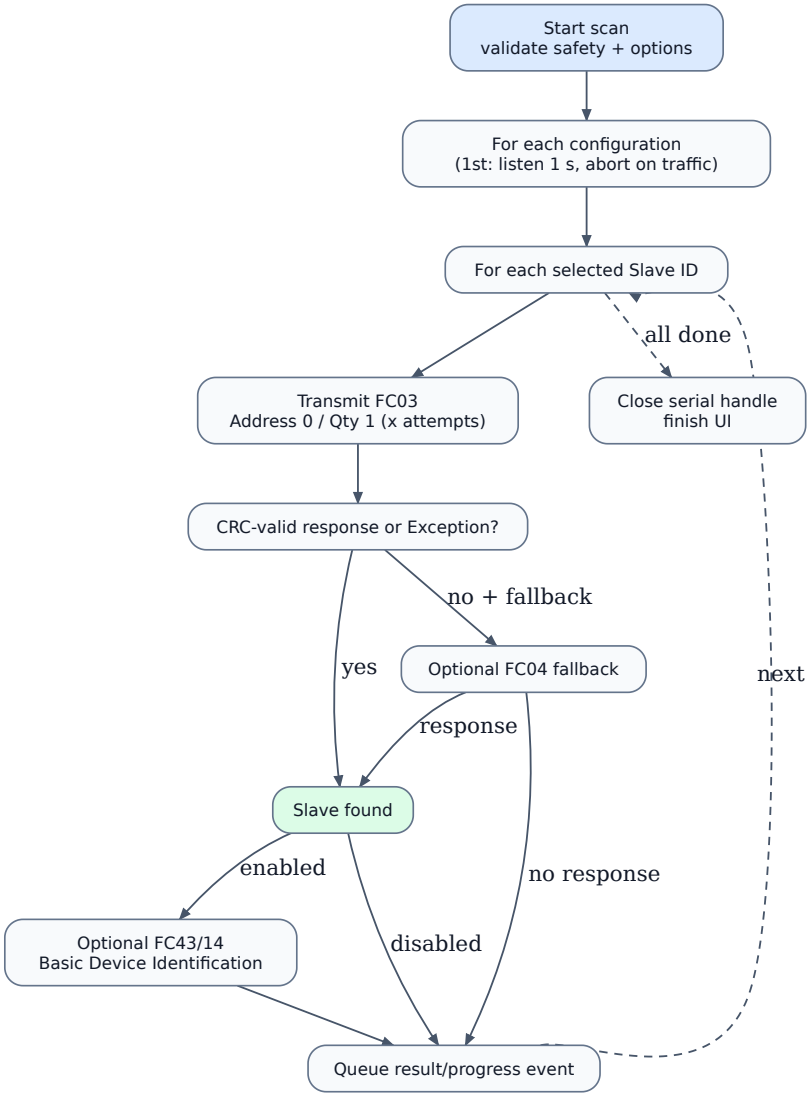
- **Slowest Slave:** highest average response time among sampled Slaves.
- **Unvalidated bytes:** bytes in explicit CRC-error frames plus RAW blocks divided by all observed bytes.
- **Timeout rate:** expired requests / observed non-broadcast requests.
- **Approximate bus utilisation:** observed bytes × configured serial bits per character / (baud × elapsed time).
- **Exceptions / Slave:** counts of Exception responses grouped by Slave ID.

> **Physical-layer boundary.** Bus utilisation and unvalidated-byte percentage are software/capture indicators. The program does not measure analog RS485 voltage, bias, reflections, ringing, common-mode level or rise/fall times.

# 7. Active Bus Slave Finder




Current active Finder UI generated from the documented source.




Discovery flow for each selected configuration and Slave ID.

# Discovery semantics

The standard probe is FC03, Address 0, Quantity 1. A CRC-valid normal response proves the Slave exists. A CRC-valid Modbus Exception also proves the Slave exists, because the addressed device had to receive and interpret the request to generate that Exception.

FC04 fallback is optional and is attempted only after FC03 produces no accepted response. **Attempts** (1–3) repeats each unanswered probe before moving on. Device Identification is also optional and is performed only after a Slave has already been found.

# Idle-bus check

Before the first transmission of a scan, `bus_activity_detected()` listens for `DISCOVERY_IDLE_CHECK_SECONDS` on the first configuration. Any byte, even one framed at another baud rate, means something else is transmitting: the worker raises an error naming the port and the first bytes, and nothing is sent. This complements the safety checkbox.

# FC43/14 Device Identification

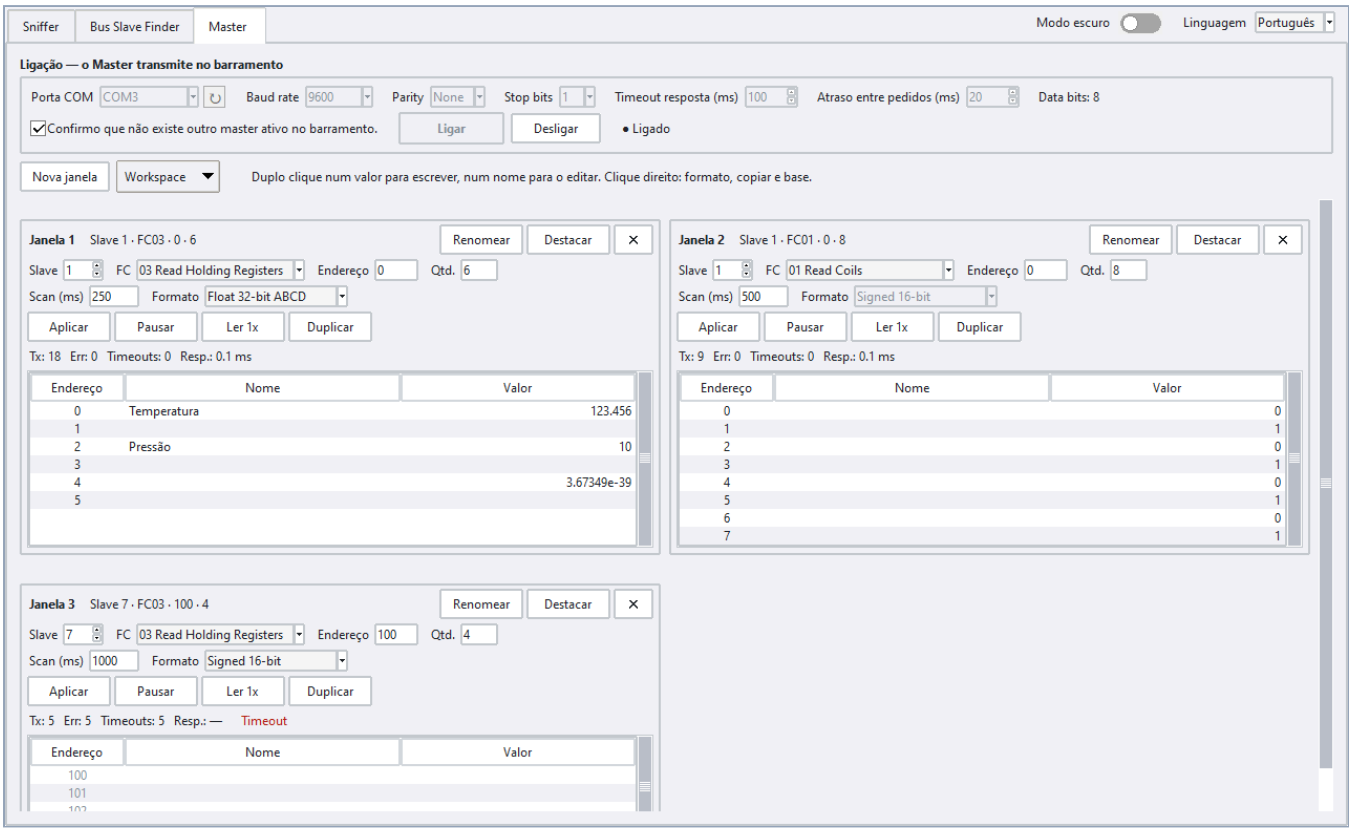
The request uses Function Code 0x2B, MEI type 0x0E and Basic Device Identification read code 0x01. The parser reads variable-length objects, validates CRC and follows `More Follows` pages up to the configured page cap. Display output focuses on object 0x00 VendorName, 0x01 ProductCode and 0x02 MajorMinorRevision.

# Driver/error handling

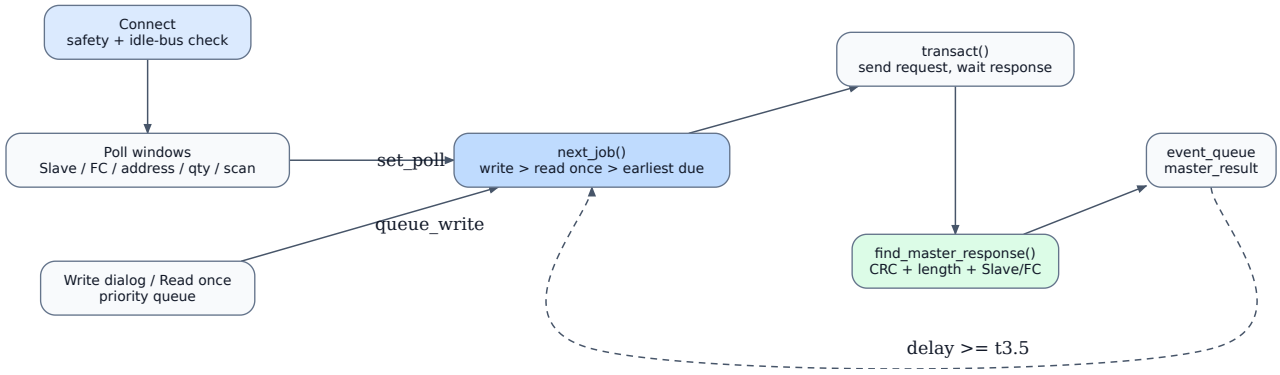
A serial write/read failure is treated as a communication/driver error, not as an ordinary "Slave did not respond" result. Write timeout recovery can cancel/clear driver buffers and retry once. Stop requests use `threading.Event` and best-effort cancel_read/cancel_write support.

> **Operational safety.** This tab actively transmits Modbus RTU requests. It must not be run on a bus that already has another active master unless the system is explicitly designed for multi-master behavior.

# 8. Master (poll) tab




Master tab connected to the in-memory test slave of tests/test_master.py.




Master engine: one request at a time over one serial port.

The Master tab reads and writes values as a Modbus RTU master, in the style of Modbus Poll. Each **poll window** describes one data area of one Slave; the state of a window lives in a `MasterWindow` object, independent of where its pane is shown.

## Scheduling on one bus

Modbus RTU allows a single request on the bus at a time, so one `MasterEngine` thread serves every window over one serial port. Its `next_job()` returns, in this order: a queued write, a queued "Read once", or the enabled window whose next due time is earliest. A window becomes due again `scan_ms` after its previous request started (0 polls continuously). After every transaction the engine leaves the configured

delay between polls on the bus, never less than Modbus t3.5. Response time and the response timeout count from the end of the transmitted request.

# Frames and responses

`build_read_request()` and `build_write_request()` build FC01–04 and FC05/06/15/16 frames within the Modbus limits (2000 bits / 125 registers per read, 1968 coils / 123 registers per write). `find_master_response()` scans the received bytes for the CRC-valid frame of the expected length from the addressed Slave, or a 5-byte exception. Read requests echoed by an RS485 adapter are skipped because their length and CRC do not fit. Write responses must echo the request (FC05/06) or its address/quantity (FC15/16). Broadcast writes (Slave 0) are not answered and are reported as sent.

# Data formats

| Format | Meaning |
|---|---|
| Signed / Unsigned 16-bit | One register as a two's-complement or unsigned integer. |
| Hex / Binary 16-bit | One register as `0xHHHH` or two binary bytes. |
| Signed / Unsigned / Float 32-bit | Two registers. Register n carries bytes A B and register n+1 bytes C D; the order names the value's bytes: ABCD big-endian, CDAB word swap, BADC byte swap, DCBA little-endian. The value is shown on the first register's row. |
| Signed / Unsigned 64-bit, Double | Four registers (bytes A B, C D, E F, G H): ABCDEFGH big-endian, GHEFCDAB registers in reverse order, BADCFEHG byte swap, HGFEDCBA little-endian. Doubles show up to 15 significant digits. |

# Per-address formats

The window's **Format** is the default. `MasterWindow.formats` maps the PDU address where a value starts to its own format, chosen in the right-click menu for the selected rows (drag with the mouse, or Ctrl/Shift, for several; ttk has no rubber-band selection, so `_master_drag_start/move/stop` implement it, with auto-scroll past the table edges). `apply_format_to_rows()` applies 16-bit formats to every selected row and 32/64-bit formats to consecutive rows in multiples of 2/4, removing overlapping per-address formats; "Reset to window format" removes them. `register_groups()` splits the window into values; a default multi-register value cut by a per-address format, or running past the window, shows "—". The same menu copies the selected rows (also Ctrl+C) as tab-separated text and sets the address base (0 = PDU Address, 1 = manual numbering).

# Scaling

`MasterWindow.scales` maps the PDU address where a value starts to a scale built by `validate_scale()`: two raw/engineering points, 0–6 decimals and a unit. `format_window_values()` shows scaled values as `e1 + (raw − r1) × (e2 − e1) / (r2 − r1)` with the unit; Hex and Binary stay raw. When writing, the typed engineering value is converted back with `raw_text_for_scaled()` (rounded for integer formats).

# Open in Master

`master_definition_from_record()` takes Slave, function, address and quantity from a Traffic REQUEST or from the request matched to a RESPONSE/EXCEPTION (FC01–FC04 only). `master_open_definition()` creates the window, shows the Master tab and, when the Master is not connected, copies the port and serial parameters. The Bus Slave Finder opens a found Slave the same way from its result row (FC03/FC04, address 0, 10 registers, the baud/parity/stop it answered on).

# Reconnection

The engine receives a `reopen` callable. A serial or driver failure during the session closes the port, posts `master_reconnecting` and retries every `MASTER_RECONNECT_INTERVAL_SECONDS` until the port opens again (`master_reconnected`) or the user disconnects; the values turn grey in the meantime.

# Writing

Double-clicking a value in an FC01 or FC03 window opens the write dialog. `master_build_write()` validates the Slave (0–247), the address in the window's base and the values, encodes them with the format of the clicked value and builds an FC05, FC06, FC15 or FC16 request; 32/64-bit formats require FC16. Several values separated by ";" go to consecutive addresses. An accepted write queues a "Read once" of the window so the table shows the new value.

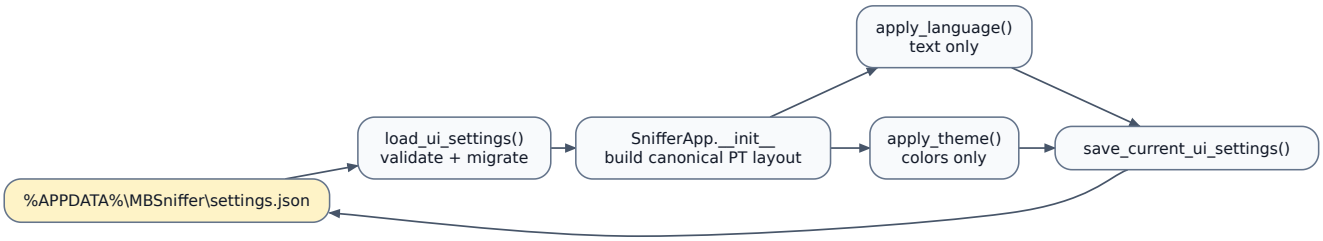
# Window life cycle

- **Apply** validates the fields and updates the engine; a changed data area resets the counters. Results of a request sent with the previous definition are ignored.
- Each pane has a header with the window name (**Rename** or double-click), a Slave/FC/address/quantity summary, **Pop out** and a × close button, so closing never depends on the pane width.
- **Pop out** rebuilds the pane inside a `tk.Toplevel`; closing it or **Dock** returns it to the grid. The grid uses as many equal columns as the tab width allows, measured from the panes' requested width (which grows with Windows display scaling).
- Panes created after start-up receive the application's mouse-wheel guard tag: the wheel scrolls the window grid and never changes combobox/spinbox values.
- The **Workspace** menu saves named `.mbsw` files (JSON) with the connection parameters, every window, its name and address names in `%APPDATA%\MBSniffer\Workspaces`, beside `settings.json`; it lists them to open or delete. Invalid entries are skipped with a warning.
- Changes to the open workspace mark it "(modified)". `settings.json` keeps `master.last_workspace`, reopened at start-up without connecting; on exit, `master_confirm_close()` asks Yes/No/Cancel only for a saved workspace with unsaved changes.

> **Operational safety.** Like Bus Slave Finder, the Master transmits. Connecting requires the safety confirmation and an idle-bus check; while it is connected the global busy flag locks Sniffer, Simulation and Finder.

> **Phase boundary.** Logging, scaling, conditional colours, charts, Test Center, FC22/23 and Modbus TCP are planned later phases and are not part of v3.2.

# 9. UI, language and theme design




Settings, language and theme lifecycle.

## Canonical-language strategy

Portuguese (Portugal) is the canonical static UI language. `canonical_ui_text()` can map known English captions back to PT-PT, then `translate_text()` renders either canonical Portuguese or an explicit English translation. Technical English terms without an explicit mapping are left unchanged.

Messages with live values are stored as PT-PT templates in `PT_TO_EN` and rendered with `SnifferApp.trf(template, **values)`. Code never branches on `current_language`; a regression test enforces this.

## Runtime language changes

`apply_language()` updates widget text, table headings, context menus, Help content and live status strings. Protocol bytes and parser data are not translated; only presentation fragments such as `Registos` / `Registers` are changed.

## Language-independent geometry

The UI is built using the canonical Portuguese captions, then `_freeze_language_layout_geometry()` measures key dimensions and freezes them. This prevents shorter/longer English captions from moving the surrounding rectangles or controls.

## Light/Dark invariant

The application uses one ttk `clam` construction. `mb_theme.py` configures geometry once and palette functions only change colors/state maps. Custom Canvas widgets receive palette updates separately.

## Mouse-wheel behavior

An application bindtag lets Treeviews/Text/Listboxes/scrollbars consume wheel events normally while stopping wheel propagation elsewhere. This preserves table scrolling without allowing the Notebook to change tabs with the wheel.

# 10. Persistence, logs and export

## User settings

On Windows, preferences are stored in `%APPDATA%\MBSniffer\settings.json`. Loading is defensive and every persisted choice/range is validated before use. Saves use a temporary file and replace operation.

Persisted data includes language, Light/Dark, last main tab, serial format, frame-gap mode/value, Pending timeout, auto-scroll, transaction separators, highlighting and Finder options (including attempts and the minimum timeout, migrated from the old 40 ms default to 100 ms) and the Master connection parameters. Master windows are not part of the settings: they are saved explicitly in workspace files. COM names, window geometry, traffic/results and the Finder safety confirmation are intentionally not persisted.

## TXT capture log

Real-capture logs are lazy: the file is created only when traffic actually arrives. The disk log is complete and independent of table filters, sorting and the bounded GUI history.

## CSV export

CSV exports the records currently retained in `_frame_history`, not necessarily an arbitrarily long complete session. It includes timing, channel, type, address metadata, CRC, raw bytes, timeout status and anomaly classification.

## Bounded GUI history

The UI retains up to approximately `MAX_UI_FRAMES = 20000` records and prunes in chunks after a hysteresis threshold. Raw Text has a similar bounded-line policy. This prevents the GUI from growing indefinitely while keeping the capture log complete.

# 11. Module-by-module reference

The following chapters provide a complete callable inventory for every Python module. Each function/method is tied to its exact source-line range. The complete source itself appears as screenshots in Appendix A.

# MBSniffer.py

**Application launcher and top-level entry point.** It sets the development simulation switch before importing the GUI, applies the Windows application identity, creates SnifferApp and enters Tkinter's main event loop.

| Source lines | SHA-256 |
| --- | --- |
| 29 | `c5eae72b053b4d96a112c492ec200b5a046724ff0a0893b3e50f86ae1df564bb` |

**Imports:** `mb_config, mb_config.set_windows_app_user_model_id, mb_gui.SnifferApp`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `debug_sim` | `0` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `main`<br>main() | 23-25 | Main. | `set_windows_app_user_model_id, SnifferApp().mainloop, SnifferApp` |

# mb_config.py

**Central configuration, constants, user settings validation and filesystem paths.** It defines versioned application constants, discovery limits, GUI history limits, Modbus exception metadata, persistent settings behavior, application/log directories and PyInstaller resource lookup.

| Source lines | SHA-256 |
| --- | --- |
| 486 | `98a2340445ccd04c8983fca1589878d9fc629a44a152ad7201f65958532d5a72` |

**Imports:** `ctypes, json, os, sys, pathlib.Path`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `APP_NAME` | `'MBSniffer'` |
| `APP_VERSION` | `'3.2'` |
| `debug_sim` | `0` |
| `DISCOVERY_BAUD_RATES` | `(1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200)` |
| `DISCOVERY_PARITIES` | `('None', 'Even', 'Odd')` |
| `DISCOVERY_STOP_BITS` | `('1', '2')` |
| `DISCOVERY_DEFAULT_MIN_TIMEOUT_MS` | `100` |
| `DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS` | `40` |
| `DISCOVERY_MIN_TIMEOUT_MS` | `10` |
| `DISCOVERY_MAX_TIMEOUT_MS` | `2000` |
| `DISCOVERY_PROBE_ADDRESS` | `0` |
| `DISCOVERY_PROBE_QTY` | `1` |
| `DISCOVERY_WRITE_TIMEOUT_SECONDS` | `1.5` |
| `DISCOVERY_WRITE_RETRIES` | `1` |
| `DISCOVERY_RX_BUFFER_LIMIT` | `1024` |
| `DISCOVERY_MIN_ATTEMPTS` | `1` |
| `DISCOVERY_MAX_ATTEMPTS` | `3` |
| `DISCOVERY_DEFAULT_ATTEMPTS` | `1` |
| `DISCOVERY_IDLE_CHECK_SECONDS` | `1.0` |

| Name | Definition |
| --- | --- |
| UI_QUEUE_BATCH_SIZE | 200 |
| UI_QUEUE_IDLE_INTERVAL_MS | 50 |
| UI_QUEUE_BUSY_INTERVAL_MS | 10 |
| MIN_PENDING_REQUEST_TIMEOUT_SECONDS | 1.0 |
| DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS | 10.0 |
| MAX_PENDING_REQUEST_TIMEOUT_SECONDS | 300.0 |
| PENDING_REQUEST_TIMEOUT_SECONDS | DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS |
| MASTER_MAX_READ_BITS | 2000 |
| MASTER_MAX_READ_REGISTERS | 125 |
| MASTER_MAX_WRITE_BITS | 1968 |
| MASTER_MAX_WRITE_REGISTERS | 123 |
| MASTER_DEFAULT_RESPONSE_TIMEOUT_MS | 1000 |
| MASTER_MIN_RESPONSE_TIMEOUT_MS | 10 |
| MASTER_MAX_RESPONSE_TIMEOUT_MS | 10000 |
| MASTER_DEFAULT_DELAY_MS | 20 |
| MASTER_MIN_DELAY_MS | 0 |
| MASTER_MAX_DELAY_MS | 10000 |
| MASTER_DEFAULT_SCAN_MS | 1000 |
| MASTER_MAX_SCAN_MS | 3600000 |
| MASTER_MAX_WINDOWS | 32 |
| MASTER_RECONNECT_INTERVAL_SECONDS | 2.0 |
| MASTER_WORKSPACE_FORMAT | 'MBSniffer Master workspace' |
| MASTER_WORKSPACE_VERSION | 1 |
| SNIFFER_SPLIT_FRAME_HOLD_MS | 40.0 |
| SLOW_RESPONSE_THRESHOLD_MS | 500.0 |
| MODBUS_EXCEPTION_CODES | {1: ('Illegal Function', 'A função Modbus pedida não é suportada/permitida pelo dispositivo.'), 2: ('Illegal Data Address', 'O endereço ou a gama d… |
| MAX_UI_FRAMES | 20000 |

| Name | Definition |
|---|---|
| `MAX_RAW_TEXT_LINES` | `20000` |
| `UI_PRUNE_CHUNK_FRAMES` | `2000` |
| `RAW_TEXT_PRUNE_CHUNK_LINES` | `2000` |
| `RAW_TRANSACTION_SEPARATOR` | `'—' * 72 + '\n'` |
| `SIMULATION_FUNCTION_CODES` | `(1, 2, 3, 4)` |
| `SIMULATION_EXCEPTION_CODES` | `(1, 2, 3, 4, 6)` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `resource_path`<br>resource_path(filename) | 129-135 | Return a bundled/source resource path for normal Python or PyInstaller. | `Path, Path(__file__).resolve` |
| `set_windows_app_user_model_id`<br>set_windows_app_user_model_id() | 137-149 | Give Windows a stable application identity so the taskbar can associate the Tk window with MBSniffer instead of a generic Python process. | `ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID` |
| `get_settings_file`<br>get_settings_file() | 153-180 | Return the persistent per-user settings file. | `folder.mkdir, os.environ.get, Path.home, Path` |
| `_validated_choice`<br>_validated_choice(value, allowed, default) | 183-184 | Validated choice. | — |
| `_validated_bool`<br>_validated_bool(value, default) | 187-188 | Validated bool. | `isinstance` |
| `_validated_number_string`<br>_validated_number_string(value, minimum, maximum, default) | 191-198 | Validated number string. | `str, float` |
| `_validated_int_string`<br>_validated_int_string(value, minimum, maximum, default) | 201-208 | Validated int string. | `str, int` |
| `load_ui_settings`<br>load_ui_settings() | 211-414 | Reads settings.json defensively and validates every persisted choice/range. | `get_settings_file, isinstance, data.get, json.loads, sniffer_in.get` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `save_ui_settings` save_ui_settings(settings=None, **updates) | 417-441 | Merges updates with validated current settings and uses a temporary file plus replace operation for atomic persistence; settings failures never prevent normal application use. | `get_settings_file, load_ui_settings, isinstance, current.update, path.parent.mkdir` |
| `get_application_directory` get_application_directory() | 444-465 | Resolves a user-facing application directory consistently for PyInstaller EXE, packaged BAT/source mode and development mode. | `getattr, (launcher_dir / 'MBSniffer.bat').is_file, Path(__file__).resolve, Path.cwd, Path(sys.executable).resolve` |
| `get_workspaces_folder` get_workspaces_folder() | 467-475 | Return/create the Master workspace folder beside settings.json:. | `folder.mkdir, get_settings_file` |
| `get_logs_folder` get_logs_folder() | 478-486 | Creates and returns the MBSniffer Logs directory beside the launcher/EXE. | `folder.mkdir, get_application_directory` |

## Detailed behavior of the key paths

**`load_ui_settings` lines 211-414**

Reads settings.json defensively and validates every persisted choice/range. Invalid/corrupt data falls back safely. It also migrates the older highlight_anomalies key to highlight_results and the old 40 ms Finder timeout default to 100 ms.

**`save_ui_settings` lines 417-441**

Merges updates with validated current settings and uses a temporary file plus replace operation for atomic persistence; settings failures never prevent normal application use.

**`get_application_directory` lines 444-465**

Resolves a user-facing application directory consistently for PyInstaller EXE, packaged BAT/source mode and development mode.

**`get_logs_folder` lines 478-486**

Creates and returns the MBSniffer Logs directory beside the launcher/EXE.

# mb_protocol.py

**Protocol engine for passive Modbus RTU parsing and session metrics.** It implements CRC16, RTU t3.5 timing, frame-shape recognition, resynchronisation, decoding, request/response pairing, timeout accounting, echo-write and broadcast classification, split-frame detection, Bus Health metrics and debug traffic generation.

**Source lines**
1007

**SHA-256**
`8c4c41051e41f0c998692206ed0204d91f057cce5fce726b15ad2cce503c1ef2`

**Imports:** `math, random, time, bisect.bisect_right, collections.defaultdict, deque, datetime.datetime, mb_config.MODBUS_EXCEPTION_CODES, PENDING_REQUEST_TIMEOUT_SECONDS, SIMULATION_FUNCTION_CODES, SIMULATION_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS`

## Module constants / configuration data

| Name | Definition |
|---|---|
| `_CRC16_TABLE` | `_build_crc16_table()` |
| `KNOWN_FRAME_FUNCTION_CODES` | `(1, 2, 3, 4, 5, 6, 7, 8, 15, 16, 22, 23)` |
| `EXACT_ECHO_FUNCTION_CODES` | `(5, 6, 22)` |
| `BROADCAST_UNIT_ID` | `0` |
| `READ_BITS_DETAILS_LIMIT` | `32` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `_build_crc16_table`<br>_build_crc16_table() | 19-29 | Build crc16 table. | `range, tuple, table.append` |
| `crc16_modbus`<br>crc16_modbus(data) | 36-41 | Computes the standard Modbus CRC-16 value (initial 0xFFFF, reflected polynomial 0xA001) with one lookup in the precomputed 256-entry _CRC16_TABLE per byte. append_crc() stores the low CRC byte first, matching Modbus RTU wire order. | — |
| `crc_ok`<br>crc_ok(frame) | 43-48 | Crc ok. | `crc16_modbus, len` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `append_crc`<br>append_crc(payload) | 50-52 | Append crc. | `crc16_modbus, bytes` |
| `hex_bytes`<br>hex_bytes(data) | 54-55 | Hex bytes. | `' '.join` |
| `modbus_rtu_interframe_gap_seconds`<br>modbus_rtu_interframe_gap_seconds(baud, bits_per_char) | 57-72 | Implements the automatic t3.5 rule used by capture and discovery: 3.5 character times at or below 19200 bit/s, fixed 1.750 ms above 19200 bit/s. | `int, float, ValueError` |
| `plausible_request_qty`<br>plausible_request_qty(fc, qty) | 74-79 | Plausible request qty. | — |
| `_plausible_unit_id`<br>_plausible_unit_id(value) | 89-94 | Return True for Modbus RTU unit addresses that can appear on the wire. | `int` |
| `_candidate_prefixes_at`<br>_candidate_prefixes_at(buf, start=0) | 97-168 | Infers only structurally plausible lengths for supported Modbus frame shapes. | `set, len, int, _plausible_unit_id, out.append` |
| `candidate_prefixes`<br>candidate_prefixes(buf) | 171-173 | Compatibility wrapper for callers/tests that inspect offset zero. | `_candidate_prefixes_at` |
| `_crc_ok_at`<br>_crc_ok_at(buf, start, length) | 176-183 | CRC check without copying the whole remaining capture buffer. | `crc16_modbus, len, memoryview` |
| `find_valid_prefix`<br>find_valid_prefix(buf, start=0) | 186-191 | Find a CRC-valid, structurally recognised Modbus frame at *start*. | `_candidate_prefixes_at, _crc_ok_at, len` |
| `guess_invalid_whole_frame`<br>guess_invalid_whole_frame(buf) | 194-201 | Classify a complete known-shape frame whose CRC is invalid. | `_candidate_prefixes_at, len, _plausible_unit_id` |
| `_unknown_whole_frame_is_valid`<br>_unknown_whole_frame_is_valid(buf) | 204-217 | Conservative fallback for one unsupported FC occupying the whole burst. | `_candidate_prefixes_at, crc_ok, _plausible_unit_id, len` |
| `split_capture_buffer`<br>split_capture_buffer(data) | 220-269 | Walks a burst from left to right. | `bytearray, find_valid_prefix, max, range, guess_invalid_whole_frame` |
| `burst_may_continue`<br>burst_may_continue(data) | 271-302 | Reports whether a burst ends with the start of a known frame that has not fully arrived yet, so the reader can wait for bytes delayed by a USB adapter latency timer instead of emitting two RAW rows. | `split_capture_buffer, any, _plausible_unit_id, len, _candidate_prefixes_at` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `read_bits_from_response`<br>read_bits_from_response(frame, qty=None) | 305-322 | Return FC01/FC02 coil/input states, least-significant bit first. | `len, range, int` |
| `read_bits_details`<br>read_bits_details(frame, qty=None) | 330-338 | Formats FC01/FC02 response bits in address order, grouped per data byte, trimmed to the request quantity when known and capped at 32 visible bits. | `read_bits_from_response, ''.join, ' '.join, len, str` |
| `decode_frame`<br>decode_frame(frame, kind_hint) | 341-449 | Extracts Slave, FC, PDU address, one-based address, quantity and selected payload details from a split frame. | `str(kind_hint).startswith, str, len, MODBUS_EXCEPTION_CODES.get, read_bits_details` |
| `format_clock_time`<br>format_clock_time(epoch) | 451-452 | Format clock time. | `datetime.fromtimestamp(float(epoch)).strftime, datetime.fromtimestamp, float` |
| `parse_burst`<br>parse_burst(data, timestamp, channel='', event_time=None, chunk_times=None) | 455-498 | Transforms split frames into the row dictionary consumed by the rest of the program. | `split_capture_buffer, time.time, float, len, decode_frame` |
| **`class SessionMetrics`** | **500-872** | **Tracks pairing, counters and bus-health diagnostics for one session.** | **Class container** |
| `SessionMetrics.__init__`<br>__init__(self, pending_timeout_seconds=PENDING_REQUEST_TIMEOUT_SECONDS) | 503-505 | Init. | `float, self.reset` |
| `SessionMetrics.reset`<br>reset(self) | 507-531 | Reset. | `set, defaultdict, deque` |
| `SessionMetrics._key`<br>_key(self, row) | 533-543 | Key. | `int, row.get` |
| `SessionMetrics.pending_count`<br>pending_count(self) | 545-546 | Pending count. | `sum, len, self.pending.values` |
| `SessionMetrics.pop_expired_requests`<br>pop_expired_requests(self) | 548-551 | Pop expired requests. | — |
| `SessionMetrics.expire_pending`<br>expire_pending(self, now=None) | 553-591 | Expire stale requests and retain their sequence IDs for the GUI. | `float, list, time.time, self.pending.keys, self.pending.pop` |
| `SessionMetrics.force_timeout`<br>force_timeout(self, slave, fc) | 593-617 | Debug/simulation helper: expire the oldest pending request for one key. | `self.pending.get, pending_list.pop, dict, self.pending.pop, int` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SessionMetrics._resolve_echo_direction`<br>_resolve_echo_direction(self, row) | 619-651 | Classifies FC05/06/08/22 frames as REQUEST or RESPONSE from the pending queue: same key, identical bytes for exact echoes and the opposite channel in RS232 dual RX. | `self._key, self.pending.get, row.get, candidate.get` |
| `SessionMetrics._record_response_time`<br>_record_response_time(self, row, response_ms) | 653-664 | Record response time. | `float, self.response_times_ms.append, self.response_times_by_slave[slave].append, int, row.get` |
| `SessionMetrics.process`<br>process(self, row) | 666-765 | This is the session accounting hot path. | `float, self.expire_pending, self.pop_expired_requests, row.get, self._record_response_time` |
| `SessionMetrics._p95`<br>_p95(values) | 768-773 | P95. | `sorted, max, min, float, len` |
| `SessionMetrics.health_summary`<br>health_summary(self, *, baud=9600, data_bits=8, parity='None', stop_bits=1, now=None) | 775-872 | Builds the Bus Health model: request rate, response average/min/max/P95, slowest Slave, unvalidated-byte rate, timeout rate, active Slave count, approximate bus utilisation and exception distribution. | `list, self._p95, self.response_times_by_slave.items, time.time, max` |
| `u16_bytes`<br>u16_bytes(value) | 874-876 | U16 bytes. | `bytes, int` |
| `corrupt_crc`<br>corrupt_crc(frame) | 878-882 | Corrupt crc. | `bytearray, bytes, len` |
| `build_simulation_transaction`<br>build_simulation_transaction(slave, fc, rng) | 884-928 | Build one syntactically valid Modbus request/response pair. | `int, rng.randint, append_crc, bytes, b''.join` |
| `build_random_simulation_plan`<br>build_random_simulation_plan(rng=None) | 930-952 | Build exactly four simulation transactions: FC01, FC02, FC03 and FC04, one each. | `rng.sample, zip, random.Random, range, build_simulation_transaction` |
| `build_anomaly_simulation_plan`<br>build_anomaly_simulation_plan(rng=None) | 956-1006 | Creates the deterministic categories required to preview diagnostic highlighting: normal response, slow response, Exception, bad CRC, timeout and RAW bytes. | `rng.sample, build_simulation_transaction, append_crc, corrupt_crc, random.Random` |

## Detailed behavior of the key paths

**`crc16_modbus` lines 36-41**

Computes the standard Modbus CRC-16 value (initial 0xFFFF, reflected polynomial 0xA001) with one lookup in the precomputed 256-entry _CRC16_TABLE per byte. append_crc() stores the low CRC byte first, matching Modbus RTU wire order.

**`modbus_rtu_interframe_gap_seconds` lines 57-72**

Implements the automatic t3.5 rule used by capture and discovery: 3.5 character times at or below 19200 bit/s, fixed 1.750 ms above 19200 bit/s.

**`_candidate_prefixes_at` lines 97-168**

Infers only structurally plausible lengths for supported Modbus frame shapes. This constrains resynchronisation and avoids the former brute-force CRC scan across arbitrary lengths.

**`split_capture_buffer` lines 220-269**

Walks a burst from left to right. A valid recognised frame is emitted immediately; otherwise the parser searches later offsets for a recognised CRC-valid frame. Bytes before the recovered frame become CRC ERROR if they exactly match a known frame shape, otherwise RAW/UNSYNC. If nothing can be recovered, the remainder becomes CRC ERROR, UNKNOWN/OK, or RAW/UNPARSED depending on its structure and CRC.

**`burst_may_continue` lines 271-302**

Reports whether a burst ends with the start of a known frame that has not fully arrived yet, so the reader can wait for bytes delayed by a USB adapter latency timer instead of emitting two RAW rows.

**`read_bits_details` lines 330-338**

Formats FC01/FC02 response bits in address order, grouped per data byte, trimmed to the request quantity when known and capped at 32 visible bits.

**`decode_frame` lines 341-449**

Extracts Slave, FC, PDU address, one-based address, quantity and selected payload details from a split frame. It handles Exceptions and common read/write Function Codes without modifying the raw bytes.

**`parse_burst` lines 455-498**

Transforms split frames into the row dictionary consumed by the rest of the program. It attaches display time, channel, decoded fields, raw hex, frame length and a floating epoch timestamp; with chunk_times each frame gets the arrival time of its own serial read.

**`SessionMetrics.process` lines 666-765**

This is the session accounting hot path. It expires old pending requests first, resolves REQ/RESP frames, updates byte/counter statistics, stores valid non-broadcast requests by (Slave, FC), FIFO-pairs responses/exceptions to pending requests, calculates response time and returns timing/pairing information to the view.

**`SessionMetrics._resolve_echo_direction` lines 619-651**

Classifies FC05/06/08/22 frames as REQUEST or RESPONSE from the pending queue: same key, identical bytes for exact echoes and the opposite channel in RS232 dual RX. Broadcast frames are always requests.

**`SessionMetrics.health_summary` lines 775-872**

Builds the Bus Health model: request rate, response average/min/max/P95, slowest Slave, unvalidated-byte rate, timeout rate, active Slave count, approximate bus utilisation and exception distribution.

**`build_anomaly_simulation_plan` lines 956-1006**

Creates the deterministic categories required to preview diagnostic highlighting: normal response, slow response, Exception, bad CRC, timeout and RAW bytes.

> **Design note.** Important design boundary: this module contains no Tkinter code and can be regression-tested independently. That separation is valuable if the project is later rebuilt for a thesis or extended with additional front ends.

# mb_capture.py

**Serial-port management and passive acquisition pipeline.** It opens selected COM ports, starts reader threads, converts inter-frame silence into capture bursts, parses those bursts, queues GUI events, controls simulation, handles COM restart and performs orderly shutdown.

**Source lines**
889

**SHA-256**
`cc61c566dd04c4ec3ce2bd61b6fef50738ccce2f85d53b7ffe1c0650f652ddb2`

**Imports:** `queue, random, threading, time, traceback, datetime.datetime, tkinter, tkinter.messagebox, mb_config.APP_NAME, APP_VERSION, MIN_PENDING_REQUEST_TIMEOUT_SECONDS, MAX_PENDING_REQUEST_TIMEOUT_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS, UI_QUEUE_BATCH_SIZE, UI_QUEUE_IDLE_INTERVAL_MS, UI_QUEUE_BUSY_INTERVAL_MS, debug_sim, mb_protocol.parse_burst, burst_may_continue, build_anomaly_simulation_plan, modbus_rtu_interframe_gap_seconds, mb_slave_finder.cancel_serial_io`

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| **`class CaptureMixin`** | **33-888** | **Methods for serial capture, COM management and simulation.** | **Class container** |
| `CaptureMixin.set_sniffer_refresh_state`<br>set_sniffer_refresh_state(self, state) | 36-41 | Apply the same state to both Sniffer COM refresh buttons. | `getattr, button.configure` |
| `CaptureMixin.update_mode_ui`<br>update_mode_ui(self) | 43-63 | Update mode ui. | `self.mode_var.get, self.port_b_combo.configure, self.update_com_status, self.mode_help_var.set, self.tr` |
| `CaptureMixin.update_gap_state`<br>update_gap_state(self) | 65-68 | Update gap state. | `self.gap_entry.configure, self.gap_mode_var.get` |
| `CaptureMixin.get_serial_modules`<br>get_serial_modules(self) | 70-76 | Get serial modules. | — |
| `CaptureMixin.refresh_ports`<br>refresh_ports(self) | 78-115 | Refresh ports. | `self.get_serial_modules, hasattr, self.update_com_status, self.port_a_var.set, self.port_b_var.set` |

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `CaptureMixin.detected_ports`<br>detected_ports(self) | 117-125 | Return the set of COM device names currently reported by Windows/pyserial. | `self.get_serial_modules, set, list_ports.comports` |
| `CaptureMixin.port_status_text`<br>port_status_text(self, label, port, enabled=True) | 127-143 | Port status text. | `self.serial_by_port.get, self.detected_ports, self.tr` |
| `CaptureMixin.update_com_status`<br>update_com_status(self) | 145-164 | Update COM status without opening/probing a port. | `self.port_a_status_var.set, self.port_b_status_var.set, hasattr, self.mode_var.get, self.port_status_text` |
| `CaptureMixin.poll_com_status`<br>poll_com_status(self) | 166-178 | Poll com status. | `self.update_com_status, hasattr, self.update_stats_labels, self.after` |
| `CaptureMixin.selected_port_plan`<br>selected_port_plan(self) | 180-198 | Translates physical mode into one or two receive channels. | `self.mode_var.get, self.port_a_var.get().strip, self.port_b_var.get().strip, ValueError, plan.append` |
| `CaptureMixin.open_selected_ports`<br>open_selected_ports(self, serial, config) | 200-213 | Open the selected capture ports and return [(channel, serial_obj), ...]. | `self.selected_port_plan, opened.append, self.open_serial_port, ser_obj.close` |
| `CaptureMixin.install_opened_ports`<br>install_opened_ports(self, opened) | 215-220 | Install opened ports. | `self.update_com_status, getattr` |
| `CaptureMixin.start_reader_threads`<br>start_reader_threads(self, opened, gap_ms) | 222-232 | Start reader threads. | `threading.Thread, self.reader_threads.append, t.start` |
| `CaptureMixin.restart_com_port`<br>restart_com_port(self) | 234-330 | Close and reopen the selected COM port(s). | `self.get_serial_modules, self.restart_btn.configure, self.status_var.set, self.stop_event.set, list` |
| `CaptureMixin.set_simulation_button_state`<br>set_simulation_button_state(self, state) | 332-335 | Change Simulation button state only when the optional button exists. | `self.sim_btn.configure` |
| `CaptureMixin.selected_pending_timeout_seconds`<br>selected_pending_timeout_seconds(self) | 337-356 | Selected pending timeout seconds. | `float, ValueError, self.pending_timeout_var.get().strip().replace, self.trf, self.tr` |
| `CaptureMixin.calculated_gap_ms`<br>calculated_gap_ms(self) | 358-370 | Returns the manual gap after validation, or computes the automatic RTU t3.5 value from baud/data/parity/stop settings using the shared protocol timing helper. | `int, float, self.gap_mode_var.get, self.baud_var.get, self.data_var.get` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `CaptureMixin.serial_config`<br>serial_config(self, serial) | 372-394 | Serial config. | `int, self.baud_var.get, self.parity_var.get, self.stop_var.get, self.data_var.get` |
| `CaptureMixin.open_serial_port`<br>open_serial_port(self, serial, port, config) | 396-428 | Open serial port. | `serial.Serial, ser.open, ser.reset_input_buffer` |
| `CaptureMixin.start_capture`<br>start_capture(self) | 430-488 | Validates state and serial settings, opens the selected port(s), resets the current session, prepares lazy logging, starts reader workers and updates the GUI to capturing state. | `self.get_serial_modules, self.mode_var.get, self.port_a_var.get().strip, self.port_b_var.get().strip, self.install_opened_ports` |
| `CaptureMixin.reader_loop`<br>reader_loop(self, ser, gap_ms, channel, generation) | 490-567 | Runs in a daemon thread per capture channel. | `max, min, bytearray, parse_burst, self.event_queue.put` |
| `CaptureMixin.stop_capture`<br>stop_capture(self) | 569-572 | Stop capture. | `self.stop_event.set, self.status_var.set, self.tr` |
| `CaptureMixin.run_simulation`<br>run_simulation(self) | 574-675 | Runs the internal diagnostic scenario without opening serial hardware. | `self.reset_session_metrics, self.close_log, self.status_var.set, self.start_btn.configure, self.restart_btn.configure` |
| `CaptureMixin.process_queue`<br>process_queue(self) | 677-822 | Main-thread bridge between workers and Tkinter. | `range, flush_frames, self.metrics.expire_pending, time.time, self.metrics.pop_expired_requests` |
| `CaptureMixin.on_close`<br>on_close(self) | 824-888 | Performs defensive shutdown: saves settings, signals all workers, cancels serial I/O, joins threads with timeouts, closes serial handles/logs and destroys the Tk root. | `getattr, self.stop_event.set, self.discovery_stop_event.set, cancel_serial_io, callable` |

## Detailed behavior of the key paths

**`selected_port_plan` lines 180-198**

Translates physical mode into one or two receive channels. RS485 and single-RX RS232 use COM A; dual-RX RS232 requires distinct COM A and COM B and labels their directions A→B/B→A.

**`calculated_gap_ms` lines 358-370**

Returns the manual gap after validation, or computes the automatic RTU t3.5 value from baud/data/parity/stop settings using the shared protocol timing helper.

**`start_capture` lines 430-488**

Validates state and serial settings, opens the selected port(s), resets the current session, prepares lazy logging, starts reader workers and updates the GUI to capturing state.

**`reader_loop` lines 490-567**

Runs in a daemon thread per capture channel. It accumulates serial bytes and their read times into a burst, considers the burst complete after the configured inter-frame silence (or after the split-frame hold when the burst ends mid-frame), then calls parse_burst() and posts frame events to the GUI queue.

**`run_simulation` lines 574-675**

Runs the internal diagnostic scenario without opening serial hardware. It feeds the same parser/event path used by real capture, including forced timeout handling.

**`process_queue` lines 677-822**

Main-thread bridge between workers and Tkinter. It drains events in bounded batches, batches frame insertions for performance, handles capture/discovery status events, expires requests and reschedules itself with a faster cadence when backlog exists.

**`on_close` lines 824-888**

Performs defensive shutdown: saves settings, signals all workers, cancels serial I/O, joins threads with timeouts, closes serial handles/logs and destroys the Tk root.

> **Design note.** Tkinter widgets are never updated directly by reader threads. Workers communicate through event_queue; process_queue() runs on the Tk main thread.

# mb_diagnostics.py

**Presentation-oriented diagnostics derived from captured records.** It maps Function Codes to names, recalculates/display CRC fields, extracts payloads, classifies anomalies/results and builds the structured Frame Inspector text.

| Source lines | SHA-256 |
| --- | --- |
| 286 | `d5edc15bf0259d130fdf9b42ffaf2caca770782101aeed1886f980e0fcf24bda` |

**Imports:** `mb_config.MODBUS_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS, mb_protocol.crc16_modbus, read_bits_from_response, mb_i18n.LANGUAGE_PORTUGUESE, translate_details, translate_text`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `FUNCTION_NAMES` | `{1: 'Read Coils', 2: 'Read Discrete Inputs', 3: 'Read Holding Registers', 4: 'Read Input Registers', 5: 'Write Single Coil', 6: 'Write Single Regis…` |
| `RESULT_LABELS` | `{'success': ('✓ Sucesso', 'Verde'), 'slow': ('⚠ Resposta lenta', 'Amarelo'), 'exception': ('Exception', 'Vermelho'), 'crc': ('× CRC Error', 'Vermel…` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `raw_hex_to_bytes`<br>raw_hex_to_bytes(raw) | 26-30 | Raw hex to bytes. | `bytes.fromhex, str` |
| `fc_number`<br>fc_number(record) | 33-41 | Fc number. | `str(record.get('fc', '') or '').strip, int, record.get, str` |
| `function_label`<br>function_label(record, language=LANGUAGE_PORTUGUESE) | 44-49 | Function label. | `fc_number, FUNCTION_NAMES.get, translate_text` |
| `crc_fields`<br>crc_fields(record) | 52-62 | Crc fields. | `str(record.get('type', '')).startswith, raw_hex_to_bytes, crc16_modbus, record.get, len` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `response_payload`<br>response_payload(record, language=LANGUAGE_PORTUGUESE) | 65-127 | Return (byte_count, data_hex, decoded_data) where available. | `raw_hex_to_bytes, str, record.get, len, MODBUS_EXCEPTION_CODES.get` |
| `anomaly_kind`<br>anomaly_kind(record, threshold_ms=SLOW_RESPONSE_THRESHOLD_MS) | 130-153 | Classifies a record with explicit priority: bad CRC, timeout, Exception, RAW, then slow response. | `bool, str(record.get('type', '')).startswith, str, record.get, float` |
| `result_info`<br>result_info(record, successful_request=False, language=LANGUAGE_PORTUGUESE) | 168-207 | Turns the anomaly/pairing state into a semantic result key, user-facing label and color name. | `anomaly_kind, str, record.get, translate_text` |
| `inspector_values`<br>inspector_values(record, successful_request=False, language=LANGUAGE_PORTUGUESE) | 210-264 | Builds every Frame Inspector field, including recalculated CRC received/calculated values, decoded payload, timeout suffix and translated result text. | `crc_fields, response_payload, str, result_info, record.get` |
| `decoded_record_text`<br>decoded_record_text(record, successful_request=False, language=LANGUAGE_PORTUGUESE) | 267-286 | Serialises the same Frame Inspector information into newline-separated text for the copy-decoded-frame context command. | `inspector_values, '\n'.join` |

## Detailed behavior of the key paths

**`anomaly_kind` lines 130-153**

Classifies a record with explicit priority: bad CRC, timeout, Exception, RAW, then slow response. The priority prevents conflicting colors/results.

**`result_info` lines 168-207**

Turns the anomaly/pairing state into a semantic result key, user-facing label and color name. Successful completed pairs are distinguished from pending requests.

**`inspector_values` lines 210-264**

Builds every Frame Inspector field, including recalculated CRC received/calculated values, decoded payload, timeout suffix and translated result text.

**`decoded_record_text` lines 267-286**

Serialises the same Frame Inspector information into newline-separated text for the copy-decoded-frame context command.

# mb_view.py

**Traffic view, filtering, sorting, Frame Inspector, Bus Health updates, CSV export and TXT logging.** It owns the bounded in-memory frame history, converts parser rows into richer view records, pairs display state with SessionMetrics, maintains table/raw views and keeps the real capture log independent from GUI filtering.

| Source lines | SHA-256 |
| --- | --- |
| 1871 | `0c65dd97ebf6d836431fa4dc0b729e3aab9f1351559de04f3dd2b713ce781601` |

**Imports:** `csv, math, os, subprocess, sys, time, datetime.datetime, tkinter, tkinter.filedialog, messagebox, mb_config.MAX_UI_FRAMES, MAX_RAW_TEXT_LINES, UI_PRUNE_CHUNK_FRAMES, RAW_TEXT_PRUNE_CHUNK_LINES, SLOW_RESPONSE_THRESHOLD_MS, get_application_directory, get_logs_folder, mb_diagnostics.anomaly_kind, decoded_record_text, inspector_values, mb_i18n.canonical_ui_text, translate_details, mb_protocol.read_bits_details`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `TRAFFIC_MENU_OPEN_IN_MASTER` | `8` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| **`class ViewMixin`** | **36-1870** | **Methods for rendering, filtering, statistics and logging.** | **Class container** |
| `ViewMixin.reset_session_metrics`<br>reset_session_metrics(self) | 39-41 | Reset session metrics. | `self.metrics.reset, self.update_stats_labels` |
| `ViewMixin.update_stats_labels`<br>update_stats_labels(self) | 43-108 | Converts SessionMetrics into both compact session counters and Bus Health values. | `self.stat_requests_var.set, self.stat_responses_var.set, self.stat_pending_var.set, self.stat_timeouts_var.set, self.stat_crc_var.set` |
| `ViewMixin.pulse_bus_activity`<br>pulse_bus_activity(self) | 113-132 | Pulse bus activity. | `getattr, var.set, self.after, self.after_cancel` |
| `ViewMixin._clear_bus_activity_pulse`<br>_clear_bus_activity_pulse(self) | 134-138 | Clear bus activity pulse. | `getattr, var.set` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.schedule_advanced_filter_refresh`<br>schedule_advanced_filter_refresh(self, *_args) | 140-153 | Schedules advanced filter refresh. | `getattr, self.after, self.after_cancel` |
| `ViewMixin.clear_advanced_filters`<br>clear_advanced_filters(self) | 155-175 | Clear advanced filters. | `getattr, self.advanced_filter_type_var.set, self.advanced_filter_fc_var.set, self.advanced_filter_response_ms_var.set, self.advanced_filter…` |
| `ViewMixin.normalized_fc_filter`<br>normalized_fc_filter(value) | 178-190 | Normalized fc filter. | `str(value or '').strip().upper, text.startswith, int, str(value or '').strip, str` |
| `ViewMixin.record_passes_advanced_filters`<br>record_passes_advanced_filters(self, record) | 192-253 | Implements Type, FC, response-time and free-text filtering. | `getattr, canonical_ui_text, str, self.normalized_fc_filter, self.advanced_filter_response_ms_var.get().strip` |
| `ViewMixin.configure_anomaly_tags`<br>configure_anomaly_tags(self, dark=None) | 255-286 | Configure anomaly tags. | `palette.items, hasattr, bool, self.tree.tag_configure, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get` |
| `ViewMixin.anomaly_tag_for_record`<br>anomaly_tag_for_record(self, record) | 288-325 | Maps the semantic result of a record to the Treeview tag used for colour highlighting, including completed normal REQUEST/RESPONSE pairs. | `anomaly_kind, str, getattr, self.highlight_anomalies_var.get, record.get` |
| `ViewMixin.on_highlight_anomalies_changed`<br>on_highlight_anomalies_changed(self) | 327-334 | Event handler for highlight anomalies changed. | `self.configure_anomaly_tags, self.configure_inspector_result_styles, self.render_filtered_view, self.update_frame_inspector, getattr` |
| `ViewMixin.mark_timed_out_records`<br>mark_timed_out_records(self, expired, rerender=True) | 336-362 | Finds original REQUEST records by their sequence IDs and marks them timed_out rather than inventing artificial timeout frames. | `len, item.get, changed_records.append, canonical_ui_text, self.render_filtered_view` |
| `ViewMixin.refresh_record_tags`<br>refresh_record_tags(self, records) | 364-376 | Re-applies the result tag to the Treeview rows of the given records and refreshes the Frame Inspector when one of them is selected. | `self.selected_record, self.anomaly_tag_for_record, self._tree_iids_by_seq.get, any, self.update_frame_inspector` |
| `ViewMixin.selected_record`<br>selected_record(self) | 378-382 | Selected record. | `self.tree.selection, self._record_by_iid.get` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.update_frame_inspector`<br>update_frame_inspector(self, record=None) | 384-413 | Builds the structured inspector values for the selected traffic record and applies result styling while preserving the raw frame. | `bool, inspector_values, self.inspector_vars.items, self.selected_raw_var.set, getattr` |
| `ViewMixin.toggle_frame_inspector`<br>toggle_frame_inspector(self) | 415-427 | Toggle frame inspector. | `getattr, callable, self.inspector_body.grid, self.inspector_toggle_btn.configure, self.inspector_body.grid_remove` |
| `ViewMixin.copy_to_clipboard`<br>copy_to_clipboard(self, text) | 429-435 | Copy to clipboard. | `self.clipboard_clear, self.clipboard_append, self.update_idletasks, str` |
| `ViewMixin.copy_selected_raw`<br>copy_selected_raw(self) | 437-440 | Copy selected raw. | `self.selected_record, self.copy_to_clipboard, record.get` |
| `ViewMixin.copy_selected_decoded`<br>copy_selected_decoded(self) | 442-455 | Copy selected decoded. | `self.selected_record, bool, self.copy_to_clipboard, decoded_record_text, record.get` |
| `ViewMixin.context_filter_selected_slave`<br>context_filter_selected_slave(self) | 457-470 | Context filter selected slave. | `self.selected_record, str, self.slave_filter_vars.items, self.update_slave_heading, self.render_filtered_view` |
| `ViewMixin.context_filter_selected_fc`<br>context_filter_selected_fc(self) | 472-486 | Context filter selected fc. | `self.selected_record, str, self.advanced_filter_fc_var.set, self.render_filtered_view, record.get` |
| `ViewMixin.context_show_selected_transaction`<br>context_show_selected_transaction(self) | 488-517 | Context show selected transaction. | `self.selected_record, record.get, self.slave_filter_vars.values, self.update_slave_heading, self.advanced_filter_type_var.set` |
| `ViewMixin.clear_transaction_filter`<br>clear_transaction_filter(self) | 519-521 | Clear transaction filter. | `self.render_filtered_view` |
| `ViewMixin.context_open_selected_in_master`<br>context_open_selected_in_master(self) | 523-526 | Context open selected in master. | `self.selected_record, self.master_open_from_record` |
| `ViewMixin.show_traffic_context_menu`<br>show_traffic_context_menu(self, event) | 528-552 | Show traffic context menu. | `self.tree.identify_row, self.tree.selection_set, self.tree.focus, self.on_row_select, getattr` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.export_session_csv`<br>export_session_csv(self, path=None) | 554-656 | Exports the frames retained by the GUI history, including timing, decoded addressing, raw data and anomaly/timeout fields. | `list, datetime.now().strftime, filedialog.asksaveasfilename, messagebox.showinfo, open` |
| `ViewMixin.tree_record_column_values`<br>tree_record_column_values(self, record) | 658-669 | Tree record column values. | `record.get, translate_details` |
| `ViewMixin.measure_tree_column_width`<br>measure_tree_column_width(self, column, records, heading_text=None) | 671-689 | Measure the pixel width required by a non-wrapped Treeview column. | `self.tree_heading_font.measure, self.tree_headings.get, str, self.tree_record_column_values(record).get, max` |
| `ViewMixin.available_details_width`<br>available_details_width(self) | 691-709 | Return the width that can be assigned to Details without intentionally forcing horizontal scrolling in the current Treeview viewport. | `int, max, self.tree.winfo_width, self.tree.column` |
| `ViewMixin.wrap_text_to_pixels`<br>wrap_text_to_pixels(self, value, max_px) | 711-761 | Word-wrap text using the actual Tk font width. | `str, max, '\n'.join, int, text.splitlines` |
| `ViewMixin.details_horizontal_rule`<br>details_horizontal_rule(self) | 763-775 | Return a rule that fills the Details column as far as the Treeview cell permits. | `max, int, self.tree_cell_font.measure, self.tree.column, math.ceil` |
| `ViewMixin.completed_pairs_for_records`<br>completed_pairs_for_records(self, records) | 777-817 | Return complete visible REQUEST/RESPONSE pairs in current display order. | `pairs.sort, record.get, response.get, seq_to_index.get, min` |
| `ViewMixin.separator_after_sequences`<br>separator_after_sequences(self, records) | 819-836 | Return logical frame seq ids that need a separator AFTER them. | `self.completed_pairs_for_records, self.separate_transactions_var.get, set, len` |
| `ViewMixin.tree_separator_values`<br>tree_separator_values(self) | 838-843 | Tree separator values. | `self.details_horizontal_rule` |
| `ViewMixin.insert_tree_separator_after_seq`<br>insert_tree_separator_after_seq(self, seq) | 845-863 | Insert one Details-width separator after the visual rows of seq. | `self._tree_iids_by_seq.get, self.tree.insert, self.tree.index, self.tree_separator_values` |
| `ViewMixin.sync_tree_transaction_separators`<br>sync_tree_transaction_separators(self, records=None) | 865-885 | Synchronize standalone transaction separator rows without rebuilding normal traffic rows. | `self.separator_after_sequences, set, self.records_for_display, self._tree_separator_iid_by_seq.pop, self._raw_by_iid.pop` |
| `ViewMixin.detail_lines_for_record`<br>detail_lines_for_record(self, record) | 887-909 | Return wrapped Details as separate visual Treeview rows. | `max, self.wrap_text_to_pixels, wrapped.splitlines, translate_details, int` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.set_tree_rowheight_for_records`<br>set_tree_rowheight_for_records(self, records) | 911-916 | Treeview row height stays fixed; wrapping uses continuation rows. | `self.tree_style.configure` |
| `ViewMixin.update_tree_rowheight_for_record`<br>update_tree_rowheight_for_record(self, record) | 918-920 | No-op by design: one logical frame may use several compact rows. | — |
| `ViewMixin.autofit_tree_columns`<br>autofit_tree_columns(self, records=None) | 922-965 | Non-Details columns fit the longest visible value/title. | `self.measure_tree_column_width, max, self.tree.column, self.set_tree_rowheight_for_records, self.records_for_display` |
| `ViewMixin.expand_tree_columns_for_record`<br>expand_tree_columns_for_record(self, record) | 967-1017 | Incrementally enlarge only the compact non-Details columns. | `self.tree_record_column_values, self.tree_heading_font.measure, self.tree_cell_font.measure, int, self.measure_tree_column_width` |
| `ViewMixin.refresh_tree_heading_width`<br>refresh_tree_heading_width(self, column) | 1019-1035 | Dynamic Hora/Slave headings remain compact. | `self.records_for_display, self.measure_tree_column_width, self.tree.column` |
| `ViewMixin.schedule_tree_reflow`<br>schedule_tree_reflow(self, _event=None) | 1037-1051 | Debounce window/Treeview resizing and then re-wrap Details to the new available width. | `self.after, self.after_cancel` |
| `ViewMixin.reflow_tree_after_resize`<br>reflow_tree_after_resize(self) | 1053-1057 | Reflow tree after resize. | `self.render_filtered_view, hasattr` |
| `ViewMixin._slave_sort_key`<br>_slave_sort_key(self, slave) | 1059-1063 | Slave sort key. | `int, str` |
| `ViewMixin.rebuild_slave_filter_menu`<br>rebuild_slave_filter_menu(self) | 1065-1092 | Rebuild slave filter menu. | `self.slave_filter_menu.delete, self.slave_filter_menu.add_command, self.update_slave_heading, self.slave_filter_menu.add_separator, sorted` |
| `ViewMixin.update_slave_heading`<br>update_slave_heading(self) | 1094-1111 | Update slave heading. | `self.tree.heading, self.refresh_tree_heading_width, self.tr, self.slave_filter_vars.items, var.get` |
| `ViewMixin.show_slave_filter_menu`<br>show_slave_filter_menu(self) | 1113-1126 | Open the slave filter menu at the position where the Slave heading was clicked. | `self.winfo_pointerx, self.slave_filter_menu.tk_popup, self.winfo_pointery, self.slave_filter_menu.grab_release` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.register_slave_for_filter`<br>register_slave_for_filter(self, slave) | 1128-1139 | Register slave for filter. | `str(slave or '').strip, tk.BooleanVar, self.rebuild_slave_filter_menu, str` |
| `ViewMixin.select_all_slaves`<br>select_all_slaves(self) | 1141-1146 | Select all slaves. | `self.slave_filter_vars.values, self.update_slave_heading, self.render_filtered_view, var.set` |
| `ViewMixin.select_no_slaves`<br>select_no_slaves(self) | 1148-1153 | Select no slaves. | `self.slave_filter_vars.values, self.update_slave_heading, self.render_filtered_view, var.set` |
| `ViewMixin.on_slave_filter_changed`<br>on_slave_filter_changed(self) | 1155-1165 | Event handler for slave filter changed. | `self.update_slave_heading, self.render_filtered_view, var.get, bool, all` |
| `ViewMixin.selected_slave_ids`<br>selected_slave_ids(self) | 1167-1171 | Selected slave ids. | `self.slave_filter_vars.items, var.get` |
| `ViewMixin.record_passes_slave_filter`<br>record_passes_slave_filter(self, record) | 1173-1183 | Record passes slave filter. | `self.selected_slave_ids, str, record.get` |
| `ViewMixin.time_order_descending`<br>time_order_descending(self) | 1185-1186 | Time order descending. | `canonical_ui_text(self.time_order_var.get()).startswith, canonical_ui_text, self.time_order_var.get` |
| `ViewMixin.on_time_order_changed`<br>on_time_order_changed(self) | 1188-1197 | Event handler for time order changed. | `self.tree.heading, self.render_filtered_view, self.time_order_descending, self.tr` |
| `ViewMixin.toggle_time_order`<br>toggle_time_order(self) | 1199-1204 | Toggle time order. | `self.time_order_descending, self.on_time_order_changed, self.time_order_var.set, self.tr` |
| `ViewMixin.records_for_display`<br>records_for_display(self) | 1206-1216 | Applies current filtering and time-order state to the bounded source-of-truth frame history. | `self.time_order_descending, records.reverse, self.record_passes_slave_filter, self.record_passes_advanced_filters` |
| `ViewMixin.raw_line_for_record`<br>raw_line_for_record(self, record) | 1218-1241 | Raw line for record. | `record.get, translate_details` |
| `ViewMixin.raw_separator_width_chars`<br>raw_separator_width_chars(self, raw_lines=None) | 1243-1263 | Width of one Raw Hex separator in visible character cells. | `max, self.raw_text_font.measure, int, self.raw_text.winfo_width` |
| `ViewMixin.raw_transaction_separator`<br>raw_transaction_separator(self, width_chars=None) | 1265-1268 | Raw transaction separator. | `max, int` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.schedule_raw_separator_reflow`<br>schedule_raw_separator_reflow(self, _event=None) | 1270-1280 | Schedules raw separator reflow. | `self.after, self.after_cancel` |
| `ViewMixin.reflow_raw_separator_after_resize`<br>reflow_raw_separator_after_resize(self) | 1282-1285 | Reflow raw separator after resize. | `self.separate_transactions_var.get, self.render_raw_text_only` |
| `ViewMixin.insert_tree_record`<br>insert_tree_record(self, record, index='end', allow_separator=True) | 1287-1355 | Insert tree record. | `self.detail_lines_for_record, visual_rows.append, self.anomaly_tag_for_record, list, self.tree.insert` |
| `ViewMixin.render_raw_text_only`<br>render_raw_text_only(self) | 1357-1410 | Render raw text only. | `self.records_for_display, self.separator_after_sequences, list, self.raw_separator_width_chars, self.raw_transaction_separator` |
| `ViewMixin.render_filtered_view`<br>render_filtered_view(self) | 1412-1472 | Render filtered view. | `self.selected_record, self.tree.get_children, self._tree_history.clear, self._tree_iids_by_seq.clear, self._tree_separator_iid_by_seq.clear` |
| `ViewMixin.add_frame_row`<br>add_frame_row(self, row) | 1474-1476 | Add frame row. | `self.add_frame_rows_batch` |
| `ViewMixin.add_frame_rows_batch`<br>add_frame_rows_batch(self, rows) | 1478-1676 | Central view ingestion path. | `self.pulse_bus_activity, ''.join, self.update_stats_labels, self.prune_ui_history, self.metrics.process` |
| `ViewMixin.prune_ui_history`<br>prune_ui_history(self) | 1678-1701 | Keeps memory/UI work bounded by removing old records in chunks after a hysteresis threshold. | `self._successful_request_seqs.intersection_update, self.render_filtered_view, len, self._frame_history.popleft, record.get` |
| `ViewMixin.on_row_select`<br>on_row_select(self, _event=None) | 1703-1705 | Event handler for row select. | `self.selected_record, self.update_frame_inspector` |
| `ViewMixin.finish_capture_ui`<br>finish_capture_ui(self) | 1707-1720 | Finish capture ui. | `self.status_var.set, self.start_btn.configure, self.stop_btn.configure, self.restart_btn.configure, self.set_simulation_button_state` |
| `ViewMixin.app_directory`<br>app_directory(self) | 1722-1723 | App directory. | `get_application_directory` |
| `ViewMixin.prepare_log_session`<br>prepare_log_session(self, prefix, started_at, header) | 1725-1738 | Prepare a real-capture log session without creating a file. | `self.close_log, started_at.strftime` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ViewMixin.ensure_log_started`<br>ensure_log_started(self) | 1740-1779 | Creates the TXT capture log lazily only when real traffic actually arrives, preventing empty logs from start/stop operations with no data. | `target.open, get_logs_folder, self.log_file.write, messagebox.showerror, ''.join` |
| `ViewMixin.queue_log_note`<br>queue_log_note(self, text) | 1781-1789 | Write a capture note to an already-open log, or keep it in memory until the first traffic frame creates the log. | `self.log_line, self.pending_log_notes.append` |
| `ViewMixin.log_line`<br>log_line(self, text) | 1791-1796 | Log line. | `self.log_file.write` |
| `ViewMixin.close_log`<br>close_log(self) | 1798-1813 | Close log. | `self.log_file.write, self.log_file.close, datetime.now().isoformat, datetime.now` |
| `ViewMixin.open_log_folder`<br>open_log_folder(self) | 1815-1825 | Open log folder. | `get_logs_folder, os.startfile, messagebox.showerror, str, subprocess.Popen` |
| `ViewMixin.clear_view`<br>clear_view(self) | 1827-1870 | Clear view. | `self.tree.get_children, self.raw_text.delete, self.selected_raw_var.set, self._tree_history.clear, self._tree_iids_by_seq.clear` |

## Detailed behavior of the key paths

**`update_stats_labels` lines 43-108**

Converts SessionMetrics into both compact session counters and Bus Health values. It recalculates health using the currently configured serial parameters so approximate bus utilisation reflects the selected format.

**`record_passes_advanced_filters` lines 192-253**

Implements Type, FC, response-time and free-text filtering. Filtering is display-only; it does not alter SessionMetrics or the on-disk capture log.

**`anomaly_tag_for_record` lines 288-325**

Maps the semantic result of a record to the Treeview tag used for colour highlighting, including completed normal REQUEST/RESPONSE pairs.

**`mark_timed_out_records`** **lines 336-362**

Finds original REQUEST records by their sequence IDs and marks them timed_out rather than inventing artificial timeout frames. Visible rows are retagged in place; the table is rebuilt only while the Timeouts filter is active.

**`refresh_record_tags`** **lines 364-376**

Re-applies the result tag to the Treeview rows of the given records and refreshes the Frame Inspector when one of them is selected.

**`update_frame_inspector`** **lines 384-413**

Builds the structured inspector values for the selected traffic record and applies result styling while preserving the raw frame.

**`export_session_csv`** **lines 554-656**

Exports the frames retained by the GUI history, including timing, decoded addressing, raw data and anomaly/timeout fields. This differs intentionally from the complete TXT capture log.

**`records_for_display`** **lines 1206-1216**

Applies current filtering and time-order state to the bounded source-of-truth frame history.

**`add_frame_rows_batch`** **lines 1478-1676**

Central view ingestion path. It assigns stable sequence IDs, runs SessionMetrics, enriches parser rows into view records (including the matched request address/quantity for read responses), tracks successful pairs, writes complete real traffic to disk, updates visible table/raw views, prunes history and auto-scrolls.

**`prune_ui_history`** **lines 1678-1701**

Keeps memory/UI work bounded by removing old records in chunks after a hysteresis threshold. The disk log is independent and remains complete.

**`ensure_log_started`** **lines 1740-1779**

Creates the TXT capture log lazily only when real traffic actually arrives, preventing empty logs from start/stop operations with no data.

**Design note.** There are deliberately two histories: the bounded GUI history and the complete TXT log. Sorting/filtering only changes presentation; statistics are derived before filtering.

# mb_slave_finder.py

**Active Modbus RTU Slave discovery and Device Identification.** It checks that the bus is idle, builds safe read-only FC03/FC04 probes with configurable attempts, calculates adaptive timeouts, performs robust serial writes/reads, accepts normal or CRC-valid Exception responses as proof of a Slave, optionally reads FC43/14 Basic Device Identification and drives the Finder UI through queued events.

| Source lines | SHA-256 |
| --- | --- |
| 1625 | `f192a6305486ea73411d920d9b6b9de19091bc612afef60ea3f5871fcd66938b` |

**Imports:** `threading, time, tkinter, tkinter.ttk, messagebox, mb_config.DISCOVERY_BAUD_RATES, DISCOVERY_PARITIES, DISCOVERY_STOP_BITS, DISCOVERY_DEFAULT_MIN_TIMEOUT_MS, DISCOVERY_MIN_TIMEOUT_MS, DISCOVERY_MAX_TIMEOUT_MS, DISCOVERY_PROBE_ADDRESS, DISCOVERY_PROBE_QTY, DISCOVERY_WRITE_TIMEOUT_SECONDS, DISCOVERY_WRITE_RETRIES, DISCOVERY_RX_BUFFER_LIMIT, DISCOVERY_MIN_ATTEMPTS, DISCOVERY_MAX_ATTEMPTS, DISCOVERY_DEFAULT_ATTEMPTS, DISCOVERY_IDLE_CHECK_SECONDS, MODBUS_EXCEPTION_CODES, mb_protocol.append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds, mb_widgets.RefreshButton`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `DEVICE_ID_FC` | `43` |
| `DEVICE_ID_MEI_TYPE` | `14` |
| `DEVICE_ID_READ_BASIC` | `1` |
| `DEVICE_ID_MORE_FOLLOWS` | `255` |
| `DEVICE_ID_BASIC_OBJECTS` | `(0, 1, 2)` |
| `DEVICE_ID_MAX_PAGES` | `8` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `build_discovery_probe`<br>build_discovery_probe(slave, fc) | 35-48 | Constructs a read-only FC03/FC04 request for Address 0, Quantity 1 and appends the RTU CRC. | `bytes, append_crc, ValueError, int` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `build_device_identification_probe`<br>build_device_identification_probe(slave, object_id=0) | 57-71 | Builds FC43 / MEI type 14 / Basic Device Identification requests, beginning at a selectable Object ID. | `int, append_crc, ValueError, bytes` |
| `_decode_device_id_value`<br>_decode_device_id_value(value) | 73-75 | Decode a Modbus Device Identification ASCII object for display. | `bytes(value).decode('ascii', errors='replace').strip, bytes(value).decode, bytes` |
| `find_device_identification_response`<br>find_device_identification_response(buffer, slave) | 77-158 | Parses FC43/14 responses and Exceptions from a receive buffer, validates CRC, decodes variable-length ASCII objects and exposes More Follows / Next Object ID for paged identification. | `bytes, int, range, max, len` |
| `read_device_identification_response`<br>read_device_identification_response(ser, slave, timeout_s, stop_event) | 160-182 | Read until a valid FC43/14 response/exception is received or timed out. | `bytearray, find_device_identification_response, time.perf_counter, max, stop_event.is_set` |
| `device_identification_timeout_seconds`<br>device_identification_timeout_seconds(baud, parity, stopbits, minimum_timeout_ms=DISCOVERY_DEFAULT_MIN_TIMEOUT_MS) | 184-201 | Conservative timeout for one FC43/14 response, up to one RTU frame. | `max, discovery_bits_per_char, int, float, modbus_rtu_interframe_gap_seconds` |
| `device_identification_request_transmit_seconds`<br>device_identification_request_transmit_seconds(baud, parity, stopbits) | 203-207 | Estimated on-wire duration of the 7-byte FC43/14 request. | `float, discovery_bits_per_char, max, int` |
| `format_device_identification`<br>format_device_identification(result) | 209-224 | Return compact Device Identification text for the Finder result table. | `any, result.get, int, objects.get, ' \| '.join` |
| `read_basic_device_identification`<br>read_basic_device_identification(ser, slave, baud, parity, stopbits, minimum_timeout_ms, stop_event, serial_module) | 226-301 | Repeats FC43/14 requests while More Follows is set, merging objects until VendorName, ProductCode and MajorMinorRevision are available or the paging limit/response terminates. | `range, stop_event.is_set, build_device_identification_probe, write_discovery_request, read_device_identification_response` |
| `discovery_bits_per_char`<br>discovery_bits_per_char(parity, stopbits) | 303-307 | Return serial bits/character: start + 8 data + parity + stop. | `float` |
| `discovery_response_timeout_seconds`<br>discovery_response_timeout_seconds(baud, parity, stopbits, minimum_timeout_ms=DISCOVERY_DEFAULT_MIN_TIMEOUT_MS) | 309-330 | Calculates a response deadline from serial character time, the expected 7-byte FC03/FC04 response and an inter-frame/processing margin, never below the user-configured minimum. | `max, discovery_bits_per_char, int, float, modbus_rtu_interframe_gap_seconds` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `discovery_request_transmit_seconds`<br>discovery_request_transmit_seconds(baud, parity, stopbits) | 332-334 | Estimated on-wire duration of the fixed 8-byte discovery request. | `float, discovery_bits_per_char, max, int` |
| `find_discovery_response`<br>find_discovery_response(buffer, slave, fc) | 336-379 | Scans received bytes for the requested Slave and Function Code. | `bytes, int, range, max, len` |
| `read_discovery_response`<br>read_discovery_response(ser, slave, fc, timeout_s, stop_event) | 381-407 | Read non-blocking serial bytes until a valid probe response or timeout. | `bytearray, find_discovery_response, time.perf_counter, max, stop_event.is_set` |
| `bus_activity_detected`<br>bus_activity_detected(ser, listen_s, stop_event) | 409-427 | Listens passively for the idle-check period and returns any bytes received; a non-empty result means another device is transmitting and the scan must not start. | `bytearray, bytes, time.perf_counter, max, stop_event.is_set` |
| `cancel_serial_io`<br>cancel_serial_io(ser) | 430-440 | Best-effort cancellation of pending serial I/O. | `getattr, callable, method` |
| `recover_discovery_write`<br>recover_discovery_write(ser) | 442-457 | Best-effort cleanup after a discovery write timeout. | `getattr, callable, ser.reset_output_buffer, ser.reset_input_buffer, cancel` |
| `write_discovery_request`<br>write_discovery_request(ser, request, baud, parity, stopbits, stop_event, serial_module) | 459-523 | Transmits one probe without using ser.flush(), retries one write timeout after driver/buffer recovery, waits for estimated on-wire transmission time and raises actual driver errors instead of disguising them as no-response Slaves. | `bytes, getattr, discovery_bits_per_char, max, range` |
| `format_duration`<br>format_duration(seconds) | 525-531 | Format duration. | `max, divmod, int, round, float` |
| **`class SlaveFinderMixin`** | **534-1624** | **Methods that implement the active Bus Slave Finder tab.** | **Class container** |
| `SlaveFinderMixin.build_discovery_tab`<br>build_discovery_tab(self, page) | 537-921 | Build the full-window active Modbus RTU slave-discovery workspace. | `getattr(self, 'ui_settings', {}).get, set, page.columnconfigure, page.rowconfigure, ttk.LabelFrame` |
| `SlaveFinderMixin._show_discovery_context_menu`<br>_show_discovery_context_menu(self, event) | 923-932 | Show discovery context menu. | `self.discovery_tree.identify_row, self.discovery_tree.selection_set, self.discovery_context_menu.entryconfigure, self.discovery_context_men…` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SlaveFinderMixin.discovery_open_selected_in_master` discovery_open_selected_in_master(self) | 934-950 | Open the selected found Slave in a new Master window. | `self.discovery_tree.selection, self.discovery_tree.item, {'N': 'None', 'E': 'Even', 'O': 'Odd'}.get, self.master_open_definition, int` |
| `SlaveFinderMixin.fit_discovery_columns_to_viewport` fit_discovery_columns_to_viewport(self, _event=None) | 952-981 | Fit Slave Finder columns to the visible Treeview width. | `int, max, sum, len, enumerate` |
| `SlaveFinderMixin.on_discovery_options_changed` on_discovery_options_changed(self) | 983-986 | Event handler for discovery options changed. | `hasattr, self.update_discovery_estimate, self.update_discovery_start_state` |
| `SlaveFinderMixin.selected_discovery_configs` selected_discovery_configs(self) | 988-997 | Selected discovery configs. | `self.discovery_baud_vars[baud].get, self.discovery_parity_vars[p].get, self.discovery_stop_vars[s].get` |
| `SlaveFinderMixin.selected_discovery_slaves` selected_discovery_slaves(self) | 999-1006 | Selected discovery slaves. | `int, list, self.discovery_slave_start_var.get, self.discovery_slave_end_var.get, ValueError` |
| `SlaveFinderMixin.selected_discovery_min_timeout_ms` selected_discovery_min_timeout_ms(self) | 1008-1018 | Selected discovery min timeout ms. | `float, self.discovery_min_timeout_var.get, ValueError, self.trf` |
| `SlaveFinderMixin.selected_discovery_attempts` selected_discovery_attempts(self) | 1020-1030 | Selected discovery attempts. | `int, self.discovery_attempts_var.get, ValueError, self.trf` |
| `SlaveFinderMixin.discovery_config_label` discovery_config_label(self, parity, stopbits) | 1032-1034 | Discovery config label. | — |
| `SlaveFinderMixin.update_discovery_estimate` update_discovery_estimate(self) | 1036-1093 | Update discovery estimate. | `bool, self.discovery_progress_text_var.set, self.discovery_estimate_var.set, self.selected_discovery_configs, self.selected_discovery_slaves` |
| `SlaveFinderMixin.update_discovery_start_state` update_discovery_start_state(self) | 1095-1111 | Update discovery start state. | `self.discovery_start_btn.configure, hasattr, bool, self.selected_discovery_slaves, self.selected_discovery_min_timeout_ms` |

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `SlaveFinderMixin.clear_discovery_results`<br>clear_discovery_results(self) | 1113-1122 | Clear discovery results. | `self.discovery_tree.get_children, self.discovery_found_var.set, self.discovery_progress_var.set, self.discovery_status_var.set, self.update…` |
| `SlaveFinderMixin.open_discovery_serial_port`<br>open_discovery_serial_port(self, serial, port, baud, parity, stopbits) | 1124-1162 | Open discovery serial port. | `serial.Serial, int, ser.open, ser.reset_input_buffer, ser.reset_output_buffer` |
| `SlaveFinderMixin.start_discovery_scan`<br>start_discovery_scan(self) | 1164-1255 | Validates the safety confirmation, COM selection, Slave range, attempts, selected serial configurations and global busy state. | `self.get_serial_modules, bool, self.clear_discovery_results, self.discovery_stop_event.clear, self.discovery_progress_var.set` |
| `SlaveFinderMixin.stop_discovery_scan`<br>stop_discovery_scan(self) | 1257-1264 | Stop discovery scan. | `self.discovery_status_var.set, self.discovery_stop_event.set, self.discovery_stop_btn.configure, cancel_serial_io, self.tr` |
| `SlaveFinderMixin.discovery_worker`<br>discovery_worker(self, serial, port, configs, slaves, minimum_timeout_ms, fallback_fc04, identify_devices, attempts=DISCOVERY_DEFAULT_ATTEMPTS) | 1266-1545 | Iterates selected serial configurations and Slave IDs. | `time.perf_counter, str, self.event_queue.put, len, self.discovery_stop_event.is_set` |
| `SlaveFinderMixin.apply_discovery_progress`<br>apply_discovery_progress(self, payload) | 1547-1562 | Apply discovery progress. | `int, max, self.discovery_progress_var.set, self.discovery_progress_text_var.set, self.discovery_found_var.set` |
| `SlaveFinderMixin.add_discovery_result`<br>add_discovery_result(self, payload) | 1564-1581 | Add discovery result. | `self.discovery_found_var.set, self.discovery_tree.insert, self.trf, payload.get` |
| `SlaveFinderMixin.finish_discovery_ui`<br>finish_discovery_ui(self, payload) | 1583-1624 | Returns both Finder and Sniffer controls to idle state and presents completed/stopped/error status without leaving the application locked in a busy state. | `self.discovery_stop_event.clear, self.discovery_stop_btn.configure, self.discovery_clear_btn.configure, self.discovery_refresh_btn.configur…` |

## Detailed behavior of the key paths

**`build_discovery_probe` lines 35-48**

Constructs a read-only FC03/FC04 request for Address 0, Quantity 1 and appends the RTU CRC. Input validation restricts Slave IDs to 1..247 and FC to 03/04.

**`find_discovery_response` lines 336-379**

Scans received bytes for the requested Slave and Function Code. A CRC-valid normal register response or CRC-valid Modbus Exception is accepted; echoes and unrelated traffic are ignored.

**`discovery_response_timeout_seconds` lines 309-330**

Calculates a response deadline from serial character time, the expected 7-byte FC03/FC04 response and an inter-frame/processing margin, never below the user-configured minimum.

**`bus_activity_detected` lines 409-427**

Listens passively for the idle-check period and returns any bytes received; a non-empty result means another device is transmitting and the scan must not start.

**`write_discovery_request` lines 459-523**

Transmits one probe without using ser.flush(), retries one write timeout after driver/buffer recovery, waits for estimated on-wire transmission time and raises actual driver errors instead of disguising them as no-response Slaves.

**`build_device_identification_probe` lines 57-71**

Builds FC43 / MEI type 14 / Basic Device Identification requests, beginning at a selectable Object ID.

**`find_device_identification_response` lines 77-158**

Parses FC43/14 responses and Exceptions from a receive buffer, validates CRC, decodes variable-length ASCII objects and exposes More Follows / Next Object ID for paged identification.

**`read_basic_device_identification` lines 226-301**

Repeats FC43/14 requests while More Follows is set, merging objects until VendorName, ProductCode and MajorMinorRevision are available or the paging limit/response terminates.

**`start_discovery_scan` lines 1164-1255**

Validates the safety confirmation, COM selection, Slave range, attempts, selected serial configurations and global busy state. It disables conflicting controls and launches the discovery worker.

`discovery_worker` **lines 1266-1545**

Iterates selected serial configurations and Slave IDs. It opens one configuration at a time, checks once that the bus is idle, probes FC03 then optional FC04 with the selected attempts, optionally performs Device Identification for found Slaves, publishes progress/results, and always closes the active serial handle.

`finish_discovery_ui` **lines 1583-1624**

Returns both Finder and Sniffer controls to idle state and presents completed/stopped/error status without leaving the application locked in a busy state.

**Design note.** The Finder is active traffic generation, unlike the passive Sniffer. The UI requires an explicit confirmation that another master is not active and the global busy lock prevents simultaneous capture/scan inside this application.

# mb_master.py

**Modbus RTU master engine, frames and data formats.** It builds FC01-04 read and FC05/06/15/16 write requests, matches responses and exceptions, converts registers to and from 16/32-bit formats in four byte orders and runs MasterEngine, which serves every poll window over one serial port one request at a time. It contains no widgets.

| Source lines | SHA-256 |
| --- | --- |
| 686 | `79ad42ac5c1f3a658b4252c2196ccb9829ad4cfdbcc5237052ef8a3a3423221e` |

**Imports:** `math, struct, threading, time, collections.deque, dataclasses.dataclass, mb_config.MASTER_RECONNECT_INTERVAL_SECONDS, MASTER_MAX_READ_BITS, MASTER_MAX_READ_REGISTERS, MASTER_MAX_WRITE_BITS, MASTER_MAX_WRITE_REGISTERS, mb_protocol.append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds, mb_slave_finder.bus_activity_detected, cancel_serial_io, discovery_bits_per_char, write_discovery_request`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `READ_FUNCTION_CODES` | `(1, 2, 3, 4)` |
| `BIT_FUNCTION_CODES` | `(1, 2)` |
| `WRITABLE_READ_FCS` | `{1: (5, 15), 3: (6, 16)}` |
| `BYTE_ORDERS` | `('ABCD', 'CDAB', 'BADC', 'DCBA')` |
| `BYTE_ORDERS_64` | `('ABCDEFGH', 'GHEFCDAB', 'BADCFEHG', 'HGFEDCBA')` |
| `WIRE_LETTERS` | `'ABCDEFGH'` |
| `REGISTER_FORMATS` | `(('int16', 'Signed 16-bit', 1), ('uint16', 'Unsigned 16-bit', 1), ('hex16', 'Hex 16-bit', 1), ('bin16', 'Binary 16-bit', 1)) + tuple(((f'{kind}_{or…` |
| `FORMAT_LABELS` | `{key: label for key, label, _size in REGISTER_FORMATS}` |
| `FORMAT_KEYS_BY_LABEL` | `{label: key for key, label, _size in REGISTER_FORMATS}` |
| `FORMAT_SIZES` | `{key: size for key, _label, size in REGISTER_FORMATS}` |
| `DEFAULT_FORMAT` | `'int16'` |
| `_STRUCT_CODES` | `{'int32': '>i', 'uint32': '>I', 'float32': '>f', 'int64': '>q', 'uint64': '>Q', 'float64': '>d'}` |

| Name | Definition |
|---|---|
| `SCALE_MAX_DECIMALS` | `6` |
| `SCALE_MAX_UNIT_LENGTH` | `12` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `_value_bytes`<br>_value_bytes(registers, order) | 81-84 | Reorder the wire bytes of 2 or 4 registers into big-endian value bytes. | `b''.join, bytes, WIRE_LETTERS.index` |
| `_registers_from_value_bytes`<br>_registers_from_value_bytes(value, order) | 87-91 | Registers from value bytes. | `bytearray, enumerate, len, WIRE_LETTERS.index, range` |
| `format_float`<br>format_float(value, digits=7) | 94-99 | Format float. | `math.isnan, math.isinf` |
| `decode_value`<br>decode_value(registers, fmt) | 102-110 | Numeric value of one value (None for the Hex/Binary display formats). | `fmt.split, struct.unpack, _value_bytes` |
| `validate_scale`<br>validate_scale(raw_min, raw_max, eng_min, eng_max, decimals=1, unit='') | 117-133 | Return a scale dict from user text/numbers; raise ValueError (PT text). | `any, int, ValueError, str(unit or '').strip, float` |
| `scale_value`<br>scale_value(number, scale) | 136-139 | Linear map from raw units to engineering units through two points. | — |
| `unscale_value`<br>unscale_value(value, scale) | 142-147 | Inverse of scale_value(): engineering units back to raw units. | `ValueError` |
| `format_scaled`<br>format_scaled(number, scale) | 150-152 | Format scaled. | `scale.get, scale_value` |
| `format_value`<br>format_value(registers, fmt) | 155-173 | Display text of one value occupying ``FORMAT_SIZES[fmt]`` registers. | `fmt.split, str, struct.unpack, format_float, _value_bytes` |
| `register_groups`<br>register_groups(qty, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None) | 176-205 | Split a window's registers into values: [(offset, size, fmt or None)]. | `overrides.get, groups.append, next, range, min` |
| `format_window_values`<br>format_window_values(registers, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None, scales=None) | 208-233 | Return one display string per register, honouring per-address formats. | `register_groups, len, scales.get, out.extend, decode_value` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `raw_text_for_scaled`<br>raw_text_for_scaled(value_text, scale, fmt) | 236-245 | Convert an engineering value typed by the user into raw text for *fmt*. | `unscale_value, fmt.split('_')[0].startswith, str, float, repr` |
| `format_register_values`<br>format_register_values(registers, fmt) | 248-250 | Turns raw registers into display strings for the selected format; 32-bit values are shown on the first register of each pair and an unpaired last register shows a dash. | `format_window_values` |
| `_parse_int`<br>_parse_int(text) | 253-260 | Parse int. | `str(text).strip().replace(' ', '').replace, text.lower, lowered.startswith, int, str(text).strip().replace` |
| `encode_register_value`<br>encode_register_value(text, fmt) | 263-307 | Converts one user-entered value into the register(s) it occupies in the selected format and byte order, rejecting values that do not fit. | `str(text).strip, ValueError, fmt.split, kind.startswith, _registers_from_value_bytes` |
| `encode_coil_value`<br>encode_coil_value(text) | 310-316 | Encode coil value. | `str(text).strip().lower, ValueError, str(text).strip, str` |
| `_u16`<br>_u16(value) | 323-324 | U16. | `bytes` |
| `read_limit`<br>read_limit(fc) | 327-328 | Read limit. | — |
| `validate_read_definition`<br>validate_read_definition(slave, fc, address, qty) | 331-342 | Raise ValueError (PT canonical message) for an invalid poll definition. | `ValueError, int, read_limit` |
| `build_read_request`<br>build_read_request(slave, fc, address, qty) | 345-347 | Build read request. | `validate_read_definition, append_crc, _u16, bytes, int` |
| `build_write_request`<br>build_write_request(slave, fc, address, values) | 350-381 | Builds FC05/06/15/16 requests within the Modbus write limits; Slave 0 is accepted for broadcast. | `append_crc, int, ValueError, len, _u16` |
| `expected_response_length`<br>expected_response_length(request) | 384-392 | Expected response length. | — |
| `find_master_response`<br>find_master_response(buffer, request) | 399-447 | Finds the CRC-valid response of the expected length from the addressed Slave, or a Modbus exception, and decodes read values. | `bytes, expected_response_length, range, len, crc_ok` |
| **`class PollDefinition`** | **455-465** | **PollDefinition.** | **Class container** |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `PollDefinition.request`<br>request(self) | 464-465 | Request. | `build_read_request` |
| **`class MasterEngine`** | **468-686** | **Serve all poll windows over one serial port, one transaction at a time.** | **Class container** |
| `MasterEngine.__init__`<br>__init__(self, ser, event_queue, *, baud, parity, stopbits, response_timeout_ms, delay_ms, serial_module=None, idle_check_seconds=0.0, reopen=None, reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS) | 483-508 | Init. | `float, int, max, modbus_rtu_interframe_gap_seconds, threading.Lock` |
| `MasterEngine.set_poll`<br>set_poll(self, definition) | 511-514 | Set poll. | `self._wake.set` |
| `MasterEngine.remove_poll`<br>remove_poll(self, window_id) | 516-519 | Remove poll. | `self._polls.pop, deque` |
| `MasterEngine.read_once`<br>read_once(self, window_id) | 521-525 | Read once. | `self._wake.set, self._once.append` |
| `MasterEngine.queue_write`<br>queue_write(self, window_id, request, description='') | 527-530 | Queue write. | `self._wake.set, self._writes.append, bytes` |
| `MasterEngine.start`<br>start(self) | 532-534 | Start. | `threading.Thread, self.thread.start` |
| `MasterEngine.stop`<br>stop(self, join_timeout=1.0) | 536-542 | Stop. | `self.stop_event.set, self._wake.set, cancel_serial_io, thread.is_alive, thread.join` |
| `MasterEngine.next_job`<br>next_job(self, now=None) | 545-568 | Chooses the next transaction: queued writes first, then "Read once" requests, then the enabled window whose next due time is earliest. | `time.perf_counter, min, self._once.popleft, self._polls.get, self._writes.popleft` |
| `MasterEngine.run`<br>run(self) | 571-599 | Worker loop: optional idle-bus check, then one transaction at a time with the configured delay between polls, posting master_ready, master_result and master_stopped events. | `self.event_queue.put, bus_activity_detected, self.stop_event.is_set, self._serve_jobs, hex_bytes` |
| `MasterEngine._serve_jobs`<br>_serve_jobs(self) | 601-623 | Serve writes / reads until stopped; serial errors propagate. | `self.stop_event.is_set, self.next_job, self.event_queue.put, self.stop_event.wait, self._wake.wait` |
| `MasterEngine._reconnect`<br>_reconnect(self, detail) | 625-640 | Close the failed port and reopen it; False when stopped meanwhile. | `self.event_queue.put, self.ser.close, self.stop_event.wait, self.reopen` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterEngine.transact`<br>transact(self, request) | 642-686 | Sends one request and waits for its response until the response timeout, classifying the result as ok, exception, invalid, timeout or broadcast. | `write_discovery_request, time.perf_counter, bytearray, self.stop_event.is_set, hex_bytes` |

## Detailed behavior of the key paths

**`format_register_values` lines 248-250**

Turns raw registers into display strings for the selected format; 32-bit values are shown on the first register of each pair and an unpaired last register shows a dash.

**`encode_register_value` lines 263-307**

Converts one user-entered value into the register(s) it occupies in the selected format and byte order, rejecting values that do not fit.

**`build_write_request` lines 350-381**

Builds FC05/06/15/16 requests within the Modbus write limits; Slave 0 is accepted for broadcast.

**`find_master_response` lines 399-447**

Finds the CRC-valid response of the expected length from the addressed Slave, or a Modbus exception, and decodes read values. Adapter echoes of read requests are skipped.

**`MasterEngine.next_job` lines 545-568**

Chooses the next transaction: queued writes first, then "Read once" requests, then the enabled window whose next due time is earliest.

**`MasterEngine.run` lines 571-599**

Worker loop: optional idle-bus check, then one transaction at a time with the configured delay between polls, posting master_ready, master_result and master_stopped events.

**`MasterEngine.transact` lines 642-686**

Sends one request and waits for its response until the response timeout, classifying the result as ok, exception, invalid, timeout or broadcast.

> **Design note.** The engine owns the serial port while the Master is connected. Widgets only change definitions through set_poll(), read_once() and queue_write(); results come back through event_queue.

# mb_master_ui.py

**Master (poll) tab user interface.** MasterMixin builds the connection bar and the grid of poll windows, validates window definitions, renders values and counters, opens the write and name dialogs, pops windows out to their own top-level windows and saves or opens workspace files.

**Source lines**
1993

**SHA-256**
`be93855e2ca6df91634d4dfa36609a4e5a15e7a28de330ac63b50d4c56f3ee98`

**Imports:** `json, os, subprocess, sys, time, tkinter, tkinter.font, tkinter.messagebox, ttk, mb_config.DISCOVERY_BAUD_RATES, DISCOVERY_IDLE_CHECK_SECONDS, MASTER_DEFAULT_DELAY_MS, MASTER_DEFAULT_RESPONSE_TIMEOUT_MS, MASTER_DEFAULT_SCAN_MS, MASTER_MAX_DELAY_MS, MASTER_MAX_RESPONSE_TIMEOUT_MS, MASTER_MAX_SCAN_MS, MASTER_MAX_WINDOWS, MASTER_MIN_DELAY_MS, MASTER_MIN_RESPONSE_TIMEOUT_MS, MASTER_WORKSPACE_FORMAT, MASTER_WORKSPACE_VERSION, MODBUS_EXCEPTION_CODES, get_workspaces_folder, mb_diagnostics.FUNCTION_NAMES, mb_master.BIT_FUNCTION_CODES, DEFAULT_FORMAT, FORMAT_KEYS_BY_LABEL, FORMAT_LABELS, FORMAT_SIZES, READ_FUNCTION_CODES, REGISTER_FORMATS, WRITABLE_READ_FCS, MasterEngine, PollDefinition, build_write_request, encode_coil_value, encode_register_value, format_window_values, raw_text_for_scaled, register_groups, validate_read_definition, validate_scale, mb_theme.theme_colors, mb_widgets.RefreshButton`

## Module constants / configuration data

| Name | Definition |
|---|---|
| `MASTER_PARITIES` | `('None', 'Even', 'Odd')` |
| `MASTER_STOP_BITS` | `('1', '2')` |
| `MASTER_PANE_MIN_WIDTH` | `600` |
| `MASTER_PANE_MIN_ROWS` | `3` |
| `MASTER_PANE_MAX_ROWS` | `15` |
| `MASTER_OK_MESSAGE_SECONDS` | `3.0` |
| `MASTER_STRIPE_LIGHT` | `'#f2f4f6'` |
| `MASTER_STRIPE_DARK` | `'#262d34'` |
| `MASTER_MAX_TITLE_LENGTH` | `40` |
| `WORKSPACE_SUFFIX` | `'.mbsw'` |
| `INVALID_FILENAME_CHARS` | `'<>:"/\\|?*'` |
| `RESERVED_FILENAMES` | `{'CON', 'PRN', 'AUX', 'NUL'} \| {f'COM{i}' for i in range(1, 10)} \| {f'LPT{i}' for i in range(1, 10)}` |
| `WHEEL_GUARD_TAG` | `'MBSniffer.MouseWheelGuard'` |

| Name | Definition |
|---|---|
| `MASTER_WHEEL_TAG` | `'MBSniffer.MasterWheel'` |
| `FC_CHOICES` | `tuple((fc_choice_label(fc) for fc in READ_FUNCTION_CODES))` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `fc_choice_label`<br>fc_choice_label(fc) | 78-79 | Fc choice label. | — |
| `parse_int_field`<br>parse_int_field(text, minimum, maximum, message) | 85-92 | Parse int field. | `int, ValueError, str(text).strip, str` |
| `validate_workspace_name`<br>validate_workspace_name(name) | 95-105 | Return a cleaned workspace name usable as a Windows file name. | `str(name or '').strip().rstrip, ValueError, any, name.split('.')[0].upper, str(name or '').strip` |
| `workspace_path`<br>workspace_path(name) | 108-109 | Workspace path. | `get_workspaces_folder, validate_workspace_name` |
| `list_workspaces`<br>list_workspaces() | 112-118 | Names of the saved workspaces, sorted alphabetically. | `get_workspaces_folder().glob, sorted, get_workspaces_folder, f.is_file` |
| **`class MasterWindow`** | **121-226** | **State of one poll window (independent of where its pane is shown).** | **Class container** |
| `MasterWindow.__init__`<br>__init__(self, window_id, number, slave=1, fc=3, address=0, qty=10, scan_ms=MASTER_DEFAULT_SCAN_MS, base=0, fmt=DEFAULT_FORMAT, names=None, enabled=True, title='', formats=None, scales=None) | 124-147 | Init. | `dict, self.reset_counters, str(title or '').strip, int, (formats or {}).items` |
| `MasterWindow.reset_counters`<br>reset_counters(self) | 149-159 | Reset counters. | — |
| `MasterWindow.definition`<br>definition(self) | 161-163 | Definition. | `PollDefinition` |
| `MasterWindow.to_json`<br>to_json(self) | 165-174 | To json. | `str, sorted, self.names.items, self.formats.items, self.scales.items` |
| `MasterWindow.groups`<br>groups(self) | 176-178 | [(row offset, size, fmt or None)] of the values shown in this window. | `register_groups` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterWindow.group_of_row`<br>group_of_row(self, row) | 180-184 | Group of row. | `self.groups` |
| `MasterWindow.validated_kwargs`<br>validated_kwargs(data) | 187-226 | Validate one workspace window entry; raise ValueError if invalid. | `validate_read_definition, int, data.get, isinstance, ValueError` |
| `apply_format_to_rows`<br>apply_format_to_rows(window, rows, fmt) | 229-260 | Set the per-address format of the selected rows of *window*. | `sorted, list, set, ValueError, window.formats.items` |
| **`class MasterMixin`** | **263-1989** | **Master (poll) tab GUI.** | **Class container** |
| `MasterMixin.build_master_tab`<br>build_master_tab(self, page) | 267-429 | Builds the connection bar, toolbar and scrollable grid of poll windows, restoring the saved connection parameters. | `getattr(self, 'ui_settings', {}).get, page.columnconfigure, page.rowconfigure, ttk.LabelFrame, connection.grid` |
| `MasterMixin._master_restore_workspace`<br>_master_restore_workspace(self, name) | 431-434 | Master restore workspace. | `self.master_open_workspace, self.master_add_window` |
| `MasterMixin._master_install_wheel_tags`<br>_master_install_wheel_tags(self, root_widget) | 437-451 | Give new widgets the global wheel guard; plain panels scroll the grid. | `stack.pop, stack.extend, tuple, isinstance, widget.winfo_children` |
| `MasterMixin._master_wheel`<br>_master_wheel(self, event) | 453-470 | Master wheel. | `getattr, canvas.yview, canvas.yview_scroll` |
| `MasterMixin._master_window_name`<br>_master_window_name(self, window) | 472-473 | Master window name. | `self.trf` |
| `MasterMixin._master_window_summary`<br>_master_window_summary(self, window) | 475-477 | Master window summary. | — |
| `MasterMixin._master_window_title`<br>_master_window_title(self, window) | 479-480 | Master window title. | `self._master_window_name, self._master_window_summary` |
| `MasterMixin._master_counters_text`<br>_master_counters_text(self, window) | 482-485 | Master counters text. | — |
| `MasterMixin._master_set_message`<br>_master_set_message(self, window, template, style='normal', **values) | 487-490 | Master set message. | `time.monotonic, self._master_render_status` |
| `MasterMixin._master_render_status`<br>_master_render_status(self, window) | 492-514 | Master render status. | `self._master_panes.get, self.trf, pane['counters_var'].set, pane['message_var'].set, pane['message_label'].configure` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin._master_build_pane`<br>_master_build_pane(self, window, parent) | 517-663 | Master build pane. | `ttk.LabelFrame, frame.columnconfigure, frame.rowconfigure, ttk.Frame, header.grid` |
| `MasterMixin._master_update_format_state`<br>_master_update_format_state(self, window) | 665-669 | Master update format state. | `self._master_panes.get, pane['format_combo'].configure` |
| `MasterMixin._master_fill_rows`<br>_master_fill_rows(self, window) | 671-682 | Master fill rows. | `self._master_panes.get, tree.delete, tree.configure, range, self._master_render_values` |
| `MasterMixin._master_render_values`<br>_master_render_values(self, window) | 684-702 | Master render values. | `self._master_panes.get, enumerate, format_window_values, tree.item, tree.set` |
| `MasterMixin._master_schedule_reflow`<br>_master_schedule_reflow(self, _event=None) | 704-710 | Master schedule reflow. | `self.after, self.after_cancel` |
| `MasterMixin._master_reflow`<br>_master_reflow(self) | 712-739 | Arrange docked panes in as many equal columns as the width allows. | `max, canvas.itemconfigure, range, enumerate, canvas.winfo_width` |
| `MasterMixin.master_add_window`<br>master_add_window(self, kwargs=None) | 742-760 | Master add window. | `next, MasterWindow, self._master_build_pane, self._master_reflow, self._master_mark_changed` |
| `MasterMixin.master_duplicate_window`<br>master_duplicate_window(self, window_id) | 762-766 | Master duplicate window. | `self.master_windows.get, MasterWindow.validated_kwargs, self.master_add_window, window.to_json` |
| `MasterMixin.master_close_window`<br>master_close_window(self, window_id) | 768-789 | Master close window. | `self.master_windows.pop, self._master_panes.pop, self._master_reflow, self.master_engine.remove_poll, pane.get` |
| `MasterMixin.master_apply_window`<br>master_apply_window(self, window_id) | 791-828 | Validates the fields of one window, updates its model, rebuilds its rows and hands the new definition to the running engine. | `self.master_windows.get, self._master_panes.get, FORMAT_KEYS_BY_LABEL.get, self._master_fill_rows, self._master_update_format_state` |
| `MasterMixin.master_toggle_pause`<br>master_toggle_pause(self, window_id) | 830-840 | Master toggle pause. | `self.master_windows.get, self._master_render_status, self._master_mark_changed, self.master_engine.set_poll, window.definition` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin.master_read_once`<br>master_read_once(self, window_id) | 842-846 | Master read once. | `self.master_engine.read_once, messagebox.showinfo, self.tr` |
| `MasterMixin.master_toggle_popout`<br>master_toggle_popout(self, window_id) | 848-876 | Moves a window between the grid and its own top-level window by rebuilding its pane from the model. | `self.master_windows.get, self._master_panes.pop, pane.get, self._master_render_status, self._master_reflow` |
| `MasterMixin._master_on_double_click`<br>_master_on_double_click(self, window_id, event) | 879-892 | Master on double click. | `self._master_panes.get, tree.identify_row, tree.selection_set, tree.identify_column, self._master_edit_name` |
| `MasterMixin._master_dialog`<br>_master_dialog(self, title) | 894-902 | Master dialog. | `tk.Toplevel, dialog.title, dialog.transient, dialog.resizable, self._master_apply_toplevel_palette` |
| `MasterMixin._master_edit_name`<br>_master_edit_name(self, window_id, row) | 904-931 | Master edit name. | `self.master_windows.get, self._master_dialog, ttk.Label(body, text=self.trf('Nome do endereço {address}', address=pdu + window.base)).grid,…` |
| `MasterMixin._master_edit_value`<br>_master_edit_value(self, window_id) | 933-1034 | Master edit value. | `self.master_windows.get, self._master_panes.get, pane['tree'].selection, int, WRITABLE_READ_FCS.get` |
| `MasterMixin.master_build_write`<br>master_build_write(self, window, slave_text, address_text, values_text, fc, fmt=None, scale=None) | 1036-1064 | Validates the write dialog and encodes the values with the window format into an FC05/06/15/16 request. | `parse_int_field, build_write_request, ValueError, str(values_text).split, p.strip` |
| `MasterMixin._master_connection_settings`<br>_master_connection_settings(self) | 1067-1082 | Master connection settings. | `parse_int_field, self.master_timeout_var.get, self.master_delay_var.get, self.master_port_var.get().strip, int` |
| `MasterMixin.master_connect`<br>master_connect(self) | 1084-1146 | Checks the busy flag, safety confirmation and settings, opens the port, starts MasterEngine with the idle-bus check and locks the other active features. | `self.get_serial_modules, list, MasterEngine, self.master_windows.values, self._master_lock_other_features` |
| `MasterMixin.master_disconnect`<br>master_disconnect(self) | 1148-1153 | Master disconnect. | `self.master_status_var.set, self.master_disconnect_btn.configure, self.master_engine.stop, self.tr` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin._master_lock_other_features`<br>_master_lock_other_features(self, locked) | 1155-1170 | Master lock other features. | `self.master_connect_btn.configure, self.master_disconnect_btn.configure, self.master_refresh_btn.configure, self.start_btn.configure, self.…` |
| `MasterMixin._master_finish_session`<br>_master_finish_session(self, payload) | 1172-1205 | Master finish session. | `self._master_lock_other_features, self.master_status_var.set, self.master_windows.values, (payload or {}).get, engine.stop` |
| `MasterMixin.handle_master_event`<br>handle_master_event(self, event, payload) | 1208-1228 | Handle master event. | `self.master_status_var.set, self.tr, self.master_windows.values, self.trf, self._master_render_values` |
| `MasterMixin._master_apply_result`<br>_master_apply_result(self, result) | 1230-1279 | Updates counters, values and the status line of the window that issued a transaction, ignoring results of an outdated definition. | `self.master_windows.get, result.get, self._master_render_values, self._master_set_message, self.master_engine.read_once` |
| `MasterMixin.master_workspace_data`<br>master_workspace_data(self) | 1282-1296 | Master workspace data. | `self.master_port_var.get, self.master_baud_var.get, self.master_parity_var.get, self.master_stop_var.get, self.master_timeout_var.get` |
| `MasterMixin._master_update_workspace_label`<br>_master_update_workspace_label(self) | 1298-1306 | Master update workspace label. | `getattr, self.master_workspace_var.set, self.trf` |
| `MasterMixin._master_mark_changed`<br>_master_mark_changed(self) | 1308-1314 | Record a change to the open workspace (ignored while loading one). | `getattr, self._master_update_workspace_label` |
| `MasterMixin._master_build_workspace_menu`<br>_master_build_workspace_menu(self) | 1316-1342 | Rebuilds the Workspace menu each time it opens, listing the saved workspaces under Open and Delete. | `menu.delete, menu.winfo_children, list_workspaces, menu.add_command, tk.Menu` |
| `MasterMixin.master_save_workspace`<br>master_save_workspace(self, name=None) | 1344-1375 | Save all windows as a named workspace in the workspace folder. | `workspace_path, self._master_update_workspace_label, self._schedule_ui_settings_save, self._master_ask_text, validate_workspace_name` |
| `MasterMixin.master_delete_workspace`<br>master_delete_workspace(self, name, confirm=True) | 1377-1393 | Master delete workspace. | `workspace_path(name).unlink, self._master_update_workspace_label, self._schedule_ui_settings_save, messagebox.askyesno, messagebox.showerror` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin.master_open_workspace_folder`<br>master_open_workspace_folder(self) | 1395-1405 | Master open workspace folder. | `get_workspaces_folder, os.startfile, messagebox.showerror, str, subprocess.Popen` |
| `MasterMixin.master_open_workspace`<br>master_open_workspace(self, name, quiet=False) | 1407-1426 | Loads a named workspace from the workspace folder, validating every window and skipping invalid entries with a warning. | `self._master_load_workspace_data, open, json.load, ValueError, workspace_path` |
| `MasterMixin._master_load_workspace_data`<br>_master_load_workspace_data(self, name, data) | 1428-1463 | Master load workspace data. | `list, data.get, self._master_update_workspace_label, self._schedule_ui_settings_save, self.master_close_window` |
| `MasterMixin._master_ask_text`<br>_master_ask_text(self, title, caption, initial) | 1466-1489 | Small modal text prompt; returns the text or None when cancelled. | `self._master_dialog, ttk.Label(body, text=caption).grid, tk.StringVar, ttk.Entry, entry.grid` |
| `MasterMixin.master_rename_window`<br>master_rename_window(self, window_id, name=None) | 1491-1508 | Gives a window its own name, shown in its header and saved in the workspace; an empty name restores the default. | `self.master_windows.get, self._master_render_status, self._master_mark_changed, self._master_ask_text, name.strip` |
| `MasterMixin._master_fields_changed`<br>_master_fields_changed(self, window, pane) | 1511-1523 | True when the definition fields differ from what was last applied. | `pane['slave_var'].get().strip, str, pane['fc_var'].get, fc_choice_label, pane['address_var'].get().strip` |
| `MasterMixin._master_update_apply_state`<br>_master_update_apply_state(self, window_id) | 1525-1535 | Master update apply state. | `self.master_windows.get, self._master_panes.get, self._master_fields_changed, pane['apply_btn'].configure` |
| `MasterMixin._master_row_tags`<br>_master_row_tags(index) | 1538-1540 | Alternate rows get a slightly darker background (zebra striping). | — |
| `MasterMixin._master_configure_tree_tags`<br>_master_configure_tree_tags(self, tree) | 1542-1555 | Master configure tree tags. | `bool, theme_colors, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, tree.tag_configure, isinstance` |
| `MasterMixin._master_build_row_menu`<br>_master_build_row_menu(self) | 1558-1559 | Master build row menu. | `tk.Menu` |
| `MasterMixin._master_drag_start`<br>_master_drag_start(self, window_id, event) | 1566-1578 | Master drag start. | `self._master_panes.get, tree.identify_row, bool, tuple, tree.selection` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin._master_drag_move`<br>_master_drag_move(self, window_id, event) | 1580-1587 | Master drag move. | `self._master_panes.get, self._master_drag_update, pane.get` |
| `MasterMixin._master_drag_update`<br>_master_drag_update(self, window_id) | 1589-1609 | Master drag update. | `self._master_panes.get, tree.winfo_height, tree.identify_row, tree.get_children, sorted` |
| `MasterMixin._master_drag_stop`<br>_master_drag_stop(self, window_id) | 1611-1620 | Master drag stop. | `self._master_panes.get, pane.get, drag.get, self.after_cancel` |
| `MasterMixin._master_selected_rows`<br>_master_selected_rows(self, window_id) | 1622-1626 | Master selected rows. | `self._master_panes.get, sorted, int, pane['tree'].selection, iid.startswith` |
| `MasterMixin._master_show_row_menu`<br>_master_show_row_menu(self, window_id, event) | 1628-1682 | Master show row menu. | `self.master_windows.get, self._master_panes.get, tree.identify_row, menu.delete, menu.winfo_children` |
| `MasterMixin.master_set_row_format`<br>master_set_row_format(self, window_id, fmt) | 1684-1697 | Applies a format (or the window default) to the selected rows through apply_format_to_rows() and redraws the values. | `self.master_windows.get, self._master_render_values, self._master_mark_changed, apply_format_to_rows, self._master_selected_rows` |
| `MasterMixin.master_definition_from_record`<br>master_definition_from_record(self, record) | 1700-1723 | Read definition (slave, fc, address, qty) behind a Traffic record, or None. | `int, record.get, validate_read_definition, str` |
| `MasterMixin.master_open_from_record`<br>master_open_from_record(self, record) | 1725-1740 | Master open from record. | `self.master_definition_from_record, self.master_open_definition, messagebox.showinfo, getattr, self.port_a_var.get().strip` |
| `MasterMixin.master_open_definition`<br>master_open_definition(self, definition, port='', baud='', parity='', stopbits='') | 1742-1762 | Creates a Master window from a definition coming from the Sniffer or the Finder and copies the serial parameters when disconnected. | `self.master_add_window, self.main_notebook.select, str, self.master_baud_var.set, self.master_parity_var.set` |
| `MasterMixin._master_selected_group_starts`<br>_master_selected_group_starts(self, window) | 1764-1772 | PDU addresses where the values of the selected rows start. | `self._master_selected_rows, window.group_of_row, starts.append` |
| `MasterMixin.master_edit_scale`<br>master_edit_scale(self, window_id) | 1774-1852 | Asks for a linear scale and applies it to the values of the selected rows. | `self.master_windows.get, self._master_selected_group_starts, self._master_dialog, ttk.Label(body, text=self.trf('Escala de {count} valor(es…` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterMixin.master_remove_scale`<br>master_remove_scale(self, window_id) | 1854-1864 | Master remove scale. | `self.master_windows.get, self._master_selected_group_starts, self._master_render_values, self._master_mark_changed, window.scales.pop` |
| `MasterMixin.master_copy_rows`<br>master_copy_rows(self, window_id) | 1866-1876 | Copy the selected rows (all rows if none) as tab-separated text. | `self._master_panes.get, list, '\t'.join, self.clipboard_clear, self.clipboard_append` |
| `MasterMixin.master_set_base`<br>master_set_base(self, window_id, base) | 1878-1888 | Master set base. | `self.master_windows.get, self._master_panes.get, pane['address_var'].set, self._master_fill_rows, self._master_render_status` |
| `MasterMixin.master_confirm_close`<br>master_confirm_close(self) | 1891-1908 | Asks whether to save a saved workspace that has unsaved changes before the application closes; Cancel keeps it open. | `messagebox.askyesnocancel, self.tr, self.trf, self.master_save_workspace, getattr` |
| `MasterMixin.master_apply_language`<br>master_apply_language(self) | 1911-1928 | Master apply language. | `self._master_update_workspace_label, self.master_windows.values, hasattr, self._master_panes.get, self._master_render_status` |
| `MasterMixin._master_apply_toplevel_palette`<br>_master_apply_toplevel_palette(self, top) | 1930-1935 | Master apply toplevel palette. | `bool, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, top.configure, getattr, tk.BooleanVar` |
| `MasterMixin._master_apply_menu_palette`<br>_master_apply_menu_palette(self) | 1937-1957 | Master apply menu palette. | `getattr, bool, theme_colors, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, list` |
| `MasterMixin.master_apply_palette`<br>master_apply_palette(self, dark) | 1959-1975 | Master apply palette. | `self._master_apply_menu_palette, theme_colors, hasattr, getattr(self, '_master_panes', {}).values, getattr` |
| `MasterMixin.master_shutdown`<br>master_shutdown(self) | 1977-1989 | Master shutdown. | `getattr, engine.stop, ser.close` |
| `fc_choice_label_write`<br>fc_choice_label_write(fc) | 1992-1993 | Fc choice label write. | — |

## Detailed behavior of the key paths

**`build_master_tab` lines 267-429**

Builds the connection bar, toolbar and scrollable grid of poll windows, restoring the saved connection parameters.

**`master_apply_window` lines 791-828**

Validates the fields of one window, updates its model, rebuilds its rows and hands the new definition to the running engine.

**`master_build_write` lines 1036-1064**

Validates the write dialog and encodes the values with the window format into an FC05/06/15/16 request.

**`master_connect` lines 1084-1146**

Checks the busy flag, safety confirmation and settings, opens the port, starts MasterEngine with the idle-bus check and locks the other active features.

**`_master_apply_result` lines 1230-1279**

Updates counters, values and the status line of the window that issued a transaction, ignoring results of an outdated definition.

**`master_toggle_popout` lines 848-876**

Moves a window between the grid and its own top-level window by rebuilding its pane from the model.

**`master_open_workspace` lines 1407-1426**

Loads a named workspace from the workspace folder, validating every window and skipping invalid entries with a warning.

**`_master_build_workspace_menu` lines 1316-1342**

Rebuilds the Workspace menu each time it opens, listing the saved workspaces under Open and Delete.

**`master_rename_window` lines 1491-1508**

Gives a window its own name, shown in its header and saved in the workspace; an empty name restores the default.

`master_set_row_format` **lines 1684-1697**

Applies a format (or the window default) to the selected rows through apply_format_to_rows() and redraws the values.

`master_edit_scale` **lines 1774-1852**

Asks for a linear scale and applies it to the values of the selected rows.

`master_open_definition` **lines 1742-1762**

Creates a Master window from a definition coming from the Sniffer or the Finder and copies the serial parameters when disconnected.

`master_confirm_close` **lines 1891-1908**

Asks whether to save a saved workspace that has unsaved changes before the application closes; Cancel keeps it open.

> **Design note.** Window state lives in MasterWindow objects, so a pane can be destroyed and rebuilt (pop out / dock, language switch) without losing data.

# mb_i18n.py

**Runtime Portuguese/English translation layer.** Portuguese (Portugal) is the canonical UI text. Static translations, runtime-status patterns, protocol-detail fragments and the complete English Help content are kept separate from protocol logic so switching language affects presentation only.

| Source lines | SHA-256 |
|---|---|
| 610 | `c189b677eeb59213b98d3717d9187b99999c0f0853c8c5cb46b327d1d3544be8` |

**Imports:** `mb_config.DISCOVERY_IDLE_CHECK_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS`

## Module constants / configuration data

| Name | Definition |
|---|---|
| `LANGUAGE_PORTUGUESE` | `'Português'` |
| `LANGUAGE_ENGLISH` | `'English'` |
| `LANGUAGE_CHOICES` | `(LANGUAGE_PORTUGUESE, LANGUAGE_ENGLISH)` |
| `PT_TO_EN` | `{'Linguagem': 'Language', 'Modo escuro': 'Dark mode', 'Configuração': 'Configuration', 'Modo físico': 'Physical mode', 'RS232 dual RX: COM A escuta…` |
| `EN_TO_PT` | `{english: portuguese for portuguese, english in PT_TO_EN.items()}` |
| `EXCEPTION_DESCRIPTION_EN` | `{1: 'The requested Modbus function is not supported or permitted by the device.', 2: 'The requested address or address range is not valid for this …` |
| `DETAIL_FRAGMENTS_PT_TO_EN` | `(('Registos (PDU Address=valor):', 'Registers (PDU Address=value):'), ('Bits (PDU Address=valor):', 'Bits (PDU Address=value):'), ('Registos=', 'Re…` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `normalize_language`<br>normalize_language(value) | 326-327 | Normalize language. | — |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `canonical_ui_text`<br>canonical_ui_text(text) | 330-333 | Normalises a known English caption back to its canonical PT-PT form. | `str, EN_TO_PT.get` |
| `translate_text`<br>translate_text(text, language) | 336-341 | Returns the English translation for known canonical PT-PT UI strings when English is selected; unknown/technical terms are returned unchanged. | `canonical_ui_text, normalize_language, PT_TO_EN.get` |
| `translate_details`<br>translate_details(text, language) | 352-361 | Translates selected human-readable fragments embedded inside protocol detail strings (DETAIL_FRAGMENTS_PT_TO_EN) while preserving numbers, addresses, register values and other protocol data. | `str, normalize_language, value.replace` |
| `exception_description`<br>exception_description(code, portuguese_text, language) | 364-367 | Exception description. | `normalize_language, EXCEPTION_DESCRIPTION_EN.get` |
| `english_help_blocks`<br>english_help_blocks(max_ui_frames_text, max_raw_lines_text, min_pending, max_pending, debug_sim=False) | 371-581 | Returns the complete English Help / Connections document as structured blocks so Help switches language together with the rest of the UI. | `blocks.extend` |
| `translate_runtime_text`<br>translate_runtime_text(text, language) | 584-610 | Uses regular expressions to translate live status strings that contain counters, elapsed time, selected configuration or estimated duration. | `str, translate_text, normalize_language, re.match, re.sub` |

## Detailed behavior of the key paths

**`canonical_ui_text` lines 330-333**

Normalises a known English caption back to its canonical PT-PT form. This makes language switching reversible instead of translating an already translated string repeatedly.

**`translate_text` lines 336-341**

Returns the English translation for known canonical PT-PT UI strings when English is selected; unknown/technical terms are returned unchanged.

**`translate_details`** **lines 352-361**

Translates selected human-readable fragments embedded inside protocol detail strings (DETAIL_FRAGMENTS_PT_TO_EN) while preserving numbers, addresses, register values and other protocol data.

**`english_help_blocks`** **lines 371-581**

Returns the complete English Help / Connections document as structured blocks so Help switches language together with the rest of the UI.

**`translate_runtime_text`** **lines 584-610**

Uses regular expressions to translate live status strings that contain counters, elapsed time, selected configuration or estimated duration.

> **Design note.** The translation design treats PT-PT as canonical and leaves technical English terms unchanged unless a translation is explicitly defined.

# mb_theme.py

**Fixed-geometry Light/Dark theme implementation.** It configures a single ttk clam layout once, installs stable checkbox geometry and changes only palette values at runtime. This is deliberately designed so theme switching does not move or resize controls.

| Source lines | SHA-256 |
| --- | --- |
| 692 | `88c0c2592f4b6e0666aa42e4d163ae120845b154f7faa3b9f6a007fef7ed8265` |

**Imports:** `tkinter, tkinter.ttk`

## Module constants / configuration data

| Name | Definition |
| --- | --- |
| `BOOTSTRAP_ACTIVE` | `False` |
| `THEME_ENGINE` | `'clam'` |
| `LIGHT` | `{'root': '#f3f4f6', 'panel': '#f3f4f6', 'panel_alt': '#ffffff', 'input': '#ffffff', 'input_readonly': '#f5f6f7', 'text': '#1f2328', 'muted': '#5963…` |
| `DARK` | `{'root': '#171a1f', 'panel': '#171a1f', 'panel_alt': '#1f242a', 'input': '#20262c', 'input_readonly': '#252c33', 'text': '#e8eaed', 'muted': '#aab2…` |
| `_CHECK_IMAGES` | `None` |
| `_CHECK_ELEMENT` | `'MBS.Check.indicator'` |
| `_FIXED_GEOMETRY_READY` | `False` |
| `_INTERP_ID` | `None` |

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `theme_colors`<br>theme_colors(root=None, dark=False) | 70-71 | Theme colors. | `dict` |
| `_ensure_engine`<br>_ensure_engine(style) | 74-95 | Select/configure the fixed engine once per Tk interpreter. | `id, _configure_fixed_geometry, style.theme_use, style.theme_names` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `_configure_fixed_geometry`<br>_configure_fixed_geometry(style) | 98-296 | Defines ttk element layouts, paddings, borders, row heights and widget metrics once. | `style.configure, style.map, _install_fixed_checkbutton_element, style.layout` |
| `_paint_checkbox_image`<br>_paint_checkbox_image(image, *, fill, border, check=None) | 299-314 | Repaint one fixed-size checkbox image without changing its dimensions. | `image.put` |
| `_install_fixed_checkbutton_element`<br>_install_fixed_checkbutton_element(style) | 317-367 | Creates the custom checkbox indicator element/images used to keep checkbox dimensions stable across palette changes. | `style.layout, repr, tk.PhotoImage, style.element_names, style.element_create` |
| `_repaint_checkboxes`<br>_repaint_checkboxes(colors, dark) | 370-408 | Repaint checkboxes. | `_paint_checkbox_image` |
| `_apply_palette`<br>_apply_palette(root, style, colors, dark) | 411-666 | Changes ttk colors/state maps and classic Tk colors without changing layout definitions, supporting the invariant that theme switching changes appearance but not geometry. | `style.configure, style.map, _repaint_checkboxes, root.configure` |
| `apply_theme`<br>apply_theme(root, dark=False, style=None) | 669-677 | Change palette only. | `_ensure_engine, _apply_palette, ttk.Style, bool` |
| `apply_light_theme`<br>apply_light_theme(root, style=None) | 680-681 | Apply light theme. | `apply_theme` |
| `apply_dark_theme`<br>apply_dark_theme(root, style=None) | 684-685 | Apply dark theme. | `apply_theme` |
| `apply_modern_theme`<br>apply_modern_theme(root, dark=False) | 688-692 | Public startup helper that selects the fixed clam engine and applies either the Light or Dark palette. | `ttk.Style, _ensure_engine, _apply_palette, bool` |

## Detailed behavior of the key paths

**`_configure_fixed_geometry` lines 98-296**

Defines ttk element layouts, paddings, borders, row heights and widget metrics once. These values are intentionally independent of Light/Dark selection.

**`_install_fixed_checkbutton_element` lines 317-367**

Creates the custom checkbox indicator element/images used to keep checkbox dimensions stable across palette changes.

`_apply_palette` **lines 411-666**

Changes ttk colors/state maps and classic Tk colors without changing layout definitions, supporting the invariant that theme switching changes appearance but not geometry.

`apply_modern_theme` **lines 688-692**

Public startup helper that selects the fixed clam engine and applies either the Light or Dark palette.

**Design note.** The theme module is geometry-sensitive: changing colors must not rebuild ttk layouts. This is why theme engine selection and fixed geometry are separated from palette application.

# mb_widgets.py

**Custom reusable Tkinter widgets.** RefreshButton draws a stable square refresh control on a Canvas; ToggleSwitch draws the Light/Dark switch. Both implement their own state, hover/press behavior and palette updates.

| Source lines | SHA-256 |
|---|---|
| 418 | `fb45d18d621a734220afa46d92dfb882228a5cba2702e4201ccf02120b2bff97` |

**Imports:** `tkinter`

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| **`class RefreshButton`** | **7-250** | **Canvas-based square button that draws the ↻ glyph and recentres it from its actual rendered bounding box.** | **Class container** |
| `RefreshButton.__init__`<br>__init__(self, master, command, size=26, cursor='hand2', **kwargs) | 20-53 | Init. | `super().__init__, self.bind, self._redraw, self._schedule_final_redraw, super` |
| `RefreshButton._system_color`<br>_system_color(widget, name, fallback) | 56-61 | System color. | `widget.winfo_rgb` |
| `RefreshButton._schedule_final_redraw`<br>_schedule_final_redraw(self) | 63-77 | Redraw after Tk has settled requested widget dimensions. | `self.after, self.after_cancel` |
| `RefreshButton._final_redraw`<br>_final_redraw(self) | 79-84 | Final redraw. | `self._redraw` |
| `RefreshButton.configure`<br>configure(self, cnf=None, **kwargs) | 86-97 | Configure. | `kwargs.pop, super().configure, str, self._redraw, self._schedule_final_redraw` |
| `RefreshButton.cget`<br>cget(self, key) | 101-104 | Cget. | `super().cget, super` |
| `RefreshButton.invoke`<br>invoke(self) | 106-109 | Invoke. | `callable, self._command` |
| `RefreshButton.set_dark_mode`<br>set_dark_mode(self, enabled) | 111-114 | Set dark mode. | `bool, self._redraw, self._schedule_final_redraw` |
| `RefreshButton._palette`<br>_palette(self) | 116-146 | Palette. | `self._system_color` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `RefreshButton._center_item_bbox`<br>_center_item_bbox(self, item, width, height) | 148-181 | Center the visible Tk bbox of one Canvas item on the square. | `range, self.bbox, self.move, round, abs` |
| `RefreshButton._redraw`<br>_redraw(self, _event=None) | 183-220 | Redraw. | `self._palette, self.delete, self.create_rectangle, self.create_text, self._center_item_bbox` |
| `RefreshButton._on_enter`<br>_on_enter(self, _event) | 222-225 | Event handler for enter. | `self._redraw` |
| `RefreshButton._on_leave`<br>_on_leave(self, _event) | 227-230 | Event handler for leave. | `self._redraw` |
| `RefreshButton._on_press`<br>_on_press(self, _event) | 232-235 | Event handler for press. | `self._redraw` |
| `RefreshButton._on_release`<br>_on_release(self, event) | 237-250 | Event handler for release. | `self._redraw, callable, self._command, self.winfo_width, self.winfo_height` |
| **`class ToggleSwitch`** | **254-418** | **Canvas-based boolean switch bound to a Tk variable.** | **Class container** |
| `ToggleSwitch.__init__`<br>__init__(self, master, variable, command=None, width=42, height=22, cursor='hand2', **kwargs) | 257-292 | Init. | `super().__init__, self.bind, self.variable.trace_add, self._redraw, self._schedule_final_redraw` |
| `ToggleSwitch._schedule_final_redraw`<br>_schedule_final_redraw(self) | 294-300 | Schedules final redraw. | `self.after, self.after_cancel` |
| `ToggleSwitch._final_redraw`<br>_final_redraw(self) | 302-307 | Final redraw. | `self._redraw` |
| `ToggleSwitch.set_dark_mode`<br>set_dark_mode(self, enabled) | 309-312 | Set dark mode. | `bool, self._redraw, self._schedule_final_redraw` |
| `ToggleSwitch._parent_bg`<br>_parent_bg(self) | 314-321 | Parent bg. | `self.winfo_rgb` |
| `ToggleSwitch._press`<br>_press(self, _event=None) | 323-326 | Press. | — |
| `ToggleSwitch._toggle`<br>_toggle(self, _event=None) | 328-332 | Toggle. | `self.variable.set, callable, self.command, bool, self.variable.get` |
| `ToggleSwitch._enter`<br>_enter(self, _event=None) | 334-336 | Enter. | `self._redraw` |
| `ToggleSwitch._leave`<br>_leave(self, _event=None) | 338-340 | Leave. | `self._redraw` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ToggleSwitch._redraw`<br>_redraw(self, _event=None) | 342-418 | Redraw. | `bool, self.configure, self.delete, self.create_rectangle, self.create_oval` |

## Detailed behavior of the key paths

**`RefreshButton` lines 7-250**

Canvas-based square button that draws the ↻ glyph and recentres it from its actual rendered bounding box. It supports normal/hover/pressed/disabled states and palette changes.

**`ToggleSwitch` lines 254-418**

Canvas-based boolean switch bound to a Tk variable. It toggles the variable, invokes the callback and redraws track/knob colors while preserving fixed dimensions.

# mb_gui.py

**Main application composition and global UI behavior.** SnifferApp combines MasterMixin, SlaveFinderMixin, CaptureMixin and ViewMixin with tk.Tk. It creates all widgets, initializes state, freezes language-independent geometry, performs runtime translation, manages theme/language persistence, prevents mouse-wheel tab switching and keeps Bus Health content responsive.

**Source lines**
2584

**SHA-256**
`f9489ca19586a7ede032d90a59d632e1c777da560f48895c9f0778b81c10e62d`

**Imports:** `queue, threading, collections.deque, tkinter, tkinter.ttk, tkinter.font, mb_config.APP_NAME, APP_VERSION, DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS, DISCOVERY_IDLE_CHECK_SECONDS, MAX_PENDING_REQUEST_TIMEOUT_SECONDS, MAX_RAW_TEXT_LINES, MAX_UI_FRAMES, MIN_PENDING_REQUEST_TIMEOUT_SECONDS, SLOW_RESPONSE_THRESHOLD_MS, SNIFFER_SPLIT_FRAME_HOLD_MS, debug_sim, load_ui_settings, resource_path, save_ui_settings, mb_protocol.SessionMetrics, mb_slave_finder.SlaveFinderMixin, mb_master_ui.MasterMixin, mb_capture.CaptureMixin, mb_view.TRAFFIC_MENU_OPEN_IN_MASTER, ViewMixin, mb_widgets.RefreshButton, ToggleSwitch, mb_theme.apply_modern_theme, apply_theme, theme_colors, mb_i18n.LANGUAGE_CHOICES, LANGUAGE_ENGLISH, LANGUAGE_PORTUGUESE, canonical_ui_text, english_help_blocks, normalize_language, translate_text, translate_runtime_text`

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| **`class SnifferApp`** | **47-2583** | **SnifferApp.** | **Class container** |
| `SnifferApp.__init__`<br>__init__(self) | 48-153 | Creates the root window, loads persisted preferences before widget construction, initializes capture/discovery/log/history state, constructs the UI, freezes language geometry, applies language, installs event guards/persistence and schedules queue/status polling. | `super().__init__, self.title, load_ui_settings, normalize_language, bool` |
| `SnifferApp._freeze_language_layout_geometry`<br>_freeze_language_layout_geometry(self) | 156-216 | Measures the canonical Portuguese layout and freezes key container dimensions/minimums so changing to English changes text only, not panel/control geometry. | `getattr, self.update_idletasks, enumerate, max, overlay.place_configure` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SnifferApp._guard_mouse_wheel`<br>_guard_mouse_wheel(self, event) | 218-234 | Allows normal wheel scrolling only on truly scrollable widgets and consumes wheel events elsewhere so a ttk Notebook cannot change tabs from mouse-wheel input. | `getattr, isinstance` |
| `SnifferApp._install_mouse_wheel_guard`<br>_install_mouse_wheel_guard(self) | 236-270 | Prepend one application bindtag to every existing widget. | `stack.pop, self.bind_class, tuple, stack.extend, widget.bindtags` |
| `SnifferApp._apply_classic_widget_palette`<br>_apply_classic_widget_palette(self, dark) | 272-325 | Apply palette to the small set of classic Tk widgets. | `theme_colors, hasattr, self.raw_text.configure, self.help_text.configure, getattr` |
| `SnifferApp.configure_inspector_result_styles`<br>configure_inspector_result_styles(self) | 327-373 | Match the Inspector Result foreground to the Traffic result palette. | `theme_colors, self.ui_style.configure, hasattr, bool, tag_map.items` |
| `SnifferApp.tr`<br>tr(self, text) | 375-377 | Translate one canonical UI string to the currently selected language. | `translate_text` |
| `SnifferApp.trf`<br>trf(self, text, **values) | 379-381 | Translate a format template and substitute values afterwards. | `self.tr(text).format, self.tr` |
| `SnifferApp._capture_portuguese_help`<br>_capture_portuguese_help(self) | 383-401 | Store the original PT-PT Help text and tag ranges exactly as built. | `hasattr, self.help_text.get, self.help_text.tag_names, list, self.help_text.tag_ranges` |
| `SnifferApp._render_help_language`<br>_render_help_language(self) | 403-431 | Render Help in English or restore the original PT-PT text byte-for-byte. | `hasattr, self.help_text.configure, self.help_text.delete, self.help_text.see, f'{MAX_UI_FRAMES:_}'.replace` |
| `SnifferApp._translate_widget_tree`<br>_translate_widget_tree(self) | 433-465 | Translate static widget captions without changing widget geometry. | `stack.pop, isinstance, stack.extend, widget.keys, widget.winfo_children` |
| `SnifferApp._translate_traffic_context_menu`<br>_translate_traffic_context_menu(self) | 467-484 | Translate traffic context menu. | `getattr, menu.entryconfigure, self.tr` |
| `SnifferApp._apply_table_language`<br>_apply_table_language(self) | 486-538 | Refresh headings and filter values that are not normal widget captions. | `hasattr, canonical_ui_text, self.advanced_filter_type_combo.configure, self.advanced_filter_type_var.set, self.time_order_var.set` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SnifferApp.apply_language`<br>apply_language(self, save=True) | 540-601 | Performs runtime translation of static widgets, tables, context menus, help content and live status strings, then re-applies saved/fixed geometry and optionally persists the selected language. | `getattr, self._translate_widget_tree, self._apply_table_language, self._translate_traffic_context_menu, self._render_help_language` |
| `SnifferApp.on_language_changed`<br>on_language_changed(self, _event=None) | 603-604 | Event handler for language changed. | `self.apply_language` |
| `SnifferApp.toggle_dark_mode`<br>toggle_dark_mode(self) | 606-623 | Applies the alternate palette using the already-created ttk geometry and updates custom Canvas widgets; the control structure itself is not rebuilt. | `bool, apply_theme, self._apply_classic_widget_palette, self.save_current_ui_settings, self.dark_mode_var.get` |
| `SnifferApp._restore_saved_main_tab`<br>_restore_saved_main_tab(self) | 625-631 | Restore saved main tab. | `self.ui_settings.get, self.main_notebook.select, pages.get` |
| `SnifferApp._current_main_tab_name`<br>_current_main_tab_name(self) | 633-642 | Current main tab name. | `self.main_notebook.select, str` |
| `SnifferApp._schedule_ui_settings_save`<br>_schedule_ui_settings_save(self, *_args) | 644-661 | Schedules ui settings save. | `getattr, self.after, self.after_cancel` |
| `SnifferApp.save_current_ui_settings`<br>save_current_ui_settings(self) | 663-738 | Collects validated user-facing preferences from both Sniffer and Finder and writes them to the shared settings file. | `getattr, save_ui_settings, self.after_cancel, int, bool` |
| `SnifferApp._install_ui_settings_persistence`<br>_install_ui_settings_persistence(self) | 740-784 | Debounced persistence for all roaming-safe user preferences. | `self.main_notebook.bind, variable.trace_add, mapping.values, self._schedule_ui_settings_save` |
| `SnifferApp._schedule_bus_health_layout_sync`<br>_schedule_bus_health_layout_sync(self, _event=None) | 786-800 | Debounce Bus Health reflow after notebook/window/Inspector resizing. | `getattr, self.after_idle, self.after_cancel` |
| `SnifferApp._initialize_bus_health_layout`<br>_initialize_bus_health_layout(self) | 802-824 | Measure the natural full Bus Health height before adaptive reflow. | `getattr, self._sync_bus_health_layout, self.update_idletasks, card.winfo_reqheight, max` |
| `SnifferApp._apply_bus_health_layout`<br>_apply_bus_health_layout(self, compact) | 826-969 | Re-grids Bus Health labels/values into equal-height rows. | `getattr, bool, range, label_widget.grid_forget, value_widget.grid_forget` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SnifferApp._sync_bus_health_layout`<br>_sync_bus_health_layout(self) | 971-1024 | Chooses full versus compact Bus Health arrangement from real available height with hysteresis, then reapplies wrapping/column behavior on width changes. | `getattr, health_frame.winfo_height, max, bool, health_frame.winfo_exists` |
| `SnifferApp._build_ui`<br>_build_ui(self) | 1026-2583 | Constructs all top-level tabs and controls. | `self.ui_settings.get, self.columnconfigure, self.rowconfigure, ttk.Notebook, self.main_notebook.grid` |

## Detailed behavior of the key paths

**`SnifferApp.__init__` lines 48-153**

Creates the root window, loads persisted preferences before widget construction, initializes capture/discovery/log/history state, constructs the UI, freezes language geometry, applies language, installs event guards/persistence and schedules queue/status polling.

**`_freeze_language_layout_geometry` lines 156-216**

Measures the canonical Portuguese layout and freezes key container dimensions/minimums so changing to English changes text only, not panel/control geometry.

**`_guard_mouse_wheel` lines 218-234**

Allows normal wheel scrolling only on truly scrollable widgets and consumes wheel events elsewhere so a ttk Notebook cannot change tabs from mouse-wheel input.

**`apply_language` lines 540-601**

Performs runtime translation of static widgets, tables, context menus, help content and live status strings, then re-applies saved/fixed geometry and optionally persists the selected language.

**`toggle_dark_mode` lines 606-623**

Applies the alternate palette using the already-created ttk geometry and updates custom Canvas widgets; the control structure itself is not rebuilt.

**`save_current_ui_settings` lines 663-738**

Collects validated user-facing preferences from both Sniffer and Finder and writes them to the shared settings file. COM port names, window geometry and safety confirmation are intentionally excluded.

`_apply_bus_health_layout` **lines 826-969**

Re-grids Bus Health labels/values into equal-height rows. Full mode uses one metric pair per row; compact mode uses two pairs per row and controlled wrapping to keep content inside each rectangle.

`_sync_bus_health_layout` **lines 971-1024**

Chooses full versus compact Bus Health arrangement from real available height with hysteresis, then reapplies wrapping/column behavior on width changes.

`_build_ui` **lines 1026-2583**

Constructs all top-level tabs and controls. It is intentionally a large declarative layout method: global language/theme overlay, Sniffer configuration/actions/statistics, Traffic/Raw/Bus Health/Help notebook, filters/context menu, Frame Inspector and the Bus Slave Finder page.

> **Design note.** SnifferApp uses mixins to keep protocol capture, Finder, Master and view behavior in separate files while sharing one Tk root and common state.

# test_regression.py

**Automated regression suite.** The tests protect RTU timing, parser/resynchronisation behavior, pairing/timeouts, broadcast and echo-write classification, split-frame reassembly, in-place timeout marking, Slave Finder attempts, idle-bus check and FC43/14 logic, plus reversible Portuguese/English translation. Fake serial classes keep the tests deterministic and hardware-independent.

| Source lines | SHA-256 |
| --- | --- |
| 787 | `b5a3f7e75c0b385f8cb874fdf3b16c8e867108c59406078d966f333db4dba70c` |

**Imports:** `json, queue, random, tempfile, threading, time, unittest, collections.deque, pathlib.Path, mb_config, mb_slave_finder, mb_capture.CaptureMixin, mb_protocol.SessionMetrics, append_crc, burst_may_continue, crc16_modbus, crc_ok, modbus_rtu_interframe_gap_seconds, parse_burst, read_bits_details, split_capture_buffer, mb_slave_finder.SlaveFinderMixin, mb_view.ViewMixin, mb_diagnostics.inspector_values, mb_i18n.LANGUAGE_ENGLISH, LANGUAGE_PORTUGUESE, english_help_blocks, normalize_language, translate_details, translate_runtime_text, translate_text, mb_slave_finder.build_device_identification_probe, build_discovery_probe, device_identification_timeout_seconds, find_device_identification_response, find_discovery_response, format_device_identification, read_basic_device_identification`

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
| --- | --- | --- | --- |
| `with_bad_crc`<br>with_bad_crc(frame) | 49-52 | With bad crc. | `bytearray, bytes` |
| `device_id_response`<br>device_id_response(slave=1, objects=None, *, more_follows=False, next_object_id=0, conformity=1) | 55-83 | Device id response. | `bytearray, objects.items, append_crc, bytes, payload.extend` |
| **`class FakeSerial`** | **86-116** | **Minimal non-blocking serial fake for active Device ID regression tests.** | **Class container** |
| `FakeSerial.__init__`<br>__init__(self, response_by_object) | 89-92 | Init. | `dict, bytearray` |
| `FakeSerial.in_waiting`<br>in_waiting(self) | 95-96 | In waiting. | `len` |
| `FakeSerial.reset_input_buffer`<br>reset_input_buffer(self) | 98-99 | Reset input buffer. | `self.rx.clear` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `FakeSerial.reset_output_buffer`<br>reset_output_buffer(self) | 101-102 | Reset output buffer. | — |
| `FakeSerial.write`<br>write(self, data) | 104-110 | Write. | `bytes, self.writes.append, self.response_by_object.get, self.rx.extend, len` |
| `FakeSerial.read`<br>read(self, size) | 112-116 | Read. | `min, bytes, int, len` |
| **`class FakeSerialModule`** | **119-124** | **FakeSerialModule.** | **Class container** |
| **`class StaticVar`** | **127-132** | **StaticVar.** | **Class container** |
| `StaticVar.__init__`<br>__init__(self, value) | 128-129 | Init. | — |
| `StaticVar.get`<br>get(self) | 131-132 | Get. | — |
| **`class DummyCapture`** | **135-144** | **DummyCapture.** | **Class container** |
| `DummyCapture.__init__`<br>__init__(self, *, mode='Auto', baud='9600', data='8', parity='None', stop='1', manual='2.0') | 138-144 | Init. | `StaticVar` |
| **`class TimingRegressionTests`** | **147-179** | **TimingRegressionTests.** | **Class container** |
| `TimingRegressionTests.test_t35_uses_character_time_at_or_below_19200`<br>test_t35_uses_character_time_at_or_below_19200(self) | 148-158 | Regression test: t35 uses character time at or below 19200. | `self.assertAlmostEqual, modbus_rtu_interframe_gap_seconds` |
| `TimingRegressionTests.test_t35_is_fixed_above_19200`<br>test_t35_is_fixed_above_19200(self) | 160-167 | Regression test: t35 is fixed above 19200. | `self.subTest, self.assertAlmostEqual, modbus_rtu_interframe_gap_seconds` |
| `TimingRegressionTests.test_sniffer_auto_gap_uses_fixed_1750_us_above_19200`<br>test_sniffer_auto_gap_uses_fixed_1750_us_above_19200(self) | 169-171 | Regression test: sniffer auto gap uses fixed 1750 us above 19200. | `DummyCapture, self.assertAlmostEqual, capture.calculated_gap_ms` |
| `TimingRegressionTests.test_sniffer_manual_gap_is_unchanged`<br>test_sniffer_manual_gap_is_unchanged(self) | 173-175 | Regression test: sniffer manual gap is unchanged. | `DummyCapture, self.assertAlmostEqual, capture.calculated_gap_ms` |
| `TimingRegressionTests.test_device_id_timeout_covers_a_full_rtu_frame`<br>test_device_id_timeout_covers_a_full_rtu_frame(self) | 177-179 | Regression test: device id timeout covers a full rtu frame. | `device_identification_timeout_seconds, self.assertGreaterEqual` |
| **`class ParserRegressionTests`** | **182-284** | **ParserRegressionTests.** | **Class container** |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ParserRegressionTests.test_crc_known_request`<br>test_crc_known_request(self) | 183-186 | Regression test: crc known request. | `build_discovery_probe, self.assertTrue, self.assertEqual, crc_ok, bytes.fromhex` |
| `ParserRegressionTests.test_invalid_frame_before_valid_frame_stays_crc_error`<br>test_invalid_frame_before_valid_frame_stays_crc_error(self) | 188-193 | Regression test: invalid frame before valid frame stays crc error. | `with_bad_crc, build_discovery_probe, split_capture_buffer, self.assertEqual` |
| `ParserRegressionTests.test_long_unsynchronised_noise_recovers_next_valid_frame`<br>test_long_unsynchronised_noise_recovers_next_valid_frame(self) | 195-201 | Regression test: long unsynchronised noise recovers next valid frame. | `build_discovery_probe, split_capture_buffer, self.assertEqual, self.assertTrue, bytes` |
| `ParserRegressionTests.test_supported_function_code_shapes`<br>test_supported_function_code_shapes(self) | 203-235 | Regression test: supported function code shapes. | `frames.items, append_crc, bytes.fromhex, self.subTest, split_capture_buffer` |
| `ParserRegressionTests.test_random_noise_never_crashes_and_preserves_all_bytes`<br>test_random_noise_never_crashes_and_preserves_all_bytes(self) | 237-244 | Regression test: random noise never crashes and preserves all bytes. | `random.Random, range, rng.randint, bytes, split_capture_buffer` |
| `ParserRegressionTests.test_request_response_pairing_and_response_time`<br>test_request_response_pairing_and_response_time(self) | 246-269 | Regression test: request response pairing and response time. | `SessionMetrics, metrics.process, self.assertEqual, self.assertAlmostEqual, parse_burst` |
| `ParserRegressionTests.test_timeout_regression`<br>test_timeout_regression(self) | 271-284 | Regression test: timeout regression. | `SessionMetrics, metrics.process, self.assertEqual, metrics.pop_expired_requests, parse_burst` |
| **`class SlaveFinderRegressionTests`** | **287-369** | **SlaveFinderRegressionTests.** | **Class container** |
| `SlaveFinderRegressionTests.test_normal_discovery_response`<br>test_normal_discovery_response(self) | 288-292 | Regression test: normal discovery response. | `append_crc, find_discovery_response, self.assertIsNotNone, self.assertEqual, bytes.fromhex` |
| `SlaveFinderRegressionTests.test_discovery_exception_counts_as_response`<br>test_discovery_exception_counts_as_response(self) | 294-299 | Regression test: discovery exception counts as response. | `append_crc, find_discovery_response, self.assertIsNotNone, self.assertEqual, bytes.fromhex` |
| `SlaveFinderRegressionTests.test_device_identification_request`<br>test_device_identification_request(self) | 301-305 | Regression test: device identification request. | `build_device_identification_probe, self.assertEqual, self.assertTrue, bytes.fromhex, len` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `SlaveFinderRegressionTests.test_device_identification_response_parser` test_device_identification_response_parser(self) | 307-318 | Regression test: device identification response parser. | `device_id_response, find_device_identification_response, self.assertIsNotNone, self.assertEqual, format_device_identification` |
| `SlaveFinderRegressionTests.test_device_identification_parser_ignores_request_echo` test_device_identification_parser_ignores_request_echo(self) | 320-325 | Regression test: device identification parser ignores request echo. | `build_device_identification_probe, device_id_response, find_device_identification_response, self.assertIsNotNone, self.assertEqual` |
| `SlaveFinderRegressionTests.test_device_identification_exception` test_device_identification_exception(self) | 327-333 | Regression test: device identification exception. | `append_crc, find_device_identification_response, self.assertIsNotNone, self.assertEqual, self.assertIn` |
| `SlaveFinderRegressionTests.test_device_identification_rejects_bad_crc` test_device_identification_rejects_bad_crc(self) | 335-337 | Regression test: device identification rejects bad crc. | `with_bad_crc, self.assertIsNone, device_id_response, find_device_identification_response` |
| `SlaveFinderRegressionTests.test_device_identification_follows_more_follows` test_device_identification_follows_more_follows(self) | 339-369 | Regression test: device identification follows more follows. | `device_id_response, FakeSerial, read_basic_device_identification, self.assertIsNotNone, self.assertEqual` |
| **`class InternationalizationRegressionTests`** | **372-422** | **InternationalizationRegressionTests.** | **Class container** |
| `InternationalizationRegressionTests.test_language_normalization_defaults_to_portuguese` test_language_normalization_defaults_to_portuguese(self) | 373-375 | Regression test: language normalization defaults to portuguese. | `self.assertEqual, normalize_language` |
| `InternationalizationRegressionTests.test_static_ui_translation_is_reversible` test_static_ui_translation_is_reversible(self) | 377-383 | Regression test: static ui translation is reversible. | `self.assertEqual, translate_text` |
| `InternationalizationRegressionTests.test_dynamic_status_translation_is_reversible` test_dynamic_status_translation_is_reversible(self) | 385-389 | Regression test: dynamic status translation is reversible. | `self.assertEqual, translate_runtime_text` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `InternationalizationRegressionTests.test_register_details_follow_language`<br>test_register_details_follow_language(self) | 391-399 | Regression test: register details follow language. | `self.assertEqual, translate_details` |
| `InternationalizationRegressionTests.test_inspector_english_labels_and_crc`<br>test_inspector_english_labels_and_crc(self) | 401-411 | Regression test: inspector english labels and crc. | `append_crc, row.update, inspector_values, self.assertTrue, self.assertIn` |
| `InternationalizationRegressionTests.test_english_help_contains_no_portuguese_diacritics`<br>test_english_help_contains_no_portuguese_diacritics(self) | 413-422 | Regression test: english help contains no portuguese diacritics. | `''.join, self.assertIn, self.assertNotIn, text.casefold, english_help_blocks` |
| `bitwise_crc16`<br>bitwise_crc16(data) | 425-432 | Reference CRC used up to v3.0, kept to verify the table-driven CRC. | `range` |
| `process_frames`<br>process_frames(metrics, frames) | 435-442 | Run (bytes, channel, epoch) frames through SessionMetrics. | `enumerate, parse_burst, out.append, metrics.process` |
| **`class ProtocolV31RegressionTests`** | **445-538** | **ProtocolV31RegressionTests.** | **Class container** |
| `ProtocolV31RegressionTests.test_crc_table_matches_bitwise_reference`<br>test_crc_table_matches_bitwise_reference(self) | 446-450 | Regression test: crc table matches bitwise reference. | `random.Random, bytes, self.assertEqual, crc16_modbus, bitwise_crc16` |
| `ProtocolV31RegressionTests.test_broadcast_request_never_times_out`<br>test_broadcast_request_never_times_out(self) | 452-461 | Regression test: broadcast request never times out. | `append_crc, SessionMetrics, process_frames, self.assertEqual, metrics.expire_pending` |
| `ProtocolV31RegressionTests.test_fc06_echo_pair_is_request_then_response`<br>test_fc06_echo_pair_is_request_then_response(self) | 463-473 | Regression test: fc06 echo pair is request then response. | `append_crc, SessionMetrics, process_frames, self.assertEqual, self.assertAlmostEqual` |
| `ProtocolV31RegressionTests.test_unanswered_fc05_write_times_out`<br>test_unanswered_fc05_write_times_out(self) | 475-480 | Regression test: unanswered fc05 write times out. | `append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex` |
| `ProtocolV31RegressionTests.test_different_fc06_writes_are_both_requests`<br>test_different_fc06_writes_are_both_requests(self) | 482-491 | Regression test: different fc06 writes are both requests. | `append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `ProtocolV31RegressionTests.test_dual_rx_echo_must_come_from_opposite_channel`<br>test_dual_rx_echo_must_come_from_opposite_channel(self) | 493-504 | Regression test: dual rx echo must come from opposite channel. | `append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex` |
| `ProtocolV31RegressionTests.test_split_frame_is_held_until_complete`<br>test_split_frame_is_held_until_complete(self) | 506-511 | Regression test: split frame is held until complete. | `append_crc, self.assertTrue, self.assertFalse, burst_may_continue, bytes.fromhex` |
| `ProtocolV31RegressionTests.test_merged_frames_keep_their_chunk_arrival_times`<br>test_merged_frames_keep_their_chunk_arrival_times(self) | 513-527 | Regression test: merged frames keep their chunk arrival times. | `append_crc, parse_burst, self.assertEqual, self.assertAlmostEqual, SessionMetrics` |
| `ProtocolV31RegressionTests.test_read_bits_details_use_request_qty`<br>test_read_bits_details_use_request_qty(self) | 529-538 | Regression test: read bits details use request qty. | `append_crc, self.assertEqual, bytes.fromhex, read_bits_details` |
| **`class _ChunkedSerial`** | **541-568** | **Delivers byte chunks at fixed delays, like a USB latency timer.** | **Class container** |
| `_ChunkedSerial.__init__`<br>__init__(self, chunks, stop_event) | 544-547 | Init. | `time.perf_counter, list` |
| `_ChunkedSerial._due`<br>_due(self) | 549-555 | Due. | `time.perf_counter, self.stop_event.set` |
| `_ChunkedSerial.in_waiting`<br>in_waiting(self) | 558-559 | In waiting. | `len, self._due` |
| `_ChunkedSerial.read`<br>read(self, size) | 561-565 | Read. | `self._due, self.chunks.pop` |
| `_ChunkedSerial.close`<br>close(self) | 567-568 | Close. | — |
| **`class _ReaderStub`** | **571-574** | **_ReaderStub.** | **Class container** |
| `_ReaderStub.__init__`<br>__init__(self) | 572-574 | Init. | `threading.Event, queue.Queue` |
| **`class ReaderV31RegressionTests`** | **577-591** | **ReaderV31RegressionTests.** | **Class container** |
| `ReaderV31RegressionTests.test_frame_split_by_usb_latency_is_reassembled`<br>test_frame_split_by_usb_latency_is_reassembled(self) | 578-591 | Regression test: frame split by usb latency is reassembled. | `append_crc, _ReaderStub, _ChunkedSerial, reader.reader_loop, self.assertEqual` |
| **`class _FakeVar`** | **594-599** | **_FakeVar.** | **Class container** |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `_FakeVar.__init__`<br>__init__(self, value) | 595-596 | Init. | — |
| `_FakeVar.get`<br>get(self) | 598-599 | Get. | — |
| **`class _FakeTree`** | **602-607** | **_FakeTree.** | **Class container** |
| `_FakeTree.__init__`<br>__init__(self) | 603-604 | Init. | — |
| `_FakeTree.item`<br>item(self, iid, tags=()) | 606-607 | Item. | — |
| **`class _TimeoutView`** | **610-627** | **_TimeoutView.** | **Class container** |
| `_TimeoutView.__init__`<br>__init__(self, type_filter) | 611-621 | Init. | `_FakeTree, _FakeVar, set, deque` |
| `_TimeoutView.render_filtered_view`<br>render_filtered_view(self) | 623-624 | Render filtered view. | — |
| `_TimeoutView.selected_record`<br>selected_record(self) | 626-627 | Selected record. | — |
| **`class ViewV31RegressionTests`** | **630-657** | **ViewV31RegressionTests.** | **Class container** |
| `ViewV31RegressionTests.test_timeout_retags_rows_without_full_rebuild`<br>test_timeout_retags_rows_without_full_rebuild(self) | 631-636 | Regression test: timeout retags rows without full rebuild. | `_TimeoutView, self.assertEqual, view.mark_timed_out_records` |
| `ViewV31RegressionTests.test_timeouts_filter_still_rebuilds_view`<br>test_timeouts_filter_still_rebuilds_view(self) | 638-641 | Regression test: timeouts filter still rebuilds view. | `_TimeoutView, view.mark_timed_out_records, self.assertEqual` |
| `ViewV31RegressionTests.test_inspector_shows_address_of_each_register`<br>test_inspector_shows_address_of_each_register(self) | 643-650 | Regression test: inspector shows address of each register. | `append_crc, inspector_values, self.assertIn, bytes.fromhex, parse_burst` |
| `ViewV31RegressionTests.test_inspector_shows_address_of_each_bit`<br>test_inspector_shows_address_of_each_bit(self) | 652-657 | Regression test: inspector shows address of each bit. | `append_crc, row.update, inspector_values, self.assertIn, bytes.fromhex` |
| **`class _FakeFinderSerial`** | **660-685** | **_FakeFinderSerial.** | **Class container** |
| `_FakeFinderSerial.__init__`<br>__init__(self, incoming=b'') | 661-663 | Init. | `bytearray` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `_FakeFinderSerial.in_waiting`<br>in_waiting(self) | 666-667 | In waiting. | `len` |
| `_FakeFinderSerial.read`<br>read(self, size) | 669-672 | Read. | `bytes` |
| `_FakeFinderSerial.write`<br>write(self, data) | 674-676 | Write. | `self.writes.append, len, bytes` |
| `_FakeFinderSerial.reset_input_buffer`<br>reset_input_buffer(self) | 678-679 | Reset input buffer. | — |
| `_FakeFinderSerial.reset_output_buffer`<br>reset_output_buffer(self) | 681-682 | Reset output buffer. | — |
| `_FakeFinderSerial.close`<br>close(self) | 684-685 | Close. | — |
| **`class _FinderStub`** | **688-714** | **_FinderStub.** | **Class container** |
| `_FinderStub.__init__`<br>__init__(self, fake_serial) | 691-695 | Init. | `queue.Queue, threading.Event` |
| `_FinderStub.tr`<br>tr(self, text) | 697-698 | Tr. | `translate_text` |
| `_FinderStub.trf`<br>trf(self, text, **values) | 700-701 | Trf. | `self.tr(text).format, self.tr` |
| `_FinderStub.open_discovery_serial_port`<br>open_discovery_serial_port(self, serial, port, baud, parity, stopbits) | 703-704 | Open discovery serial port. | — |
| `_FinderStub.run_worker`<br>run_worker(self, slaves, attempts) | 706-714 | Run worker. | `self.discovery_worker, self.event_queue.empty, events.append, self.event_queue.get` |
| **`class FinderV31RegressionTests`** | **717-763** | **FinderV31RegressionTests.** | **Class container** |
| `FinderV31RegressionTests.setUp`<br>setUp(self) | 718-720 | SetUp. | — |
| `FinderV31RegressionTests.tearDown`<br>tearDown(self) | 722-723 | TearDown. | — |
| `FinderV31RegressionTests.test_each_silent_slave_gets_all_attempts`<br>test_each_silent_slave_gets_all_attempts(self) | 725-730 | Regression test: each silent slave gets all attempts. | `_FinderStub, stub.run_worker, self.assertIsNone, self.assertEqual, _FakeFinderSerial` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `FinderV31RegressionTests.test_traffic_before_scan_blocks_transmission`<br>test_traffic_before_scan_blocks_transmission(self) | 732-737 | Regression test: traffic before scan blocks transmission. | `_FinderStub, stub.run_worker, self.assertIn, self.assertEqual, _FakeFinderSerial` |
| `FinderV31RegressionTests.test_legacy_default_timeout_migrates`<br>test_legacy_default_timeout_migrates(self) | 739-763 | Regression test: legacy default timeout migrates. | `tempfile.TemporaryDirectory, Path, path.write_text, self.assertEqual, json.dumps` |
| **`class SourceHygieneV31Tests`** | **766-783** | **SourceHygieneV31Tests.** | **Class container** |
| `SourceHygieneV31Tests.test_no_inline_language_branches`<br>test_no_inline_language_branches(self) | 767-773 | Regression test: no inline language branches. | `package.glob, source.read_text, self.assertNotIn, Path(__file__).resolve, Path` |
| `SourceHygieneV31Tests.test_message_templates_translate`<br>test_message_templates_translate(self) | 775-783 | Regression test: message templates translate. | `self.assertEqual, translate_text('Encontrados: {count}', LANGUAGE_ENGLISH).format, translate_text` |

# test_master.py

**Master regression tests.** An in-memory Modbus slave behind a fake serial port answers reads and writes, so the tests cover data formats, frames from the Modbus specification, response matching, scheduling, writes, broadcast, timeouts, the idle-bus check and workspace validation without hardware.

| Source lines | SHA-256 |
|---|---|
| 563 | `23f733b881f14e4300d50ab5587347ea1e02eedcc933691a6d060d2c1c066c05` |

**Imports:** `json, os, queue, struct, tempfile, time, unittest, pathlib.Path, mb_config, mb_master_ui, mb_master.BYTE_ORDERS, BYTE_ORDERS_64, format_value, format_window_values, raw_text_for_scaled, register_groups, validate_scale, MasterEngine, PollDefinition, build_read_request, build_write_request, encode_coil_value, encode_register_value, find_master_response, format_register_values, mb_master_ui.MasterMixin, MasterWindow, apply_format_to_rows, mb_protocol.append_crc, crc_ok`

## Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.2 source snapshot documented here.

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| **`class FakeSlaveSerial`** | **33-115** | **In-memory Modbus RTU slave behind a fake serial port.** | **Class container** |
| `FakeSlaveSerial.__init__`<br>__init__(self, slave_id=1, *, echo=False, bus_traffic=b'', exception_for=None) | 41-48 | Init. | `bytearray, range` |
| `FakeSlaveSerial.in_waiting`<br>in_waiting(self) | 51-52 | In waiting. | `len` |
| `FakeSlaveSerial.read`<br>read(self, size) | 54-57 | Read. | `bytes` |
| `FakeSlaveSerial.reset_input_buffer`<br>reset_input_buffer(self) | 59-60 | Reset input buffer. | `self.rx.clear` |
| `FakeSlaveSerial.reset_output_buffer`<br>reset_output_buffer(self) | 62-63 | Reset output buffer. | — |
| `FakeSlaveSerial.cancel_read`<br>cancel_read(self) | 65-66 | Cancel read. | — |
| `FakeSlaveSerial.cancel_write`<br>cancel_write(self) | 68-69 | Cancel write. | — |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `FakeSlaveSerial.close`<br>close(self) | 71-72 | Close. | — |
| `FakeSlaveSerial.write`<br>write(self, data) | 74-83 | Write. | `bytes, self.requests.append, len, self.rx.extend, crc_ok` |
| `FakeSlaveSerial._answer`<br>_answer(self, req) | 85-115 | Answer. | `append_crc, b''.join, bytearray, range, bytes` |
| `drain`<br>drain(event_queue) | 118-122 | Drain. | `event_queue.empty, events.append, event_queue.get` |
| `make_engine`<br>make_engine(serial, **overrides) | 125-129 | Make engine. | `dict, options.update, MasterEngine, queue.Queue` |
| **`class MasterFormatTests`** | **132-168** | **MasterFormatTests.** | **Class container** |
| `MasterFormatTests.test_32_bit_formats_round_trip_in_every_byte_order`<br>test_32_bit_formats_round_trip_in_every_byte_order(self) | 133-140 | Regression test: 32 bit formats round trip in every byte order. | `encode_register_value, self.assertEqual, len, format_register_values` |
| `MasterFormatTests.test_float_byte_orders_match_wire_layout`<br>test_float_byte_orders_match_wire_layout(self) | 142-148 | Regression test: float byte orders match wire layout. | `struct.pack, self.assertEqual, encode_register_value, bytes.fromhex` |
| `MasterFormatTests.test_16_bit_formats`<br>test_16_bit_formats(self) | 150-157 | Regression test: 16 bit formats. | `self.assertEqual, format_register_values, encode_register_value` |
| `MasterFormatTests.test_values_outside_the_format_are_rejected`<br>test_values_outside_the_format_are_rejected(self) | 159-165 | Regression test: values outside the format are rejected. | `self.assertRaises, encode_coil_value, encode_register_value` |
| `MasterFormatTests.test_unpaired_register_in_32_bit_format`<br>test_unpaired_register_in_32_bit_format(self) | 167-168 | Regression test: unpaired register in 32 bit format. | `self.assertEqual, format_register_values` |
| **`class MasterFrameTests`** | **171-216** | **MasterFrameTests.** | **Class container** |
| `MasterFrameTests.test_write_frames_match_modbus_specification_examples`<br>test_write_frames_match_modbus_specification_examples(self) | 172-182 | Regression test: write frames match modbus specification examples. | `self.assertEqual, bytes.fromhex, build_write_request` |
| `MasterFrameTests.test_invalid_definitions_are_rejected`<br>test_invalid_definitions_are_rejected(self) | 184-190 | Regression test: invalid definitions are rejected. | `self.assertRaises, build_write_request, build_read_request` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterFrameTests.test_read_response_found_after_adapter_echo`<br>test_read_response_found_after_adapter_echo(self) | 192-197 | Regression test: read response found after adapter echo. | `build_read_request, append_crc, find_master_response, self.assertEqual, bytes.fromhex` |
| `MasterFrameTests.test_coil_response_is_trimmed_to_quantity`<br>test_coil_response_is_trimmed_to_quantity(self) | 199-202 | Regression test: coil response is trimmed to quantity. | `build_read_request, find_master_response, self.assertEqual, append_crc, bytes.fromhex` |
| `MasterFrameTests.test_exception_and_bad_crc`<br>test_exception_and_bad_crc(self) | 204-210 | Regression test: exception and bad crc. | `build_read_request, append_crc, self.assertEqual, bytearray, self.assertIsNone` |
| `MasterFrameTests.test_write_response_must_match_request`<br>test_write_response_must_match_request(self) | 212-216 | Regression test: write response must match request. | `build_write_request, self.assertIsNotNone, append_crc, self.assertIsNone, find_master_response` |
| **`class MasterEngineTests`** | **219-288** | **MasterEngineTests.** | **Class container** |
| `MasterEngineTests.run_engine`<br>run_engine(self, engine, seconds) | 220-224 | Run engine. | `engine.start, time.sleep, engine.stop, drain` |
| `MasterEngineTests.test_scheduler_serves_writes_then_read_once_then_due_polls`<br>test_scheduler_serves_writes_then_read_once_then_due_polls(self) | 226-238 | Regression test: scheduler serves writes then read once then due polls. | `make_engine, engine.set_poll, engine.read_once, engine.queue_write, self.assertEqual` |
| `MasterEngineTests.test_fast_and_slow_windows_share_one_bus`<br>test_fast_and_slow_windows_share_one_bus(self) | 240-254 | Regression test: fast and slow windows share one bus. | `FakeSlaveSerial, make_engine, engine.set_poll, self.run_engine, self.assertGreater` |
| `MasterEngineTests.test_write_is_executed_and_reported`<br>test_write_is_executed_and_reported(self) | 256-262 | Regression test: write is executed and reported. | `FakeSlaveSerial, make_engine, engine.queue_write, self.assertEqual, build_write_request` |
| `MasterEngineTests.test_broadcast_write_does_not_wait_for_a_response`<br>test_broadcast_write_does_not_wait_for_a_response(self) | 264-272 | Regression test: broadcast write does not wait for a response. | `FakeSlaveSerial, make_engine, engine.queue_write, time.perf_counter, engine.transact` |
| `MasterEngineTests.test_timeout_exception_and_invalid_results`<br>test_timeout_exception_and_invalid_results(self) | 274-279 | Regression test: timeout exception and invalid results. | `make_engine, self.assertEqual, engine.transact, FakeSlaveSerial, build_read_request` |
| `MasterEngineTests.test_traffic_on_the_bus_blocks_the_master`<br>test_traffic_on_the_bus_blocks_the_master(self) | 281-288 | Regression test: traffic on the bus blocks the master. | `FakeSlaveSerial, make_engine, engine.set_poll, self.run_engine, self.assertEqual` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| **`class MasterWindowTests`** | **291-308** | **MasterWindowTests.** | **Class container** |
| `MasterWindowTests.test_workspace_entry_round_trip_and_validation`<br>test_workspace_entry_round_trip_and_validation(self) | 292-300 | Regression test: workspace entry round trip and validation. | `MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json, self.assertRaises` |
| `MasterWindowTests.test_write_dialog_builds_fc16_for_float_values`<br>test_write_dialog_builds_fc16_for_float_values(self) | 302-308 | Regression test: write dialog builds fc16 for float values. | `MasterWindow, MasterMixin.master_build_write, self.assertEqual, bytes.fromhex, self.assertRaises` |
| **`class MasterWorkspaceFolderTests`** | **311-347** | **MasterWorkspaceFolderTests.** | **Class container** |
| `MasterWorkspaceFolderTests.setUp`<br>setUp(self) | 312-315 | SetUp. | `os.environ.get, tempfile.TemporaryDirectory` |
| `MasterWorkspaceFolderTests.tearDown`<br>tearDown(self) | 317-322 | TearDown. | `self._folder.cleanup, os.environ.pop` |
| `MasterWorkspaceFolderTests.test_workspaces_live_beside_the_settings`<br>test_workspaces_live_beside_the_settings(self) | 324-328 | Regression test: workspaces live beside the settings. | `mb_config.get_workspaces_folder, self.assertEqual, self.assertTrue, folder.is_dir, mb_config.get_settings_file` |
| `MasterWorkspaceFolderTests.test_saved_workspaces_are_listed_by_name`<br>test_saved_workspaces_are_listed_by_name(self) | 330-335 | Regression test: saved workspaces are listed by name. | `(mb_config.get_workspaces_folder() / 'notas.txt').write_text, self.assertEqual, mb_master_ui.workspace_path(name).write_text, mb_master_ui.…` |
| `MasterWorkspaceFolderTests.test_workspace_names_must_be_valid_file_names`<br>test_workspace_names_must_be_valid_file_names(self) | 337-341 | Regression test: workspace names must be valid file names. | `self.assertEqual, mb_master_ui.validate_workspace_name, self.assertRaises` |
| `MasterWorkspaceFolderTests.test_window_title_is_saved_in_the_workspace`<br>test_window_title_is_saved_in_the_workspace(self) | 343-347 | Regression test: window title is saved in the workspace. | `MasterWindow, self.assertEqual, MasterWindow.validated_kwargs, window.to_json` |
| **`class MasterPerAddressFormatTests`** | **350-407** | **MasterPerAddressFormatTests.** | **Class container** |
| `MasterPerAddressFormatTests.test_64_bit_formats_round_trip_in_every_byte_order`<br>test_64_bit_formats_round_trip_in_every_byte_order(self) | 351-358 | Regression test: 64 bit formats round trip in every byte order. | `encode_register_value, self.assertEqual, len, format_value` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterPerAddressFormatTests.test_double_byte_orders_match_wire_layout`<br>test_double_byte_orders_match_wire_layout(self) | 360-365 | Regression test: double byte orders match wire layout. | `self.assertEqual, encode_register_value` |
| `MasterPerAddressFormatTests.test_per_address_formats_override_the_window_format`<br>test_per_address_formats_override_the_window_format(self) | 367-372 | Regression test: per address formats override the window format. | `format_window_values, self.assertEqual, register_groups` |
| `MasterPerAddressFormatTests.test_default_multi_register_values_cut_by_an_override_show_a_dash`<br>test_default_multi_register_values_cut_by_an_override_show_a_dash(self) | 374-376 | Regression test: default multi register values cut by an override show a dash. | `format_window_values, self.assertEqual` |
| `MasterPerAddressFormatTests.test_selection_rules_for_multi_register_formats`<br>test_selection_rules_for_multi_register_formats(self) | 378-393 | Regression test: selection rules for multi register formats. | `MasterWindow, apply_format_to_rows, self.assertEqual, self.assertNotIn, self.assertRaises` |
| `MasterPerAddressFormatTests.test_formats_are_saved_and_invalid_ones_dropped`<br>test_formats_are_saved_and_invalid_ones_dropped(self) | 395-401 | Regression test: formats are saved and invalid ones dropped. | `MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json` |
| `MasterPerAddressFormatTests.test_write_uses_the_format_of_the_clicked_value`<br>test_write_uses_the_format_of_the_clicked_value(self) | 403-407 | Regression test: write uses the format of the clicked value. | `MasterWindow, MasterMixin.master_build_write, self.assertEqual, bytes.fromhex` |
| **`class MasterScaleTests`** | **410-449** | **MasterScaleTests.** | **Class container** |
| `MasterScaleTests.test_scaled_values_are_shown_in_engineering_units`<br>test_scaled_values_are_shown_in_engineering_units(self) | 413-417 | Regression test: scaled values are shown in engineering units. | `format_window_values, self.assertEqual` |
| `MasterScaleTests.test_scale_applies_to_multi_register_values`<br>test_scale_applies_to_multi_register_values(self) | 419-423 | Regression test: scale applies to multi register values. | `encode_register_value, validate_scale, self.assertEqual, format_window_values` |
| `MasterScaleTests.test_written_engineering_values_are_converted_to_raw`<br>test_written_engineering_values_are_converted_to_raw(self) | 425-432 | Regression test: written engineering values are converted to raw. | `self.assertEqual, MasterWindow, MasterMixin.master_build_write, raw_text_for_scaled, bytes.fromhex` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterScaleTests.test_invalid_scales_are_rejected`<br>test_invalid_scales_are_rejected(self) | 434-440 | Regression test: invalid scales are rejected. | `self.assertRaises, raw_text_for_scaled, validate_scale` |
| `MasterScaleTests.test_scales_are_saved_and_invalid_ones_dropped`<br>test_scales_are_saved_and_invalid_ones_dropped(self) | 442-449 | Regression test: scales are saved and invalid ones dropped. | `MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json, list` |
| **`class MasterOpenFromTrafficTests`** | **452-474** | **MasterOpenFromTrafficTests.** | **Class container** |
| `MasterOpenFromTrafficTests.definition`<br>definition(self, record) | 453-454 | Definition. | `MasterMixin.master_definition_from_record` |
| `MasterOpenFromTrafficTests.test_request_and_matched_response_give_the_read_definition`<br>test_request_and_matched_response_give_the_read_definition(self) | 456-465 | Regression test: request and matched response give the read definition. | `self.assertEqual, self.definition` |
| `MasterOpenFromTrafficTests.test_frames_without_a_read_definition_are_refused`<br>test_frames_without_a_read_definition_are_refused(self) | 467-474 | Regression test: frames without a read definition are refused. | `self.assertIsNone, self.definition` |
| **`class FlakySlaveSerial`** | **477-492** | **Fake slave whose port fails (like an unplugged USB adapter).** | **Class container** |
| `FlakySlaveSerial.__init__`<br>__init__(self, failures, **kwargs) | 480-483 | Init. | `super().__init__, super` |
| `FlakySlaveSerial.write`<br>write(self, data) | 485-489 | Write. | `super().write, OSError, super` |
| `FlakySlaveSerial.close`<br>close(self) | 491-492 | Close. | — |
| **`class MasterReconnectTests`** | **495-542** | **MasterReconnectTests.** | **Class container** |
| `MasterReconnectTests.test_engine_reopens_the_port_after_a_failure`<br>test_engine_reopens_the_port_after_a_failure(self) | 496-514 | Regression test: engine reopens the port after a failure. | `FlakySlaveSerial, FakeSlaveSerial, make_engine, engine.set_poll, engine.start` |
| `MasterReconnectTests.test_engine_keeps_retrying_until_stopped`<br>test_engine_keeps_retrying_until_stopped(self) | 516-532 | Regression test: engine keeps retrying until stopped. | `make_engine, engine.set_poll, engine.start, time.sleep, engine.stop` |

| Callable | Lines | Responsibility | Representative direct calls |
|---|---|---|---|
| `MasterReconnectTests.test_without_reopen_a_failure_ends_the_session`<br>test_without_reopen_a_failure_ends_the_session(self) | 534-542 | Regression test: without reopen a failure ends the session. | `make_engine, engine.set_poll, engine.start, time.sleep, engine.stop` |
| **`class MasterSettingsTests`** | **545-559** | **MasterSettingsTests.** | **Class container** |
| `MasterSettingsTests.test_last_workspace_is_validated`<br>test_last_workspace_is_validated(self) | 546-559 | Regression test: last workspace is validated. | `tempfile.TemporaryDirectory, Path, path.write_text, self.assertEqual, json.dumps` |

# 12. Regression-test suite

The documented package was executed with `python -m unittest discover -s tests -p "test_*.py" -v`. All 86 tests passed.

> Verification result: **86 tests run - 86 passed - 0 failures/errors**.

## TimingRegressionTests

Protects RTU timing and automatic/manual gap calculations, including the fixed 1.750 ms t3.5 rule above 19200 bit/s and FC43 response timing.

| Test | Lines | What it protects |
|---|---|---|
| `test_t35_uses_character_time_at_or_below_19200` | 148-158 | Regression test: t35 uses character time at or below 19200. |
| `test_t35_is_fixed_above_19200` | 160-167 | Regression test: t35 is fixed above 19200. |
| `test_sniffer_auto_gap_uses_fixed_1750_us_above_19200` | 169-171 | Regression test: sniffer auto gap uses fixed 1750 us above 19200. |
| `test_sniffer_manual_gap_is_unchanged` | 173-175 | Regression test: sniffer manual gap is unchanged. |
| `test_device_id_timeout_covers_a_full_rtu_frame` | 177-179 | Regression test: device id timeout covers a full rtu frame. |

## ParserRegressionTests

Protects CRC, resynchronisation, supported frame shapes, random-noise robustness, request/response pairing and timeout behavior.

| Test | Lines | What it protects |
|---|---|---|
| `test_crc_known_request` | 183-186 | Regression test: crc known request. |
| `test_invalid_frame_before_valid_frame_stays_crc_error` | 188-193 | Regression test: invalid frame before valid frame stays crc error. |
| `test_long_unsynchronised_noise_recovers_next_valid_frame` | 195-201 | Regression test: long unsynchronised noise recovers next valid frame. |
| `test_supported_function_code_shapes` | 203-235 | Regression test: supported function code shapes. |

| Test | Lines | What it protects |
|---|---|---|
| `test_random_noise_never_crashes_and_preserves_all_bytes` | 237-244 | Regression test: random noise never crashes and preserves all bytes. |
| `test_request_response_pairing_and_response_time` | 246-269 | Regression test: request response pairing and response time. |
| `test_timeout_regression` | 271-284 | Regression test: timeout regression. |

## SlaveFinderRegressionTests

Protects FC03/FC04 discovery and FC43/14 Device Identification parsing, CRC validation, Exceptions, echo rejection and More Follows paging.

| Test | Lines | What it protects |
|---|---|---|
| `test_normal_discovery_response` | 288-292 | Regression test: normal discovery response. |
| `test_discovery_exception_counts_as_response` | 294-299 | Regression test: discovery exception counts as response. |
| `test_device_identification_request` | 301-305 | Regression test: device identification request. |
| `test_device_identification_response_parser` | 307-318 | Regression test: device identification response parser. |
| `test_device_identification_parser_ignores_request_echo` | 320-325 | Regression test: device identification parser ignores request echo. |
| `test_device_identification_exception` | 327-333 | Regression test: device identification exception. |
| `test_device_identification_rejects_bad_crc` | 335-337 | Regression test: device identification rejects bad crc. |
| `test_device_identification_follows_more_follows` | 339-369 | Regression test: device identification follows more follows. |

## InternationalizationRegressionTests

Protects language normalization, reversible static/runtime translation, translated register details, Frame Inspector English text and English Help content.

| Test | Lines | What it protects |
|---|---|---|
| `test_language_normalization_defaults_to_portuguese` | 373-375 | Regression test: language normalization defaults to portuguese. |

| Test | Lines | What it protects |
|---|---|---|
| `test_static_ui_translation_is_reversible` | 377-383 | Regression test: static ui translation is reversible. |
| `test_dynamic_status_translation_is_reversible` | 385-389 | Regression test: dynamic status translation is reversible. |
| `test_register_details_follow_language` | 391-399 | Regression test: register details follow language. |
| `test_inspector_english_labels_and_crc` | 401-411 | Regression test: inspector english labels and crc. |
| `test_english_help_contains_no_portuguese_diacritics` | 413-422 | Regression test: english help contains no portuguese diacritics. |

## ProtocolV31RegressionTests

Protects the v3.1 protocol fixes: table CRC against the bit-wise reference, broadcast accounting, FC05/06 echo pairing, dual-RX direction, split-burst detection, per-read timestamps and FC01/02 bit formatting.

| Test | Lines | What it protects |
|---|---|---|
| `test_crc_table_matches_bitwise_reference` | 446-450 | Regression test: crc table matches bitwise reference. |
| `test_broadcast_request_never_times_out` | 452-461 | Regression test: broadcast request never times out. |
| `test_fc06_echo_pair_is_request_then_response` | 463-473 | Regression test: fc06 echo pair is request then response. |
| `test_unanswered_fc05_write_times_out` | 475-480 | Regression test: unanswered fc05 write times out. |
| `test_different_fc06_writes_are_both_requests` | 482-491 | Regression test: different fc06 writes are both requests. |
| `test_dual_rx_echo_must_come_from_opposite_channel` | 493-504 | Regression test: dual rx echo must come from opposite channel. |
| `test_split_frame_is_held_until_complete` | 506-511 | Regression test: split frame is held until complete. |
| `test_merged_frames_keep_their_chunk_arrival_times` | 513-527 | Regression test: merged frames keep their chunk arrival times. |
| `test_read_bits_details_use_request_qty` | 529-538 | Regression test: read bits details use request qty. |

## ReaderV31RegressionTests

Feeds a frame split by a simulated 16 ms USB latency into the real reader_loop() and requires one CRC-valid RESPONSE.

| Test | Lines | What it protects |
|---|---|---|
| `test_frame_split_by_usb_latency_is_reassembled` | 578-591 | Regression test: frame split by usb latency is reassembled. |

## ViewV31RegressionTests

Protects in-place timeout retagging, the Timeouts-filter rebuild and the address-mapped register/bit data in the Frame Inspector.

| Test | Lines | What it protects |
|---|---|---|
| `test_timeout_retags_rows_without_full_rebuild` | 631-636 | Regression test: timeout retags rows without full rebuild. |
| `test_timeouts_filter_still_rebuilds_view` | 638-641 | Regression test: timeouts filter still rebuilds view. |
| `test_inspector_shows_address_of_each_register` | 643-650 | Regression test: inspector shows address of each register. |
| `test_inspector_shows_address_of_each_bit` | 652-657 | Regression test: inspector shows address of each bit. |

## FinderV31RegressionTests

Runs the real discovery worker against fake serial ports to check attempts, the idle-bus abort and the 40 to 100 ms settings migration.

| Test | Lines | What it protects |
|---|---|---|
| `test_each_silent_slave_gets_all_attempts` | 725-730 | Regression test: each silent slave gets all attempts. |
| `test_traffic_before_scan_blocks_transmission` | 732-737 | Regression test: traffic before scan blocks transmission. |
| `test_legacy_default_timeout_migrates` | 739-763 | Regression test: legacy default timeout migrates. |

## SourceHygieneV31Tests

Rejects inline language branches in the package and checks that message templates translate.

| Test | Lines | What it protects |
|---|---|---|
| `test_no_inline_language_branches` | 767-773 | Regression test: no inline language branches. |
| `test_message_templates_translate` | 775-783 | Regression test: message templates translate. |

## MasterFormatTests

Protects the 16/32-bit data formats, the four byte orders and the rejection of values that do not fit.

| Test | Lines | What it protects |
|---|---|---|
| `test_32_bit_formats_round_trip_in_every_byte_order` | 133-140 | Regression test: 32 bit formats round trip in every byte order. |
| `test_float_byte_orders_match_wire_layout` | 142-148 | Regression test: float byte orders match wire layout. |
| `test_16_bit_formats` | 150-157 | Regression test: 16 bit formats. |
| `test_values_outside_the_format_are_rejected` | 159-165 | Regression test: values outside the format are rejected. |
| `test_unpaired_register_in_32_bit_format` | 167-168 | Regression test: unpaired register in 32 bit format. |

## MasterFrameTests

Checks write frames against the Modbus specification examples, definition limits, adapter echoes, exceptions, bad CRC and write response matching.

| Test | Lines | What it protects |
|---|---|---|
| `test_write_frames_match_modbus_specification_examples` | 172-182 | Regression test: write frames match modbus specification examples. |
| `test_invalid_definitions_are_rejected` | 184-190 | Regression test: invalid definitions are rejected. |
| `test_read_response_found_after_adapter_echo` | 192-197 | Regression test: read response found after adapter echo. |
| `test_coil_response_is_trimmed_to_quantity` | 199-202 | Regression test: coil response is trimmed to quantity. |
| `test_exception_and_bad_crc` | 204-210 | Regression test: exception and bad crc. |
| `test_write_response_must_match_request` | 212-216 | Regression test: write response must match request. |

## MasterEngineTests

Runs MasterEngine against an in-memory slave: scheduling order, fast and slow windows on one bus, writes, broadcast, timeouts, exceptions and the idle-bus abort.

| Test | Lines | What it protects |
|---|---|---|
| `test_scheduler_serves_writes_then_read_once_then_due_polls` | 226-238 | Regression test: scheduler serves writes then read once then due polls. |

| Test | Lines | What it protects |
|---|---|---|
| `test_fast_and_slow_windows_share_one_bus` | 240-254 | Regression test: fast and slow windows share one bus. |
| `test_write_is_executed_and_reported` | 256-262 | Regression test: write is executed and reported. |
| `test_broadcast_write_does_not_wait_for_a_response` | 264-272 | Regression test: broadcast write does not wait for a response. |
| `test_timeout_exception_and_invalid_results` | 274-279 | Regression test: timeout exception and invalid results. |
| `test_traffic_on_the_bus_blocks_the_master` | 281-288 | Regression test: traffic on the bus blocks the master. |

## MasterWindowTests

Protects workspace entry validation and the FC16 float encoding of the write dialog.

| Test | Lines | What it protects |
|---|---|---|
| `test_workspace_entry_round_trip_and_validation` | 292-300 | Regression test: workspace entry round trip and validation. |
| `test_write_dialog_builds_fc16_for_float_values` | 302-308 | Regression test: write dialog builds fc16 for float values. |

## MasterWorkspaceFolderTests

Checks that workspaces live beside settings.json, are listed alphabetically and use valid Windows file names, and that window names are saved.

| Test | Lines | What it protects |
|---|---|---|
| `test_workspaces_live_beside_the_settings` | 324-328 | Regression test: workspaces live beside the settings. |
| `test_saved_workspaces_are_listed_by_name` | 330-335 | Regression test: saved workspaces are listed by name. |
| `test_workspace_names_must_be_valid_file_names` | 337-341 | Regression test: workspace names must be valid file names. |
| `test_window_title_is_saved_in_the_workspace` | 343-347 | Regression test: window title is saved in the workspace. |

## MasterPerAddressFormatTests

Protects the 64-bit formats and byte orders, per-address formats, value grouping, the selection rules and the write encoding of the clicked value.

| Test | Lines | What it protects |
|---|---|---|
| `test_64_bit_formats_round_trip_in_every_byte_order` | 351-358 | Regression test: 64 bit formats round trip in every byte order. |
| `test_double_byte_orders_match_wire_layout` | 360-365 | Regression test: double byte orders match wire layout. |
| `test_per_address_formats_override_the_window_format` | 367-372 | Regression test: per address formats override the window format. |
| `test_default_multi_register_values_cut_by_an_override_show_a_dash` | 374-376 | Regression test: default multi register values cut by an override show a dash. |
| `test_selection_rules_for_multi_register_formats` | 378-393 | Regression test: selection rules for multi register formats. |
| `test_formats_are_saved_and_invalid_ones_dropped` | 395-401 | Regression test: formats are saved and invalid ones dropped. |
| `test_write_uses_the_format_of_the_clicked_value` | 403-407 | Regression test: write uses the format of the clicked value. |

## MasterScaleTests

Protects scaled display, multi-register scaling, the conversion of written engineering values and scale validation.

| Test | Lines | What it protects |
|---|---|---|
| `test_scaled_values_are_shown_in_engineering_units` | 413-417 | Regression test: scaled values are shown in engineering units. |
| `test_scale_applies_to_multi_register_values` | 419-423 | Regression test: scale applies to multi register values. |
| `test_written_engineering_values_are_converted_to_raw` | 425-432 | Regression test: written engineering values are converted to raw. |
| `test_invalid_scales_are_rejected` | 434-440 | Regression test: invalid scales are rejected. |
| `test_scales_are_saved_and_invalid_ones_dropped` | 442-449 | Regression test: scales are saved and invalid ones dropped. |

## MasterOpenFromTrafficTests

Checks which Traffic records can open a Master window and with which definition.

| Test | Lines | What it protects |
|---|---|---|
| `test_request_and_matched_response_give_the_read_definition` | 456-465 | Regression test: request and matched response give the read definition. |

| Test | Lines | What it protects |
|---|---|---|
| `test_frames_without_a_read_definition_are_refused` | 467-474 | Regression test: frames without a read definition are refused. |

## MasterReconnectTests

Simulates an unplugged adapter: the engine reopens the port and resumes, keeps retrying until stopped, or ends without reopen.

| Test | Lines | What it protects |
|---|---|---|
| `test_engine_reopens_the_port_after_a_failure` | 496-514 | Regression test: engine reopens the port after a failure. |
| `test_engine_keeps_retrying_until_stopped` | 516-532 | Regression test: engine keeps retrying until stopped. |
| `test_without_reopen_a_failure_ends_the_session` | 534-542 | Regression test: without reopen a failure ends the session. |

## MasterSettingsTests

Validates the last workspace name stored in settings.json.

| Test | Lines | What it protects |
|---|---|---|
| `test_last_workspace_is_validated` | 546-559 | Regression test: last workspace is validated. |

# 13. Build and launch automation

The package contains three Windows batch files. Their complete source is reproduced in Appendix A.

## MBSniffer.bat

Windows batch automation included in the package.

| Lines | SHA-256 |
| --- | --- |
| 210 | `3c1a036c6ef094c26390c883d8f59aa5e226da5394100edad2e523410fdf5a1d` |

- Sets the expected versioned source folder and verifies that MBSniffer.py exists.
- Detects Python 3 through py -3, python on PATH, or standard Python 3.13 install locations.
- If Python is absent and WinGet exists, attempts a silent per-user Python 3.13 installation.
- Verifies tkinter; if missing, attempts a Python repair/reinstall.
- Verifies pyserial and installs it with pip/ensurepip when required.
- Prefers pythonw.exe so the GUI opens without a console window, with python as fallback.

## MBSniffer v3.2/build_exe.bat

Windows batch automation included in the package.

| Lines | SHA-256 |
| --- | --- |
| 151 | `fbe1caf24e1c48fe6a55f959a61ecc0d0f6a50b376f02297bb6c7d642b8b0775` |

- Normalizes application/distribution paths and finds Python 3.
- Installs/upgrades pyserial and PyInstaller.
- Runs the complete regression suite before building; any failure cancels the build.
- Cleans prior build/spec/output artifacts.
- Creates a one-file, windowed MBSniffer.exe with the icon embedded when available.

## MBSniffer v3.2/run_tests.bat

Windows batch automation included in the package.

| Lines | SHA-256 |
| --- | --- |
| 45 | `8066035cd8d62a7aded1f8314b457d438795a8c59319bf1290c47bc63d34372f` |

- Finds a Python 3 interpreter.
- Runs unittest discovery for tests/test_*.py in verbose mode.
- Returns a non-zero exit code and explicit warning when tests fail, so a release should not proceed.

# 14. Known boundaries and extension points

## What the current v3.2 does not implement

- No Modbus TCP transport.
- Master tab is phase 1 only: no data logging, scaling, conditional colours, charts, Test Center, FC22/23 or OLE-style automation.
- No RTU/TCP Slave Simulator.
- No passive Ethernet packet capture.
- No analog RS485 electrical measurement.

## Current parser scope

Known frame-shape recognition covers the Function Codes encoded in `_candidate_prefixes_at()`. An unsupported FC can only be accepted conservatively as `UNKNOWN / OK` when it occupies the complete remaining burst and has a valid CRC. This intentionally favors resynchronisation reliability over aggressive guessing.

## Natural refactoring seams

- **Transport abstraction:** `mb_protocol.py` is already mostly transport-independent above RTU framing, making it the natural starting point for a future shared Modbus PDU core.
- **Active client engine:** the Finder already contains robust serial request/write/read/timeout primitives that could inform a future general Master engine, but the thesis-quality design should separate those primitives from the Finder UI.
- **Server simulator:** a future Slave Simulator should reuse a common FC encode/decode layer rather than placing protocol behavior directly inside a tab.
- **Testing:** the existing hardware-independent regression suite is a strong base; future transport/client/server work should extend the same pattern.

> **Documentation intent.** These are architectural extension points inferred from the current code structure, not features claimed to exist in v3.2.

# Appendix A - Complete source-code screenshots

Every Python and batch source file in the package is reproduced below as syntax-highlighted screenshots with original line numbers. Very long source lines are visually wrapped inside the screenshot for readability; this does not alter the source file. The SHA-256 values in the module chapters identify the exact files represented.

# MBSniffer v3.2/MBSniffer.py

29 source lines.

**MBSniffer v3.2/MBSniffer.py — lines 1-29**

```
#!/usr/bin/env python3
# MBSniffer v3.2
"""MBSniffer launcher."""

# Simulação:
#   0 = simulação inativa (o botão não aparece no GUI)
#   1 = simulação ativa  (o botão aparece no GUI)
#
# Mantido neste ficheiro para preservar o mesmo ponto de configuração usado
# antes da modularização.
debug_sim = 0

import mb_config

# Apply the launcher switch before importing GUI/capture modules, which read
# the value during import.
mb_config.debug_sim = debug_sim

from mb_config import set_windows_app_user_model_id
from mb_gui import SnifferApp


def main():
    set_windows_app_user_model_id()
    SnifferApp().mainloop()


if __name__ == "__main__":
    main()
```

# MBSniffer v3.2/mb_config.py

486 source lines.

**MBSniffer v3.2/mb_config.py — lines 1-40**

```python
#!/usr/bin/env python3
"""MBSniffer application configuration, constants and filesystem helpers."""

import ctypes
import json
import os
import sys
from pathlib import Path

APP_NAME = "MBSniffer"
APP_VERSION = "3.2"

# Simulação:
#   0 = simulação inativa (o botão não aparece no GUI)
#   1 = simulação ativa  (o botão aparece no GUI)
debug_sim = 0

# Active Modbus RTU slave discovery.
DISCOVERY_BAUD_RATES = (1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200)
DISCOVERY_PARITIES = ("None", "Even", "Odd")
DISCOVERY_STOP_BITS = ("1", "2")
DISCOVERY_DEFAULT_MIN_TIMEOUT_MS = 100
# v3.0 default; settings saved by v3.0 with this value migrate to the new default.
DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS = 40
DISCOVERY_MIN_TIMEOUT_MS = 10
DISCOVERY_MAX_TIMEOUT_MS = 2000
DISCOVERY_PROBE_ADDRESS = 0
DISCOVERY_PROBE_QTY = 1
DISCOVERY_WRITE_TIMEOUT_SECONDS = 1.5
DISCOVERY_WRITE_RETRIES = 1
DISCOVERY_RX_BUFFER_LIMIT = 1024
# Total probe attempts per FC when a Slave does not answer.
DISCOVERY_MIN_ATTEMPTS = 1
DISCOVERY_MAX_ATTEMPTS = 3
DISCOVERY_DEFAULT_ATTEMPTS = 1
# Passive listen before the first transmission; any byte aborts the scan.
DISCOVERY_IDLE_CHECK_SECONDS = 1.0

# GUI queue processing limits.
UI_QUEUE_BATCH_SIZE = 200
```

**MBSniffer v3.2/mb_config.py — lines 41-80**

```
41 UI_QUEUE_IDLE_INTERVAL_MS = 50
42 UI_QUEUE_BUSY_INTERVAL_MS = 10
43
44 # Pending-request timeout.
45 MIN_PENDING_REQUEST_TIMEOUT_SECONDS = 1.0
46 DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS = 10.0
47 MAX_PENDING_REQUEST_TIMEOUT_SECONDS = 300.0
48 PENDING_REQUEST_TIMEOUT_SECONDS = DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS
49
50 # Master (poll) tab. Modbus limits per request; RTU response timeout and the
51 # silent gap between two requests are user settings.
52 MASTER_MAX_READ_BITS = 2000
53 MASTER_MAX_READ_REGISTERS = 125
54 MASTER_MAX_WRITE_BITS = 1968
55 MASTER_MAX_WRITE_REGISTERS = 123
56 MASTER_DEFAULT_RESPONSE_TIMEOUT_MS = 1000
57 MASTER_MIN_RESPONSE_TIMEOUT_MS = 10
58 MASTER_MAX_RESPONSE_TIMEOUT_MS = 10000
59 MASTER_DEFAULT_DELAY_MS = 20
60 MASTER_MIN_DELAY_MS = 0
61 MASTER_MAX_DELAY_MS = 10000
62 MASTER_DEFAULT_SCAN_MS = 1000
63 MASTER_MAX_SCAN_MS = 3600000
64 MASTER_MAX_WINDOWS = 32
65 # When the serial port fails (e.g. USB adapter unplugged), the Master keeps
66 # trying to reopen it at this interval until it succeeds or is disconnected.
67 MASTER_RECONNECT_INTERVAL_SECONDS = 2.0
68 MASTER_WORKSPACE_FORMAT = "MBSniffer Master workspace"
69 MASTER_WORKSPACE_VERSION = 1
70
71 # Sniffer burst assembly: extra wait when a burst ends with the start of a
72 # recognised frame. USB-serial adapters deliver bytes in latency-timer
73 # chunks (FTDI default 16 ms), which can split one RTU frame in two.
74 SNIFFER_SPLIT_FRAME_HOLD_MS = 40.0
75
76 # Diagnostic thresholds.
77 SLOW_RESPONSE_THRESHOLD_MS = 500.0
78
79 # Standard Modbus exception response codes.
80 MODBUS_EXCEPTION_CODES = {
```

**MBSniffer v3.2/mb_config.py — lines 81-120**

```
    0x01: (
        "Illegal Function",
        "A função Modbus pedida não é suportada/permitida pelo dispositivo."
    ),
    0x02: (
        "Illegal Data Address",
        "O endereço ou a gama de endereços pedida não é válida neste dispositivo."
    ),
    0x03: (
        "Illegal Data Value",
        "Um valor, quantidade ou campo do pedido é inválido para esta função."
    ),
    0x04: (
        "Server Device Failure",
        "O dispositivo encontrou um erro interno ao tentar executar o pedido."
    ),
    0x05: (
        "Acknowledge",
        "O pedido foi aceite, mas necessita de mais tempo para ser processado."
    ),
    0x06: (
        "Server Device Busy",
        "O dispositivo está ocupado e não consegue processar o pedido neste momento."
    ),
    0x08: (
        "Memory Parity Error",
        "Foi detetado um erro de paridade ao aceder à memória do dispositivo."
    ),
    0x0A: (
        "Gateway Path Unavailable",
        "O gateway não conseguiu estabelecer o caminho para o dispositivo de destino."
    ),
    0x0B: (
        "Gateway Target Device Failed to Respond",
        "O gateway não recebeu resposta do dispositivo de destino."
    ),
}

# Bounded GUI history. The disk log remains complete.
MAX_UI_FRAMES = 20000
```

### MBSniffer v3.2/mb_config.py — lines 121-160

```
121 MAX_RAW_TEXT_LINES = 20000
122 UI_PRUNE_CHUNK_FRAMES = 2000
123 RAW_TEXT_PRUNE_CHUNK_LINES = 2000
124 RAW_TRANSACTION_SEPARATOR = "—" * 72 + "\n"
125
126 SIMULATION_FUNCTION_CODES = (1, 2, 3, 4)
127 SIMULATION_EXCEPTION_CODES = (1, 2, 3, 4, 6)
128
129 def resource_path(filename):
130     """Return a bundled/source resource path for normal Python or PyInstaller."""
131     try:
132         base = Path(sys._MEIPASS)
133     except (AttributeError, TypeError):
134         base = Path(__file__).resolve().parent
135     return base / filename
136
137 def set_windows_app_user_model_id():
138     """
139     Give Windows a stable application identity so the taskbar can associate
140     the Tk window with MBSniffer instead of a generic Python process.
141     """
142     if os.name != "nt":
143         return
144     try:
145         ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID(
146             f"{APP_NAME}.{APP_VERSION}"
147         )
148     except Exception:
149         pass
150
151
152
153 def get_settings_file() -> Path:
154     """
155     Return the persistent per-user settings file.
156
157     Windows:
158         %APPDATA%\\MBSniffer\\settings.json
159
160     Fallback:
```

### MBSniffer v3.2/mb_config.py — lines 161-200

```
161         ~/.mbsniffer/settings.json
162
163     This location is independent from the EXE/BAT folder, so the selected
164     Light/Dark mode survives application updates and also works when the
165     application folder is read-only.
166     """
167     try:
168         if os.name == "nt":
169             roaming_app_data = os.environ.get("APPDATA")
170             if roaming_app_data:
171                 folder = Path(roaming_app_data) / APP_NAME
172             else:
173                 folder = Path.home() / "AppData" / "Roaming" / APP_NAME
174         else:
175             folder = Path.home() / ".mbsniffer"
176
177         folder.mkdir(parents=True, exist_ok=True)
178         return folder / "settings.json"
179     except Exception:
180         return Path.home() / ".mbsniffer_settings.json"
181
182
183 def _validated_choice(value, allowed, default):
184     return value if value in allowed else default
185
186
187 def _validated_bool(value, default):
188     return value if isinstance(value, bool) else default
189
190
191 def _validated_number_string(value, minimum, maximum, default):
192     try:
193         number = float(value)
194         if minimum <= number <= maximum:
195             return f"{number:g}"
196     except (TypeError, ValueError):
197         pass
198     return str(default)
199
200
```

### MBSniffer v3.2/mb_config.py — lines 201-240

```
201 def _validated_int_string(value, minimum, maximum, default):
202     try:
203         number = int(value)
204         if minimum <= number <= maximum:
205             return str(number)
206     except (TypeError, ValueError):
207         pass
208     return str(default)
209 
210 
211 def load_ui_settings() -> dict:
212     """
213     Load validated user preferences from the single Roaming settings file.
214 
215     Machine-specific state such as COM-port names and window geometry is
216     deliberately excluded.
217     """
218     path = get_settings_file()
219     try:
220         data = json.loads(path.read_text(encoding="utf-8"))
221     except Exception:
222         return {}
223 
224     if not isinstance(data, dict):
225         return {}
226 
227     result = {}
228 
229     if isinstance(data.get("dark_mode"), bool):
230         result["dark_mode"] = data["dark_mode"]
231 
232     if data.get("language") in ("Português", "English"):
233         result["language"] = data["language"]
234 
235     if data.get("last_main_tab") in ("Sniffer", "Bus Slave Finder", "Master"):
236         result["last_main_tab"] = data["last_main_tab"]
237 
238     sniffer_in = data.get("sniffer")
239     if isinstance(sniffer_in, dict):
240         # v2.9 migration: the UI option was renamed from "Realçar anomalias"
```

**MBSniffer v3.2/mb_config.py — lines 241-280**

```
        # to "Realçar resultados". Accept the old key once so existing user
        # preferences are not lost, but expose/save only the new name.
        highlight_results_value = sniffer_in.get(
            "highlight_results",
            sniffer_in.get("highlight_anomalies"),
        )

        result["sniffer"] = {
            "physical_mode": _validated_choice(
                sniffer_in.get("physical_mode"),
                ("RS485 2-wire", "RS232 single RX", "RS232 dual RX"),
                "RS485 2-wire",
            ),
            "baud": _validated_choice(
                str(sniffer_in.get("baud", "")),
                tuple(str(v) for v in DISCOVERY_BAUD_RATES),
                "9600",
            ),
            "data_bits": _validated_choice(
                str(sniffer_in.get("data_bits", "")),
                ("5", "6", "7", "8"),
                "8",
            ),
            "parity": _validated_choice(
                sniffer_in.get("parity"),
                ("None", "Even", "Odd", "Mark", "Space"),
                "None",
            ),
            "stop_bits": _validated_choice(
                str(sniffer_in.get("stop_bits", "")),
                ("1", "1.5", "2"),
                "1",
            ),
            "frame_gap_mode": _validated_choice(
                sniffer_in.get("frame_gap_mode"),
                ("Auto", "Manual"),
                "Auto",
            ),
            "frame_gap_ms": str(sniffer_in.get("frame_gap_ms", "")),
            "pending_timeout_s": _validated_number_string(
```

### MBSniffer v3.2/mb_config.py — lines 281-320

```
                sniffer_in.get("pending_timeout_s"),
                MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
                MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
                DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS,
            ),
            "auto_scroll": _validated_bool(
                sniffer_in.get("auto_scroll"),
                True,
            ),
            "separate_transactions": _validated_bool(
                sniffer_in.get("separate_transactions"),
                True,
            ),
            "highlight_results": _validated_bool(
                highlight_results_value,
                False,
            ),
        }

    finder_in = data.get("bus_slave_finder")
    if isinstance(finder_in, dict):
        saved_bauds = finder_in.get("baud_rates")
        if isinstance(saved_bauds, list):
            baud_rates = [
                int(v) for v in saved_bauds
                if isinstance(v, (int, float, str))
                and str(v).isdigit()
                and int(v) in DISCOVERY_BAUD_RATES
            ]
        else:
            baud_rates = [9600, 19200]

        saved_parities = finder_in.get("parities")
        if isinstance(saved_parities, list):
            parities = [
                v for v in saved_parities
                if v in DISCOVERY_PARITIES
            ]
        else:
            parities = ["None", "Even"]
```

**MBSniffer v3.2/mb_config.py — lines 321-360**

```
321
322         saved_stops = finder_in.get("stop_bits")
323         if isinstance(saved_stops, list):
324             stop_bits = [
325                 str(v) for v in saved_stops
326                 if str(v) in DISCOVERY_STOP_BITS
327             ]
328         else:
329             stop_bits = ["1"]
330
331         min_timeout_ms = _validated_int_string(
332             finder_in.get("min_timeout_ms"),
333             DISCOVERY_MIN_TIMEOUT_MS,
334             DISCOVERY_MAX_TIMEOUT_MS,
335             DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
336         )
337         # Settings written before v3.1 have no "attempts" key. Their 40 ms
338         # value is the old automatic default, not a user choice.
339         if (
340             "attempts" not in finder_in
341             and min_timeout_ms == str(DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS)
342         ):
343             min_timeout_ms = str(DISCOVERY_DEFAULT_MIN_TIMEOUT_MS)
344
345         result["bus_slave_finder"] = {
346             "baud_rates": baud_rates,
347             "parities": parities,
348             "stop_bits": stop_bits,
349             "slave_start": _validated_int_string(
350                 finder_in.get("slave_start"),
351                 1,
352                 247,
353                 1,
354             ),
355             "slave_end": _validated_int_string(
356                 finder_in.get("slave_end"),
357                 1,
358                 247,
359                 247,
360             ),
```

### MBSniffer v3.2/mb_config.py — lines 361-400

```
            "min_timeout_ms": min_timeout_ms,
            "attempts": _validated_int_string(
                finder_in.get("attempts"),
                DISCOVERY_MIN_ATTEMPTS,
                DISCOVERY_MAX_ATTEMPTS,
                DISCOVERY_DEFAULT_ATTEMPTS,
            ),
            "fc04_fallback": _validated_bool(
                finder_in.get("fc04_fallback"),
                False,
            ),
            "device_identification": _validated_bool(
                finder_in.get("device_identification"),
                False,
            ),
        }

    master_in = data.get("master")
    if isinstance(master_in, dict):
        result["master"] = {
            "baud": _validated_choice(
                str(master_in.get("baud", "")),
                tuple(str(v) for v in DISCOVERY_BAUD_RATES),
                "9600",
            ),
            "parity": _validated_choice(
                master_in.get("parity"), ("None", "Even", "Odd"), "None"
            ),
            "stop_bits": _validated_choice(
                str(master_in.get("stop_bits", "")), ("1", "2"), "1"
            ),
            "response_timeout_ms": _validated_int_string(
                master_in.get("response_timeout_ms"),
                MASTER_MIN_RESPONSE_TIMEOUT_MS,
                MASTER_MAX_RESPONSE_TIMEOUT_MS,
                MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
            ),
            "delay_ms": _validated_int_string(
                master_in.get("delay_ms"),
                MASTER_MIN_DELAY_MS,
```

### MBSniffer v3.2/mb_config.py — lines 401-440

```
                MASTER_MAX_DELAY_MS,
                MASTER_DEFAULT_DELAY_MS,
            ),
            # Workspace reopened at start-up (name of a file in Workspaces).
            "last_workspace": (
                master_in["last_workspace"].strip()
                if isinstance(master_in.get("last_workspace"), str)
                and 0 < len(master_in["last_workspace"].strip()) <= 60
                and not any(ch in '<>:"/\\|?*' for ch in master_in["last_workspace"])
                else ""
            ),
        }

    return result


def save_ui_settings(settings=None, **updates) -> None:
    """
    Persist UI preferences atomically in the single Roaming settings file.

    Partial updates merge with the existing file. This lets the Dark-mode
    toggle update immediately without discarding the other remembered options.
    """
    path = get_settings_file()

    current = load_ui_settings()
    if isinstance(settings, dict):
        current.update(settings)
    current.update(updates)

    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        tmp = path.with_suffix(path.suffix + ".tmp")
        tmp.write_text(
            json.dumps(current, indent=2, sort_keys=True),
            encoding="utf-8",
        )
        tmp.replace(path)
    except Exception:
        # Remembered UI state must never break application use.
```

### MBSniffer v3.2/mb_config.py — lines 441-480

```
441         pass
442 
443 
444 def get_application_directory() -> Path:
445     """
446     Return the user-facing application directory.
447 
448     - PyInstaller EXE: directory containing MBSniffer.exe.
449     - Packaged Python/BAT mode: directory containing the root MBSniffer.bat.
450       The Python sources live inside the versioned child folder, e.g. ``MBSniffer v3.2``.
451     - Development Python mode: directory containing this module.
452     """
453     try:
454         if getattr(sys, "frozen", False):
455             return Path(sys.executable).resolve().parent
456 
457         source_dir = Path(__file__).resolve().parent
458         launcher_dir = source_dir.parent
459 
460         if (launcher_dir / "MBSniffer.bat").is_file():
461             return launcher_dir
462 
463         return source_dir
464     except Exception:
465         return Path.cwd()
466 
467 def get_workspaces_folder() -> Path:
468     """
469     Return/create the Master workspace folder beside settings.json:
470 
471         %APPDATA%\\MBSniffer\\Workspaces
472     """
473     folder = get_settings_file().parent / "Workspaces"
474     folder.mkdir(parents=True, exist_ok=True)
475     return folder
476 
477 
478 def get_logs_folder() -> Path:
479     """
480     Return/create the log directory beside MBSniffer.bat or MBSniffer.exe:
```

**MBSniffer v3.2/mb_config.py — lines 481-486**

```
481
482         <launcher/exe directory>/MBSniffer Logs
483     """
484     folder = get_application_directory() / "MBSniffer Logs"
485     folder.mkdir(parents=True, exist_ok=True)
486     return folder
```

# MBSniffer v3.2/mb_protocol.py

1007 source lines.

**MBSniffer v3.2/mb_protocol.py — lines 1-40**

```python
#!/usr/bin/env python3
"""Modbus RTU parsing, CRC, transaction metrics and simulation helpers."""

import math
import random
import time
from bisect import bisect_right
from collections import defaultdict, deque
from datetime import datetime

from mb_config import (
    MODBUS_EXCEPTION_CODES,
    PENDING_REQUEST_TIMEOUT_SECONDS,
    SIMULATION_FUNCTION_CODES,
    SIMULATION_EXCEPTION_CODES,
    SLOW_RESPONSE_THRESHOLD_MS,
)

def _build_crc16_table():
    table = []
    for value in range(256):
        crc = value
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
        table.append(crc)
    return tuple(table)

# Table-driven CRC: one lookup per byte instead of eight shift/XOR steps. The
# resynchronisation scan computes CRCs at many offsets, so this matters on
# noisy buses.
_CRC16_TABLE = _build_crc16_table()

def crc16_modbus(data: bytes) -> int:
    crc = 0xFFFF
    table = _CRC16_TABLE
    for b in data:
        crc = (crc >> 8) ^ table[(crc ^ b) & 0xFF]
```

### MBSniffer v3.2/mb_protocol.py — lines 41-80

```
41      return crc
42
43  def crc_ok(frame: bytes) -> bool:
44      if len(frame) < 4:
45          return False
46      expected = crc16_modbus(frame[:-2])
47      received = frame[-2] | (frame[-1] << 8)
48      return expected == received
49
50  def append_crc(payload: bytes) -> bytes:
51      crc = crc16_modbus(payload)
52      return payload + bytes((crc & 0xFF, (crc >> 8) & 0xFF))
53
54  def hex_bytes(data: bytes) -> str:
55      return " ".join(f"{b:02X}" for b in data)
56
57  def modbus_rtu_interframe_gap_seconds(baud: int, bits_per_char: float) -> float:
58      """Return the recommended Modbus RTU t3.5 inter-frame gap.
59
60      Modbus Serial Line V1.02 requires the character-time timers to be
61      respected at baud rates up to and including 19200 bit/s. Above 19200
62      bit/s it recommends a fixed t3.5 value of 1.750 ms.
63      """
64      baud = int(baud)
65      bits_per_char = float(bits_per_char)
66      if baud <= 0:
67          raise ValueError("Baud rate must be greater than zero")
68      if bits_per_char <= 0:
69          raise ValueError("Bits per character must be greater than zero")
70      if baud > 19200:
71          return 0.001750
72      return 3.5 * bits_per_char / float(baud)
73
74  def plausible_request_qty(fc: int, qty: int) -> bool:
75      if fc in (1, 2):
76          return 1 <= qty <= 2000
77      if fc in (3, 4):
78          return 1 <= qty <= 125
79      return True
80
```

### MBSniffer v3.2/mb_protocol.py — lines 81-120

```python
# Function Codes whose RTU frame length the parser can derive.
KNOWN_FRAME_FUNCTION_CODES = (1, 2, 3, 4, 5, 6, 7, 8, 15, 16, 22, 23)

# Function Codes whose normal response is an exact copy of the request.
EXACT_ECHO_FUNCTION_CODES = (5, 6, 22)

BROADCAST_UNIT_ID = 0

def _plausible_unit_id(value: int) -> bool:
    """Return True for Modbus RTU unit addresses that can appear on the wire."""
    try:
        return 0 <= int(value) <= 247
    except (TypeError, ValueError):
        return False


def _candidate_prefixes_at(buf, start=0):
    """Return structurally plausible known-frame lengths at *start*."""
    remaining = len(buf) - int(start)
    if remaining < 2:
        return []

    unit = buf[start]
    if not _plausible_unit_id(unit):
        return []

    fc = buf[start + 1]
    out = []

    if fc & 0x80:
        # Exception responses use FC | 0x80 and have a fixed five-byte length.
        base_fc = fc & 0x7F
        if base_fc in KNOWN_FRAME_FUNCTION_CODES:
            out.append((5, "EXCEPTION"))
        return out

    if fc in (1, 2, 3, 4):
        if remaining >= 6:
            qty = (buf[start + 4] << 8) | buf[start + 5]
            if plausible_request_qty(fc, qty):
```

### MBSniffer v3.2/mb_protocol.py — lines 121-160

```
121                 out.append((8, "REQUEST"))
122
123         if remaining >= 3:
124             byte_count = buf[start + 2]
125             resp_len = 5 + byte_count
126             if fc in (3, 4):
127                 if byte_count > 0 and byte_count % 2 == 0:
128                     out.append((resp_len, "RESPONSE"))
129             elif byte_count > 0:
130                 out.append((resp_len, "RESPONSE"))
131
132     elif fc in (5, 6, 8):
133         out.append((8, "REQ/RESP"))
134
135     elif fc == 7:
136         # FC07 request: Slave + FC + CRC = 4 bytes.
137         # FC07 response: Slave + FC + Status + CRC = 5 bytes.
138         if remaining >= 5:
139             out.append((5, "RESPONSE"))
140         out.append((4, "REQUEST"))
141
142     elif fc in (15, 16):
143         out.append((8, "RESPONSE"))
144         if remaining >= 7:
145             byte_count = buf[start + 6]
146             if byte_count > 0:
147                 out.append((9 + byte_count, "REQUEST"))
148
149     elif fc == 22:
150         out.append((10, "REQ/RESP"))
151
152     elif fc == 23:
153         if remaining >= 3:
154             bc = buf[start + 2]
155             if bc > 0 and bc % 2 == 0:
156                 out.append((5 + bc, "RESPONSE"))
157         if remaining >= 11:
158             bc = buf[start + 10]
159             if bc > 0:
160                 out.append((13 + bc, "REQUEST"))
```

### MBSniffer v3.2/mb_protocol.py — lines 161-200

```
161
162     cleaned = []
163     seen = set()
164     for length, kind in out:
165         if 4 <= length <= 260 and (length, kind) not in seen:
166             seen.add((length, kind))
167             cleaned.append((length, kind))
168     return cleaned
169
170
171 def candidate_prefixes(buf: bytes):
172     """Compatibility wrapper for callers/tests that inspect offset zero."""
173     return _candidate_prefixes_at(buf, 0)
174
175
176 def _crc_ok_at(buf, start: int, length: int) -> bool:
177     """CRC check without copying the whole remaining capture buffer."""
178     if length < 4 or start < 0 or start + length > len(buf):
179         return False
180     end = start + length
181     expected = crc16_modbus(memoryview(buf)[start:end - 2])
182     received = buf[end - 2] | (buf[end - 1] << 8)
183     return expected == received
184
185
186 def find_valid_prefix(buf: bytes, start=0):
187     """Find a CRC-valid, structurally recognised Modbus frame at *start*."""
188     for length, kind in _candidate_prefixes_at(buf, start):
189         if start + length <= len(buf) and _crc_ok_at(buf, start, length):
190             return length, kind
191     return None
192
193
194 def guess_invalid_whole_frame(buf: bytes):
195     """Classify a complete known-shape frame whose CRC is invalid."""
196     if len(buf) < 4 or not _plausible_unit_id(buf[0]):
197         return None
198     for length, kind in _candidate_prefixes_at(buf, 0):
199         if len(buf) == length:
200             return kind
```

**MBSniffer v3.2/mb_protocol.py — lines 201-240**

```
201     return None
202 
203 
204 def _unknown_whole_frame_is_valid(buf: bytes) -> bool:
205     """Conservative fallback for one unsupported FC occupying the whole burst.
206 
207     Unknown frames cannot be length-validated from their FC, so CRC-only
208     recognition is deliberately restricted to the complete remaining burst.
209     It is never used while scanning arbitrary offsets for resynchronisation.
210     """
211     if not 4 <= len(buf) <= 260:
212         return False
213     if not _plausible_unit_id(buf[0]):
214         return False
215     if _candidate_prefixes_at(buf, 0):
216         return False
217     return crc_ok(buf)
218 
219 
220 def split_capture_buffer(data: bytes):
221     results = []
222     buf = bytearray(data)
223 
224     while buf:
225         match = find_valid_prefix(buf, 0)
226         if match:
227             length, kind = match
228             results.append((bytes(buf[:length]), kind, "OK"))
229             del buf[:length]
230             continue
231 
232         # Search the full remaining burst, but only for structurally recognised
233         # Modbus frames. The former generic CRC scan tried up to 256 candidate
234         # lengths at every offset and became very slow on long noise bursts.
235         # Restricting resynchronisation to known frame shapes makes the scan
236         # effectively linear in the burst size and strongly reduces accidental
237         # CRC-only "phantom" frames in random noise.
238         found_offset = None
239         search_limit = max(len(buf) - 4, 0)
240         for offset in range(1, search_limit + 1):
```

### MBSniffer v3.2/mb_protocol.py — lines 241-280

```
241             if find_valid_prefix(buf, offset):
242                 found_offset = offset
243                 break
244 
245         if found_offset is not None:
246             prefix = bytes(buf[:found_offset])
247 
248             # If the bytes immediately before the recovered frame are exactly a
249             # recognised Modbus frame shape, preserve the useful CRC diagnosis
250             # instead of hiding it as RAW/UNSYNC. Otherwise they are genuinely
251             # unsynchronised bytes and remain RAW.
252             guessed = guess_invalid_whole_frame(prefix)
253             if guessed:
254                 results.append((prefix, guessed, "ERROR"))
255             else:
256                 results.append((prefix, "RAW/UNSYNC", "UNKNOWN"))
257             del buf[:found_offset]
258             continue
259 
260         guessed = guess_invalid_whole_frame(bytes(buf))
261         if guessed:
262             results.append((bytes(buf), guessed, "ERROR"))
263         elif _unknown_whole_frame_is_valid(bytes(buf)):
264             results.append((bytes(buf), "UNKNOWN", "OK"))
265         else:
266             results.append((bytes(buf), "RAW/UNPARSED", "UNKNOWN"))
267         break
268 
269     return results
270 
271 def burst_may_continue(data: bytes) -> bool:
272     """Return True when *data* ends with the start of an unfinished frame.
273 
274     USB-serial adapters deliver received bytes in chunks, so a silence longer
275     than t3.5 can occur in the middle of one RTU frame. The reader uses this
276     to wait a little longer before closing such a burst. Only the final
277     unvalidated segment is inspected, and only at its first byte.
278     """
279     segments = split_capture_buffer(data)
280     if not segments:
```

**MBSniffer v3.2/mb_protocol.py — lines 281-320**

```
281         return False
282
283     tail, _kind, state = segments[-1]
284     if state == "OK":
285         return False
286
287     if not _plausible_unit_id(tail[0]):
288         return False
289     if len(tail) == 1:
290         return True
291
292     fc = tail[1]
293     if (fc & 0x7F) not in KNOWN_FRAME_FUNCTION_CODES:
294         return False
295     if len(tail) < 8 and not fc & 0x80:
296         # Too short for the request/response length fields to be known.
297         return True
298
299     return any(
300         length > len(tail)
301         for length, _kind in _candidate_prefixes_at(tail, 0)
302     )
303
304
305 def read_bits_from_response(frame: bytes, qty=None):
306     """Return FC01/FC02 coil/input states, least-significant bit first.
307
308     Without the matching request quantity the padding bits of the last data
309     byte cannot be distinguished, so every transmitted bit is returned.
310     """
311     if len(frame) < 5:
312         return []
313     byte_count = frame[2]
314     data = frame[3:3 + byte_count]
315     bits = [(byte >> bit) & 1 for byte in data for bit in range(8)]
316     try:
317         qty = int(qty) if qty is not None else None
318     except (TypeError, ValueError):
319         qty = None
320     if qty is not None and 0 < qty <= len(bits):
```

### MBSniffer v3.2/mb_protocol.py — lines 321-360

```
321         bits = bits[:qty]
322     return bits
323 
324 
325 # FC01/FC02 can return up to 2000 bits; the Traffic row shows only the first
326 # ones (the Frame Inspector lists them all with their addresses).
327 READ_BITS_DETAILS_LIMIT = 32
328 
329 
330 def read_bits_details(frame: bytes, qty=None) -> str:
331     """Compact Details text: bits in address order, grouped per data byte."""
332     byte_count = frame[2] if len(frame) >= 3 else 0
333     bits = read_bits_from_response(frame, qty)
334     shown = "".join(str(bit) for bit in bits[:READ_BITS_DETAILS_LIMIT])
335     grouped = " ".join(shown[i:i + 8] for i in range(0, len(shown), 8))
336     if len(bits) > READ_BITS_DETAILS_LIMIT:
337         grouped += f" …(+{len(bits) - READ_BITS_DETAILS_LIMIT})"
338     return f"ByteCount={byte_count}  Bits={grouped}"
339 
340 
341 def decode_frame(frame: bytes, kind_hint: str):
342     info = {
343         "kind": kind_hint,
344         "slave": "",
345         "fc": "",
346         "details": "",
347         "pdu_address": None,
348         "one_based": None,
349         "qty": None,
350     }
351 
352     if str(kind_hint).startswith("RAW"):
353         info["details"] = f"{len(frame)} raw bytes"
354         return info
355 
356     if len(frame) < 2:
357         info["details"] = "Raw bytes"
358         return info
359 
360     slave = frame[0]
```

**MBSniffer v3.2/mb_protocol.py — lines 361-400**

```
    fc = frame[1]
    info["slave"] = str(slave)
    info["fc"] = f"{fc:02X}"

    if fc & 0x80:
        info["kind"] = "EXCEPTION"
        code = frame[2] if len(frame) >= 3 else None

        if code is None:
            info["details"] = "Exception"
            return info

        exception_info = MODBUS_EXCEPTION_CODES.get(code)
        if exception_info is not None:
            name, _description = exception_info
            info["details"] = f"Exception 0x{code:02X} — {name}"
        else:
            info["details"] = f"Exception 0x{code:02X} — Unknown/Reserved Exception"
        return info

    if fc in (1, 2, 3, 4):
        if kind_hint == "REQUEST" and len(frame) >= 8:
            start_addr = (frame[2] << 8) | frame[3]
            qty = (frame[4] << 8) | frame[5]
            info["pdu_address"] = start_addr
            info["one_based"] = start_addr + 1
            info["qty"] = qty
            info["details"] = (
                f"PDU Address={start_addr}  1-based={start_addr + 1}  Qty={qty}"
            )
            return info

        if kind_hint == "RESPONSE" and len(frame) >= 5:
            bc = frame[2]
            details = f"ByteCount={bc}"
            if fc in (3, 4) and bc % 2 == 0 and len(frame) >= 5 + bc:
                data = frame[3:3+bc]
                regs = [(data[i] << 8) | data[i+1] for i in range(0, len(data), 2)]
                details += f"  Registos={regs}"
            elif fc in (1, 2) and len(frame) >= 5 + bc:
```

### MBSniffer v3.2/mb_protocol.py — lines 401-440

```
401                 details = read_bits_details(frame)
402             info["details"] = details
403             return info
404
405     if fc in (5, 6, 8) and len(frame) >= 8:
406         addr = (frame[2] << 8) | frame[3]
407         value = (frame[4] << 8) | frame[5]
408         info["pdu_address"] = addr
409         info["one_based"] = addr + 1
410         info["details"] = (
411             f"PDU Address={addr}  1-based={addr + 1}  Value=0x{value:04X}"
412         )
413         return info
414
415     if fc == 7:
416         if kind_hint == "REQUEST":
417             info["details"] = "Read Exception Status"
418         elif kind_hint == "RESPONSE" and len(frame) >= 5:
419             info["details"] = f"Status=0x{frame[2]:02X}"
420         else:
421             info["details"] = "Read Exception Status"
422         return info
423
424     if fc in (15, 16):
425         if kind_hint == "REQUEST" and len(frame) >= 9:
426             start_addr = (frame[2] << 8) | frame[3]
427             qty = (frame[4] << 8) | frame[5]
428             bc = frame[6]
429             info["pdu_address"] = start_addr
430             info["one_based"] = start_addr + 1
431             info["qty"] = qty
432             info["details"] = (
433                 f"PDU Address={start_addr}  1-based={start_addr + 1}  "
434                 f"Qty={qty}  ByteCount={bc}"
435             )
436             return info
437         if len(frame) >= 8:
438             start_addr = (frame[2] << 8) | frame[3]
439             qty = (frame[4] << 8) | frame[5]
440             info["pdu_address"] = start_addr
```

**MBSniffer v3.2/mb_protocol.py — lines 441-480**

```
            info["one_based"] = start_addr + 1
            info["qty"] = qty
            info["details"] = (
                f"PDU Address={start_addr}  1-based={start_addr + 1}  Qty={qty}"
            )
            return info

    info["details"] = f"{len(frame)} bytes"
    return info

def format_clock_time(epoch: float) -> str:
    return datetime.fromtimestamp(float(epoch)).strftime("%H:%M:%S.%f")[:-3]


def parse_burst(data: bytes, timestamp: str, channel: str = "", event_time=None,
                chunk_times=None):
    """Split one received burst into decoded rows.

    *chunk_times* is an optional ascending list of ``(byte_offset, epoch)``
    pairs recording when each serial read arrived. When given, every frame
    gets the arrival time of the chunk holding its first byte instead of the
    burst start time, so frames merged into one burst keep distinct times.
    """
    rows = []
    if event_time is None:
        event_time = time.time()

    chunk_offsets = [int(o) for o, _t in chunk_times] if chunk_times else []
    offset = 0

    for frame, kind, state in split_capture_buffer(data):
        frame_time = float(event_time)
        frame_timestamp = timestamp
        if chunk_offsets:
            index = bisect_right(chunk_offsets, offset) - 1
            if index > 0:
                frame_time = float(chunk_times[index][1])
                frame_timestamp = format_clock_time(frame_time)
        offset += len(frame)

```

### MBSniffer v3.2/mb_protocol.py — lines 481-520

```
481         info = decode_frame(frame, kind)
482         crc_status = "OK" if state == "OK" else ("ERROR" if state == "ERROR" else "?")
483         rows.append({
484             "time": frame_timestamp,
485             "channel": channel,
486             "type": info["kind"],
487             "slave": info["slave"],
488             "fc": info["fc"],
489             "details": info["details"],
490             "crc": crc_status,
491             "raw": hex_bytes(frame),
492             "pdu_address": info["pdu_address"],
493             "one_based": info["one_based"],
494             "qty": info["qty"],
495             "_frame_len": len(frame),
496             "_epoch": frame_time,
497         })
498     return rows
499 
500 class SessionMetrics:
501     """Tracks pairing, counters and bus-health diagnostics for one session."""
502 
503     def __init__(self, pending_timeout_seconds=PENDING_REQUEST_TIMEOUT_SECONDS):
504         self.pending_timeout_seconds = float(pending_timeout_seconds)
505         self.reset()
506 
507     def reset(self):
508         self.first_event_time = None
509         self.last_event_time = None
510         self.pending = {}
511         self.requests = 0
512         # Broadcast requests (Slave 0) never receive a response, so they are
513         # never pending and are excluded from the timeout-rate denominator.
514         self.broadcast_requests = 0
515         self.responses = 0
516         self.timeouts = 0
517         self.crc_errors = 0
518         self.exceptions = 0
519         self.raw = 0
520 
```

**MBSniffer v3.2/mb_protocol.py — lines 521-560**

```
        # Bus Health.
        self.total_bytes = 0
        self.nonvalidated_bytes = 0
        self.active_slaves = set()
        self.exception_by_slave = defaultdict(int)
        self.response_times_ms = deque(maxlen=10000)
        self.response_times_by_slave = defaultdict(lambda: deque(maxlen=2000))

        # Timeout events are exposed to the view so the original request row can
        # be marked/highlighted without inventing a fake serial frame.
        self._expired_requests = []

    def _key(self, row):
        try:
            slave = int(row.get("slave", ""))
            fc = int(row.get("fc", ""), 16)
        except (ValueError, TypeError):
            return None

        if row.get("type") == "EXCEPTION":
            fc &= 0x7F

        return (slave, fc)

    def pending_count(self):
        return sum(len(v) for v in self.pending.values())

    def pop_expired_requests(self):
        items = self._expired_requests
        self._expired_requests = []
        return items

    def expire_pending(self, now=None):
        """
        Expire stale requests and retain their sequence IDs for the GUI.

        Returns the number of newly expired requests, preserving the original
        public behavior used by CaptureMixin.
        """
        if now is None:
```

**MBSniffer v3.2/mb_protocol.py — lines 561-600**

```
561             now = time.time()
562         now = float(now)
563 
564         expired = 0
565         cutoff = now - self.pending_timeout_seconds
566 
567         for key in list(self.pending.keys()):
568             pending_list = self.pending[key]
569 
570             keep_from = 0
571             for item in pending_list:
572                 if item["time"] <= cutoff:
573                     expired += 1
574                     keep_from += 1
575                     event = dict(item)
576                     event["slave"] = key[0]
577                     event["fc"] = key[1]
578                     self._expired_requests.append(event)
579                 else:
580                     break
581 
582             if keep_from:
583                 del pending_list[:keep_from]
584 
585             if not pending_list:
586                 self.pending.pop(key, None)
587 
588         if expired:
589             self.timeouts += expired
590 
591         return expired
592 
593     def force_timeout(self, slave, fc):
594         """
595         Debug/simulation helper: expire the oldest pending request for one key.
596 
597         It uses the same pending bookkeeping and returns the expired request
598         descriptor so the GUI marks the real REQUEST row as timed out.
599         """
600         try:
```

**MBSniffer v3.2/mb_protocol.py — lines 601-640**

```
601             key = (int(slave), int(fc))
602         except (TypeError, ValueError):
603             return []
604 
605         pending_list = self.pending.get(key, [])
606         if not pending_list:
607             return []
608 
609         item = pending_list.pop(0)
610         if not pending_list:
611             self.pending.pop(key, None)
612 
613         event = dict(item)
614         event["slave"] = key[0]
615         event["fc"] = key[1]
616         self.timeouts += 1
617         return [event]
618 
619     def _resolve_echo_direction(self, row):
620         """Classify a REQ/RESP frame (FC05/06/08/22) as REQUEST or RESPONSE.
621 
622         These requests and their normal responses have the same shape, so the
623         parser cannot tell them apart. A frame is the RESPONSE when an older
624         pending request with the same (Slave, FC) exists; for exact-echo FCs
625         the bytes must also be identical. In RS232 dual RX a response must
626         arrive on the opposite channel to its request.
627         """
628         if row.get("crc") != "OK":
629             return row.get("type", "")
630 
631         key = self._key(row)
632         if key is None:
633             return row.get("type", "")
634         if key[0] == BROADCAST_UNIT_ID:
635             return "REQUEST"
636 
637         pending_list = self.pending.get(key)
638         if pending_list:
639             candidate = pending_list[0]
640             channel = row.get("channel", "")
```

### MBSniffer v3.2/mb_protocol.py — lines 641-680

```
641             same_direction = (
642                 channel in ("A→B", "B→A")
643                 and candidate.get("channel") == channel
644             )
645             same_bytes = candidate.get("raw") == row.get("raw")
646             if not same_direction and (
647                 key[1] not in EXACT_ECHO_FUNCTION_CODES or same_bytes
648             ):
649                 return "RESPONSE"
650 
651         return "REQUEST"
652 
653     def _record_response_time(self, row, response_ms):
654         if response_ms is None or row.get("crc") != "OK":
655             return
656 
657         response_ms = float(response_ms)
658         self.response_times_ms.append(response_ms)
659 
660         try:
661             slave = int(row.get("slave", ""))
662         except (TypeError, ValueError):
663             return
664         self.response_times_by_slave[slave].append(response_ms)
665 
666     def process(self, row):
667         event_time = float(row.get("_epoch", time.time()))
668 
669         # Expire stale requests BEFORE pairing the current frame.
670         self.expire_pending(event_time)
671         expired_requests = self.pop_expired_requests()
672 
673         if self.first_event_time is None:
674             self.first_event_time = event_time
675 
676         delta_ms = None
677         if self.last_event_time is not None and event_time >= self.last_event_time:
678             delta_ms = (event_time - self.last_event_time) * 1000.0
679         if self.last_event_time is None or event_time >= self.last_event_time:
680             self.last_event_time = event_time
```

**MBSniffer v3.2/mb_protocol.py — lines 681-720**

```
681
682         if row.get("type") == "REQ/RESP":
683             row["type"] = self._resolve_echo_direction(row)
684
685         row_type = row.get("type", "")
686         crc = row.get("crc", "")
687
688         try:
689             frame_len = max(0, int(row.get("_frame_len", 0)))
690         except (TypeError, ValueError):
691             frame_len = 0
692         self.total_bytes += frame_len
693
694         try:
695             slave_num = int(row.get("slave", ""))
696             if 1 <= slave_num <= 247 and not str(row_type).startswith("RAW"):
697                 self.active_slaves.add(slave_num)
698         except (TypeError, ValueError):
699             slave_num = None
700
701         if crc == "ERROR":
702             self.crc_errors += 1
703
704         # RAW bytes cannot honestly be called CRC errors: there may be no
705         # complete frame on which to validate a CRC. For Bus Health we track
706         # them together with explicit CRC failures as traffic that could not be
707         # validated as a correct Modbus RTU frame.
708         if crc == "ERROR" or str(row_type).startswith("RAW"):
709             self.nonvalidated_bytes += frame_len
710
711         if row_type == "REQUEST":
712             self.requests += 1
713             key = self._key(row)
714             if key is not None and key[0] == BROADCAST_UNIT_ID:
715                 self.broadcast_requests += 1
716             elif key is not None and crc == "OK":
717                 self.pending.setdefault(key, []).append({
718                     "time": event_time,
719                     "seq": row.get("_view_seq"),
720                     "pdu_address": row.get("pdu_address"),
```

**MBSniffer v3.2/mb_protocol.py — lines 721-760**

```
                    "one_based": row.get("one_based"),
                    "qty": row.get("qty"),
                    "channel": row.get("channel", ""),
                    "raw": row.get("raw", ""),
                })

        elif row_type == "RESPONSE":
            self.responses += 1

        elif row_type == "EXCEPTION":
            self.responses += 1
            self.exceptions += 1
            if slave_num is not None:
                self.exception_by_slave[slave_num] += 1

        elif str(row_type).startswith("RAW"):
            self.raw += 1

        response_ms = None
        matched_request = None

        if row_type in ("RESPONSE", "EXCEPTION"):
            key = self._key(row)
            if key is not None:
                pending_list = self.pending.get(key, [])
                if pending_list:
                    matched_request = pending_list.pop(0)
                    if not pending_list:
                        self.pending.pop(key, None)

                    if event_time >= matched_request["time"]:
                        response_ms = (
                            event_time - matched_request["time"]
                        ) * 1000.0

        self._record_response_time(row, response_ms)

        return {
            "delta_ms": delta_ms,
            "response_ms": response_ms,
```

### MBSniffer v3.2/mb_protocol.py — lines 761-800

```
761             "matched_request": matched_request,
762             "expired_requests": expired_requests,
763             "pending": self.pending_count(),
764             "timeouts": self.timeouts,
765         }
766 
767     @staticmethod
768     def _p95(values):
769         if not values:
770             return None
771         ordered = sorted(float(v) for v in values)
772         index = max(0, min(len(ordered) - 1, math.ceil(0.95 * len(ordered)) - 1))
773         return ordered[index]
774 
775     def health_summary(
776         self,
777         *,
778         baud=9600,
779         data_bits=8,
780         parity="None",
781         stop_bits=1,
782         now=None,
783     ):
784         """Return compact diagnostic metrics for the Bus Health view."""
785         if now is None:
786             now = time.time()
787 
788         if self.first_event_time is None:
789             elapsed = 0.0
790         else:
791             end = max(
792                 float(now),
793                 float(self.last_event_time or self.first_event_time),
794             )
795             elapsed = max(0.0, end - float(self.first_event_time))
796 
797         responses = list(self.response_times_ms)
798         response_avg = (
799             sum(responses) / len(responses)
800             if responses else None
```

**MBSniffer v3.2/mb_protocol.py — lines 801-840**

```
        )
        response_min = min(responses) if responses else None
        response_max = max(responses) if responses else None
        response_p95 = self._p95(responses)

        slowest_slave = None
        slowest_avg = None
        for slave, samples in self.response_times_by_slave.items():
            if not samples:
                continue
            avg = sum(samples) / len(samples)
            if slowest_avg is None or avg > slowest_avg:
                slowest_slave = slave
                slowest_avg = avg

        request_rate = (
            self.requests / elapsed
            if elapsed > 0.0 else 0.0
        )

        nonvalidated_rate = (
            100.0 * self.nonvalidated_bytes / self.total_bytes
            if self.total_bytes else 0.0
        )
        answerable_requests = self.requests - self.broadcast_requests
        timeout_rate = (
            100.0 * self.timeouts / answerable_requests
            if answerable_requests > 0 else 0.0
        )

        try:
            baud = max(1.0, float(baud))
        except (TypeError, ValueError):
            baud = 9600.0
        try:
            data_bits = float(data_bits)
        except (TypeError, ValueError):
            data_bits = 8.0
        try:
            stop_bits = float(stop_bits)
```

### MBSniffer v3.2/mb_protocol.py — lines 841-880

```
        except (TypeError, ValueError):
            stop_bits = 1.0

        parity_bits = 0.0 if str(parity) == "None" else 1.0
        bits_per_char = 1.0 + data_bits + parity_bits + stop_bits
        bus_load = (
            100.0 * (self.total_bytes * bits_per_char) / (baud * elapsed)
            if elapsed > 0.0 else 0.0
        )

        exception_summary = ", ".join(
            f"S{slave}:{count}"
            for slave, count in sorted(self.exception_by_slave.items())
        ) or "-"

        return {
            "elapsed_s": elapsed,
            "request_rate": request_rate,
            "response_avg_ms": response_avg,
            "response_min_ms": response_min,
            "response_max_ms": response_max,
            "response_p95_ms": response_p95,
            "nonvalidated_rate_pct": nonvalidated_rate,
            "timeout_rate_pct": timeout_rate,
            "active_slaves": len(self.active_slaves),
            "bus_load_pct": bus_load,
            "slowest_slave": slowest_slave,
            "slowest_avg_ms": slowest_avg,
            "exception_summary": exception_summary,
            "total_bytes": self.total_bytes,
            "nonvalidated_bytes": self.nonvalidated_bytes,
        }

def u16_bytes(value):
    value = int(value) & 0xFFFF
    return bytes(((value >> 8) & 0xFF, value & 0xFF))

def corrupt_crc(frame):
    damaged = bytearray(frame)
    if len(damaged) >= 2:
```

### MBSniffer v3.2/mb_protocol.py — lines 881-920

```
881         damaged[-1] ^= 0xFF
882     return bytes(damaged)
883 
884 def build_simulation_transaction(slave, fc, rng):
885     """
886     Build one syntactically valid Modbus request/response pair.
887 
888     Simulation rules:
889     - FC01..FC04 only.
890     - Address random 1..32.
891     - Qty random 1..8.
892     """
893     slave = int(slave)
894     fc = int(fc)
895     start_addr = rng.randint(1, 32)
896     qty = rng.randint(1, 8)
897 
898     if fc in (1, 2):
899         request = append_crc(
900             bytes((slave, fc)) + u16_bytes(start_addr) + u16_bytes(qty)
901         )
902 
903         byte_count = (qty + 7) // 8
904         data = bytes(rng.randrange(256) for _ in range(byte_count))
905         response = append_crc(bytes((slave, fc, byte_count)) + data)
906 
907     elif fc in (3, 4):
908         request = append_crc(
909             bytes((slave, fc)) + u16_bytes(start_addr) + u16_bytes(qty)
910         )
911 
912         values = [rng.randint(0, 65535) for _ in range(qty)]
913         payload = b"".join(u16_bytes(value) for value in values)
914         response = append_crc(
915             bytes((slave, fc, len(payload))) + payload
916         )
917 
918     else:
919         raise ValueError(f"Unsupported simulation FC: {fc}")
920 
```

**MBSniffer v3.2/mb_protocol.py — lines 921-960**

```
    return {
        "slave": slave,
        "fc": fc,
        "address": start_addr,
        "qty": qty,
        "request": request,
        "response": response,
    }

def build_random_simulation_plan(rng=None):
    """
    Build exactly four simulation transactions:
    FC01, FC02, FC03 and FC04, one each.

    Four unique random Slave IDs are used, one per transaction.
    """
    if rng is None:
        rng = random.Random()

    slaves = rng.sample(range(1, 248), 4)

    plan = []
    for slave, fc in zip(slaves, SIMULATION_FUNCTION_CODES):
        tx = build_simulation_transaction(slave, fc, rng)
        tx["outcome"] = "response"
        tx["reply"] = tx["response"]
        plan.append(tx)

    return {
        "slaves": slaves,
        "transactions": plan,
    }



def build_anomaly_simulation_plan(rng=None):
    """
    Build a deterministic set of anomaly categories for UI colour preview.

    Used whenever debug simulation is executed. "Realçar resultados" controls
```

**MBSniffer v3.2/mb_protocol.py — lines 961-1000**

```
    only whether these rows receive colours; it does not change the generated
    scenario.

    It demonstrates:
      - normal response
      - slow response
      - Modbus exception
      - CRC error
      - timeout
      - RAW/UNPARSED bytes
    """
    if rng is None:
        rng = random.Random()

    slaves = rng.sample(range(1, 248), 5)

    normal = build_simulation_transaction(slaves[0], 1, rng)
    normal["outcome"] = "response"
    normal["reply"] = normal["response"]

    slow = build_simulation_transaction(slaves[1], 2, rng)
    slow["outcome"] = "slow"
    slow["reply"] = slow["response"]

    exception = build_simulation_transaction(slaves[2], 3, rng)
    exception["outcome"] = "exception"
    exception["reply"] = append_crc(
        bytes((slaves[2], 0x83, 0x02))
    )

    crc_error = build_simulation_transaction(slaves[3], 4, rng)
    crc_error["outcome"] = "crc"
    crc_error["reply"] = corrupt_crc(crc_error["response"])

    timeout = build_simulation_transaction(slaves[4], 3, rng)
    timeout["outcome"] = "timeout"
    timeout["reply"] = None

    return {
        "slaves": slaves,
```

**MBSniffer v3.2/mb_protocol.py — lines 1001-1007**

```
1001        "transactions": [normal, slow, exception, crc_error, timeout],
1002        # Four bytes are deliberately too short to be accepted as the FC=0xFF
1003        # exception frame candidate, producing a visible RAW/UNPARSED row.
1004        "raw_noise": bytes((0x00, 0x00, 0x00, 0x00)),
1005        "slow_delay_seconds": (SLOW_RESPONSE_THRESHOLD_MS / 1000.0) + 0.15,
1006    }
1007
```

# MBSniffer v3.2/mb_capture.py

889 source lines.

### MBSniffer v3.2/mb_capture.py — lines 1-40

```
#!/usr/bin/env python3
"""Serial-port management, passive capture, simulation and event processing."""

import queue
import random
import threading
import time
import traceback
from datetime import datetime

import tkinter as tk
from tkinter import messagebox

from mb_config import (
    APP_NAME,
    APP_VERSION,
    MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
    MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
    SNIFFER_SPLIT_FRAME_HOLD_MS,
    UI_QUEUE_BATCH_SIZE,
    UI_QUEUE_IDLE_INTERVAL_MS,
    UI_QUEUE_BUSY_INTERVAL_MS,
    debug_sim,
)
from mb_protocol import (
    parse_burst,
    burst_may_continue,
    build_anomaly_simulation_plan,
    modbus_rtu_interframe_gap_seconds,
)
from mb_slave_finder import cancel_serial_io

class CaptureMixin:
    """Methods for serial capture, COM management and simulation."""

    def set_sniffer_refresh_state(self, state):
        """Apply the same state to both Sniffer COM refresh buttons."""
        for name in ("refresh_a_btn", "refresh_b_btn"):
            button = getattr(self, name, None)
            if button is not None:
```

### MBSniffer v3.2/mb_capture.py — lines 41-80

```
41                 button.configure(state=state)
42
43     def update_mode_ui(self):
44         mode = self.mode_var.get()
45         dual = mode == "RS232 dual RX"
46
47         state = "readonly" if dual else "disabled"
48         self.port_b_combo.configure(state=state)
49
50         if dual:
51             self.mode_help_var.set(
52                 self.tr("RS232 dual RX: COM A escuta uma direção; COM B escuta a direção oposta.")
53             )
54         elif mode == "RS232 single RX":
55             self.mode_help_var.set(
56                 self.tr("RS232 single RX: uma COM escuta apenas uma direção da ligação RS232.")
57             )
58         else:
59             self.mode_help_var.set(
60                 self.tr("RS485 2-wire: uma COM consegue observar pedidos e respostas no mesmo par diferencial.")
61             )
62
63         self.update_com_status()
64
65     def update_gap_state(self):
66         self.gap_entry.configure(
67             state="normal" if self.gap_mode_var.get() == "Manual" else "disabled"
68         )
69
70     def get_serial_modules(self):
71         try:
72             import serial
73             from serial.tools import list_ports
74             return serial, list_ports
75         except ImportError:
76             return None, None
77
78     def refresh_ports(self):
79         _, list_ports = self.get_serial_modules()
80         ports = [p.device for p in list_ports.comports()] if list_ports else []
```

### MBSniffer v3.2/mb_capture.py — lines 81-120

```
81
82          self.port_a_combo["values"] = ports
83          self.port_b_combo["values"] = ports
84
85          if ports:
86              if self.port_a_var.get() not in ports:
87                  self.port_a_var.set(ports[0])
88              if len(ports) > 1:
89                  if self.port_b_var.get() not in ports or self.port_b_var.get() == self.port_a_var.get():
90                      self.port_b_var.set(ports[1])
91              elif not self.port_b_var.get():
92                  self.port_b_var.set("")
93          else:
94              self.port_a_var.set("")
95              self.port_b_var.set("")
96
97          if hasattr(self, "discovery_port_combo"):
98              self.discovery_port_combo["values"] = ports
99              current_discovery = self.discovery_port_var.get().strip()
100             if ports:
101                 if current_discovery not in ports:
102                     self.discovery_port_var.set(ports[0])
103             else:
104                 self.discovery_port_var.set("")
105             self.update_discovery_start_state()
106
107         if hasattr(self, "master_port_combo"):
108             self.master_port_combo["values"] = ports
109             if ports:
110                 if self.master_port_var.get().strip() not in ports:
111                     self.master_port_var.set(ports[0])
112             elif not getattr(self, "master_active", False):
113                 self.master_port_var.set("")
114
115         self.update_com_status()
116
117     def detected_ports(self):
118         """Return the set of COM device names currently reported by Windows/pyserial."""
119         _, list_ports = self.get_serial_modules()
120         if list_ports is None:
```

### MBSniffer v3.2/mb_capture.py — lines 121-160

```
            return set()
        try:
            return {p.device for p in list_ports.comports()}
        except Exception:
            return set()

    def port_status_text(self, label, port, enabled=True):
        if not enabled:
            return f"{label} - {self.tr('Não Utilizada')}"
        if not port:
            return f"{label} - {self.tr('Não selecionada')}"

        ser = self.serial_by_port.get(port)
        if ser is not None:
            try:
                if ser.is_open:
                    return f"{label} - {self.tr('Aberta')}"
            except Exception:
                pass

        if port in self.detected_ports():
            return f"{label} - {self.tr('Detetada')}"
        return f"{label} - {self.tr('Não detetada')}"

    def update_com_status(self):
        """
        Update COM status without opening/probing a port.

        Status means:
          - Aberta: currently opened by MBSniffer
          - Detetada: reported by Windows/pyserial, but not opened by MBSniffer
          - Não detetada: selected name is no longer reported by the OS
        """
        if not hasattr(self, "port_a_status_var"):
            return

        mode = self.mode_var.get() if hasattr(self, "mode_var") else "RS485 2-wire"
        dual = mode == "RS232 dual RX"
        self.port_a_status_var.set(
            self.port_status_text("COM A", self.port_a_var.get().strip(), True)
```

### MBSniffer v3.2/mb_capture.py — lines 161-200

```
161         )
162         self.port_b_status_var.set(
163             self.port_status_text("COM B", self.port_b_var.get().strip(), dual)
164         )
165 
166     def poll_com_status(self):
167         if self._closing:
168             return
169         try:
170             self.update_com_status()
171             if hasattr(self, "stat_requests_var"):
172                 self.update_stats_labels()
173         finally:
174             if not self._closing:
175                 try:
176                     self.after(1000, self.poll_com_status)
177                 except tk.TclError:
178                     pass
179 
180     def selected_port_plan(self):
181         """Return [(channel, port), ...] for the currently selected physical mode."""
182         mode = self.mode_var.get()
183         port_a = self.port_a_var.get().strip()
184         port_b = self.port_b_var.get().strip()
185 
186         if not port_a:
187             raise ValueError(self.tr("Seleciona a COM A."))
188 
189         plan = [("BUS" if mode == "RS485 2-wire" else "A→B", port_a)]
190 
191         if mode == "RS232 dual RX":
192             if not port_b:
193                 raise ValueError(self.tr("Seleciona também a COM B."))
194             if port_a == port_b:
195                 raise ValueError(self.tr("COM A e COM B têm de ser diferentes."))
196             plan.append(("B→A", port_b))
197 
198         return plan
199 
200     def open_selected_ports(self, serial, config):
```

### MBSniffer v3.2/mb_capture.py — lines 201-240

```
201         """Open the selected capture ports and return [(channel, serial_obj), ...]."""
202         opened = []
203         try:
204             for channel, port in self.selected_port_plan():
205                 opened.append((channel, self.open_serial_port(serial, port, config)))
206             return opened
207         except Exception:
208             for _, ser_obj in opened:
209                 try:
210                     ser_obj.close()
211                 except Exception:
212                     pass
213             raise
214 
215     def install_opened_ports(self, opened):
216         self.serial_objects = [ser_obj for _, ser_obj in opened]
217         self.serial_by_port = {
218             ser_obj.port: ser_obj for _, ser_obj in opened if getattr(ser_obj, "port", None)
219         }
220         self.update_com_status()
221 
222     def start_reader_threads(self, opened, gap_ms):
223         self.reader_threads = []
224         generation = self.reader_generation
225         for channel, ser_obj in opened:
226             t = threading.Thread(
227                 target=self.reader_loop,
228                 args=(ser_obj, gap_ms, channel, generation),
229                 daemon=True
230             )
231             self.reader_threads.append(t)
232             t.start()
233 
234     def restart_com_port(self):
235         """
236         Close and reopen the selected COM port(s).
237 
238         During an active capture the capture continues in the same session/log.
239         When stopped, the selected COM port(s) are opened and closed once as a
240         restart/test of the application's serial handle.
```

### MBSniffer v3.2/mb_capture.py — lines 241-280

```
241         """
242         if self.busy and not self.capture_active:
243             return
244         if self.restart_in_progress:
245             return
246 
247         serial, _ = self.get_serial_modules()
248         if serial is None:
249             messagebox.showerror(self.tr("pyserial em falta"), self.tr("pyserial não está instalado."))
250             return
251 
252         try:
253             config = self.serial_config(serial)
254             gap_ms = self.calculated_gap_ms()
255             self.selected_port_plan()
256         except Exception as exc:
257             messagebox.showerror(self.tr("Porta COM"), str(exc))
258             return
259 
260         self.restart_in_progress = True
261         self.restart_btn.configure(state="disabled")
262 
263         if not self.capture_active:
264             opened = []
265             try:
266                 opened = self.open_selected_ports(serial, config)
267                 self.install_opened_ports(opened)
268                 self.update_idletasks()
269             except Exception as exc:
270                 messagebox.showerror(self.tr("Reiniciar COM"), self.trf("Não foi possível reiniciar a porta COM:\n\n{error}", error=exc))
271             finally:
272                 for _, ser_obj in opened:
273                     try:
274                         ser_obj.close()
275                     except Exception:
276                         pass
277                 self.serial_objects = []
278                 self.serial_by_port = {}
279                 self.restart_in_progress = False
280                 self.restart_btn.configure(state="normal")
```

### MBSniffer v3.2/mb_capture.py — lines 281-320

```
281                 self.refresh_ports()
282             return
283 
284         # Active capture: stop only the current serial-reader generation.
285         self.status_var.set(self.tr("● A reiniciar COM"))
286         self.reader_generation += 1
287         self.stop_event.set()
288 
289         for thread in list(self.reader_threads):
290             try:
291                 thread.join(timeout=0.5)
292             except Exception:
293                 pass
294 
295         for ser_obj in list(self.serial_objects):
296             try:
297                 ser_obj.close()
298             except Exception:
299                 pass
300 
301         self.reader_threads = []
302         self.serial_objects = []
303         self.serial_by_port = {}
304         self.stop_event.clear()
305 
306         try:
307             opened = self.open_selected_ports(serial, config)
308         except Exception as exc:
309             self.restart_in_progress = False
310             self.capture_active = False
311             self.busy = False
312             self.stop_event.set()
313             self.finish_capture_ui()
314             self.update_com_status()
315             messagebox.showerror(
316                 self.tr("Reiniciar COM"),
317                 self.trf("Não foi possível reabrir a porta COM. A captura foi parada.\n\n{error}", error=exc)
318             )
319             return
320 
```

### MBSniffer v3.2/mb_capture.py — lines 321-360

```
        self.install_opened_ports(opened)
        self.stop_event.clear()
        self.start_reader_threads(opened, gap_ms)
        self.queue_log_note(
            f"# COM restart: {datetime.now().isoformat(timespec='milliseconds')}\n"
        )
        self.status_var.set(self.tr("● A capturar"))
        self.restart_in_progress = False
        self.restart_btn.configure(state="normal")
        self.update_com_status()

    def set_simulation_button_state(self, state):
        """Change Simulation button state only when the optional button exists."""
        if self.sim_btn is not None:
            self.sim_btn.configure(state=state)

    def selected_pending_timeout_seconds(self):
        try:
            value = float(self.pending_timeout_var.get().strip().replace(",", "."))
        except ValueError:
            raise ValueError(self.tr("Pending timeout inválido."))

        if not (
            MIN_PENDING_REQUEST_TIMEOUT_SECONDS
            <= value
            <= MAX_PENDING_REQUEST_TIMEOUT_SECONDS
        ):
            raise ValueError(
                self.trf(
                    "Pending timeout deve estar entre {minimum:g} e {maximum:g} segundos.",
                    minimum=MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
                    maximum=MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
                )
            )

        return value

    def calculated_gap_ms(self):
        if self.gap_mode_var.get() == "Manual":
            value = float(self.gap_ms_var.get().replace(",", "."))
```

### MBSniffer v3.2/mb_capture.py — lines 361-400

```
361             if value <= 0:
362                 raise ValueError(self.tr("Frame gap manual inválido."))
363             return value
364 
365         baud = int(self.baud_var.get())
366         data_bits = int(self.data_var.get())
367         parity_bits = 0 if self.parity_var.get() == "None" else 1
368         stop_bits = float(self.stop_var.get())
369         bits_per_char = 1 + data_bits + parity_bits + stop_bits
370         return modbus_rtu_interframe_gap_seconds(baud, bits_per_char) * 1000.0
371 
372     def serial_config(self, serial):
373         byte_map = {
374             5: serial.FIVEBITS, 6: serial.SIXBITS,
375             7: serial.SEVENBITS, 8: serial.EIGHTBITS,
376         }
377         parity_map = {
378             "None": serial.PARITY_NONE,
379             "Even": serial.PARITY_EVEN,
380             "Odd": serial.PARITY_ODD,
381             "Mark": serial.PARITY_MARK,
382             "Space": serial.PARITY_SPACE,
383         }
384         stop_map = {
385             "1": serial.STOPBITS_ONE,
386             "1.5": serial.STOPBITS_ONE_POINT_FIVE,
387             "2": serial.STOPBITS_TWO,
388         }
389         return {
390             "baudrate": int(self.baud_var.get()),
391             "bytesize": byte_map[int(self.data_var.get())],
392             "parity": parity_map[self.parity_var.get()],
393             "stopbits": stop_map[self.stop_var.get()],
394         }
395 
396     def open_serial_port(self, serial, port, config):
397         ser = serial.Serial()
398         ser.port = port
399         ser.baudrate = config["baudrate"]
400         ser.bytesize = config["bytesize"]
```

### MBSniffer v3.2/mb_capture.py — lines 401-440

```
401         ser.parity = config["parity"]
402         ser.stopbits = config["stopbits"]
403         ser.timeout = 0
404         ser.write_timeout = 0
405         ser.xonxoff = False
406         ser.rtscts = False
407         ser.dsrdtr = False
408 
409         try:
410             ser.rts = False
411             ser.dtr = False
412         except Exception:
413             pass
414 
415         ser.open()
416 
417         try:
418             ser.rts = False
419             ser.dtr = False
420         except Exception:
421             pass
422 
423         try:
424             ser.reset_input_buffer()
425         except Exception:
426             pass
427 
428         return ser
429 
430     def start_capture(self):
431         if self.busy:
432             return
433 
434         serial, _ = self.get_serial_modules()
435         if serial is None:
436             messagebox.showerror(
437                 self.tr("pyserial em falta"),
438                 self.tr("pyserial não está instalado.\n\nO build_exe.bat inclui-o automaticamente no EXE.")
439             )
440             return
```

### MBSniffer v3.2/mb_capture.py — lines 441-480

```
441
442         mode = self.mode_var.get()
443         port_a = self.port_a_var.get().strip()
444         port_b = self.port_b_var.get().strip()
445
446         try:
447             gap_ms = self.calculated_gap_ms()
448             pending_timeout_s = self.selected_pending_timeout_seconds()
449             config = self.serial_config(serial)
450             opened = self.open_selected_ports(serial, config)
451         except Exception as exc:
452             messagebox.showerror(self.tr("Erro ao abrir porta"), str(exc))
453             self.update_com_status()
454             return
455
456         self.reader_generation += 1
457         self.install_opened_ports(opened)
458         self.reader_threads = []
459         self.stop_event.clear()
460         self.busy = True
461         self.capture_active = True
462         self.metrics.pending_timeout_seconds = pending_timeout_s
463         self.reset_session_metrics()
464
465         capture_started = datetime.now()
466         log_header = (
467             f"# {APP_NAME} v{APP_VERSION}\n"
468             f"# Start: {capture_started.isoformat(timespec='milliseconds')}\n"
469             f"# Mode: {mode}\n"
470             f"# COM A: {port_a}\n"
471             + (f"# COM B: {port_b}\n" if mode == "RS232 dual RX" else "")
472             + f"# Serial: baud={self.baud_var.get()} data={self.data_var.get()} "
473               f"parity={self.parity_var.get()} stop={self.stop_var.get()} "
474               f"frame_gap_ms={gap_ms:.3f}\n"
475             + f"# Pending timeout: {pending_timeout_s:g} s\n"
476             + "# Passive/read-only: no Serial.write() calls.\n#\n"
477         )
478         self.prepare_log_session("MBSniffer", capture_started, log_header)
479
480         self.status_var.set(self.tr("● A capturar"))
```

### MBSniffer v3.2/mb_capture.py — lines 481-520

```
        self.start_btn.configure(state="disabled")
        self.stop_btn.configure(state="normal")
        self.restart_btn.configure(state="normal")
        self.set_simulation_button_state("disabled")
        self.set_sniffer_refresh_state("disabled")

        self.start_reader_threads(opened, gap_ms)
        self.update_com_status()

    def reader_loop(self, ser, gap_ms, channel, generation):
        gap_s = gap_ms / 1000.0
        hold_s = max(gap_s, SNIFFER_SPLIT_FRAME_HOLD_MS / 1000.0)
        poll_sleep = min(max(gap_s / 8.0, 0.0002), 0.001)

        buffer = bytearray()
        buffer_wall = None
        # (byte offset, epoch) of every read, so frames merged into one burst
        # by USB latency keep their own arrival times.
        chunk_times = []
        last_rx = None
        # Burst length already checked by burst_may_continue(); avoids
        # re-parsing an unchanged buffer on every poll of the hold window.
        hold_checked_len = None
        hold_active = False

        def flush_buffer():
            ts = (
                buffer_wall.strftime("%H:%M:%S.%f")[:-3]
                if buffer_wall else datetime.now().strftime("%H:%M:%S.%f")[:-3]
            )
            event_epoch = (
                buffer_wall.timestamp() if buffer_wall else time.time()
            )
            for row in parse_burst(
                bytes(buffer), ts, channel, event_epoch, chunk_times
            ):
                self.event_queue.put(("frame", row))

        try:
            while not self.stop_event.is_set():
```

### MBSniffer v3.2/mb_capture.py — lines 521-560

```
                waiting = ser.in_waiting

                if waiting:
                    chunk = ser.read(waiting)
                    if chunk:
                        if not buffer:
                            buffer_wall = datetime.now()
                            chunk_times.append((0, buffer_wall.timestamp()))
                        else:
                            chunk_times.append((len(buffer), time.time()))
                        buffer.extend(chunk)
                        last_rx = time.perf_counter()

                elif buffer and last_rx is not None:
                    idle_s = time.perf_counter() - last_rx
                    if idle_s >= gap_s:
                        if idle_s < hold_s:
                            if hold_checked_len != len(buffer):
                                hold_checked_len = len(buffer)
                                hold_active = burst_may_continue(bytes(buffer))
                            if hold_active:
                                time.sleep(poll_sleep)
                                continue

                        flush_buffer()
                        buffer.clear()
                        chunk_times.clear()
                        buffer_wall = None
                        last_rx = None
                        hold_checked_len = None
                        hold_active = False

                time.sleep(poll_sleep)

        except Exception as exc:
            if not self.stop_event.is_set():
                self.event_queue.put(("error", (generation, channel, str(exc))))

        finally:
            if buffer:
```

### MBSniffer v3.2/mb_capture.py — lines 561-600

```
561                 flush_buffer()
562 
563             try:
564                 ser.close()
565             except Exception:
566                 pass
567             self.event_queue.put(("reader_stopped", (generation, channel)))
568 
569     def stop_capture(self):
570         if self.capture_active:
571             self.status_var.set(self.tr("● A parar"))
572         self.stop_event.set()
573 
574     def run_simulation(self):
575         if debug_sim != 1:
576             return
577 
578         if self.busy:
579             return
580 
581         try:
582             pending_timeout_s = self.selected_pending_timeout_seconds()
583         except Exception as exc:
584             messagebox.showerror(self.tr("Configuração inválida"), str(exc))
585             return
586 
587         self.busy = True
588         self.metrics.pending_timeout_seconds = pending_timeout_s
589         self.reset_session_metrics()
590 
591         # Simulation is GUI-only. Never create or write a .txt log.
592         self.close_log()
593 
594         self.status_var.set(self.tr("● A simular"))
595         self.start_btn.configure(state="disabled")
596         self.restart_btn.configure(state="disabled")
597         self.set_simulation_button_state("disabled")
598         self.set_sniffer_refresh_state("disabled")
599 
600         def worker():
```

**MBSniffer v3.2/mb_capture.py — lines 601-640**

```
601             try:
602                 # Simulation always exercises the complete diagnostic scenario.
603                 # "Realçar resultados" affects only Treeview row colouring; it
604                 # never changes which simulated frames are generated.
605                 rng = random.Random()
606                 plan = build_anomaly_simulation_plan(rng=rng)
607 
608                 for tx in plan["transactions"]:
609                     time.sleep(rng.uniform(0.025, 0.085))
610 
611                     request_now = datetime.now()
612                     request_ts = request_now.strftime("%H:%M:%S.%f")[:-3]
613 
614                     request_channel = "SIM"
615 
616                     for row in parse_burst(
617                         tx["request"], request_ts, request_channel,
618                         request_now.timestamp()
619                     ):
620                         self.event_queue.put(("frame", row))
621 
622                     outcome = tx.get("outcome", "response")
623 
624                     if outcome == "timeout":
625                         # Do not sleep for the user-configured Pending timeout.
626                         # The queue event expires this exact pending request
627                         # through SessionMetrics and marks the REQUEST row.
628                         self.event_queue.put((
629                             "simulation_force_timeout",
630                             (tx["slave"], tx["fc"]),
631                         ))
632                         continue
633 
634                     if outcome == "slow":
635                         time.sleep(plan.get("slow_delay_seconds", 0.65))
636                     else:
637                         time.sleep(rng.uniform(0.018, 0.095))
638 
639                     reply_now = datetime.now()
640                     reply_ts = reply_now.strftime("%H:%M:%S.%f")[:-3]
```

### MBSniffer v3.2/mb_capture.py — lines 641-680

```
                    reply_channel = "SIM"

                    reply = tx.get("reply")
                    if reply:
                        for row in parse_burst(
                            reply, reply_ts, reply_channel,
                            reply_now.timestamp()
                        ):
                            self.event_queue.put(("frame", row))

                # One RAW row completes the diagnostic scenario. With anomaly
                # highlighting OFF it remains a normal, uncoloured RAW row;
                # with highlighting ON the same row receives the RAW colour.
                raw_now = datetime.now()
                raw_ts = raw_now.strftime("%H:%M:%S.%f")[:-3]
                raw_channel = "SIM"
                for row in parse_burst(
                    plan["raw_noise"],
                    raw_ts,
                    raw_channel,
                    raw_now.timestamp(),
                ):
                    self.event_queue.put(("frame", row))

                self.event_queue.put(("simulation_done", None))

            except Exception as exc:
                # Never leave the GUI permanently busy if the simulation worker
                # fails unexpectedly.
                self.event_queue.put((
                    "simulation_error",
                    f"{type(exc).__name__}: {exc}"
                ))

        threading.Thread(target=worker, daemon=True).start()

    def process_queue(self):
        if self._closing:
            return

```

### MBSniffer v3.2/mb_capture.py — lines 681-720

```
        delay = UI_QUEUE_IDLE_INTERVAL_MS
        try:
            events = []
            for _ in range(UI_QUEUE_BATCH_SIZE):
                try:
                    events.append(self.event_queue.get_nowait())
                except queue.Empty:
                    break

            frame_batch = []

            def flush_frames():
                nonlocal frame_batch
                if frame_batch:
                    self.add_frame_rows_batch(frame_batch)
                    frame_batch = []

            for event, payload in events:
                if event == "frame":
                    frame_batch.append(payload)
                    continue

                flush_frames()

                if event == "error":
                    generation, channel, error_text = payload
                    if generation != self.reader_generation:
                        continue
                    # Stop all readers in this capture generation after any
                    # reader failure; never leave a half-alive dual-RX capture.
                    self.stop_event.set()
                    self.update_com_status()
                    messagebox.showerror(
                        self.tr("Erro de captura"), f"{channel}: {error_text}"
                    )

                elif event == "reader_stopped":
                    generation, _channel = payload
                    if generation != self.reader_generation:
                        continue
```

**MBSniffer v3.2/mb_capture.py — lines 721-760**

```
                    if not any(t.is_alive() for t in self.reader_threads):
                        self.finish_capture_ui()

                elif event == "simulation_force_timeout":
                    slave, fc = payload
                    expired = self.metrics.force_timeout(slave, fc)
                    if expired:
                        self.mark_timed_out_records(expired, rerender=True)
                    self.update_stats_labels()

                elif event == "simulation_done":
                    self.busy = False
                    self.status_var.set(self.tr("● Simulação concluída"))
                    self.start_btn.configure(state="normal")
                    self.stop_btn.configure(state="disabled")
                    self.restart_btn.configure(state="normal")
                    self.set_simulation_button_state("normal")
                    self.set_sniffer_refresh_state("normal")
                    self.close_log()
                    self.update_com_status()

                elif event == "simulation_error":
                    self.busy = False
                    self.status_var.set(self.tr("● Erro na simulação"))
                    self.start_btn.configure(state="normal")
                    self.stop_btn.configure(state="disabled")
                    self.restart_btn.configure(state="normal")
                    self.set_simulation_button_state("normal")
                    self.set_sniffer_refresh_state("normal")
                    self.close_log()
                    self.update_com_status()
                    messagebox.showerror(self.tr("Erro na simulação"), str(payload))

                elif event == "discovery_status":
                    self.discovery_status_var.set(str(payload))

                elif event == "discovery_progress":
                    self.apply_discovery_progress(payload)

                elif event == "discovery_found":
```

### MBSniffer v3.2/mb_capture.py — lines 761-800

```
                    self.add_discovery_result(payload)

                elif event == "discovery_done":
                    self.finish_discovery_ui(payload)

                elif event.startswith("master_"):
                    self.handle_master_event(event, payload)

            flush_frames()

            if self.metrics.expire_pending(time.time()):
                expired = self.metrics.pop_expired_requests()
                if expired:
                    self.mark_timed_out_records(expired, rerender=True)
                self.update_stats_labels()

            if (
                self.capture_active
                and not self.restart_in_progress
                and self.stop_event.is_set()
                and self.reader_threads
                and not any(t.is_alive() for t in self.reader_threads)
                and self.stop_btn["state"] != "disabled"
            ):
                self.finish_capture_ui()

            delay = (
                UI_QUEUE_BUSY_INTERVAL_MS
                if len(events) >= UI_QUEUE_BATCH_SIZE
                else UI_QUEUE_IDLE_INTERVAL_MS
            )

        except tk.TclError:
            if self._closing:
                return
            traceback.print_exc()
            delay = UI_QUEUE_IDLE_INTERVAL_MS

        except Exception as exc:
            # One unexpected event/UI error must not permanently kill the
```

### MBSniffer v3.2/mb_capture.py — lines 801-840

```
            # periodic queue callback and make the application appear frozen.
            traceback.print_exc()
            if not self._process_queue_error_shown and not self._closing:
                self._process_queue_error_shown = True
                try:
                    messagebox.showerror(
                        self.tr("Erro interno"),
                        self.trf(
                            "Ocorreu um erro ao processar eventos da interface. O processamento continuará.\n\n{error}",
                            error=f"{type(exc).__name__}: {exc}",
                        )
                    )
                except Exception:
                    pass
            delay = UI_QUEUE_IDLE_INTERVAL_MS

        finally:
            if not self._closing:
                try:
                    self.after(delay, self.process_queue)
                except tk.TclError:
                    pass

    def on_close(self):
        if self._closing:
            return

        # A changed Master workspace may be saved first; "Cancel" keeps the
        # application open.
        confirm = getattr(self, "master_confirm_close", None)
        if callable(confirm) and not confirm():
            return

        try:
            saver = getattr(self, "save_current_ui_settings", None)
            if callable(saver):
                saver()
        except Exception:
            pass

```

### MBSniffer v3.2/mb_capture.py — lines 841-880

```
        self._closing = True
        self.busy = False
        self.capture_active = False
        self.stop_event.set()
        self.discovery_stop_event.set()

        # Cancel driver I/O before joining workers so shutdown is not held up
        # by a serial read/write timeout.
        cancel_serial_io(self.discovery_serial)
        stopper = getattr(self, "master_shutdown", None)
        if callable(stopper):
            stopper()

        if self.discovery_thread is not None:
            try:
                if self.discovery_thread.is_alive():
                    self.discovery_thread.join(timeout=0.8)
            except Exception:
                pass

        if self.discovery_serial is not None:
            try:
                self.discovery_serial.close()
            except Exception:
                pass
        self.discovery_serial = None

        for ser in list(self.serial_objects):
            cancel_serial_io(ser)

        for thread in list(self.reader_threads):
            try:
                if thread.is_alive():
                    thread.join(timeout=0.5)
            except Exception:
                pass

        for ser in list(self.serial_objects):
            try:
                ser.close()
```

**MBSniffer v3.2/mb_capture.py — lines 881-889**

```
            except Exception:
                pass

        self.close_log()
        try:
            self.destroy()
        except tk.TclError:
            pass

```

# MBSniffer v3.2/mb_diagnostics.py

286 source lines.

**MBSniffer v3.2/mb_diagnostics.py — lines 1-40**

```
#!/usr/bin/env python3
"""Diagnostic helpers for MBSniffer traffic inspection and anomaly display."""

from mb_config import MODBUS_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS
from mb_protocol import crc16_modbus, read_bits_from_response
from mb_i18n import LANGUAGE_PORTUGUESE, translate_details, translate_text


FUNCTION_NAMES = {
    0x01: "Read Coils",
    0x02: "Read Discrete Inputs",
    0x03: "Read Holding Registers",
    0x04: "Read Input Registers",
    0x05: "Write Single Coil",
    0x06: "Write Single Register",
    0x07: "Read Exception Status",
    0x08: "Diagnostics",
    0x0F: "Write Multiple Coils",
    0x10: "Write Multiple Registers",
    0x16: "Mask Write Register",
    0x17: "Read/Write Multiple Registers",
    0x2B: "Encapsulated Interface Transport",
}


def raw_hex_to_bytes(raw):
    try:
        return bytes.fromhex(str(raw or ""))
    except (TypeError, ValueError):
        return b""


def fc_number(record):
    text = str(record.get("fc", "") or "").strip()
    try:
        value = int(text, 16)
    except (TypeError, ValueError):
        return None
    if record.get("type") == "EXCEPTION":
        value &= 0x7F
```

### MBSniffer v3.2/mb_diagnostics.py — lines 41-80

```
41      return value
42
43
44  def function_label(record, language=LANGUAGE_PORTUGUESE):
45      fc = fc_number(record)
46      if fc is None:
47          return "—"
48      name = FUNCTION_NAMES.get(fc, translate_text("Desconhecida/outra", language))
49      return f"0x{fc:02X} — {name}"
50
51
52  def crc_fields(record):
53      if str(record.get("type", "")).startswith("RAW"):
54          return None, None, str(record.get("crc", "?"))
55
56      frame = raw_hex_to_bytes(record.get("raw"))
57      if len(frame) < 4:
58          return None, None, str(record.get("crc", "?"))
59
60      received = frame[-2] | (frame[-1] << 8)
61      calculated = crc16_modbus(frame[:-2])
62      return received, calculated, ("OK" if received == calculated else "ERROR")
63
64
65  def response_payload(record, language=LANGUAGE_PORTUGUESE):
66      """Return (byte_count, data_hex, decoded_data) where available."""
67      frame = raw_hex_to_bytes(record.get("raw"))
68      if len(frame) < 3:
69          return None, "", ""
70
71      kind = str(record.get("type", ""))
72      fc_raw = frame[1] if len(frame) >= 2 else 0
73      fc = fc_raw & 0x7F
74
75      if kind == "EXCEPTION":
76          code = frame[2] if len(frame) >= 3 else None
77          if code is None:
78              return None, "", "Exception"
79          info = MODBUS_EXCEPTION_CODES.get(code)
80          name = info[0] if info else "Unknown/Reserved Exception"
```

### MBSniffer v3.2/mb_diagnostics.py — lines 81-120

```
        return None, "", f"0x{code:02X} — {name}"

    if kind == "RESPONSE" and fc in (1, 2, 3, 4) and len(frame) >= 5:
        byte_count = frame[2]
        data = frame[3:3 + byte_count]
        data_hex = " ".join(f"{b:02X}" for b in data)
        decoded = ""
        start = record.get("request_pdu_address")
        if fc in (3, 4) and byte_count % 2 == 0 and len(data) == byte_count:
            regs = [
                (data[i] << 8) | data[i + 1]
                for i in range(0, len(data), 2)
            ]
            if start is not None:
                # The matched request gives the address of every value.
                pairs = ", ".join(
                    f"{start + i}={value}" for i, value in enumerate(regs)
                )
                decoded = translate_details(
                    f"Registos (PDU Address=valor): {pairs}", language
                )
            else:
                decoded = translate_details(f"Registos={regs}", language)
        elif fc in (1, 2) and len(data) == byte_count:
            bits = read_bits_from_response(frame, record.get("request_qty"))
            if start is not None:
                pairs = ", ".join(
                    f"{start + i}={value}" for i, value in enumerate(bits)
                )
                decoded = translate_details(
                    f"Bits (PDU Address=valor): {pairs}", language
                )
            else:
                decoded = "Bits=" + "".join(str(bit) for bit in bits)
        return byte_count, data_hex, decoded

    if kind == "REQUEST" and fc in (15, 16) and len(frame) >= 9:
        byte_count = frame[6]
        data = frame[7:7 + byte_count]
        return byte_count, " ".join(f"{b:02X}" for b in data), ""
```

### MBSniffer v3.2/mb_diagnostics.py — lines 121-160

```
121
122     if kind == "REQUEST" and fc == 23 and len(frame) >= 13:
123         byte_count = frame[10]
124         data = frame[11:11 + byte_count]
125         return byte_count, " ".join(f"{b:02X}" for b in data), ""
126
127     return record.get("byte_count"), str(record.get("data_hex", "") or ""), ""
128
129
130 def anomaly_kind(record, threshold_ms=SLOW_RESPONSE_THRESHOLD_MS):
131     """
132     Return one anomaly class or None.
133
134     Priority avoids ambiguous colouring when one frame satisfies several
135     conditions. CRC is strongest because the frame itself is invalid.
136     """
137     if str(record.get("crc", "")) == "ERROR":
138         return "crc"
139     if bool(record.get("timed_out")):
140         return "timeout"
141     if str(record.get("type", "")) == "EXCEPTION":
142         return "exception"
143     if str(record.get("type", "")).startswith("RAW"):
144         return "raw"
145
146     try:
147         response_ms = float(record.get("response_text", ""))
148     except (TypeError, ValueError):
149         response_ms = None
150
151     if response_ms is not None and response_ms >= float(threshold_ms):
152         return "slow"
153     return None
154
155
156 RESULT_LABELS = {
157     "success": ("✓ Sucesso", "Verde"),
158     "slow": ("⚠ Resposta lenta", "Amarelo"),
159     "exception": ("Exception", "Vermelho"),
160     "crc": ("× CRC Error", "Vermelho"),
```

### MBSniffer v3.2/mb_diagnostics.py — lines 161-200

```
161     "timeout": ("⚠ Timeout", "Laranja"),
162     "raw": ("RAW", "Roxo"),
163     "pending": ("Pendente / sem resultado", "Neutro"),
164     "neutral": ("Sem classificação", "Neutro"),
165 }
166
167
168 def result_info(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
169     """
170     Return (result_key, human_label, colour_name) for one traffic record.
171
172     The semantic result is always available to the Frame Inspector. Whether
173     the table/Inspector actually uses the category colour is controlled by
174     "Realçar resultados".
175     """
176     if not record:
177         return "neutral", "—", translate_text("Neutro", language)
178
179     kind = anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS)
180     if kind:
181         label, colour = RESULT_LABELS[kind]
182         if kind == "raw":
183             # Preserve the exact parser state in the Inspector.
184             row_type = str(record.get("type", "RAW") or "RAW")
185             label = row_type
186         return kind, translate_text(label, language), translate_text(colour, language)
187
188     row_type = str(record.get("type", ""))
189
190     if (
191         row_type == "RESPONSE"
192         and record.get("crc") == "OK"
193         and record.get("matched")
194     ):
195         label, colour = RESULT_LABELS["success"]
196         return "success", translate_text(label, language), translate_text(colour, language)
197
198     if row_type == "REQUEST":
199         if successful_request and not record.get("timed_out"):
200             label, colour = RESULT_LABELS["success"]
```

### MBSniffer v3.2/mb_diagnostics.py — lines 201-240

```
            return "success", translate_text(label, language), translate_text(colour, language)

        label, colour = RESULT_LABELS["pending"]
        return "pending", translate_text(label, language), translate_text(colour, language)

    label, colour = RESULT_LABELS["neutral"]
    return "neutral", translate_text(label, language), translate_text(colour, language)


def inspector_values(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
    if not record:
        return {
            "slave": "Slave: —",
            "type": f"{translate_text('Tipo', language)}: —",
            "function": "Function: —",
            "response": "Resp.: —",
            "result": f"{translate_text('Resultado', language)}: —",
            "result_key": "neutral",
            "pdu": "PDU Address: —",
            "one_based": "1-based: —",
            "qty": "Qty: —",
            "byte_count": "ByteCount: —",
            "crc": "CRC: —",
            "data": f"{translate_text('Dados', language)}: —",
            "raw": "",
        }

    received, calculated, crc_state = crc_fields(record)
    if received is None:
        crc_text = f"CRC: {crc_state or '—'}"
    else:
        crc_text = (
            (f"CRC: received 0x{received:04X} / calculated 0x{calculated:04X} — {crc_state}" if language == "English" else f"CRC: recebido 0x{received:04X} / calculado 0x{calcul
ated:04X} — {crc_state}")
        )

    byte_count, data_hex, decoded = response_payload(record, language=language)
    payload = decoded or data_hex or str(record.get("details", "") or "—")

    response = str(record.get("response_text", "") or "")
    if response:
```

### MBSniffer v3.2/mb_diagnostics.py — lines 241-280

```
        response += " ms"

    timeout_suffix = " — TIMEOUT" if record.get("timed_out") else ""
    result_key, result_label, _result_colour = result_info(
        record,
        successful_request=successful_request,
        language=language,
    )

    return {
        "slave": f"Slave: {record.get('slave') or '—'}",
        "type": f"{translate_text('Tipo', language)}: {record.get('type') or '—'}{timeout_suffix}",
        "function": f"Function: {function_label(record, language=language)}",
        "response": f"Resp.: {response or '—'}",
        "result": f"{translate_text('Resultado', language)}: {result_label}",
        "result_key": result_key,
        "pdu": f"PDU Address: {record.get('pdu_address') if record.get('pdu_address') is not None else '—'}",
        "one_based": f"1-based: {record.get('one_based') if record.get('one_based') is not None else '—'}",
        "qty": f"Qty: {record.get('qty') if record.get('qty') is not None else '—'}",
        "byte_count": f"ByteCount: {byte_count if byte_count is not None else '—'}",
        "crc": crc_text,
        "data": f"{translate_text('Dados', language)}: {translate_details(payload, language)}",
        "raw": str(record.get("raw", "") or ""),
    }


def decoded_record_text(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
    values = inspector_values(
        record,
        successful_request=successful_request,
        language=language,
    )
    return "\n".join((
        values["slave"],
        values["type"],
        values["function"],
        values["response"],
        values["result"],
        values["pdu"],
        values["one_based"],
```

**MBSniffer v3.2/mb_diagnostics.py — lines 281-286**

```
        values["qty"],
        values["byte_count"],
        values["crc"],
        values["data"],
        f"Raw: {values['raw']}",
    ))
```

# MBSniffer v3.2/mb_view.py

1871 source lines.

### MBSniffer v3.2/mb_view.py — lines 1-40

```
#!/usr/bin/env python3
"""Traffic rendering, filtering, Raw Hex view, statistics and log handling."""

import csv
import math
import os
import subprocess
import sys
import time
from datetime import datetime

import tkinter as tk
from tkinter import filedialog, messagebox

from mb_config import (
    MAX_UI_FRAMES,
    MAX_RAW_TEXT_LINES,
    UI_PRUNE_CHUNK_FRAMES,
    RAW_TEXT_PRUNE_CHUNK_LINES,
    SLOW_RESPONSE_THRESHOLD_MS,
    get_application_directory,
    get_logs_folder,
)
from mb_diagnostics import (
    anomaly_kind,
    decoded_record_text,
    inspector_values,
)
from mb_i18n import canonical_ui_text, translate_details
from mb_protocol import read_bits_details

# Index of "Abrir no Master" in the Traffic context menu (built in mb_gui.py).
TRAFFIC_MENU_OPEN_IN_MASTER = 8


class ViewMixin:
    """Methods for rendering, filtering, statistics and logging."""

    def reset_session_metrics(self):
        self.metrics.reset()
```

### MBSniffer v3.2/mb_view.py — lines 41-80

```
41          self.update_stats_labels()
42
43      def update_stats_labels(self):
44          self.stat_requests_var.set(f"{self.tr('Pedidos')}: {self.metrics.requests}")
45          self.stat_responses_var.set(f"{self.tr('Respostas')}: {self.metrics.responses}")
46          self.stat_pending_var.set(f"{self.tr('Pendentes')}: {self.metrics.pending_count()}")
47          self.stat_timeouts_var.set(f"Timeouts: {self.metrics.timeouts}")
48          self.stat_crc_var.set(f"{self.tr('Erros de CRC')}: {self.metrics.crc_errors}")
49          self.stat_exceptions_var.set(f"{self.tr('Exceções')}: {self.metrics.exceptions}")
50          self.stat_raw_var.set(f"RAW: {self.metrics.raw}")
51
52          try:
53              health_now = (
54                  time.time()
55                  if getattr(self, "capture_active", False) or getattr(self, "busy", False)
56                  else self.metrics.last_event_time
57              )
58              summary = self.metrics.health_summary(
59                  baud=self.baud_var.get(),
60                  data_bits=self.data_var.get(),
61                  parity=self.parity_var.get(),
62                  stop_bits=self.stop_var.get(),
63                  now=health_now,
64              )
65          except Exception:
66              summary = self.metrics.health_summary()
67
68          def ms(value):
69              return "–" if value is None else f"{value:.1f} ms"
70
71          if hasattr(self, "stat_slaves_var"):
72              self.stat_slaves_var.set(f"Slaves: {summary['active_slaves']}")
73
74          mappings = {
75              "health_requests_var": str(self.metrics.requests),
76              "health_responses_var": str(self.metrics.responses),
77              "health_crc_count_var": str(self.metrics.crc_errors),
78              "health_timeout_count_var": str(self.metrics.timeouts),
79              "health_exception_count_var": str(self.metrics.exceptions),
80              "health_req_rate_var": f"{summary['request_rate']:.2f} req/s",
```

**MBSniffer v3.2/mb_view.py — lines 81-120**

```
            "health_resp_avg_var": ms(summary["response_avg_ms"]),
            "health_resp_min_var": ms(summary["response_min_ms"]),
            "health_resp_max_var": ms(summary["response_max_ms"]),
            "health_p95_var": ms(summary["response_p95_ms"]),
            "health_nonvalidated_rate_var": f"{summary['nonvalidated_rate_pct']:.2f} %",
            "health_timeout_rate_var": f"{summary['timeout_rate_pct']:.2f} %",
            "health_bus_load_var": f"{summary['bus_load_pct']:.2f} %",
            "health_slaves_var": str(summary["active_slaves"]),
            "health_total_bytes_var": str(summary["total_bytes"]),
            "health_exception_slaves_var": summary["exception_summary"],
        }

        slowest_slave = summary["slowest_slave"]
        if slowest_slave is None:
            mappings["health_slowest_var"] = "—"
        else:
            mappings["health_slowest_var"] = (
                self.trf(
                    "Slave {slave} — média {average:.1f} ms",
                    slave=slowest_slave,
                    average=summary["slowest_avg_ms"],
                )
            )

        for attr, value in mappings.items():
            var = getattr(self, attr, None)
            if var is not None:
                var.set(value)

    # ---------------------------------------------------------------------
    # Bus activity, filters, anomaly highlighting and Frame Inspector.
    # ---------------------------------------------------------------------
    def pulse_bus_activity(self):
        var = getattr(self, "bus_activity_var", None)
        if var is None:
            return

        var.set("BUS ● RX")
        previous = getattr(self, "_bus_activity_after_id", None)
        if previous is not None:
```

### MBSniffer v3.2/mb_view.py — lines 121-160

```
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass

        try:
            self._bus_activity_after_id = self.after(
                180,
                self._clear_bus_activity_pulse,
            )
        except tk.TclError:
            self._bus_activity_after_id = None

    def _clear_bus_activity_pulse(self):
        self._bus_activity_after_id = None
        var = getattr(self, "bus_activity_var", None)
        if var is not None:
            var.set("BUS ○")

    def schedule_advanced_filter_refresh(self, *_args):
        previous = getattr(self, "_advanced_filter_after_id", None)
        if previous is not None:
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass
        try:
            self._advanced_filter_after_id = self.after(
                120,
                self.render_filtered_view,
            )
        except tk.TclError:
            self._advanced_filter_after_id = None

    def clear_advanced_filters(self):
        pending = getattr(self, "_advanced_filter_after_id", None)
        if pending is not None:
            try:
                self.after_cancel(pending)
            except tk.TclError:
```

### MBSniffer v3.2/mb_view.py — lines 161-200

```
161                 pass
162             self._advanced_filter_after_id = None
163
164         self.advanced_filter_type_var.set(self.tr("Todos"))
165         self.advanced_filter_fc_var.set("")
166         self.advanced_filter_response_ms_var.set("")
167         self.advanced_filter_text_var.set("")
168         self._transaction_filter_seqs = None
169
170         self.slave_filter_all_mode = True
171         for var in self.slave_filter_vars.values():
172             var.set(True)
173         self.update_slave_heading()
174
175         self.render_filtered_view()
176
177     @staticmethod
178     def normalized_fc_filter(value):
179         text = str(value or "").strip().upper()
180         if not text or text == "TODOS":
181             return ""
182         if text.startswith("0X"):
183             text = text[2:]
184         try:
185             number = int(text, 16)
186         except ValueError:
187             return "__INVALID__"
188         if not 0 <= number <= 0xFF:
189             return "__INVALID__"
190         return f"{number:02X}"
191
192     def record_passes_advanced_filters(self, record):
193         transaction_filter = getattr(self, "_transaction_filter_seqs", None)
194         if transaction_filter is not None:
195             if record.get("seq") not in transaction_filter:
196                 return False
197
198         kind = canonical_ui_text(self.advanced_filter_type_var.get())
199         row_type = str(record.get("type", ""))
200
```

### MBSniffer v3.2/mb_view.py — lines 201-240

```
201         if kind == "Requests" and row_type != "REQUEST":
202             return False
203         if kind == "Responses" and row_type != "RESPONSE":
204             return False
205         if kind == "Exceptions" and row_type != "EXCEPTION":
206             return False
207         if kind == "CRC errors" and record.get("crc") != "ERROR":
208             return False
209         if kind == "Timeouts" and not record.get("timed_out"):
210             return False
211         if kind == "RAW" and not row_type.startswith("RAW"):
212             return False
213 
214         fc_filter = self.normalized_fc_filter(
215             self.advanced_filter_fc_var.get()
216         )
217         if fc_filter == "__INVALID__":
218             return False
219         if fc_filter:
220             record_fc_text = str(record.get("fc", "")).upper()
221             candidates = {record_fc_text}
222             if str(record.get("type", "")) == "EXCEPTION":
223                 try:
224                     candidates.add(f"{int(record_fc_text, 16) & 0x7F:02X}")
225                 except ValueError:
226                     pass
227             if fc_filter not in candidates:
228                 return False
229 
230         response_limit = self.advanced_filter_response_ms_var.get().strip()
231         if response_limit:
232             try:
233                 minimum_ms = float(response_limit.replace(",", "."))
234                 response_ms = float(record.get("response_text", ""))
235             except (TypeError, ValueError):
236                 return False
237             if response_ms <= minimum_ms:
238                 return False
239 
240         query = self.advanced_filter_text_var.get().strip().casefold()
```

### MBSniffer v3.2/mb_view.py — lines 241-280

```
241         if query:
242             haystack = " ".join((
243                 str(record.get("details", "")),
244                 str(record.get("raw", "")),
245                 str(record.get("slave", "")),
246                 str(record.get("fc", "")),
247                 str(record.get("type", "")),
248                 str(record.get("channel", "")),
249             )).casefold()
250             if query not in haystack:
251                 return False
252 
253         return True
254 
255     def configure_anomaly_tags(self, dark=None):
256         if not hasattr(self, "tree"):
257             return
258 
259         if dark is None:
260             dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
261 
262         if dark:
263             palette = {
264                 "result_success": ("#20372a", "#a7e4b8"),
265                 "anomaly_exception": ("#3b2728", "#ffc0c5"),
266                 "anomaly_crc": ("#482427", "#ffb3ba"),
267                 "anomaly_timeout": ("#42351f", "#ffd27a"),
268                 "anomaly_slow": ("#3b3422", "#ffe19a"),
269                 "anomaly_raw": ("#30283c", "#d8c5ff"),
270             }
271         else:
272             palette = {
273                 "result_success": ("#e4f4e9", "#1f6b3a"),
274                 "anomaly_exception": ("#ffe8e9", "#8c1d24"),
275                 "anomaly_crc": ("#ffdfe1", "#8c1d24"),
276                 "anomaly_timeout": ("#fff0cf", "#774b00"),
277                 "anomaly_slow": ("#fff6dd", "#6b5200"),
278                 "anomaly_raw": ("#f0e9ff", "#5d3d8f"),
279             }
280 
```

### MBSniffer v3.2/mb_view.py — lines 281-320

```
281         for tag, (background, foreground) in palette.items():
282             self.tree.tag_configure(
283                 tag,
284                 background=background,
285                 foreground=foreground,
286             )
287 
288     def anomaly_tag_for_record(self, record):
289         """
290         Return the optional result-highlight tag for one logical traffic row.
291 
292         "Realçar resultados" is a presentation option only:
293         - diagnostic outcomes retain their existing anomaly colors;
294         - a successfully completed normal REQUEST/RESPONSE transaction is green;
295         - unmatched/pending requests remain neutral.
296         """
297         if not getattr(self, "highlight_anomalies_var", None):
298             return ""
299         if not self.highlight_anomalies_var.get():
300             return ""
301 
302         kind = anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS)
303         if kind:
304             return f"anomaly_{kind}"
305 
306         row_type = str(record.get("type", ""))
307 
308         # A normal, CRC-valid matched RESPONSE is a successful result.
309         if (
310             row_type == "RESPONSE"
311             and record.get("crc") == "OK"
312             and record.get("matched")
313         ):
314             return "result_success"
315 
316         # Once the normal response has arrived, also mark its REQUEST so the
317         # complete successful transaction pair reads as one result.
318         if (
319             row_type == "REQUEST"
320             and record.get("seq") in self._successful_request_seqs
```

### MBSniffer v3.2/mb_view.py — lines 321-360

```
            and not record.get("timed_out")
        ):
            return "result_success"

        return ""

    def on_highlight_anomalies_changed(self):
        self.configure_anomaly_tags()
        self.configure_inspector_result_styles()
        self.render_filtered_view()
        self.update_frame_inspector(self.selected_record())
        saver = getattr(self, "_schedule_ui_settings_save", None)
        if callable(saver):
            saver()

    def mark_timed_out_records(self, expired, rerender=True):
        if not expired:
            return 0

        seqs = {
            item.get("seq")
            for item in expired
            if item.get("seq") is not None
        }
        if not seqs:
            return 0

        changed_records = []
        for record in self._frame_history:
            if record.get("seq") in seqs and not record.get("timed_out"):
                record["timed_out"] = True
                changed_records.append(record)

        if changed_records and rerender:
            # The "Timeouts" filter changes which rows are visible, so only
            # then is a full rebuild needed. Otherwise a timeout changes just
            # the row colour: retag the already visible rows in place.
            if canonical_ui_text(self.advanced_filter_type_var.get()) == "Timeouts":
                self.render_filtered_view()
            else:
```

### MBSniffer v3.2/mb_view.py — lines 361-400

```
361                 self.refresh_record_tags(changed_records)
362         return len(changed_records)
363
364     def refresh_record_tags(self, records):
365         """Re-apply the result tag of visible rows without rebuilding the table."""
366         for record in records:
367             row_tag = self.anomaly_tag_for_record(record)
368             for iid in self._tree_iids_by_seq.get(record.get("seq"), []):
369                 try:
370                     self.tree.item(iid, tags=((row_tag,) if row_tag else ()))
371                 except tk.TclError:
372                     pass
373
374         selected = self.selected_record()
375         if selected is not None and any(selected is record for record in records):
376             self.update_frame_inspector(selected)
377
378     def selected_record(self):
379         selection = self.tree.selection()
380         if not selection:
381             return None
382         return self._record_by_iid.get(selection[0])
383
384     def update_frame_inspector(self, record=None):
385         if not hasattr(self, "inspector_vars"):
386             return
387
388         successful_request = bool(
389             record
390             and record.get("type") == "REQUEST"
391             and record.get("seq") in self._successful_request_seqs
392         )
393         values = inspector_values(
394             record,
395             successful_request=successful_request,
396             language=self.current_language,
397         )
398         for key, var in self.inspector_vars.items():
399             if key in values:
400                 var.set(values[key])
```

### MBSniffer v3.2/mb_view.py — lines 401-440

```
401         self.selected_raw_var.set(values.get("raw", ""))
402 
403         result_label = getattr(self, "inspector_result_label", None)
404         if result_label is not None:
405             style_key = values.get("result_key", "neutral")
406             if not self.highlight_anomalies_var.get():
407                 style_key = "neutral"
408             try:
409                 result_label.configure(
410                     style=f"MBS.Result.{style_key}.TLabel"
411                 )
412             except tk.TclError:
413                 pass
414 
415     def toggle_frame_inspector(self):
416         if getattr(self, "inspector_collapsed", False):
417             self.inspector_body.grid()
418             self.inspector_collapsed = False
419             self.inspector_toggle_btn.configure(text=self.tr("Recolher"))
420         else:
421             self.inspector_body.grid_remove()
422             self.inspector_collapsed = True
423             self.inspector_toggle_btn.configure(text=self.tr("Expandir"))
424 
425         scheduler = getattr(self, "_schedule_bus_health_layout_sync", None)
426         if callable(scheduler):
427             scheduler()
428 
429     def copy_to_clipboard(self, text):
430         try:
431             self.clipboard_clear()
432             self.clipboard_append(str(text))
433             self.update_idletasks()
434         except tk.TclError:
435             pass
436 
437     def copy_selected_raw(self):
438         record = self.selected_record()
439         if record:
440             self.copy_to_clipboard(record.get("raw", ""))
```

### MBSniffer v3.2/mb_view.py — lines 441-480

```
441
442     def copy_selected_decoded(self):
443         record = self.selected_record()
444         if record:
445             successful_request = bool(
446                 record.get("type") == "REQUEST"
447                 and record.get("seq") in self._successful_request_seqs
448             )
449             self.copy_to_clipboard(
450                 decoded_record_text(
451                     record,
452                     successful_request=successful_request,
453                     language=self.current_language,
454                 )
455             )
456
457     def context_filter_selected_slave(self):
458         record = self.selected_record()
459         if not record:
460             return
461         slave = str(record.get("slave", "") or "")
462         if not slave or slave not in self.slave_filter_vars:
463             return
464
465         self._transaction_filter_seqs = None
466         self.slave_filter_all_mode = False
467         for key, var in self.slave_filter_vars.items():
468             var.set(key == slave)
469         self.update_slave_heading()
470         self.render_filtered_view()
471
472     def context_filter_selected_fc(self):
473         record = self.selected_record()
474         if not record:
475             return
476         fc = str(record.get("fc", "") or "")
477         if not fc:
478             return
479         if str(record.get("type", "")) == "EXCEPTION":
480             try:
```

### MBSniffer v3.2/mb_view.py — lines 481-520

```
481                 fc = f"{int(fc, 16) & 0x7F:02X}"
482             except ValueError:
483                 pass
484         self.advanced_filter_fc_var.set(fc)
485         self._transaction_filter_seqs = None
486         self.render_filtered_view()
487 
488     def context_show_selected_transaction(self):
489         record = self.selected_record()
490         if not record:
491             return
492 
493         seq = record.get("seq")
494         seqs = {seq}
495 
496         if record.get("request_seq") is not None:
497             seqs.add(record["request_seq"])
498         else:
499             for candidate in self._frame_history:
500                 if candidate.get("request_seq") == seq:
501                     seqs.add(candidate.get("seq"))
502                     break
503 
504         self.slave_filter_all_mode = True
505         for var in self.slave_filter_vars.values():
506             var.set(True)
507         self.update_slave_heading()
508 
509         self.advanced_filter_type_var.set(self.tr("Todos"))
510         self.advanced_filter_fc_var.set("")
511         self.advanced_filter_response_ms_var.set("")
512         self.advanced_filter_text_var.set("")
513 
514         self._transaction_filter_seqs = {
515             value for value in seqs if value is not None
516         }
517         self.render_filtered_view()
518 
519     def clear_transaction_filter(self):
520         self._transaction_filter_seqs = None
```

### MBSniffer v3.2/mb_view.py — lines 521-560

```
521         self.render_filtered_view()
522
523     def context_open_selected_in_master(self):
524         record = self.selected_record()
525         if record is not None:
526             self.master_open_from_record(record)
527
528     def show_traffic_context_menu(self, event):
529         row_iid = self.tree.identify_row(event.y)
530         if not row_iid or row_iid not in self._record_by_iid:
531             return
532
533         self.tree.selection_set(row_iid)
534         self.tree.focus(row_iid)
535         self.on_row_select()
536
537         opener = getattr(self, "master_definition_from_record", None)
538         if callable(opener):
539             state = "normal" if opener(self._record_by_iid[row_iid]) else "disabled"
540             try:
541                 self.traffic_context_menu.entryconfigure(TRAFFIC_MENU_OPEN_IN_MASTER,
542                                                          state=state)
543             except tk.TclError:
544                 pass
545
546         try:
547             self.traffic_context_menu.tk_popup(event.x_root, event.y_root)
548         finally:
549             try:
550                 self.traffic_context_menu.grab_release()
551             except tk.TclError:
552                 pass
553
554     def export_session_csv(self, path=None):
555         interactive = path is None
556         records = list(self._frame_history)
557         if not records:
558             if path is None:
559                 messagebox.showinfo(
560                     self.tr("Exportar CSV"),
```

### MBSniffer v3.2/mb_view.py — lines 561-600

```
561                     self.tr("Não existem frames na sessão atual para exportar."),
562                 )
563             return None
564 
565         if path is None:
566             filename = datetime.now().strftime(
567                 "MBSniffer_session_%Y%m%d_%H%M%S.csv"
568             )
569             path = filedialog.asksaveasfilename(
570                 title=self.tr("Exportar sessão para CSV"),
571                 defaultextension=".csv",
572                 initialfile=filename,
573                 filetypes=[("CSV", "*.csv"), (self.tr("Todos os ficheiros"), "*.*")],
574             )
575             if not path:
576                 return None
577 
578         fieldnames = [
579             "Time",
580             "Delta ms",
581             "Response ms",
582             "Channel",
583             "Type",
584             "Slave",
585             "FC",
586             "Details",
587             "CRC",
588             "Raw",
589             "PDU Address",
590             "1-based Address",
591             "Qty",
592             "Byte Count",
593             "Timed out",
594             "Anomaly",
595         ]
596 
597         try:
598             with open(path, "w", newline="", encoding="utf-8-sig") as handle:
599                 writer = csv.DictWriter(handle, fieldnames=fieldnames)
600                 writer.writeheader()
```

**MBSniffer v3.2/mb_view.py — lines 601-640**

```
601
602                 for record in records:
603                     inspector = inspector_values(record, language=self.current_language)
604                     byte_count_text = inspector["byte_count"].split(
605                         ":", 1
606                     )[-1].strip()
607                     writer.writerow({
608                         "Time": record.get("time", ""),
609                         "Delta ms": record.get("delta_text", ""),
610                         "Response ms": record.get("response_text", ""),
611                         "Channel": record.get("channel", ""),
612                         "Type": record.get("type", ""),
613                         "Slave": record.get("slave", ""),
614                         "FC": record.get("fc", ""),
615                         "Details": translate_details(record.get("details", ""), self.current_language),
616                         "CRC": record.get("crc", ""),
617                         "Raw": record.get("raw", ""),
618                         "PDU Address": (
619                             "" if record.get("pdu_address") is None
620                             else record.get("pdu_address")
621                         ),
622                         "1-based Address": (
623                             "" if record.get("one_based") is None
624                             else record.get("one_based")
625                         ),
626                         "Qty": (
627                             "" if record.get("qty") is None
628                             else record.get("qty")
629                         ),
630                         "Byte Count": (
631                             "" if byte_count_text == "—"
632                             else byte_count_text
633                         ),
634                         "Timed out": "Yes" if record.get("timed_out") else "No",
635                         "Anomaly": anomaly_kind(
636                             record,
637                             SLOW_RESPONSE_THRESHOLD_MS,
638                         ) or "",
639                     })
640
```

### MBSniffer v3.2/mb_view.py — lines 641-680

```
641             if interactive:
642                 try:
643                     messagebox.showinfo(
644                         self.tr("Exportar CSV"),
645                         self.trf("Sessão exportada para:\n\n{path}", path=path),
646                     )
647                 except tk.TclError:
648                     pass
649             return path
650 
651         except OSError as exc:
652             messagebox.showerror(
653                 self.tr("Exportar CSV"),
654                 self.trf("Não foi possível guardar o CSV:\n\n{error}", error=exc),
655             )
656             return None
657 
658     def tree_record_column_values(self, record):
659         return {
660             "time": record.get("time", ""),
661             "delta": record.get("delta_text", ""),
662             "response": record.get("response_text", ""),
663             "channel": record.get("channel", ""),
664             "type": record.get("type", ""),
665             "slave": record.get("slave", ""),
666             "fc": record.get("fc", ""),
667             "details": translate_details(record.get("details", ""), self.current_language),
668             "crc": record.get("crc", ""),
669         }
670 
671     def measure_tree_column_width(self, column, records, heading_text=None):
672         """
673         Measure the pixel width required by a non-wrapped Treeview column.
674 
675         Details is handled separately because it uses the remaining viewport
676         width and wraps instead of expanding horizontally without limit.
677         """
678         heading = (
679             heading_text
680             if heading_text is not None
```

**MBSniffer v3.2/mb_view.py — lines 681-720**

```
            else self.tree_headings.get(column, column)
        )
        max_px = self.tree_heading_font.measure(str(heading))

        for record in records:
            value = self.tree_record_column_values(record).get(column, "")
            max_px = max(max_px, self.tree_cell_font.measure(str(value)))

        return max_px + self.tree_column_padding_px

    def available_details_width(self):
        """
        Return the width that can be assigned to Details without intentionally
        forcing horizontal scrolling in the current Treeview viewport.
        """
        tree_width = int(self.tree.winfo_width())
        if tree_width <= 20:
            tree_width = 1100  # sensible pre-layout fallback

        other_total = 0
        for column in self.tree["columns"]:
            if column == "details":
                continue
            other_total += int(self.tree.column(column, "width"))

        return max(
            self.tree_detail_min_width_px,
            tree_width - other_total - self.tree_detail_border_reserve_px
        )

    def wrap_text_to_pixels(self, value, max_px):
        """
        Word-wrap text using the actual Tk font width.

        Long individual tokens are split character-by-character as a fallback,
        so a cell can never force the Details column wider than its target.
        """
        text = str(value or "")
        if not text:
            return ""
```

**MBSniffer v3.2/mb_view.py — lines 721-760**

```
721
722         max_px = max(40, int(max_px))
723         paragraphs = text.splitlines() or [text]
724         output_lines = []
725
726         for paragraph in paragraphs:
727             words = paragraph.split()
728             if not words:
729                 output_lines.append("")
730                 continue
731
732             line = ""
733             for word in words:
734                 candidate = word if not line else f"{line} {word}"
735                 if self.tree_cell_font.measure(candidate) <= max_px:
736                     line = candidate
737                     continue
738
739                 if line:
740                     output_lines.append(line)
741                     line = ""
742
743                 # The token itself may be wider than the available space.
744                 if self.tree_cell_font.measure(word) <= max_px:
745                     line = word
746                     continue
747
748                 chunk = ""
749                 for char in word:
750                     candidate_chunk = chunk + char
751                     if chunk and self.tree_cell_font.measure(candidate_chunk) > max_px:
752                         output_lines.append(chunk)
753                         chunk = char
754                     else:
755                         chunk = candidate_chunk
756                 line = chunk
757
758             if line:
759                 output_lines.append(line)
760
```

**MBSniffer v3.2/mb_view.py — lines 761-800**

```
        return "\n".join(output_lines)

    def details_horizontal_rule(self):
        """
        Return a rule that fills the Details column as far as the Treeview cell
        permits. A slight overdraw is intentional; Treeview clips it at the
        column boundary.
        """
        details_width = max(
            40,
            int(self.tree.column("details", "width"))
        )
        char_px = max(1, self.tree_cell_font.measure("—"))
        count = max(8, int(math.ceil(details_width / char_px)) + 4)
        return "—" * count

    def completed_pairs_for_records(self, records):
        """
        Return complete visible REQUEST/RESPONSE pairs in current display order.

        Each matched RESPONSE stores the exact request sequence id. This allows
        correct boundaries in both Hora ↑ and Hora ↓ without relying on row
        adjacency alone.
        """
        seq_to_index = {
            record.get("seq"): index
            for index, record in enumerate(records)
        }

        pairs = []
        for response in records:
            request_seq = response.get("request_seq")
            response_seq = response.get("seq")

            if request_seq is None or request_seq not in seq_to_index:
                continue

            request_index = seq_to_index[request_seq]
            response_index = seq_to_index.get(response_seq)
            if response_index is None:
```

**MBSniffer v3.2/mb_view.py — lines 801-840**

```
                continue

            start_index = min(request_index, response_index)
            end_index = max(request_index, response_index)

            pairs.append({
                "request_seq": request_seq,
                "response_seq": response_seq,
                "start_index": start_index,
                "end_index": end_index,
                # Separator belongs after whichever logical frame is visually
                # the end of the pair.
                "end_seq": records[end_index].get("seq"),
            })

        pairs.sort(key=lambda item: item["start_index"])
        return pairs

    def separator_after_sequences(self, records):
        """
        Return logical frame seq ids that need a separator AFTER them.

        There is intentionally no separator after the final complete visible
        pair because there is nothing below it to divide.
        """
        if not self.separate_transactions_var.get():
            return set()

        pairs = self.completed_pairs_for_records(records)
        if len(pairs) < 2:
            return set()

        return {
            pair["end_seq"]
            for pair in pairs[:-1]
        }

    def tree_separator_values(self):
        return (
            "", "", "", "", "", "", "",
```

**MBSniffer v3.2/mb_view.py — lines 841-880**

```
            self.details_horizontal_rule(),
            ""
        )

    def insert_tree_separator_after_seq(self, seq):
        """Insert one Details-width separator after the visual rows of seq."""
        if seq in self._tree_separator_iid_by_seq:
            return

        iids = self._tree_iids_by_seq.get(seq, [])
        if not iids:
            return

        last_iid = iids[-1]
        insert_at = self.tree.index(last_iid) + 1

        iid = self.tree.insert(
            "",
            insert_at,
            values=self.tree_separator_values()
        )
        self._raw_by_iid[iid] = ""
        self._tree_separator_iid_by_seq[seq] = iid

    def sync_tree_transaction_separators(self, records=None):
        """
        Synchronize standalone transaction separator rows without rebuilding
        normal traffic rows.
        """
        if records is None:
            records = self.records_for_display()

        desired = self.separator_after_sequences(records)
        existing = set(self._tree_separator_iid_by_seq)

        # Remove boundaries that are no longer valid (filter/order changes).
        for seq in existing - desired:
            iid = self._tree_separator_iid_by_seq.pop(seq, None)
            if iid and self.tree.exists(iid):
                self.tree.delete(iid)
```

**MBSniffer v3.2/mb_view.py — lines 881-920**

```
881             self._raw_by_iid.pop(iid, None)
882 
883         # Add newly required boundaries.
884         for seq in desired - existing:
885             self.insert_tree_separator_after_seq(seq)
886 
887     def detail_lines_for_record(self, record):
888         """
889         Return wrapped Details as separate visual Treeview rows.
890 
891         ttk.Treeview has one global row height for the entire widget. Putting
892         newline-wrapped text in one item would therefore make every REQUEST and
893         RESPONSE row as tall as the largest Details message. Continuation rows
894         preserve wrapping while keeping normal rows compact.
895         """
896         content_width = max(
897             40,
898             int(self.tree.column("details", "width"))
899             - self.tree_column_padding_px
900         )
901         wrapped = self.wrap_text_to_pixels(
902             translate_details(record.get("details", ""), self.current_language),
903             content_width
904         )
905 
906         if not wrapped:
907             return [""]
908 
909         return wrapped.splitlines()
910 
911     def set_tree_rowheight_for_records(self, records):
912         """Treeview row height stays fixed; wrapping uses continuation rows."""
913         self.tree_style.configure(
914             self.tree_style_name,
915             rowheight=self.tree_single_line_rowheight
916         )
917 
918     def update_tree_rowheight_for_record(self, record):
919         """No-op by design: one logical frame may use several compact rows."""
920         return
```

**MBSniffer v3.2/mb_view.py — lines 921-960**

```
921
922     def autofit_tree_columns(self, records=None):
923         """
924         Non-Details columns fit the longest visible value/title.
925
926         Details deliberately behaves differently: it consumes the remaining
927         viewport width and wraps. This keeps the full row information visible
928         without letting long register lists create an extremely wide table.
929         """
930         if records is None:
931             records = self.records_for_display()
932
933         # First compact every non-Details column.
934         for column in self.tree["columns"]:
935             if column == "details":
936                 continue
937
938             width = self.measure_tree_column_width(column, records)
939             self.tree.column(
940                 column,
941                 width=width,
942                 minwidth=width,
943                 anchor="center",
944                 stretch=False
945             )
946
947         # Then allocate the remaining viewport to Details.
948         heading_min = self.measure_tree_column_width(
949             "details",
950             [],
951             heading_text=self.tree_headings["details"]
952         )
953         details_width = max(
954             heading_min,
955             self.available_details_width()
956         )
957         self.tree.column(
958             "details",
959             width=details_width,
960             minwidth=heading_min,
```

**MBSniffer v3.2/mb_view.py — lines 961-1000**

```
            anchor="w",
            stretch=True
        )

        self.set_tree_rowheight_for_records(records)

    def expand_tree_columns_for_record(self, record):
        """
        Incrementally enlarge only the compact non-Details columns.

        Returns True when another column became wider. If that happens the
        remaining Details width changes, so the caller can rebuild/re-wrap the
        visible table once for the batch.
        """
        values = self.tree_record_column_values(record)
        layout_changed = False

        for column in self.tree["columns"]:
            if column == "details":
                continue

            heading_px = self.tree_heading_font.measure(
                str(self.tree_headings.get(column, column))
            )
            value_px = self.tree_cell_font.measure(str(values.get(column, "")))
            required = max(heading_px, value_px) + self.tree_column_padding_px
            current = int(self.tree.column(column, "width"))

            if required > current:
                self.tree.column(
                    column,
                    width=required,
                    minwidth=required,
                    anchor="center",
                    stretch=False
                )
                layout_changed = True

        if layout_changed:
            heading_min = self.measure_tree_column_width(
```

### MBSniffer v3.2/mb_view.py — lines 1001-1040

```
1001                "details",
1002                [],
1003                heading_text=self.tree_headings["details"]
1004            )
1005            details_width = max(
1006                heading_min,
1007                self.available_details_width()
1008            )
1009            self.tree.column(
1010                "details",
1011                width=details_width,
1012                minwidth=heading_min,
1013                anchor="w",
1014                stretch=True
1015            )
1016
1017        return layout_changed
1018
1019    def refresh_tree_heading_width(self, column):
1020        """
1021        Dynamic Hora/Slave headings remain compact. Details is intentionally
1022        excluded because it is viewport-driven and wrapped.
1023        """
1024        if column == "details":
1025            return
1026
1027        records = self.records_for_display()
1028        width = self.measure_tree_column_width(column, records)
1029        self.tree.column(
1030            column,
1031            width=width,
1032            minwidth=width,
1033            anchor="center",
1034            stretch=False
1035        )
1036
1037    def schedule_tree_reflow(self, _event=None):
1038        """
1039        Debounce window/Treeview resizing and then re-wrap Details to the new
1040        available width.
```

### MBSniffer v3.2/mb_view.py — lines 1041-1080

```
1041        """
1042        if self._tree_reflow_after_id is not None:
1043            try:
1044                self.after_cancel(self._tree_reflow_after_id)
1045            except Exception:
1046                pass
1047
1048        self._tree_reflow_after_id = self.after(
1049            120,
1050            self.reflow_tree_after_resize
1051        )
1052
1053    def reflow_tree_after_resize(self):
1054        self._tree_reflow_after_id = None
1055        if not hasattr(self, "tree"):
1056            return
1057        self.render_filtered_view()
1058
1059    def _slave_sort_key(self, slave):
1060        try:
1061            return (0, int(slave))
1062        except (TypeError, ValueError):
1063            return (1, str(slave))
1064
1065    def rebuild_slave_filter_menu(self):
1066        self.slave_filter_menu.delete(0, "end")
1067
1068        self.slave_filter_menu.add_command(
1069            label=self.tr("Todos"),
1070            command=self.select_all_slaves
1071        )
1072        self.slave_filter_menu.add_command(
1073            label=self.tr("Nenhum"),
1074            command=self.select_no_slaves
1075        )
1076
1077        if self.slave_filter_vars:
1078            self.slave_filter_menu.add_separator()
1079            for slave in sorted(self.slave_filter_vars, key=self._slave_sort_key):
1080                self.slave_filter_menu.add_checkbutton(
```

### MBSniffer v3.2/mb_view.py — lines 1081-1120

```
                    label=f"Slave {slave}",
                    variable=self.slave_filter_vars[slave],
                    command=self.on_slave_filter_changed
                )
        else:
            self.slave_filter_menu.add_separator()
            self.slave_filter_menu.add_command(
                label=self.tr("Nenhum slave detetado"),
                state="disabled"
            )

        self.update_slave_heading()

    def update_slave_heading(self):
        if self.slave_filter_all_mode:
            label = "Slave ▾"
        else:
            selected = [
                slave for slave, var in self.slave_filter_vars.items()
                if var.get()
            ]
            label = (self.tr("Slave (Filtro) ▾") if selected else self.tr("Slave (Nenhum) ▾"))

        self.tree_headings["slave"] = label
        self.tree.heading(
            "slave",
            text=label,
            anchor="center",
            command=self.show_slave_filter_menu
        )
        self.refresh_tree_heading_width("slave")

    def show_slave_filter_menu(self):
        """
        Open the slave filter menu at the position where the Slave heading
        was clicked.
        """
        try:
            x = self.winfo_pointerx()
            y = self.winfo_pointery() + 8
```

### MBSniffer v3.2/mb_view.py — lines 1121-1160

```
1121            self.slave_filter_menu.tk_popup(x, y)
1122        finally:
1123            try:
1124                self.slave_filter_menu.grab_release()
1125            except tk.TclError:
1126                pass
1127
1128    def register_slave_for_filter(self, slave):
1129        slave = str(slave or "").strip()
1130        if not slave:
1131            return
1132
1133        if slave in self.slave_filter_vars:
1134            return
1135
1136        self.slave_filter_vars[slave] = tk.BooleanVar(
1137            value=self.slave_filter_all_mode
1138        )
1139        self.rebuild_slave_filter_menu()
1140
1141    def select_all_slaves(self):
1142        self.slave_filter_all_mode = True
1143        for var in self.slave_filter_vars.values():
1144            var.set(True)
1145        self.update_slave_heading()
1146        self.render_filtered_view()
1147
1148    def select_no_slaves(self):
1149        self.slave_filter_all_mode = False
1150        for var in self.slave_filter_vars.values():
1151            var.set(False)
1152        self.update_slave_heading()
1153        self.render_filtered_view()
1154
1155    def on_slave_filter_changed(self):
1156        if not self.slave_filter_vars:
1157            self.slave_filter_all_mode = True
1158        else:
1159            values = [var.get() for var in self.slave_filter_vars.values()]
1160            # If every currently known slave is checked, remain in "All" mode
```

### MBSniffer v3.2/mb_view.py — lines 1161-1200

```
1161            # so future slaves are automatically displayed too.
1162            self.slave_filter_all_mode = bool(values) and all(values)
1163
1164        self.update_slave_heading()
1165        self.render_filtered_view()
1166
1167    def selected_slave_ids(self):
1168        return {
1169            slave for slave, var in self.slave_filter_vars.items()
1170            if var.get()
1171        }
1172
1173    def record_passes_slave_filter(self, record):
1174        # "Todos" includes RAW/unsynchronised data where a slave ID could not
1175        # be identified. "Nenhum" hides absolutely all traffic.
1176        if self.slave_filter_all_mode:
1177            return True
1178
1179        selected = self.selected_slave_ids()
1180        if not selected:
1181            return False
1182
1183        return str(record.get("slave", "")) in selected
1184
1185    def time_order_descending(self):
1186        return canonical_ui_text(self.time_order_var.get()).startswith("Decrescente")
1187
1188    def on_time_order_changed(self):
1189        label = self.tr("Hora ↓") if self.time_order_descending() else self.tr("Hora ↑")
1190        self.tree_headings["time"] = label
1191        self.tree.heading(
1192            "time",
1193            text=label,
1194            anchor="center",
1195            command=self.toggle_time_order
1196        )
1197        self.render_filtered_view()
1198
1199    def toggle_time_order(self):
1200        if self.time_order_descending():
```

### MBSniffer v3.2/mb_view.py — lines 1201-1240

```
1201            self.time_order_var.set(self.tr("Crescente — antigo → recente"))
1202        else:
1203            self.time_order_var.set(self.tr("Decrescente — recente → antigo"))
1204        self.on_time_order_changed()
1205
1206    def records_for_display(self):
1207        records = [
1208            record for record in self._frame_history
1209            if (
1210                self.record_passes_slave_filter(record)
1211                and self.record_passes_advanced_filters(record)
1212            )
1213        ]
1214        if self.time_order_descending():
1215            records.reverse()
1216        return records
1217
1218    def raw_line_for_record(self, record):
1219        # Keep fixed-width timing fields so the rest of every Raw Hex line
1220        # starts at exactly the same horizontal position. When a value does not
1221        # exist, show a completely blank field instead of "-", "dt=", "resp="
1222        # or "ms".
1223        delta_field = (
1224            f"dt={record['delta_text']:>7}ms "
1225            if record.get("delta_text")
1226            else " " * 13
1227        )
1228        response_field = (
1229            f"resp={record['response_text']:>7}ms "
1230            if record.get("response_text")
1231            else " " * 15
1232        )
1233
1234        return (
1235            f"[{record['time']}] "
1236            f"{delta_field}"
1237            f"{response_field}"
1238            f"{record['channel']:<6} {record['type']:<10} "
1239            f"Slave={record['slave'] or '-':<3} FC={record['fc'] or '-':<2} "
1240            f"CRC={record['crc']:<5} | {record['raw']} | {translate_details(record['details'], self.current_language)}\n"
```

**MBSniffer v3.2/mb_view.py — lines 1241-1280**

```
        )

    def raw_separator_width_chars(self, raw_lines=None):
        """
        Width of one Raw Hex separator in visible character cells.

        Raw Hex now wraps to the viewport, so the separator must span only the
        visible width. Using the longest unwrapped data line would make the
        separator wrap onto multiple lines.
        """
        try:
            # Leave a small pixel reserve for Text internal borders/padding so
            # the last separator character does not wrap onto a second line.
            widget_px = max(1, int(self.raw_text.winfo_width()) - 14)
            char_px = max(1, self.raw_text_font.measure("—"))
            viewport_chars = max(
                1,
                int(widget_px // char_px)
            )
        except Exception:
            viewport_chars = 1

        return viewport_chars

    def raw_transaction_separator(self, width_chars=None):
        if width_chars is None:
            width_chars = self._raw_separator_width_chars
        return "—" * max(1, int(width_chars)) + "\n"

    def schedule_raw_separator_reflow(self, _event=None):
        if self._raw_reflow_after_id is not None:
            try:
                self.after_cancel(self._raw_reflow_after_id)
            except Exception:
                pass

        self._raw_reflow_after_id = self.after(
            120,
            self.reflow_raw_separator_after_resize
        )
```

**MBSniffer v3.2/mb_view.py — lines 1281-1320**

```
1281
1282    def reflow_raw_separator_after_resize(self):
1283        self._raw_reflow_after_id = None
1284        if self.separate_transactions_var.get():
1285            self.render_raw_text_only()
1286
1287    def insert_tree_record(self, record, index="end", allow_separator=True):
1288        detail_lines = self.detail_lines_for_record(record)
1289        visual_rows = []
1290
1291        # Main row: all columns + first Details line.
1292        visual_rows.append({
1293            "kind": "main",
1294            "values": (
1295                record["time"],
1296                record["delta_text"],
1297                record["response_text"],
1298                record["channel"],
1299                record["type"],
1300                record["slave"],
1301                record["fc"],
1302                detail_lines[0],
1303                record["crc"],
1304            ),
1305            "raw": record["raw"],
1306        })
1307
1308        # Wrapped continuation lines keep only the Details text.
1309        for continuation in detail_lines[1:]:
1310            visual_rows.append({
1311                "kind": "continuation",
1312                "values": (
1313                    "", "", "", "", "", "", "",
1314                    continuation,
1315                    ""
1316                ),
1317                "raw": record["raw"],
1318            })
1319
1320        inserted_main_iid = None
```

**MBSniffer v3.2/mb_view.py — lines 1321-1360**

```
        inserted_iids = []

        # index=0 requires reverse insertion to preserve visual block order.
        rows_to_insert = (
            list(reversed(visual_rows))
            if index == 0
            else visual_rows
        )

        row_tag = self.anomaly_tag_for_record(record)

        for visual in rows_to_insert:
            iid = self.tree.insert(
                "",
                index,
                values=visual["values"],
                tags=((row_tag,) if row_tag else ()),
            )

            self._raw_by_iid[iid] = visual["raw"]
            self._record_by_iid[iid] = record

            if index == 0:
                inserted_iids.insert(0, iid)
                self._tree_history.appendleft((iid, True))
            else:
                inserted_iids.append(iid)
                self._tree_history.append((iid, True))

            if visual["kind"] == "main":
                inserted_main_iid = iid

        self._tree_iids_by_seq[record["seq"]] = inserted_iids
        self._ui_frame_count += 1
        return inserted_main_iid

    def render_raw_text_only(self):
        records = self.records_for_display()
        separator_after = self.separator_after_sequences(records)

```

**MBSniffer v3.2/mb_view.py — lines 1361-1400**

```
        line_by_seq = {
            record["seq"]: self.raw_line_for_record(record)
            for record in records
        }
        all_raw_lines = list(line_by_seq.values())
        self._raw_separator_width_chars = self.raw_separator_width_chars(
            all_raw_lines
        )
        separator_line = self.raw_transaction_separator(
            self._raw_separator_width_chars
        )

        # Respect the visual line cap, counting transaction separators as one
        # line each.
        if not self.time_order_descending():
            selected_reversed = []
            line_count = 0

            for record in reversed(records):
                cost = 1 + (1 if record["seq"] in separator_after else 0)
                if line_count + cost > MAX_RAW_TEXT_LINES:
                    break
                selected_reversed.append(record)
                line_count += cost

            raw_records = list(reversed(selected_reversed))
        else:
            selected = []
            line_count = 0

            for record in records:
                cost = 1 + (1 if record["seq"] in separator_after else 0)
                if line_count + cost > MAX_RAW_TEXT_LINES:
                    break
                selected.append(record)
                line_count += cost

            raw_records = selected

        parts = []
```

**MBSniffer v3.2/mb_view.py — lines 1401-1440**

```
        for record in raw_records:
            parts.append(line_by_seq[record["seq"]])
            if record["seq"] in separator_after:
                parts.append(separator_line)

        block = "".join(parts)
        self.raw_text.delete("1.0", "end")
        if block:
            self.raw_text.insert("end", block)
        self._raw_text_line_count = block.count("\n")

    def render_filtered_view(self):
        # This is intentionally called on user-driven filter/sort changes and
        # infrequent history-prune events, not once per received frame.
        selected_record = self.selected_record()
        selected_seq = (
            selected_record.get("seq")
            if selected_record is not None
            else None
        )

        for item in self.tree.get_children():
            self.tree.delete(item)

        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._tree_history.clear()
        self._tree_iids_by_seq.clear()
        self._tree_separator_iid_by_seq.clear()
        self._ui_frame_count = 0
        self.selected_raw_var.set("")

        descending = self.time_order_descending()
        records = self.records_for_display()

        # Column sizing determines the wrap width, so it must happen before
        # inserting the visible rows.
        self.autofit_tree_columns(records)

        for record in records:
```

**MBSniffer v3.2/mb_view.py — lines 1441-1480**

```
            self.insert_tree_record(
                record,
                index="end",
                allow_separator=False
            )

        self.sync_tree_transaction_separators(records)
        self.render_raw_text_only()

        if selected_seq is not None:
            iids = self._tree_iids_by_seq.get(selected_seq, [])
            if iids:
                try:
                    self.tree.selection_set(iids[0])
                    self.tree.focus(iids[0])
                    self.update_frame_inspector(
                        self._record_by_iid.get(iids[0])
                    )
                except tk.TclError:
                    pass
            else:
                self.update_frame_inspector(None)

        if self.auto_scroll_var.get() and self._tree_history:
            if descending:
                iid = self._tree_history[0][0]
                self.tree.see(iid)
                self.raw_text.see("1.0")
            else:
                iid = self._tree_history[-1][0]
                self.tree.see(iid)
                self.raw_text.see("end")

    def add_frame_row(self, row):
        # Kept as a convenience wrapper for callers/tests.
        self.add_frame_rows_batch([row])

    def add_frame_rows_batch(self, rows):
        if not rows:
            return
```

**MBSniffer v3.2/mb_view.py — lines 1481-1520**

```
1481
1482        raw_lines_log = []
1483        any_visible = False
1484        tree_layout_changed = False
1485        pair_boundary_changed = False
1486        expired_requests = []
1487
1488        self.pulse_bus_activity()
1489
1490        for row in rows:
1491            seq = self._next_frame_seq
1492            self._next_frame_seq += 1
1493            row["_view_seq"] = seq
1494
1495            timing = self.metrics.process(row)
1496
1497            delta_text = (
1498                f"{timing['delta_ms']:.1f}" if timing["delta_ms"] is not None else ""
1499            )
1500            response_text = (
1501                f"{timing['response_ms']:.1f}"
1502                if timing["response_ms"] is not None
1503                else ""
1504            )
1505
1506            # Keep pairing for response-time/statistics, but do not copy the
1507            # request Address/1-based/Qty into RESPONSE/EXCEPTION details.
1508            # Those fields remain visible only on the REQUEST itself.
1509            matched = timing.get("matched_request")
1510            expired_requests.extend(
1511                timing.get("expired_requests", [])
1512            )
1513
1514            self.register_slave_for_filter(row.get("slave", ""))
1515
1516            # FC01/FC02: the request Qty tells which bits of the last data
1517            # byte are padding, so trim them from the response details.
1518            if (
1519                matched is not None
1520                and row.get("type") == "RESPONSE"
```

**MBSniffer v3.2/mb_view.py — lines 1521-1560**

```
                and row.get("fc") in ("01", "02")
            ):
                try:
                    row["details"] = read_bits_details(
                        bytes.fromhex(row.get("raw", "")),
                        matched.get("qty"),
                    )
                except ValueError:
                    pass

            record = {
                "seq": seq,
                "epoch": float(row.get("_epoch", time.time())),
                "time": row["time"],
                "delta_text": delta_text,
                "response_text": response_text,
                "channel": row["channel"],
                "type": row["type"],
                "slave": row["slave"],
                "fc": row["fc"],
                "details": row["details"],
                "crc": row["crc"],
                "raw": row["raw"],
                "pdu_address": row.get("pdu_address"),
                "one_based": row.get("one_based"),
                "qty": row.get("qty"),
                "frame_len": row.get("_frame_len", 0),
                "timed_out": False,
                "matched": matched is not None,
                "request_seq": (
                    matched.get("seq")
                    if matched is not None
                    else None
                ),
                # Used only by the Frame Inspector to show the address of
                # each returned value; RESPONSE Details stay response-native.
                "request_pdu_address": (
                    matched.get("pdu_address")
                    if matched is not None
                    else None
```

### MBSniffer v3.2/mb_view.py — lines 1561-1600

```
                ),
                "request_qty": (
                    matched.get("qty")
                    if matched is not None
                    else None
                ),
            }
            self._frame_history.append(record)

            # Track successful normal transaction pairs independently of whether
            # highlighting is currently enabled. This lets the user toggle
            # "Realçar resultados" on/off against the exact same session.
            if (
                record.get("type") == "RESPONSE"
                and record.get("crc") == "OK"
                and record.get("matched")
                and anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS) is None
                and record.get("request_seq") is not None
            ):
                self._successful_request_seqs.add(record["request_seq"])

                # If the REQUEST is already visible, update it immediately
                # without rebuilding the entire Treeview.
                if self.highlight_anomalies_var.get():
                    for request_iid in self._tree_iids_by_seq.get(
                        record["request_seq"], []
                    ):
                        try:
                            self.tree.item(
                                request_iid,
                                tags=("result_success",),
                            )
                        except tk.TclError:
                            pass

            # Disk log is always complete and ignores GUI filters/sorting.
            raw_lines_log.append(self.raw_line_for_record(record))

            if (
                self.record_passes_slave_filter(record)
```

**MBSniffer v3.2/mb_view.py — lines 1601-1640**

```
                and self.record_passes_advanced_filters(record)
            ):
                any_visible = True
                if self.expand_tree_columns_for_record(record):
                    tree_layout_changed = True

                self.update_tree_rowheight_for_record(record)

                if self.time_order_descending():
                    self.insert_tree_record(
                        record,
                        index=0,
                        allow_separator=False
                    )

                    raw_block = self.raw_line_for_record(record)
                    self.raw_text.insert("1.0", raw_block)
                    self._raw_text_line_count += raw_block.count("\n")
                else:
                    self.insert_tree_record(
                        record,
                        index="end",
                        allow_separator=False
                    )

                    raw_block = self.raw_line_for_record(record)
                    self.raw_text.insert("end", raw_block)
                    self._raw_text_line_count += raw_block.count("\n")

                if record.get("matched"):
                    pair_boundary_changed = True

        if expired_requests:
            # A new frame can itself cause older requests to cross the Pending
            # timeout. Mark those original REQUEST records in place.
            self.mark_timed_out_records(
                expired_requests,
                rerender=True,
            )

```

**MBSniffer v3.2/mb_view.py — lines 1641-1680**

```
        log_block = "".join(raw_lines_log)

        # Only real traffic is allowed to create a .txt log. GUI filters do not
        # change the complete saved traffic.
        if self.capture_active and log_block:
            if self.ensure_log_started():
                self.log_line(log_block)

        self.update_stats_labels()
        rebuilt = self.prune_ui_history()

        if tree_layout_changed and not rebuilt:
            self.render_filtered_view()
            rebuilt = True

        if not rebuilt and pair_boundary_changed:
            visible_records = self.records_for_display()
            self.sync_tree_transaction_separators(visible_records)
            self.render_raw_text_only()

        # Raw Hex has its own bounded line count. Rebuild it in the correct
        # chronological direction when its hysteresis threshold is reached.
        if (
            not rebuilt
            and self._raw_text_line_count
            > MAX_RAW_TEXT_LINES + RAW_TEXT_PRUNE_CHUNK_LINES
        ):
            self.render_raw_text_only()

        if self.auto_scroll_var.get() and any_visible and self._tree_history:
            if self.time_order_descending():
                self.tree.see(self._tree_history[0][0])
                self.raw_text.see("1.0")
            else:
                self.tree.see(self._tree_history[-1][0])
                self.raw_text.see("end")

    def prune_ui_history(self):
        """
        Keep the GUI source history bounded.
```

### MBSniffer v3.2/mb_view.py — lines 1681-1720

```
1681
1682        The full disk log is independent and remains complete. Pruning happens
1683        by deque chunks and only triggers a full GUI rebuild occasionally.
1684        """
1685        trigger_frames = MAX_UI_FRAMES + UI_PRUNE_CHUNK_FRAMES
1686
1687        if len(self._frame_history) <= trigger_frames:
1688            return False
1689
1690        while len(self._frame_history) > MAX_UI_FRAMES:
1691            self._frame_history.popleft()
1692
1693        retained_sequences = {
1694            record.get("seq")
1695            for record in self._frame_history
1696            if record.get("seq") is not None
1697        }
1698        self._successful_request_seqs.intersection_update(retained_sequences)
1699
1700        self.render_filtered_view()
1701        return True
1702
1703    def on_row_select(self, _event=None):
1704        record = self.selected_record()
1705        self.update_frame_inspector(record)
1706
1707    def finish_capture_ui(self):
1708        self.busy = False
1709        self.capture_active = False
1710        self.status_var.set(self.tr("● Parado"))
1711        self.start_btn.configure(state="normal")
1712        self.stop_btn.configure(state="disabled")
1713        self.restart_btn.configure(state="normal")
1714        self.set_simulation_button_state("normal")
1715        self.set_sniffer_refresh_state("normal")
1716        self.serial_objects = []
1717        self.serial_by_port = {}
1718        self.reader_threads = []
1719        self.close_log()
1720        self.update_com_status()
```

**MBSniffer v3.2/mb_view.py — lines 1721-1760**

```
1721
1722    def app_directory(self):
1723        return get_application_directory()
1724
1725    def prepare_log_session(self, prefix, started_at, header):
1726        """
1727        Prepare a real-capture log session without creating a file.
1728
1729        The .txt file is created only after the first received frame/RAW block.
1730        """
1731        self.close_log()
1732        self.pending_log_filename = started_at.strftime(
1733            prefix + "_%Y%m%d_%H%M%S.txt"
1734        )
1735        self.pending_log_header = header
1736        self.pending_log_notes = []
1737        self.log_start_failed = False
1738        self.log_path = None
1739
1740    def ensure_log_started(self):
1741        """
1742        Create the pending real-capture log on first actual traffic.
1743
1744        Returns True when the log is open. A creation failure is reported only
1745        once for the current capture session.
1746        """
1747        if self.log_file:
1748            return True
1749
1750        if not self.pending_log_filename or self.log_start_failed:
1751            return False
1752
1753        try:
1754            target = get_logs_folder() / self.pending_log_filename
1755            self.log_file = target.open("w", encoding="utf-8", buffering=1)
1756            self.log_path = target
1757
1758            if self.pending_log_header:
1759                self.log_file.write(self.pending_log_header)
1760
```

### MBSniffer v3.2/mb_view.py — lines 1761-1800

```
            if self.pending_log_notes:
                self.log_file.write("".join(self.pending_log_notes))

            self.pending_log_notes = []
            return True

        except OSError as exc:
            self.log_start_failed = True
            self.log_file = None
            self.log_path = None
            messagebox.showerror(
                self.tr("Erro ao criar log"),
                self.trf(
                    "Foi detetado tráfego, mas não foi possível criar o ficheiro de log em:\n\n{folder}\n\n{error}",
                    folder=get_logs_folder(),
                    error=exc,
                )
            )
            return False

    def queue_log_note(self, text):
        """
        Write a capture note to an already-open log, or keep it in memory until
        the first traffic frame creates the log.
        """
        if self.log_file:
            self.log_line(text)
        elif self.capture_active and self.pending_log_filename:
            self.pending_log_notes.append(text)

    def log_line(self, text):
        if self.log_file:
            try:
                self.log_file.write(text)
            except Exception:
                pass

    def close_log(self):
        if self.log_file:
            try:
```

### MBSniffer v3.2/mb_view.py — lines 1801-1840

```
                self.log_file.write(
                    f"# Stop: {datetime.now().isoformat(timespec='milliseconds')}\n"
                )
                self.log_file.close()
            except Exception:
                pass

        self.log_file = None
        self.log_path = None
        self.pending_log_filename = None
        self.pending_log_header = ""
        self.pending_log_notes = []
        self.log_start_failed = False

    def open_log_folder(self):
        try:
            folder = get_logs_folder()
            if os.name == "nt":
                os.startfile(str(folder))
            elif sys.platform == "darwin":
                subprocess.Popen(["open", str(folder)])
            else:
                subprocess.Popen(["xdg-open", str(folder)])
        except Exception as exc:
            messagebox.showerror(self.tr("Abrir pasta"), str(exc))

    def clear_view(self):
        for item in self.tree.get_children():
            self.tree.delete(item)
        self.raw_text.delete("1.0", "end")
        self.selected_raw_var.set("")
        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._tree_history.clear()
        self._tree_iids_by_seq.clear()
        self._tree_separator_iid_by_seq.clear()
        self._frame_history.clear()
        self._next_frame_seq = 1
        self._ui_frame_count = 0
        self._raw_text_line_count = 0
```

### MBSniffer v3.2/mb_view.py — lines 1841-1871

```
1841        self._raw_separator_width_chars = 1
1842
1843        self.slave_filter_vars.clear()
1844        self.slave_filter_all_mode = True
1845        self.rebuild_slave_filter_menu()
1846
1847        self._transaction_filter_seqs = None
1848        self._successful_request_seqs.clear()
1849        if hasattr(self, "advanced_filter_type_var"):
1850            self.advanced_filter_type_var.set(self.tr("Todos"))
1851            self.advanced_filter_fc_var.set("")
1852            self.advanced_filter_response_ms_var.set("")
1853            self.advanced_filter_text_var.set("")
1854        self.update_frame_inspector(None)
1855        if hasattr(self, "bus_activity_var"):
1856            self.bus_activity_var.set("BUS ○")
1857
1858        self.time_order_var.set(self.tr("Crescente — antigo → recente"))
1859        self.tree_headings["time"] = self.tr("Hora ↑")
1860        self.tree.heading(
1861            "time",
1862            text=self.tree_headings["time"],
1863            anchor="center",
1864            command=self.toggle_time_order
1865        )
1866
1867        # With no traffic, widths/row height return to their default layout.
1868        self.autofit_tree_columns([])
1869
1870        self.reset_session_metrics()
1871
```

# MBSniffer v3.2/mb_slave_finder.py

1625 source lines.

**MBSniffer v3.2/mb_slave_finder.py — lines 1-40**

```python
#!/usr/bin/env python3
"""Active Modbus RTU Bus Slave Finder logic and GUI mixin."""

import threading
import time
import tkinter as tk
from tkinter import ttk, messagebox

from mb_config import (
    DISCOVERY_BAUD_RATES,
    DISCOVERY_PARITIES,
    DISCOVERY_STOP_BITS,
    DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
    DISCOVERY_MIN_TIMEOUT_MS,
    DISCOVERY_MAX_TIMEOUT_MS,
    DISCOVERY_PROBE_ADDRESS,
    DISCOVERY_PROBE_QTY,
    DISCOVERY_WRITE_TIMEOUT_SECONDS,
    DISCOVERY_WRITE_RETRIES,
    DISCOVERY_RX_BUFFER_LIMIT,
    DISCOVERY_MIN_ATTEMPTS,
    DISCOVERY_MAX_ATTEMPTS,
    DISCOVERY_DEFAULT_ATTEMPTS,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MODBUS_EXCEPTION_CODES,
)
from mb_protocol import (
    append_crc,
    crc_ok,
    hex_bytes,
    modbus_rtu_interframe_gap_seconds,
)
from mb_widgets import RefreshButton

def build_discovery_probe(slave: int, fc: int) -> bytes:
    """Build a read-only discovery probe: FC03/FC04, Address 0, Qty 1."""
    if not (1 <= int(slave) <= 247):
        raise ValueError("Slave ID fora do intervalo 1..247")
    if int(fc) not in (3, 4):
        raise ValueError("A descoberta ativa suporta apenas FC03/FC04")
```

### MBSniffer v3.2/mb_slave_finder.py — lines 41-80

```
41
42      payload = bytes((
43          int(slave),
44          int(fc),
45          0x00, DISCOVERY_PROBE_ADDRESS & 0xFF,
46          0x00, DISCOVERY_PROBE_QTY & 0xFF,
47      ))
48      return append_crc(payload)
49
50  DEVICE_ID_FC = 0x2B
51  DEVICE_ID_MEI_TYPE = 0x0E
52  DEVICE_ID_READ_BASIC = 0x01
53  DEVICE_ID_MORE_FOLLOWS = 0xFF
54  DEVICE_ID_BASIC_OBJECTS = (0x00, 0x01, 0x02)
55  DEVICE_ID_MAX_PAGES = 8
56
57  def build_device_identification_probe(slave: int, object_id: int = 0x00) -> bytes:
58      """Build FC43/MEI 14 Basic Device Identification request."""
59      slave = int(slave)
60      object_id = int(object_id)
61      if not (1 <= slave <= 247):
62          raise ValueError("Slave ID fora do intervalo 1..247")
63      if not (0 <= object_id <= 0xFF):
64          raise ValueError("Object ID fora do intervalo 0..255")
65      return append_crc(bytes((
66          slave,
67          DEVICE_ID_FC,
68          DEVICE_ID_MEI_TYPE,
69          DEVICE_ID_READ_BASIC,
70          object_id,
71      )))
72
73  def _decode_device_id_value(value: bytes) -> str:
74      """Decode a Modbus Device Identification ASCII object for display."""
75      return bytes(value).decode("ascii", errors="replace").strip("\x00 ")
76
77  def find_device_identification_response(buffer: bytes, slave: int):
78      """Find one CRC-valid FC43/14 response or Modbus exception in *buffer*."""
79      data = bytes(buffer)
80      slave = int(slave)
```

**MBSniffer v3.2/mb_slave_finder.py — lines 81-120**

```
81
82      for offset in range(max(0, len(data) - 3)):
83          if data[offset] != slave or offset + 2 > len(data):
84              continue
85
86          response_fc = data[offset + 1]
87
88          # FC43 exception response: Slave + 0xAB + Exception + CRC.
89          if response_fc == (DEVICE_ID_FC | 0x80):
90              end = offset + 5
91              if end <= len(data):
92                  frame = data[offset:end]
93                  if crc_ok(frame):
94                      return {
95                          "kind": "EXCEPTION",
96                          "frame": frame,
97                          "exception_code": frame[2],
98                          "objects": {},
99                          "more_follows": False,
100                         "next_object_id": 0,
101                     }
102             continue
103
104         if response_fc != DEVICE_ID_FC:
105             continue
106
107         # Normal response fixed header before the object list:
108         # Slave, FC, MEI, ReadCode, Conformity, MoreFollows, NextObject, Count.
109         if offset + 8 > len(data):
110             continue
111         if data[offset + 2] != DEVICE_ID_MEI_TYPE:
112             continue
113         if data[offset + 3] != DEVICE_ID_READ_BASIC:
114             continue
115
116         more_follows = data[offset + 5] == DEVICE_ID_MORE_FOLLOWS
117         next_object_id = data[offset + 6]
118         object_count = data[offset + 7]
119         pos = offset + 8
120         objects = {}
```

**MBSniffer v3.2/mb_slave_finder.py — lines 121-160**

```
121         complete = True
122
123         for _ in range(object_count):
124             if pos + 2 > len(data):
125                 complete = False
126                 break
127             object_id = data[pos]
128             object_len = data[pos + 1]
129             pos += 2
130             if pos + object_len > len(data):
131                 complete = False
132                 break
133             objects[object_id] = _decode_device_id_value(
134                 data[pos:pos + object_len]
135             )
136             pos += object_len
137
138         if not complete:
139             continue
140
141         end = pos + 2
142         if end > len(data):
143             continue
144         frame = data[offset:end]
145         if not crc_ok(frame):
146             continue
147
148         return {
149             "kind": "DEVICE_ID",
150             "frame": frame,
151             "exception_code": None,
152             "objects": objects,
153             "more_follows": more_follows,
154             "next_object_id": next_object_id,
155             "conformity_level": data[offset + 4],
156         }
157
158     return None
159
160 def read_device_identification_response(ser, slave, timeout_s, stop_event):
```

### MBSniffer v3.2/mb_slave_finder.py — lines 161-200

```
161     """Read until a valid FC43/14 response/exception is received or timed out."""
162     deadline = time.perf_counter() + max(0.001, float(timeout_s))
163     buffer = bytearray()
164 
165     while time.perf_counter() < deadline:
166         if stop_event.is_set():
167             return None
168 
169         waiting = int(getattr(ser, "in_waiting", 0) or 0)
170         if waiting:
171             chunk = ser.read(waiting)
172             if chunk:
173                 buffer.extend(chunk)
174                 if len(buffer) > DISCOVERY_RX_BUFFER_LIMIT:
175                     del buffer[:-DISCOVERY_RX_BUFFER_LIMIT]
176                 found = find_device_identification_response(buffer, slave)
177                 if found is not None:
178                     return found
179         elif stop_event.wait(0.001):
180             return None
181 
182     return find_device_identification_response(buffer, slave)
183 
184 def device_identification_timeout_seconds(
185     baud: int,
186     parity: str,
187     stopbits: str,
188     minimum_timeout_ms: float = DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
189 ) -> float:
190     """Conservative timeout for one FC43/14 response, up to one RTU frame."""
191     baud = max(1, int(baud))
192     bits_per_char = discovery_bits_per_char(parity, stopbits)
193     char_s = bits_per_char / float(baud)
194     # A Modbus RTU ADU is at most 256 bytes. Allow one full response frame,
195     # the recommended t3.5 gap and a small device-processing margin.
196     calculated = (
197         256.0 * char_s
198         + modbus_rtu_interframe_gap_seconds(baud, bits_per_char)
199         + 0.020
200     )
```

### MBSniffer v3.2/mb_slave_finder.py — lines 201-240

```
    return max(float(minimum_timeout_ms) / 1000.0, calculated)

def device_identification_request_transmit_seconds(
    baud: int, parity: str, stopbits: str
) -> float:
    """Estimated on-wire duration of the 7-byte FC43/14 request."""
    return 7.0 * discovery_bits_per_char(parity, stopbits) / float(max(1, int(baud)))

def format_device_identification(result) -> str:
    """Return compact Device Identification text for the Finder result table."""
    if result is None:
        return "No response"
    if result.get("kind") == "EXCEPTION":
        code = int(result.get("exception_code", 0))
        name = MODBUS_EXCEPTION_CODES.get(
            code, ("Unknown/Reserved Exception", "")
        )[0]
        return f"Exception 0x{code:02X} – {name}"

    objects = result.get("objects", {}) or {}
    values = [objects.get(obj_id, "") for obj_id in DEVICE_ID_BASIC_OBJECTS]
    if any(values):
        return " | ".join(value or "—" for value in values)
    return "Response – no Basic Device Identification objects"

def read_basic_device_identification(
    ser, slave, baud, parity, stopbits, minimum_timeout_ms, stop_event, serial_module
):
    """Read Basic Device Identification, following More Follows when required."""
    object_id = 0x00
    merged_objects = {}
    frames = []
    conformity_level = None

    for _page in range(DEVICE_ID_MAX_PAGES):
        if stop_event.is_set():
            return None

        try:
            ser.reset_input_buffer()
```

### MBSniffer v3.2/mb_slave_finder.py — lines 241-280

```
        except Exception:
            raise

        request = build_device_identification_probe(slave, object_id)
        sent = write_discovery_request(
            ser, request, baud, parity, stopbits, stop_event, serial_module
        )
        if not sent or stop_event.is_set():
            return None

        result = read_device_identification_response(
            ser,
            slave,
            device_identification_timeout_seconds(
                baud, parity, stopbits, minimum_timeout_ms
            ),
            stop_event,
        )
        if result is None or result.get("kind") == "EXCEPTION":
            return result

        frames.append(result["frame"])
        merged_objects.update(result.get("objects", {}))
        conformity_level = result.get("conformity_level")

        # Basic identification is complete as soon as VendorName, ProductCode
        # and MajorMinorRevision have all been received.
        if all(obj_id in merged_objects for obj_id in DEVICE_ID_BASIC_OBJECTS):
            return {
                "kind": "DEVICE_ID",
                "frame": b"".join(frames),
                "exception_code": None,
                "objects": merged_objects,
                "more_follows": bool(result.get("more_follows")),
                "next_object_id": int(result.get("next_object_id", 0)),
                "conformity_level": conformity_level,
            }

        if not result.get("more_follows"):
            break
```

**MBSniffer v3.2/mb_slave_finder.py — lines 281-320**

```
281
282         next_object_id = int(result.get("next_object_id", 0))
283         if next_object_id == object_id:
284             break
285         object_id = next_object_id
286
287         gap_s = modbus_rtu_interframe_gap_seconds(
288             baud, discovery_bits_per_char(parity, stopbits)
289         )
290         if stop_event.wait(gap_s):
291             return None
292
293     return {
294         "kind": "DEVICE_ID",
295         "frame": b"".join(frames),
296         "exception_code": None,
297         "objects": merged_objects,
298         "more_follows": False,
299         "next_object_id": object_id,
300         "conformity_level": conformity_level,
301     } if frames else None
302
303 def discovery_bits_per_char(parity: str, stopbits: str) -> float:
304     """Return serial bits/character: start + 8 data + parity + stop."""
305     parity_bits = 0 if parity == "None" else 1
306     stop_value = float(stopbits)
307     return 1.0 + 8.0 + parity_bits + stop_value
308
309 def discovery_response_timeout_seconds(
310     baud: int,
311     parity: str,
312     stopbits: str,
313     minimum_timeout_ms: float = DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
314 ) -> float:
315     """
316     Adaptive response deadline after the request has left the serial driver.
317
318     The deadline includes the on-wire duration of a normal 7-byte FC03/FC04
319     response plus processing/inter-frame margin, with a user-configurable
320     minimum.
```

### MBSniffer v3.2/mb_slave_finder.py — lines 321-360

```
321     """
322     baud = max(1, int(baud))
323     bits_per_char = discovery_bits_per_char(parity, stopbits)
324     char_s = bits_per_char / float(baud)
325     normal_response_s = 7.0 * char_s
326     serial_margin_s = max(
327         0.012, modbus_rtu_interframe_gap_seconds(baud, bits_per_char)
328     )
329     calculated = normal_response_s + serial_margin_s
330     return max(float(minimum_timeout_ms) / 1000.0, calculated)
331
332 def discovery_request_transmit_seconds(baud: int, parity: str, stopbits: str) -> float:
333     """Estimated on-wire duration of the fixed 8-byte discovery request."""
334     return 8.0 * discovery_bits_per_char(parity, stopbits) / float(max(1, int(baud)))
335
336 def find_discovery_response(buffer: bytes, slave: int, fc: int):
337     """
338     Find a valid normal or exception response for one discovery probe.
339
340     Any CRC-valid Modbus exception counts as proof that the slave exists.
341     The scanner intentionally ignores other traffic and request echoes.
342     """
343     data = bytes(buffer)
344     slave = int(slave)
345     fc = int(fc)
346
347     for offset in range(max(0, len(data) - 3)):
348         if data[offset] != slave or offset + 2 > len(data):
349             continue
350
351         response_fc = data[offset + 1]
352
353         if response_fc == (fc | 0x80):
354             end = offset + 5
355             if end <= len(data):
356                 frame = data[offset:end]
357                 if crc_ok(frame):
358                     code = frame[2]
359                     return {
360                         "kind": "EXCEPTION",
```

### MBSniffer v3.2/mb_slave_finder.py — lines 361-400

```
                        "frame": frame,
                        "exception_code": code,
                    }

        if response_fc == fc and offset + 3 <= len(data):
            byte_count = data[offset + 2]
            # FC03/04 register responses must contain an even positive byte count.
            if byte_count > 0 and byte_count % 2 == 0 and byte_count <= 250:
                end = offset + 5 + byte_count
                if end <= len(data):
                    frame = data[offset:end]
                    if crc_ok(frame):
                        return {
                            "kind": "RESPONSE",
                            "frame": frame,
                            "exception_code": None,
                        }

    return None

def read_discovery_response(ser, slave, fc, timeout_s, stop_event):
    """Read non-blocking serial bytes until a valid probe response or timeout."""
    deadline = time.perf_counter() + max(0.001, float(timeout_s))
    buffer = bytearray()

    while time.perf_counter() < deadline:
        if stop_event.is_set():
            return None

        # Do not hide serial-driver failures as ordinary slave timeouts.
        waiting = int(getattr(ser, "in_waiting", 0) or 0)

        if waiting:
            chunk = ser.read(waiting)
            if chunk:
                buffer.extend(chunk)
                if len(buffer) > DISCOVERY_RX_BUFFER_LIMIT:
                    del buffer[:-DISCOVERY_RX_BUFFER_LIMIT]
                found = find_discovery_response(buffer, slave, fc)
                if found is not None:
```

### MBSniffer v3.2/mb_slave_finder.py — lines 401-440

```
401                     return found
402         else:
403             # Event.wait() keeps the Stop button responsive.
404             if stop_event.wait(0.001):
405                 return None
406 
407     return find_discovery_response(buffer, slave, fc)
408 
409 def bus_activity_detected(ser, listen_s, stop_event):
410     """Listen passively for *listen_s* seconds; return the bytes received.
411 
412     Any byte, even one framed at another baud rate, means something else is
413     transmitting, so the active scan must not start.
414     """
415     deadline = time.perf_counter() + max(0.0, float(listen_s))
416     received = bytearray()
417     while time.perf_counter() < deadline:
418         if stop_event.is_set():
419             break
420         waiting = int(getattr(ser, "in_waiting", 0) or 0)
421         if waiting:
422             received.extend(ser.read(waiting) or b"")
423             if received:
424                 break
425         elif stop_event.wait(0.005):
426             break
427     return bytes(received)
428 
429 
430 def cancel_serial_io(ser):
431     """Best-effort cancellation of pending serial I/O."""
432     if ser is None:
433         return
434     for method_name in ("cancel_read", "cancel_write"):
435         method = getattr(ser, method_name, None)
436         if callable(method):
437             try:
438                 method()
439             except Exception:
440                 pass
```

### MBSniffer v3.2/mb_slave_finder.py — lines 441-480

```
441
442 def recover_discovery_write(ser):
443     """Best-effort cleanup after a discovery write timeout."""
444     cancel = getattr(ser, "cancel_write", None)
445     if callable(cancel):
446         try:
447             cancel()
448         except Exception:
449             pass
450     try:
451         ser.reset_output_buffer()
452     except Exception:
453         pass
454     try:
455         ser.reset_input_buffer()
456     except Exception:
457         pass
458
459 def write_discovery_request(
460     ser, request, baud, parity, stopbits, stop_event, serial_module
461 ):
462     """
463     Transmit one discovery request robustly.
464
465     A write timeout is retried once after cancelling/clearing TX. ser.flush()
466     is deliberately avoided because some USB-serial drivers can block there.
467     The helper also waits for the request's estimated physical on-wire time so
468     the slave response timeout starts at a sensible point on slow baud rates.
469     """
470     request = bytes(request)
471     timeout_exc = getattr(serial_module, "SerialTimeoutException", TimeoutError)
472     serial_exc = getattr(serial_module, "SerialException", OSError)
473     attempts = DISCOVERY_WRITE_RETRIES + 1
474     last_error = None
475     bits_per_char = discovery_bits_per_char(parity, stopbits)
476     char_time_s = bits_per_char / float(max(1, int(baud)))
477     request_wire_s = len(request) * char_time_s
478     retry_silence_s = max(
479         0.050,
480         request_wire_s + modbus_rtu_interframe_gap_seconds(baud, bits_per_char),
```

**MBSniffer v3.2/mb_slave_finder.py — lines 481-520**

```
481     )
482 
483     for attempt in range(attempts):
484         if stop_event.is_set():
485             return False
486 
487         started_write = time.perf_counter()
488         try:
489             written_total = 0
490             while written_total < len(request):
491                 if stop_event.is_set():
492                     return False
493                 written = ser.write(request[written_total:])
494                 written = 0 if written is None else int(written)
495                 if written <= 0:
496                     raise timeout_exc(
497                         "A porta série não aceitou bytes para transmissão."
498                     )
499                 written_total += written
500 
501             elapsed_write_s = time.perf_counter() - started_write
502             remaining_wire_s = max(0.0, request_wire_s - elapsed_write_s)
503             if remaining_wire_s > 0 and stop_event.wait(remaining_wire_s):
504                 return False
505             return True
506 
507         except timeout_exc as exc:
508             last_error = exc
509             recover_discovery_write(ser)
510             if attempt + 1 >= attempts or stop_event.is_set():
511                 raise
512             # If any fragment reached the wire before the driver timed out,
513             # leave enough silence for that fragment to finish and for a full
514             # Modbus RTU frame gap before retrying.
515             if stop_event.wait(retry_silence_s):
516                 return False
517 
518         except serial_exc:
519             raise
520 
```

### MBSniffer v3.2/mb_slave_finder.py — lines 521-560

```
521     if last_error is not None:
522         raise last_error
523     return False
524 
525 def format_duration(seconds: float) -> str:
526     total = max(0, int(round(float(seconds))))
527     hours, rem = divmod(total, 3600)
528     minutes, secs = divmod(rem, 60)
529     if hours:
530         return f"{hours:d}:{minutes:02d}:{secs:02d}"
531     return f"{minutes:02d}:{secs:02d}"
532 
533 
534 class SlaveFinderMixin:
535     """Methods that implement the active Bus Slave Finder tab."""
536 
537     def build_discovery_tab(self, page):
538         """Build the full-window active Modbus RTU slave-discovery workspace."""
539         finder_saved = getattr(self, "ui_settings", {}).get(
540             "bus_slave_finder", {}
541         )
542         if not isinstance(finder_saved, dict):
543             finder_saved = {}
544 
545         saved_bauds = set(
546             finder_saved.get("baud_rates", [9600, 19200])
547         )
548         saved_parities = set(
549             finder_saved.get("parities", ["None", "Even"])
550         )
551         saved_stops = set(
552             finder_saved.get("stop_bits", ["1"])
553         )
554 
555         page.columnconfigure(0, weight=1)
556         page.rowconfigure(3, weight=1)
557 
558         warning = ttk.LabelFrame(page, text="Pesquisa ativa — transmite no barramento")
559         warning.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
560         warning.columnconfigure(0, weight=1)
```

### MBSniffer v3.2/mb_slave_finder.py — lines 561-600

```
561         ttk.Label(
562             warning,
563             text=(
564                 "Esta ferramenta atua temporariamente como master Modbus RTU e envia apenas "
565                 "pedidos de leitura. Não a utilizes com outro master ativo no mesmo barramento."
566             ),
567             wraplength=1100,
568         ).grid(row=0, column=0, sticky="w", padx=10, pady=(8, 4))
569 
570         self.discovery_safety_var = tk.BooleanVar(value=False)
571         safety_cb = ttk.Checkbutton(
572             warning,
573             text="Confirmo que não existe outro master ativo no barramento.",
574             variable=self.discovery_safety_var,
575             command=self.update_discovery_start_state,
576         )
577         safety_cb.grid(row=1, column=0, sticky="w", padx=10, pady=(0, 8))
578 
579         options = ttk.LabelFrame(page, text="Configuração da pesquisa")
580         self._discovery_options_frame = options
581         options.grid(row=1, column=0, sticky="ew", padx=12, pady=6)
582         options.columnconfigure(0, weight=1)
583 
584         top_row = ttk.Frame(options)
585         top_row.grid(row=0, column=0, sticky="ew", padx=8, pady=(8, 4))
586         top_row.columnconfigure(3, weight=1)
587 
588         ttk.Label(top_row, text="Porta COM").grid(
589             row=0, column=0, sticky="w", padx=(0, 4)
590         )
591         self.discovery_port_var = tk.StringVar()
592         self.discovery_port_combo = ttk.Combobox(
593             top_row,
594             textvariable=self.discovery_port_var,
595             width=13,
596             state="readonly",
597         )
598         self.discovery_port_combo.grid(
599             row=0, column=1, sticky="w", padx=(0, 0)
600         )
```

### MBSniffer v3.2/mb_slave_finder.py — lines 601-640

```
601         self.discovery_port_combo.bind(
602             "<<ComboboxSelected>>",
603             lambda _e: self.update_discovery_start_state(),
604         )
605 
606         self.discovery_refresh_btn = RefreshButton(
607             top_row,
608             command=self.refresh_ports,
609             size=24,
610         )
611         self.discovery_refresh_btn.grid(
612             row=0, column=2, sticky="w", padx=(4, 0)
613         )
614 
615         def _sync_discovery_refresh_square(_event=None):
616             height = self.discovery_port_combo.winfo_height()
617             if height > 1:
618                 self.discovery_refresh_btn.configure(
619                     width=height, height=height
620                 )
621 
622         self.discovery_port_combo.bind(
623             "<Configure>", _sync_discovery_refresh_square, add="+"
624         )
625 
626         ttk.Label(
627             top_row,
628             text="Data bits: 8 (fixo)",
629         ).grid(row=0, column=4, sticky="w", padx=(28, 20))
630         ttk.Label(
631             top_row,
632             text="Probe: FC03 / Address 0 / Qty 1",
633         ).grid(row=0, column=5, sticky="w")
634 
635         ttk.Label(top_row, text="Tentativas").grid(
636             row=0, column=6, sticky="w", padx=(28, 4)
637         )
638         self.discovery_attempts_var = tk.StringVar(
639             value=finder_saved.get("attempts", str(DISCOVERY_DEFAULT_ATTEMPTS))
640         )
```

**MBSniffer v3.2/mb_slave_finder.py — lines 641-680**

```
641         self.discovery_attempts_spin = ttk.Spinbox(
642             top_row,
643             from_=DISCOVERY_MIN_ATTEMPTS,
644             to=DISCOVERY_MAX_ATTEMPTS,
645             increment=1,
646             width=4,
647             textvariable=self.discovery_attempts_var,
648         )
649         self.discovery_attempts_spin.grid(row=0, column=7, sticky="w")
650 
651         # Baud-rate checkboxes. Defaults keep a normal industrial scan quick.
652         baud_frame = ttk.LabelFrame(options, text="Baud rate")
653         baud_frame.grid(row=1, column=0, sticky="ew", padx=8, pady=4)
654         baud_frame.rowconfigure(0, weight=1)
655         self.discovery_baud_vars = {}
656         for idx in range(len(DISCOVERY_BAUD_RATES)):
657             baud_frame.columnconfigure(idx, weight=1, uniform="baud")
658         for idx, baud in enumerate(DISCOVERY_BAUD_RATES):
659             var = tk.BooleanVar(value=baud in saved_bauds)
660             self.discovery_baud_vars[baud] = var
661             cb = ttk.Checkbutton(baud_frame, text=str(baud), variable=var)
662             cb.grid(row=0, column=idx, sticky="", padx=4, pady=(8, 7))
663             var.trace_add("write", lambda *_a: self.on_discovery_options_changed())
664 
665         serial_frame = ttk.Frame(options)
666         serial_frame.grid(row=2, column=0, sticky="ew", padx=8, pady=4)
667         # Equal thirds independent of translated child captions.
668         for column in range(3):
669             serial_frame.columnconfigure(
670                 column, weight=1, uniform="finder_serial_columns"
671             )
672 
673         parity_box = ttk.LabelFrame(serial_frame, text="Parity")
674         parity_box.grid(row=0, column=0, sticky="nsew", padx=(0, 5))
675         parity_box.rowconfigure(0, weight=1)
676         for idx in range(len(DISCOVERY_PARITIES)):
677             parity_box.columnconfigure(idx, weight=1, uniform="parity")
678         self.discovery_parity_vars = {}
679         for idx, parity in enumerate(DISCOVERY_PARITIES):
680             var = tk.BooleanVar(value=parity in saved_parities)
```

**MBSniffer v3.2/mb_slave_finder.py — lines 681-720**

```
            self.discovery_parity_vars[parity] = var
            ttk.Checkbutton(parity_box, text=parity, variable=var).grid(
                row=0, column=idx, sticky="", padx=4, pady=(8, 7)
            )
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())
        stop_box = ttk.LabelFrame(serial_frame, text="Stop bits")
        stop_box.grid(row=0, column=1, sticky="nsew", padx=5)
        stop_box.rowconfigure(0, weight=1)
        for idx in range(len(DISCOVERY_STOP_BITS)):
            stop_box.columnconfigure(idx, weight=1, uniform="stop")
        self.discovery_stop_vars = {}
        for idx, stopbits in enumerate(DISCOVERY_STOP_BITS):
            var = tk.BooleanVar(value=stopbits in saved_stops)
            self.discovery_stop_vars[stopbits] = var
            ttk.Checkbutton(stop_box, text=stopbits, variable=var).grid(
                row=0, column=idx, sticky="", padx=4, pady=(8, 7)
            )
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())
        range_box = ttk.LabelFrame(serial_frame, text="Slave IDs / velocidade")
        range_box.grid(row=0, column=2, sticky="nsew", padx=(5, 0))
        range_box.columnconfigure(0, weight=1)
        range_box.rowconfigure(0, weight=1)

        # Keep the range and timeout controls on one centred line so this
        # LabelFrame stays as compact as the neighbouring Parity/Stop bits
        # boxes.  The option checkboxes keep their own second line so the long
        # Device Identification caption remains fully inside the rectangle.
        range_content = ttk.Frame(range_box)
        range_content.grid(
            row=0, column=0, sticky="nsew", padx=8, pady=(8, 7)
        )
        range_content.columnconfigure(0, weight=1)
        for row in range(2):
            range_content.rowconfigure(row, weight=1, uniform="range_rows")

        id_row = ttk.Frame(range_content)
        id_row.grid(row=0, column=0, sticky="")
        ttk.Label(id_row, text="De").grid(row=0, column=0, padx=(0, 3))
        self.discovery_slave_start_var = tk.StringVar(value=finder_saved.get("slave_start", "1"))
        self.discovery_slave_start_spin = ttk.Spinbox(
```

**MBSniffer v3.2/mb_slave_finder.py — lines 721-760**

```
            id_row, from_=1, to=247, width=5, textvariable=self.discovery_slave_start_var
        )
        self.discovery_slave_start_spin.grid(row=0, column=1, padx=3)
        ttk.Label(id_row, text="até").grid(row=0, column=2, padx=3)
        self.discovery_slave_end_var = tk.StringVar(value=finder_saved.get("slave_end", "247"))
        self.discovery_slave_end_spin = ttk.Spinbox(
            id_row, from_=1, to=247, width=5, textvariable=self.discovery_slave_end_var
        )
        self.discovery_slave_end_spin.grid(row=0, column=3, padx=3)

        ttk.Label(id_row, text="Timeout mín.").grid(
            row=0, column=4, padx=(14, 3)
        )
        self.discovery_min_timeout_var = tk.StringVar(
            value=finder_saved.get(
                "min_timeout_ms",
                str(DISCOVERY_DEFAULT_MIN_TIMEOUT_MS),
            )
        )
        self.discovery_min_timeout_spin = ttk.Spinbox(
            id_row,
            from_=DISCOVERY_MIN_TIMEOUT_MS,
            to=DISCOVERY_MAX_TIMEOUT_MS,
            increment=5,
            width=6,
            textvariable=self.discovery_min_timeout_var,
        )
        self.discovery_min_timeout_spin.grid(row=0, column=5, padx=3)
        ttk.Label(id_row, text="ms").grid(row=0, column=6, padx=3)

        toggles = ttk.Frame(range_content)
        # Small vertical separation from the Slave range/timeout row.
        # Keep this deliberately modest so the panel remains compact.
        toggles.grid(row=1, column=0, sticky="", pady=(3, 0))

        self.discovery_fc04_fallback_var = tk.BooleanVar(value=finder_saved.get("fc04_fallback", False))
        fallback_cb = ttk.Checkbutton(
            toggles,
            text="FC04 fallback",
            variable=self.discovery_fc04_fallback_var,
```

### MBSniffer v3.2/mb_slave_finder.py — lines 761-800

```
        )
        fallback_cb.grid(row=0, column=0, sticky="w", padx=(0, 10))

        self.discovery_device_identification_var = tk.BooleanVar(
            value=finder_saved.get("device_identification", False)
        )
        device_id_cb = ttk.Checkbutton(
            toggles,
            text="Device Identification (FC43/14)",
            variable=self.discovery_device_identification_var,
        )
        device_id_cb.grid(row=0, column=1, sticky="w")

        for var in (
            self.discovery_slave_start_var,
            self.discovery_slave_end_var,
            self.discovery_min_timeout_var,
            self.discovery_attempts_var,
            self.discovery_fc04_fallback_var,
            self.discovery_device_identification_var,
        ):
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())

        ttk.Label(
            options,
            text=(
                "FC03 é o modo rápido. Uma resposta normal OU uma Modbus Exception com CRC válido "
                "conta como slave encontrado. Ativa FC04 fallback apenas para dispositivos que "
                "possam ignorar FC03 sem devolver Exception. Device Identification envia FC43/14 "
                "apenas aos slaves encontrados e tenta ler VendorName, ProductCode e Revision. "
                "Tentativas repete cada pedido sem resposta. Antes de transmitir, a pesquisa escuta "
                "o barramento durante 1 s e não arranca se detetar tráfego."
            ),
            wraplength=1120,
        ).grid(row=3, column=0, sticky="w", padx=10, pady=(4, 8))

        controls = ttk.LabelFrame(page, text="Pesquisa")
        controls.grid(row=2, column=0, sticky="ew", padx=12, pady=6)
        controls.columnconfigure(4, weight=1)

```

### MBSniffer v3.2/mb_slave_finder.py — lines 801-840

```
        self.discovery_start_btn = ttk.Button(
            controls, text="Iniciar pesquisa", command=self.start_discovery_scan
        )
        self.discovery_start_btn.grid(row=0, column=0, padx=(10, 5), pady=8)
        self.discovery_stop_btn = ttk.Button(
            controls, text="Parar", command=self.stop_discovery_scan, state="disabled"
        )
        self.discovery_stop_btn.grid(row=0, column=1, padx=5, pady=8)
        self.discovery_clear_btn = ttk.Button(
            controls, text="Limpar resultados", command=self.clear_discovery_results
        )
        self.discovery_clear_btn.grid(row=0, column=2, padx=5, pady=8)

        self.discovery_found_var = tk.StringVar(value="Encontrados: 0")
        ttk.Label(controls, textvariable=self.discovery_found_var).grid(
            row=0, column=3, padx=(18, 8), pady=8, sticky="w"
        )

        self.discovery_status_var = tk.StringVar(value="Parado")
        ttk.Label(controls, textvariable=self.discovery_status_var).grid(
            row=0, column=4, padx=8, pady=8, sticky="w"
        )

        self.discovery_progress_var = tk.DoubleVar(value=0.0)
        self.discovery_progress = ttk.Progressbar(
            controls, variable=self.discovery_progress_var, maximum=100.0
        )
        self.discovery_progress.grid(
            row=1, column=0, columnspan=5, sticky="ew", padx=10, pady=(0, 5)
        )

        self.discovery_progress_text_var = tk.StringVar(value="0 / 0")
        self.discovery_estimate_var = tk.StringVar(value="")
        ttk.Label(controls, textvariable=self.discovery_progress_text_var).grid(
            row=2, column=0, columnspan=2, sticky="w", padx=10, pady=(0, 8)
        )
        ttk.Label(controls, textvariable=self.discovery_estimate_var).grid(
            row=2, column=2, columnspan=3, sticky="e", padx=10, pady=(0, 8)
        )

```

### MBSniffer v3.2/mb_slave_finder.py — lines 841-880

```
841         results = ttk.LabelFrame(page, text="Slaves encontrados")
842         results.grid(row=3, column=0, sticky="nsew", padx=12, pady=(6, 12))
843         results.columnconfigure(0, weight=1)
844         results.rowconfigure(0, weight=1)
845 
846         columns = (
847             "slave", "baud", "config", "fc", "result", "resp", "device_id", "raw"
848         )
849         self.discovery_tree = ttk.Treeview(
850             results, columns=columns, show="headings", selectmode="browse"
851         )
852         headings = {
853             "slave": "Slave",
854             "baud": "Baud",
855             "config": "Config.",
856             "fc": "FC",
857             "result": "Resultado",
858             "resp": "Resp. (ms)",
859             "device_id": "Device Identification",
860             "raw": "Raw Hex",
861         }
862         # Match the Traffic table behaviour: no horizontal scrollbar. Keep
863         # short fields compact and let the three text-heavy columns share the
864         # remaining viewport width. Column widths are recalculated whenever
865         # the Treeview is resized so the complete table remains visible.
866         self.discovery_fixed_widths = {
867             "slave": 70,
868             "baud": 90,
869             "config": 85,
870             "fc": 55,
871             "resp": 95,
872         }
873         self.discovery_flex_columns = ("result", "device_id", "raw")
874         self.discovery_flex_weights = {
875             "result": 0.27,
876             "device_id": 0.38,
877             "raw": 0.35,
878         }
879         self.discovery_flex_min_width = 90
880 
```

**MBSniffer v3.2/mb_slave_finder.py — lines 881-920**

```
        initial_widths = {
            **self.discovery_fixed_widths,
            "result": 220,
            "device_id": 330,
            "raw": 300,
        }
        for col in columns:
            self.discovery_tree.heading(col, text=headings[col], anchor="center")
            self.discovery_tree.column(
                col,
                width=initial_widths[col],
                minwidth=(
                    self.discovery_flex_min_width
                    if col in self.discovery_flex_columns
                    else self.discovery_fixed_widths[col]
                ),
                anchor="w" if col in self.discovery_flex_columns else "center",
                stretch=False,
            )

        discovery_y = ttk.Scrollbar(
            results, orient="vertical", command=self.discovery_tree.yview
        )
        self.discovery_tree.configure(yscrollcommand=discovery_y.set)
        self.discovery_tree.grid(row=0, column=0, sticky="nsew")
        discovery_y.grid(row=0, column=1, sticky="ns")
        self.discovery_tree.bind(
            "<Configure>", self.fit_discovery_columns_to_viewport, add="+"
        )
        # Right-click / double-click a found Slave to read it in the Master tab.
        self.discovery_context_menu = tk.Menu(self, tearoff=False)
        self.discovery_context_menu.add_command(
            label="Abrir no Master", command=self.discovery_open_selected_in_master
        )
        self.discovery_tree.bind("<Button-3>", self._show_discovery_context_menu)
        self.discovery_tree.bind(
            "<Double-1>", lambda _e: self.discovery_open_selected_in_master()
        )

        self.update_discovery_estimate()
```

### MBSniffer v3.2/mb_slave_finder.py — lines 921-960

```
921         self.update_discovery_start_state()
922 
923     def _show_discovery_context_menu(self, event):
924         row = self.discovery_tree.identify_row(event.y)
925         if not row:
926             return
927         self.discovery_tree.selection_set(row)
928         try:
929             self.discovery_context_menu.entryconfigure(0, label=self.tr("Abrir no Master"))
930             self.discovery_context_menu.tk_popup(event.x_root, event.y_root)
931         finally:
932             self.discovery_context_menu.grab_release()
933 
934     def discovery_open_selected_in_master(self):
935         """Open the selected found Slave in a new Master window."""
936         selection = self.discovery_tree.selection()
937         if not selection:
938             return
939         values = self.discovery_tree.item(selection[0], "values")
940         try:
941             slave, baud, config, fc = int(values[0]), str(values[1]), str(values[2]), int(values[3])
942         except (IndexError, ValueError):
943             return
944         parity = {"N": "None", "E": "Even", "O": "Odd"}.get(config[1:2], "None")
945         stopbits = config[2:] or "1"
946         self.master_open_definition(
947             {"slave": slave, "fc": fc, "address": 0, "qty": 10},
948             port=self.discovery_port_var.get().strip(),
949             baud=baud, parity=parity, stopbits=stopbits,
950         )
951 
952     def fit_discovery_columns_to_viewport(self, _event=None):
953         """Fit Slave Finder columns to the visible Treeview width."""
954         if not hasattr(self, "discovery_tree"):
955             return
956 
957         viewport_width = int(self.discovery_tree.winfo_width())
958         if viewport_width <= 1:
959             return
960 
```

**MBSniffer v3.2/mb_slave_finder.py — lines 961-1000**

```
        # Reserve a few pixels for Treeview borders so Tk never needs a hidden
        # horizontal overflow area. The vertical scrollbar is a sibling and is
        # therefore already excluded from winfo_width().
        usable_width = max(1, viewport_width - 4)
        fixed_total = sum(self.discovery_fixed_widths.values())
        flex_count = len(self.discovery_flex_columns)
        min_flex_total = self.discovery_flex_min_width * flex_count

        flex_total = max(min_flex_total, usable_width - fixed_total)
        allocated = 0
        for index, col in enumerate(self.discovery_flex_columns):
            if index == flex_count - 1:
                width = flex_total - allocated
            else:
                width = int(round(flex_total * self.discovery_flex_weights[col]))
                allocated += width
            width = max(self.discovery_flex_min_width, width)
            self.discovery_tree.column(col, width=width)

        for col, width in self.discovery_fixed_widths.items():
            self.discovery_tree.column(col, width=width)

    def on_discovery_options_changed(self):
        if hasattr(self, "discovery_estimate_var"):
            self.update_discovery_estimate()
            self.update_discovery_start_state()

    def selected_discovery_configs(self):
        bauds = [baud for baud in DISCOVERY_BAUD_RATES if self.discovery_baud_vars[baud].get()]
        parities = [p for p in DISCOVERY_PARITIES if self.discovery_parity_vars[p].get()]
        stops = [s for s in DISCOVERY_STOP_BITS if self.discovery_stop_vars[s].get()]
        return [
            (baud, parity, stopbits)
            for baud in bauds
            for parity in parities
            for stopbits in stops
        ]

    def selected_discovery_slaves(self):
        start = int(self.discovery_slave_start_var.get())
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1001-1040

```
1001        end = int(self.discovery_slave_end_var.get())
1002        if not (1 <= start <= 247 and 1 <= end <= 247):
1003            raise ValueError(self.tr("Os Slave IDs têm de estar entre 1 e 247."))
1004        if start > end:
1005            raise ValueError(self.tr("O Slave ID inicial não pode ser superior ao final."))
1006        return list(range(start, end + 1))
1007
1008    def selected_discovery_min_timeout_ms(self):
1009        value = float(self.discovery_min_timeout_var.get())
1010        if not (DISCOVERY_MIN_TIMEOUT_MS <= value <= DISCOVERY_MAX_TIMEOUT_MS):
1011            raise ValueError(
1012                self.trf(
1013                    "O timeout mínimo tem de estar entre {minimum} e {maximum} ms.",
1014                    minimum=DISCOVERY_MIN_TIMEOUT_MS,
1015                    maximum=DISCOVERY_MAX_TIMEOUT_MS,
1016                )
1017            )
1018        return value
1019
1020    def selected_discovery_attempts(self):
1021        value = int(self.discovery_attempts_var.get())
1022        if not (DISCOVERY_MIN_ATTEMPTS <= value <= DISCOVERY_MAX_ATTEMPTS):
1023            raise ValueError(
1024                self.trf(
1025                    "Tentativas tem de estar entre {minimum} e {maximum}.",
1026                    minimum=DISCOVERY_MIN_ATTEMPTS,
1027                    maximum=DISCOVERY_MAX_ATTEMPTS,
1028                )
1029            )
1030        return value
1031
1032    def discovery_config_label(self, parity, stopbits):
1033        parity_letter = {"None": "N", "Even": "E", "Odd": "O"}[parity]
1034        return f"8{parity_letter}{stopbits}"
1035
1036    def update_discovery_estimate(self):
1037        try:
1038            configs = self.selected_discovery_configs()
1039            slaves = self.selected_discovery_slaves()
1040            minimum_timeout_ms = self.selected_discovery_min_timeout_ms()
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1041-1080

```
            attempts = self.selected_discovery_attempts()
        except Exception:
            self.discovery_estimate_var.set(self.tr("Máx. estimado: —"))
            self.discovery_progress_text_var.set("0 / 0")
            return

        total_units = len(configs) * len(slaves)
        probes_per_unit = 2 if self.discovery_fc04_fallback_var.get() else 1
        identify_devices = bool(self.discovery_device_identification_var.get())

        seconds = 0.0
        for baud, parity, stopbits in configs:
            per_probe = (
                discovery_request_transmit_seconds(baud, parity, stopbits)
                + discovery_response_timeout_seconds(
                    baud, parity, stopbits, minimum_timeout_ms
                )
            )
            # Worst case: no Slave answers, so every probe uses all attempts.
            seconds += len(slaves) * probes_per_unit * attempts * per_probe
            if identify_devices:
                # Worst case: every probed Slave exists and therefore receives
                # one optional Basic Device Identification query.
                seconds += len(slaves) * (
                    device_identification_request_transmit_seconds(
                        baud, parity, stopbits
                    )
                    + device_identification_timeout_seconds(
                        baud, parity, stopbits, minimum_timeout_ms
                    )
                )
            seconds += 0.025  # small reopen/reconfigure allowance
        seconds += DISCOVERY_IDLE_CHECK_SECONDS if configs else 0.0

        self.discovery_progress_text_var.set(f"0 / {total_units}")
        suffix_parts = []
        if probes_per_unit == 2:
            suffix_parts.append("FC04 fallback")
        if identify_devices:
            suffix_parts.append("Device Identification")
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1081-1120

```
        suffix = (
            self.trf(" (com {options})", options=" + ".join(suffix_parts))
            if suffix_parts else ""
        )
        self.discovery_estimate_var.set(
            self.trf(
                "{configs} config. × {slaves} slaves — máx. estimado: {duration}",
                configs=len(configs),
                slaves=len(slaves),
                duration=format_duration(seconds),
            )
            + suffix
        )

    def update_discovery_start_state(self):
        if not hasattr(self, "discovery_start_btn"):
            return
        enabled = (
            not self.discovery_active
            and not self.busy
            and bool(self.discovery_port_var.get().strip())
            and bool(self.discovery_safety_var.get())
        )
        try:
            enabled = enabled and bool(self.selected_discovery_configs())
            self.selected_discovery_slaves()
            self.selected_discovery_min_timeout_ms()
            self.selected_discovery_attempts()
        except Exception:
            enabled = False
        self.discovery_start_btn.configure(state="normal" if enabled else "disabled")

    def clear_discovery_results(self):
        if self.discovery_active:
            return
        for iid in self.discovery_tree.get_children():
            self.discovery_tree.delete(iid)
        self.discovery_found_count = 0
        self.discovery_found_var.set(self.trf("Encontrados: {count}", count=0))
        self.discovery_progress_var.set(0.0)
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1121-1160

```
1121        self.discovery_status_var.set(self.tr("Parado"))
1122        self.update_discovery_estimate()
1123
1124    def open_discovery_serial_port(self, serial, port, baud, parity, stopbits):
1125        parity_map = {
1126            "None": serial.PARITY_NONE,
1127            "Even": serial.PARITY_EVEN,
1128            "Odd": serial.PARITY_ODD,
1129        }
1130        stop_map = {
1131            "1": serial.STOPBITS_ONE,
1132            "2": serial.STOPBITS_TWO,
1133        }
1134
1135        ser = serial.Serial()
1136        ser.port = port
1137        ser.baudrate = int(baud)
1138        ser.bytesize = serial.EIGHTBITS
1139        ser.parity = parity_map[parity]
1140        ser.stopbits = stop_map[stopbits]
1141        ser.timeout = 0
1142
1143        ser.write_timeout = DISCOVERY_WRITE_TIMEOUT_SECONDS
1144        ser.xonxoff = False
1145        ser.rtscts = False
1146        ser.dsrdtr = False
1147
1148        # Do not force RTS/DTR levels. Many USB-RS485 adapters manage direction
1149        # internally and forcing modem-control lines can interfere with some
1150        # adapters/drivers. Hardware flow control is disabled above.
1151        ser.open()
1152
1153        try:
1154            ser.reset_input_buffer()
1155        except Exception:
1156            pass
1157        try:
1158            ser.reset_output_buffer()
1159        except Exception:
1160            pass
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1161-1200

```
1161
1162        return ser
1163
1164    def start_discovery_scan(self):
1165        if self.busy or self.discovery_active:
1166            return
1167
1168        serial, _ = self.get_serial_modules()
1169        if serial is None:
1170            messagebox.showerror(
1171                self.tr("pyserial em falta"),
1172                self.tr("pyserial não está instalado. O build_exe.bat inclui-o automaticamente no EXE.")
1173            )
1174            return
1175
1176        if not self.discovery_safety_var.get():
1177            messagebox.showwarning(
1178                self.tr("Pesquisa ativa"),
1179                self.tr("Confirma primeiro que não existe outro master ativo no barramento.")
1180            )
1181            return
1182
1183        try:
1184            port = self.discovery_port_var.get().strip()
1185            if not port:
1186                raise ValueError(self.tr("Seleciona uma porta COM."))
1187            configs = self.selected_discovery_configs()
1188            if not configs:
1189                raise ValueError(self.tr("Seleciona pelo menos uma configuração série."))
1190            slaves = self.selected_discovery_slaves()
1191            minimum_timeout_ms = self.selected_discovery_min_timeout_ms()
1192            attempts = self.selected_discovery_attempts()
1193        except Exception as exc:
1194            messagebox.showerror(self.tr("Configuração inválida"), str(exc))
1195            return
1196
1197        fallback_fc04 = bool(self.discovery_fc04_fallback_var.get())
1198        identify_devices = bool(self.discovery_device_identification_var.get())
1199
1200        self.clear_discovery_results()
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1201-1240

```
        self.discovery_active = True
        self.busy = True
        self.discovery_stop_event.clear()
        self.discovery_found_count = 0
        self.discovery_progress_var.set(0.0)
        self.discovery_status_var.set(self.tr("A iniciar pesquisa…"))

        self.discovery_start_btn.configure(state="disabled")
        self.discovery_stop_btn.configure(state="normal")
        self.discovery_clear_btn.configure(state="disabled")
        self.discovery_refresh_btn.configure(state="disabled")
        self.discovery_port_combo.configure(state="disabled")

        # Prevent passive capture/simulation from being started concurrently.
        self.start_btn.configure(state="disabled")
        self.stop_btn.configure(state="disabled")
        self.restart_btn.configure(state="disabled")
        self.set_sniffer_refresh_state("disabled")
        self.set_simulation_button_state("disabled")

        immutable_configs = list(configs)
        immutable_slaves = list(slaves)

        self.discovery_thread = threading.Thread(
            target=self.discovery_worker,
            args=(
                serial, port, immutable_configs, immutable_slaves,
                minimum_timeout_ms, fallback_fc04, identify_devices, attempts
            ),
            daemon=True,
        )
        try:
            self.discovery_thread.start()
        except Exception as exc:
            # Restore the interface if the worker cannot even be started.
            self.discovery_active = False
            self.busy = False
            self.discovery_thread = None
            self.discovery_stop_btn.configure(state="disabled")
            self.discovery_clear_btn.configure(state="normal")
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1241-1280

```
            self.discovery_refresh_btn.configure(state="normal")
            self.discovery_port_combo.configure(state="readonly")
            self.start_btn.configure(state="normal")
            self.restart_btn.configure(state="normal")
            self.set_sniffer_refresh_state("normal")
            self.set_simulation_button_state("normal")
            self.discovery_status_var.set(self.tr("Erro ao iniciar pesquisa"))
            self.update_discovery_start_state()
            messagebox.showerror(
                self.tr("Erro na Bus Slave Finder"),
                self.trf(
                    "Não foi possível iniciar a thread de pesquisa:\n\n{error}",
                    error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                )
            )

    def stop_discovery_scan(self):
        if self.discovery_active:
            self.discovery_status_var.set(self.tr("A parar…"))
            self.discovery_stop_event.set()
            self.discovery_stop_btn.configure(state="disabled")
            # Wake/cancel pending driver I/O without closing the handle from
            # the Tk thread while the discovery worker may still be using it.
            cancel_serial_io(self.discovery_serial)

    def discovery_worker(
        self, serial, port, configs, slaves, minimum_timeout_ms, fallback_fc04,
        identify_devices, attempts=DISCOVERY_DEFAULT_ATTEMPTS
    ):
        started = time.perf_counter()
        total_units = len(configs) * len(slaves)
        completed = 0
        found_count = 0
        fatal_error = None
        context = str(port)
        idle_checked = False

        try:
            for baud, parity, stopbits in configs:
                if self.discovery_stop_event.is_set():
```

**MBSniffer v3.2/mb_slave_finder.py — lines 1281-1320**

```
1281                    break
1282
1283                config_label = self.discovery_config_label(parity, stopbits)
1284                context = f"{port} — {baud} / {config_label}"
1285                self.event_queue.put((
1286                    "discovery_status",
1287                    self.trf(
1288                        "A configurar {baud} / {config}…",
1289                        baud=baud,
1290                        config=config_label,
1291                    )
1292                ))
1293
1294                ser = None
1295                try:
1296                    try:
1297                        ser = self.open_discovery_serial_port(
1298                            serial, port, baud, parity, stopbits
1299                        )
1300                    except Exception as exc:
1301                        raise RuntimeError(
1302                            self.trf(
1303                                "Não foi possível abrir {port} em {baud} / {config}: {error}",
1304                                port=port,
1305                                baud=baud,
1306                                config=config_label,
1307                                error=f"{type(exc).__name__}: {self.tr(str(exc))}",
1308                            )
1309                        ) from exc
1310
1311                    self.discovery_serial = ser
1312                    if self.discovery_stop_event.wait(0.025):
1313                        break
1314
1315                    if not idle_checked:
1316                        # Last line of defence behind the safety checkbox:
1317                        # never transmit onto a bus that is already busy.
1318                        idle_checked = True
1319                        self.event_queue.put((
1320                            "discovery_status",
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1321-1360

```
                            self.tr("A verificar se o barramento está livre…"),
                        ))
                        traffic = bus_activity_detected(
                            ser,
                            DISCOVERY_IDLE_CHECK_SECONDS,
                            self.discovery_stop_event,
                        )
                        if traffic:
                            raise RuntimeError(
                                self.trf(
                                    "Foi detetado tráfego em {port} antes de transmitir "
                                    "({count} bytes: {raw}). Pode existir outro master "
                                    "ativo no barramento; a pesquisa não foi iniciada.",
                                    port=port,
                                    count=len(traffic),
                                    raw=hex_bytes(traffic[:16]),
                                )
                            )
                        if self.discovery_stop_event.is_set():
                            break
                        self.event_queue.put((
                            "discovery_status",
                            self.trf(
                                "A configurar {baud} / {config}…",
                                baud=baud,
                                config=config_label,
                            ),
                        ))

                    response_timeout_s = discovery_response_timeout_seconds(
                        baud, parity, stopbits, minimum_timeout_ms
                    )
                    char_gap_s = modbus_rtu_interframe_gap_seconds(
                        baud, discovery_bits_per_char(parity, stopbits)
                    )

                    for slave in slaves:
                        if self.discovery_stop_event.is_set():
                            break

```

### MBSniffer v3.2/mb_slave_finder.py — lines 1361-1400

```
                    result = None
                    used_fc = None
                    elapsed_ms = None
                    probe_fcs = [3, 4] if fallback_fc04 else [3]

                    probe_plan = [
                        (fc, attempt)
                        for fc in probe_fcs
                        for attempt in range(1, attempts + 1)
                    ]
                    for fc, attempt in probe_plan:
                        if self.discovery_stop_event.is_set():
                            break

                        context = (
                            f"{port} — {baud} / {config_label} / "
                            f"Slave {slave} / FC{fc:02X}"
                        )
                        if attempts > 1:
                            context += f" / {attempt}/{attempts}"

                        # If this fails, the COM/driver is unhealthy. Do not
                        # disguise it as a normal no-response slave timeout.
                        try:
                            ser.reset_input_buffer()
                        except Exception as exc:
                            raise RuntimeError(
                                self.trf(
                                    "Falha ao preparar a receção em {context}: {error}",
                                    context=context,
                                    error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                )
                            ) from exc

                        request = build_discovery_probe(slave, fc)
                        started_probe = time.perf_counter()
                        try:
                            sent = write_discovery_request(
                                ser, request, baud, parity, stopbits,
                                self.discovery_stop_event, serial
```

**MBSniffer v3.2/mb_slave_finder.py — lines 1401-1440**

```
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha ao transmitir em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc

                            if not sent or self.discovery_stop_event.is_set():
                                break

                            try:
                                result = read_discovery_response(
                                    ser, slave, fc, response_timeout_s,
                                    self.discovery_stop_event
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha ao receber em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc

                            if result is not None:
                                elapsed_ms = (
                                    time.perf_counter() - started_probe
                                ) * 1000.0
                                used_fc = fc
                                if char_gap_s > 0:
                                    self.discovery_stop_event.wait(
                                        min(char_gap_s, 0.05)
                                    )
                                break

                        if result is not None and used_fc is not None:
                            found_count += 1
```

**MBSniffer v3.2/mb_slave_finder.py — lines 1441-1480**

```
                        if result["kind"] == "EXCEPTION":
                            code = result["exception_code"]
                            name = MODBUS_EXCEPTION_CODES.get(
                                code, ("Unknown/Reserved Exception", "")
                            )[0]
                            result_text = f"Exception 0x{code:02X} – {name}"
                        else:
                            result_text = "Response"

                        device_id_text = ""
                        if identify_devices and not self.discovery_stop_event.is_set():
                            context = (
                                f"{port} – {baud} / {config_label} / "
                                f"Slave {slave} / FC43/14"
                            )
                            try:
                                device_result = read_basic_device_identification(
                                    ser,
                                    slave,
                                    baud,
                                    parity,
                                    stopbits,
                                    minimum_timeout_ms,
                                    self.discovery_stop_event,
                                    serial,
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha em Device Identification em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc
                            device_id_text = format_device_identification(
                                device_result
                            )
                            if char_gap_s > 0:
                                self.discovery_stop_event.wait(
                                    min(char_gap_s, 0.05)
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1481-1520

```
                                    )

                            self.event_queue.put((
                                "discovery_found",
                                {
                                    "slave": slave,
                                    "baud": baud,
                                    "config": config_label,
                                    "fc": f"{used_fc:02X}",
                                    "result": result_text,
                                    "response_ms": elapsed_ms,
                                    "device_id": device_id_text,
                                    "raw": hex_bytes(result["frame"]),
                                }
                            ))

                        completed += 1
                        elapsed = time.perf_counter() - started
                        eta = (
                            max(
                                0.0,
                                (elapsed / completed) * (total_units - completed)
                            )
                            if completed > 0 else 0.0
                        )

                        self.event_queue.put((
                            "discovery_progress",
                            {
                                "completed": completed,
                                "total": total_units,
                                "baud": baud,
                                "config": config_label,
                                "slave": slave,
                                "elapsed": elapsed,
                                "eta": eta,
                                "found": found_count,
                            }
                        ))

```

### MBSniffer v3.2/mb_slave_finder.py — lines 1521-1560

```
                finally:
                    if ser is not None:
                        cancel_serial_io(ser)
                        try:
                            ser.close()
                        except Exception:
                            pass
                    self.discovery_serial = None

        except Exception as exc:
            if not self.discovery_stop_event.is_set():
                fatal_error = str(exc) or f"{context}: {type(exc).__name__}"

        elapsed = time.perf_counter() - started
        self.event_queue.put((
            "discovery_done",
            {
                "stopped": self.discovery_stop_event.is_set(),
                "error": fatal_error,
                "elapsed": elapsed,
                "completed": completed,
                "total": total_units,
                "found": found_count,
            }
        ))

    def apply_discovery_progress(self, payload):
        completed = int(payload["completed"])
        total = max(1, int(payload["total"]))
        self.discovery_progress_var.set(100.0 * completed / total)
        self.discovery_progress_text_var.set(
            f"{completed} / {payload['total']} — "
            f"{payload['baud']} / {payload['config']} / Slave {payload['slave']}"
        )
        self.discovery_found_var.set(self.trf("Encontrados: {count}", count=payload["found"]))
        self.discovery_status_var.set(
            self.trf(
                "Decorrido {elapsed} — restante ~{remaining}",
                elapsed=format_duration(payload["elapsed"]),
                remaining=format_duration(payload["eta"]),
```

### MBSniffer v3.2/mb_slave_finder.py — lines 1561-1600

```
            )
        )

    def add_discovery_result(self, payload):
        self.discovery_found_count += 1
        self.discovery_found_var.set(
            self.trf("Encontrados: {count}", count=self.discovery_found_count)
        )
        self.discovery_tree.insert(
            "", "end",
            values=(
                payload["slave"],
                payload["baud"],
                payload["config"],
                payload["fc"],
                payload["result"],
                f"{payload['response_ms']:.1f}" if payload.get("response_ms") is not None else "",
                payload.get("device_id", ""),
                payload["raw"],
            )
        )

    def finish_discovery_ui(self, payload):
        self.discovery_active = False
        self.busy = False
        self.discovery_stop_event.clear()
        self.discovery_serial = None
        self.discovery_thread = None

        self.discovery_stop_btn.configure(state="disabled")
        self.discovery_clear_btn.configure(state="normal")
        self.discovery_refresh_btn.configure(state="normal")
        self.discovery_port_combo.configure(state="readonly")

        self.start_btn.configure(state="normal")
        self.stop_btn.configure(state="disabled")
        self.restart_btn.configure(state="normal")
        self.set_sniffer_refresh_state("normal")
        self.set_simulation_button_state("normal")

```

**MBSniffer v3.2/mb_slave_finder.py — lines 1601-1625**

```
        if payload.get("error"):
            self.discovery_status_var.set(self.tr("Erro na pesquisa"))
            messagebox.showerror(self.tr("Erro na Bus Slave Finder"), payload["error"])
        elif payload.get("stopped"):
            self.discovery_status_var.set(
                self.trf(
                    "Pesquisa parada — {found} encontrados — {elapsed}",
                    found=payload["found"],
                    elapsed=format_duration(payload["elapsed"]),
                )
            )
        else:
            self.discovery_progress_var.set(100.0)
            self.discovery_status_var.set(
                self.trf(
                    "Pesquisa concluída — {found} encontrados — {elapsed}",
                    found=payload["found"],
                    elapsed=format_duration(payload["elapsed"]),
                )
            )

        self.update_discovery_estimate()
        self.update_discovery_start_state()
        self.update_com_status()

```

# MBSniffer v3.2/mb_master.py

686 source lines.

**MBSniffer v3.2/mb_master.py — lines 1-40**

```python
#!/usr/bin/env python3
"""Modbus RTU master (poll) engine: frames, data formats and scheduling.

This module has no widgets. ``MasterEngine`` owns one open serial port and
serves every poll window over it: Modbus RTU allows a single request on the
bus at a time, so windows are polled in turn according to their scan rate,
with queued writes and "read once" requests served first. Results are
posted to the application's ``event_queue`` like the other workers.
"""

import math
import struct
import threading
import time
from collections import deque
from dataclasses import dataclass

from mb_config import (
    MASTER_RECONNECT_INTERVAL_SECONDS,
    MASTER_MAX_READ_BITS,
    MASTER_MAX_READ_REGISTERS,
    MASTER_MAX_WRITE_BITS,
    MASTER_MAX_WRITE_REGISTERS,
)
from mb_protocol import append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds
from mb_slave_finder import (
    bus_activity_detected,
    cancel_serial_io,
    discovery_bits_per_char,
    write_discovery_request,
)

READ_FUNCTION_CODES = (1, 2, 3, 4)
BIT_FUNCTION_CODES = (1, 2)
WRITABLE_READ_FCS = {1: (5, 15), 3: (6, 16)}  # read window FC -> write FCs

# ---------------------------------------------------------------------------
# Data formats
# ---------------------------------------------------------------------------

```

### MBSniffer v3.2/mb_master.py — lines 41-80

```
41 BYTE_ORDERS = ("ABCD", "CDAB", "BADC", "DCBA")
42 # 64-bit orders, in the same sequence: big-endian, word swap (registers in
43 # reverse order), byte swap inside each register, little-endian.
44 BYTE_ORDERS_64 = ("ABCDEFGH", "GHEFCDAB", "BADCFEHG", "HGFEDCBA")
45 WIRE_LETTERS = "ABCDEFGH"
46
47 # (key, label, registers per value).  Labels are technical terms and are not
48 # translated.  Byte order letters follow the wire order of the registers:
49 # register n = A B, register n+1 = C D, ...; "ABCD" / "ABCDEFGH" is big-endian.
50 REGISTER_FORMATS = (
51     ("int16", "Signed 16-bit", 1),
52     ("uint16", "Unsigned 16-bit", 1),
53     ("hex16", "Hex 16-bit", 1),
54     ("bin16", "Binary 16-bit", 1),
55 ) + tuple(
56     (f"{kind}_{order}", f"{label} {order}", 2)
57     for kind, label in (
58         ("int32", "Signed 32-bit"),
59         ("uint32", "Unsigned 32-bit"),
60         ("float32", "Float 32-bit"),
61     )
62     for order in BYTE_ORDERS
63 ) + tuple(
64     (f"{kind}_{order}", f"{label} {order}", 4)
65     for kind, label in (
66         ("int64", "Signed 64-bit"),
67         ("uint64", "Unsigned 64-bit"),
68         ("float64", "Double 64-bit"),
69     )
70     for order in BYTE_ORDERS_64
71 )
72 FORMAT_LABELS = {key: label for key, label, _size in REGISTER_FORMATS}
73 FORMAT_KEYS_BY_LABEL = {label: key for key, label, _size in REGISTER_FORMATS}
74 FORMAT_SIZES = {key: size for key, _label, size in REGISTER_FORMATS}
75 DEFAULT_FORMAT = "int16"
76
77 _STRUCT_CODES = {"int32": ">i", "uint32": ">I", "float32": ">f",
78                  "int64": ">q", "uint64": ">Q", "float64": ">d"}
79
80
```

### MBSniffer v3.2/mb_master.py — lines 81-120

```
81 def _value_bytes(registers, order):
82     """Reorder the wire bytes of 2 or 4 registers into big-endian value bytes."""
83     wire = b"".join(bytes(((r >> 8) & 0xFF, r & 0xFF)) for r in registers)
84     return bytes(wire[WIRE_LETTERS.index(letter)] for letter in order)
85 
86 
87 def _registers_from_value_bytes(value, order):
88     wire = bytearray(len(order))
89     for index, letter in enumerate(order):
90         wire[WIRE_LETTERS.index(letter)] = value[index]
91     return [(wire[i] << 8) | wire[i + 1] for i in range(0, len(wire), 2)]
92 
93 
94 def format_float(value, digits=7):
95     if math.isnan(value):
96         return "NaN"
97     if math.isinf(value):
98         return "-Inf" if value < 0 else "Inf"
99     return f"{value:.{digits}g}"
100 
101 
102 def decode_value(registers, fmt):
103     """Numeric value of one value (None for the Hex/Binary display formats)."""
104     if fmt in ("hex16", "bin16"):
105         return None
106     if FORMAT_SIZES[fmt] == 1:
107         value = registers[0] & 0xFFFF
108         return value - 0x10000 if fmt == "int16" and value & 0x8000 else value
109     kind, order = fmt.split("_")
110     return struct.unpack(_STRUCT_CODES[kind], _value_bytes(registers, order))[0]
111 
112 
113 SCALE_MAX_DECIMALS = 6
114 SCALE_MAX_UNIT_LENGTH = 12
115 
116 
117 def validate_scale(raw_min, raw_max, eng_min, eng_max, decimals=1, unit=""):
118     """Return a scale dict from user text/numbers; raise ValueError (PT text)."""
119     try:
120         values = [float(str(v).strip().replace(",", ".")) for v in
```

### MBSniffer v3.2/mb_master.py — lines 121-160

```
                  (raw_min, raw_max, eng_min, eng_max)]
        decimals = int(str(decimals).strip())
    except ValueError:
        raise ValueError("A escala tem de ter números válidos.") from None
    if any(math.isnan(v) or math.isinf(v) for v in values):
        raise ValueError("A escala tem de ter números válidos.")
    if values[0] == values[1]:
        raise ValueError("O bruto mínimo e o bruto máximo têm de ser diferentes.")
    if not 0 <= decimals <= SCALE_MAX_DECIMALS:
        raise ValueError("As casas decimais têm de estar entre 0 e 6.")
    unit = str(unit or "").strip()[:SCALE_MAX_UNIT_LENGTH]
    return {"raw": [values[0], values[1]], "eng": [values[2], values[3]],
            "decimals": decimals, "unit": unit}


def scale_value(number, scale):
    """Linear map from raw units to engineering units through two points."""
    (r1, r2), (e1, e2) = scale["raw"], scale["eng"]
    return e1 + (number - r1) * (e2 - e1) / (r2 - r1)


def unscale_value(value, scale):
    """Inverse of scale_value(): engineering units back to raw units."""
    (r1, r2), (e1, e2) = scale["raw"], scale["eng"]
    if e1 == e2:
        raise ValueError("Esta escala não permite escrever: os valores de engenharia são iguais.")
    return r1 + (value - e1) * (r2 - r1) / (e2 - e1)


def format_scaled(number, scale):
    text = f"{scale_value(number, scale):.{scale['decimals']}f}"
    return f"{text} {scale['unit']}" if scale.get("unit") else text


def format_value(registers, fmt):
    """Display text of one value occupying ``FORMAT_SIZES[fmt]`` registers."""
    if FORMAT_SIZES[fmt] == 1:
        value = registers[0] & 0xFFFF
        if fmt == "int16":
            return str(value - 0x10000 if value & 0x8000 else value)
```

**MBSniffer v3.2/mb_master.py — lines 161-200**

```
        if fmt == "uint16":
            return str(value)
        if fmt == "hex16":
            return f"0x{value:04X}"
        return f"{value >> 8:08b} {value & 0xFF:08b}"

    kind, order = fmt.split("_")
    number = struct.unpack(_STRUCT_CODES[kind], _value_bytes(registers, order))[0]
    if kind == "float32":
        return format_float(number, 7)
    if kind == "float64":
        return format_float(number, 15)
    return str(number)


def register_groups(qty, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None):
    """Split a window's registers into values: [(offset, size, fmt or None)].

    *overrides* maps a PDU address to the format of the value starting there
    (per-address formats). Other registers use *default_fmt*. A value that
    would run past the window, or a default multi-register value cut by an
    override, becomes ``(offset, size, None)`` and is shown as "—".
    """
    overrides = overrides or {}
    default_fmt = default_fmt if default_fmt in FORMAT_SIZES else DEFAULT_FORMAT
    groups = []
    offset = 0
    while offset < qty:
        fmt = overrides.get(start_address + offset)
        if fmt not in FORMAT_SIZES:
            fmt = default_fmt
            size = FORMAT_SIZES[fmt]
            cut = next((j for j in range(offset + 1, min(offset + size, qty))
                        if start_address + j in overrides), None)
            if cut is not None:
                groups.append((offset, cut - offset, None))
                offset = cut
                continue
        size = FORMAT_SIZES[fmt]
        if offset + size > qty:
```

**MBSniffer v3.2/mb_master.py — lines 201-240**

```
            groups.append((offset, qty - offset, None))
            break
        groups.append((offset, size, fmt))
        offset += size
    return groups


def format_window_values(registers, start_address=0, default_fmt=DEFAULT_FORMAT,
                         overrides=None, scales=None):
    """Return one display string per register, honouring per-address formats.

    A multi-register value is shown on the row of its first register; the
    other rows of the group are empty. Incomplete groups show "—". *scales*
    maps the PDU address where a value starts to a scale (validate_scale());
    scaled values are shown in engineering units (Hex/Binary stay raw).
    """
    scales = scales or {}
    out = []
    for offset, size, fmt in register_groups(len(registers), start_address,
                                             default_fmt, overrides):
        if fmt is None:
            out.extend(["—"] + [""] * (size - 1))
            continue
        group = registers[offset:offset + size]
        scale = scales.get(start_address + offset)
        number = decode_value(group, fmt) if scale else None
        if scale and number is not None and not (isinstance(number, float)
                                                 and (math.isnan(number) or math.isinf(number))):
            out.append(format_scaled(number, scale))
        else:
            out.append(format_value(group, fmt))
        out.extend([""] * (size - 1))
    return out


def raw_text_for_scaled(value_text, scale, fmt):
    """Convert an engineering value typed by the user into raw text for *fmt*."""
    try:
        value = float(str(value_text).strip().replace(",", "."))
    except ValueError:
```

### MBSniffer v3.2/mb_master.py — lines 241-280

```
241         raise ValueError("Valor fora do formato selecionado.") from None
242     raw = unscale_value(value, scale)
243     if fmt.split("_")[0].startswith("float"):
244         return repr(raw)
245     return str(int(round(raw)))
246
247
248 def format_register_values(registers, fmt):
249     """Display strings for registers that all use one format."""
250     return format_window_values(registers, 0, fmt, None)
251
252
253 def _parse_int(text):
254     text = str(text).strip().replace(" ", "").replace("_", "")
255     lowered = text.lower()
256     if lowered.startswith(("0x", "-0x")):
257         return int(text, 16)
258     if lowered.startswith(("0b", "-0b")):
259         return int(text, 2)
260     return int(text, 10)
261
262
263 def encode_register_value(text, fmt):
264     """Convert one user-entered value to the registers it occupies.
265
266     Raises ValueError with a message key (PT canonical text) when the value
267     does not fit the format.
268     """
269     fmt = fmt if fmt in FORMAT_SIZES else DEFAULT_FORMAT
270     text = str(text).strip()
271     if not text:
272         raise ValueError("Valor vazio.")
273     try:
274         if fmt == "hex16":
275             value = int(text[2:] if text.lower().startswith("0x") else text, 16)
276             if not 0 <= value <= 0xFFFF:
277                 raise ValueError
278             return [value]
279         if fmt == "bin16":
280             value = int(text.replace(" ", "")[2:] if text.lower().startswith("0b")
```

### MBSniffer v3.2/mb_master.py — lines 281-320

```
                        else text.replace(" ", ""), 2)
            if not 0 <= value <= 0xFFFF:
                raise ValueError
            return [value]
        if fmt == "int16":
            value = _parse_int(text)
            if not -0x8000 <= value <= 0x7FFF:
                raise ValueError
            return [value & 0xFFFF]
        if fmt == "uint16":
            value = _parse_int(text)
            if not 0 <= value <= 0xFFFF:
                raise ValueError
            return [value]

        kind, order = fmt.split("_")
        code = _STRUCT_CODES[kind]
        if kind.startswith("float"):
            number = float(text.replace(",", "."))
            raw = struct.pack(code, number)
            if math.isinf(struct.unpack(code, raw)[0]) and not math.isinf(number):
                raise ValueError
        else:
            raw = struct.pack(code, _parse_int(text))
        return _registers_from_value_bytes(raw, order)
    except (ValueError, OverflowError, struct.error):
        raise ValueError("Valor fora do formato selecionado.") from None


def encode_coil_value(text):
    value = str(text).strip().lower()
    if value in ("1", "on", "true"):
        return 1
    if value in ("0", "off", "false"):
        return 0
    raise ValueError("Uma coil só aceita 0 ou 1.")


# ---------------------------------------------------------------------------
# Request frames
```

### MBSniffer v3.2/mb_master.py — lines 321-360

```
# ------------------------------------------------------------------------

def _u16(value):
    return bytes(((value >> 8) & 0xFF, value & 0xFF))


def read_limit(fc):
    return MASTER_MAX_READ_BITS if fc in BIT_FUNCTION_CODES else MASTER_MAX_READ_REGISTERS


def validate_read_definition(slave, fc, address, qty):
    """Raise ValueError (PT canonical message) for an invalid poll definition."""
    if not 1 <= int(slave) <= 247:
        raise ValueError("O Slave ID de leitura tem de estar entre 1 e 247.")
    if int(fc) not in READ_FUNCTION_CODES:
        raise ValueError("Função de leitura não suportada.")
    if not 0 <= int(address) <= 0xFFFF:
        raise ValueError("O endereço tem de estar entre 0 e 65535.")
    if not 1 <= int(qty) <= read_limit(int(fc)):
        raise ValueError("Quantidade fora do limite desta função.")
    if int(address) + int(qty) > 0x10000:
        raise ValueError("O intervalo de endereços ultrapassa 65535.")


def build_read_request(slave, fc, address, qty):
    validate_read_definition(slave, fc, address, qty)
    return append_crc(bytes((int(slave), int(fc))) + _u16(int(address)) + _u16(int(qty)))


def build_write_request(slave, fc, address, values):
    """Build an FC05/06/15/16 request. *values* are 0/1 bits or 16-bit registers."""
    slave, fc, address = int(slave), int(fc), int(address)
    values = [int(v) for v in values]
    if not 0 <= slave <= 247:
        raise ValueError("O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.")
    if not 0 <= address <= 0xFFFF or address + max(1, len(values)) > 0x10000:
        raise ValueError("O endereço tem de estar entre 0 e 65535.")
    if fc == 5:
        if len(values) != 1:
            raise ValueError("FC05 escreve exatamente uma coil.")
```

### MBSniffer v3.2/mb_master.py — lines 361-400

```
361         payload = _u16(address) + (b"\xFF\x00" if values[0] else b"\x00\x00")
362     elif fc == 6:
363         if len(values) != 1:
364             raise ValueError("FC06 escreve exatamente um registo.")
365         payload = _u16(address) + _u16(values[0] & 0xFFFF)
366     elif fc == 15:
367         if not 1 <= len(values) <= MASTER_MAX_WRITE_BITS:
368             raise ValueError("Quantidade fora do limite desta função.")
369         data = bytearray((len(values) + 7) // 8)
370         for index, bit in enumerate(values):
371             if bit:
372                 data[index // 8] |= 1 << (index % 8)
373         payload = _u16(address) + _u16(len(values)) + bytes((len(data),)) + bytes(data)
374     elif fc == 16:
375         if not 1 <= len(values) <= MASTER_MAX_WRITE_REGISTERS:
376             raise ValueError("Quantidade fora do limite desta função.")
377         data = b"".join(_u16(v & 0xFFFF) for v in values)
378         payload = _u16(address) + _u16(len(values)) + bytes((len(data),)) + data
379     else:
380         raise ValueError("Função de escrita não suportada.")
381     return append_crc(bytes((slave, fc)) + payload)
382 
383 
384 def expected_response_length(request):
385     fc = request[1]
386     if fc in BIT_FUNCTION_CODES:
387         qty = (request[4] << 8) | request[5]
388         return 5 + (qty + 7) // 8
389     if fc in (3, 4):
390         qty = (request[4] << 8) | request[5]
391         return 5 + 2 * qty
392     return 8  # FC05/06/15/16
393 
394 
395 # ---------------------------------------------------------------------------
396 # Response matching
397 # ---------------------------------------------------------------------------
398 
399 def find_master_response(buffer, request):
400     """Find the response to *request* in the received bytes.
```

**MBSniffer v3.2/mb_master.py — lines 401-440**

```
401
402     Returns a dict with ``kind`` RESPONSE or EXCEPTION, the ``frame`` and,
403     for reads, the decoded ``values`` (bits or registers); None when no
404     complete, CRC-valid, matching frame is present yet. A request echoed by
405     an RS485 adapter is skipped for reads because its length and CRC do not
406     fit the expected response. For FC05/06 a slave reply is itself an exact
407     echo, so an adapter echo cannot be distinguished from it.
408     """
409     data = bytes(buffer)
410     slave, fc = request[0], request[1]
411     length = expected_response_length(request)
412
413     for offset in range(len(data)):
414         if data[offset] != slave or offset + 2 > len(data):
415             continue
416         response_fc = data[offset + 1]
417
418         if response_fc == (fc | 0x80) and offset + 5 <= len(data):
419             frame = data[offset:offset + 5]
420             if crc_ok(frame):
421                 return {"kind": "EXCEPTION", "frame": frame, "exception_code": frame[2],
422                         "values": None}
423
424         if response_fc != fc or offset + length > len(data):
425             continue
426         frame = data[offset:offset + length]
427         if not crc_ok(frame):
428             continue
429
430         if fc in READ_FUNCTION_CODES:
431             if frame[2] != length - 5:
432                 continue
433             payload = frame[3:-2]
434             qty = (request[4] << 8) | request[5]
435             if fc in BIT_FUNCTION_CODES:
436                 values = [(payload[i // 8] >> (i % 8)) & 1 for i in range(qty)]
437             else:
438                 values = [(payload[i] << 8) | payload[i + 1] for i in range(0, len(payload), 2)]
439             return {"kind": "RESPONSE", "frame": frame, "exception_code": None, "values": values}
440
```

### MBSniffer v3.2/mb_master.py — lines 441-480

```
441         if fc in (5, 6) and frame != request:
442             continue
443         if fc in (15, 16) and frame[2:6] != request[2:6]:
444             continue
445         return {"kind": "RESPONSE", "frame": frame, "exception_code": None, "values": None}
446 
447     return None
448 
449 
450 # ---------------------------------------------------------------------------
451 # Poll definitions and engine
452 # ---------------------------------------------------------------------------
453 
454 @dataclass
455 class PollDefinition:
456     window_id: int
457     slave: int
458     fc: int
459     address: int
460     qty: int
461     scan_ms: int
462     enabled: bool = True
463 
464     def request(self):
465         return build_read_request(self.slave, self.fc, self.address, self.qty)
466 
467 
468 class MasterEngine:
469     """Serve all poll windows over one serial port, one transaction at a time.
470 
471     Order of service: queued writes, then "read once" requests, then the
472     enabled poll whose next due time is earliest. A poll becomes due again
473     ``scan_ms`` after its previous transaction started (0 = continuously).
474     After every transaction the bus is left silent for at least the
475     configured delay between polls, and never less than Modbus t3.5.
476 
477     With *reopen* (a callable returning a new open serial object), a serial or
478     driver failure does not end the session: the engine closes the port, posts
479     ``master_reconnecting`` and retries every MASTER_RECONNECT_INTERVAL_SECONDS
480     until the port opens again (``master_reconnected``) or the engine stops.
```

### MBSniffer v3.2/mb_master.py — lines 481-520

```
481     """
482
483     def __init__(self, ser, event_queue, *, baud, parity, stopbits,
484                  response_timeout_ms, delay_ms, serial_module=None,
485                  idle_check_seconds=0.0, reopen=None,
486                  reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS):
487         self.ser = ser
488         self.reopen = reopen
489         self.reconnect_interval_s = float(reconnect_interval_s)
490         self.event_queue = event_queue
491         self.baud = int(baud)
492         self.parity = parity
493         self.stopbits = stopbits
494         self.response_timeout_s = max(0.001, float(response_timeout_ms) / 1000.0)
495         gap_s = modbus_rtu_interframe_gap_seconds(
496             self.baud, discovery_bits_per_char(parity, stopbits)
497         )
498         self.delay_s = max(gap_s, float(delay_ms) / 1000.0)
499         self.serial_module = serial_module
500         self.idle_check_seconds = float(idle_check_seconds)
501
502         self._lock = threading.Lock()
503         self._polls = {}           # window_id -> [PollDefinition, next_due]
504         self._writes = deque()     # (window_id, request, description)
505         self._once = deque()       # window_id
506         self._wake = threading.Event()
507         self.stop_event = threading.Event()
508         self.thread = None
509
510     # -- control (any thread) -----------------------------------------------
511     def set_poll(self, definition):
512         with self._lock:
513             self._polls[definition.window_id] = [definition, 0.0]
514         self._wake.set()
515
516     def remove_poll(self, window_id):
517         with self._lock:
518             self._polls.pop(window_id, None)
519             self._once = deque(w for w in self._once if w != window_id)
520
```

### MBSniffer v3.2/mb_master.py — lines 521-560

```
521     def read_once(self, window_id):
522         with self._lock:
523             if window_id in self._polls and window_id not in self._once:
524                 self._once.append(window_id)
525         self._wake.set()
526 
527     def queue_write(self, window_id, request, description=""):
528         with self._lock:
529             self._writes.append((window_id, bytes(request), description))
530         self._wake.set()
531 
532     def start(self):
533         self.thread = threading.Thread(target=self.run, daemon=True)
534         self.thread.start()
535 
536     def stop(self, join_timeout=1.0):
537         self.stop_event.set()
538         self._wake.set()
539         cancel_serial_io(self.ser)
540         thread = self.thread
541         if thread is not None and thread.is_alive() and thread is not threading.current_thread():
542             thread.join(timeout=join_timeout)
543 
544     # -- scheduling -----------------------------------------------------------
545     def next_job(self, now=None):
546         """Return ('write', item), ('read', definition, once) or ('wait', seconds)."""
547         now = time.perf_counter() if now is None else now
548         with self._lock:
549             if self._writes:
550                 return ("write", self._writes.popleft())
551             while self._once:
552                 window_id = self._once.popleft()
553                 entry = self._polls.get(window_id)
554                 if entry is not None:
555                     return ("read", entry[0], True)
556             due = [
557                 (entry[1], window_id)
558                 for window_id, entry in self._polls.items()
559                 if entry[0].enabled
560             ]
```

### MBSniffer v3.2/mb_master.py — lines 561-600

```
561             if not due:
562                 return ("wait", 0.5)
563             next_due, window_id = min(due)
564             if next_due > now:
565                 return ("wait", min(0.5, next_due - now))
566             entry = self._polls[window_id]
567             entry[1] = now + max(0, entry[0].scan_ms) / 1000.0
568             return ("read", entry[0], False)
569 
570     # -- worker ------------------------------------------------------------
571     def run(self):
572         error = None
573         try:
574             if self.idle_check_seconds > 0:
575                 traffic = bus_activity_detected(
576                     self.ser, self.idle_check_seconds, self.stop_event
577                 )
578                 if traffic:
579                     error = ("traffic", hex_bytes(traffic[:16]), len(traffic))
580                     return
581             self.event_queue.put(("master_ready", None))
582 
583             while not self.stop_event.is_set():
584                 try:
585                     self._serve_jobs()
586                 except Exception as exc:  # serial/driver failure
587                     if self.stop_event.is_set():
588                         break
589                     detail = f"{type(exc).__name__}: {exc}"
590                     if self.reopen is None:
591                         error = ("serial", detail, 0)
592                         break
593                     if not self._reconnect(detail):
594                         break
595         except Exception as exc:  # failure during the idle check
596             if not self.stop_event.is_set():
597                 error = ("serial", f"{type(exc).__name__}: {exc}", 0)
598         finally:
599             self.event_queue.put(("master_stopped", {"error": error}))
600 
```

**MBSniffer v3.2/mb_master.py — lines 601-640**

```
601     def _serve_jobs(self):
602         """Serve writes / reads until stopped; serial errors propagate."""
603         while not self.stop_event.is_set():
604             job = self.next_job()
605             if job[0] == "wait":
606                 self._wake.wait(job[1])
607                 self._wake.clear()
608                 continue
609 
610             if job[0] == "write":
611                 window_id, request, description = job[1]
612                 result = self.transact(request)
613                 result.update(kind="write", window_id=window_id, description=description)
614             else:
615                 definition, once = job[1], job[2]
616                 result = self.transact(definition.request())
617                 result.update(kind="read", window_id=definition.window_id, once=once,
618                               fc=definition.fc, address=definition.address)
619             if self.stop_event.is_set():
620                 return
621             self.event_queue.put(("master_result", result))
622             if self.stop_event.wait(self.delay_s):
623                 return
624 
625     def _reconnect(self, detail):
626         """Close the failed port and reopen it; False when stopped meanwhile."""
627         try:
628             self.ser.close()
629         except Exception:
630             pass
631         self.event_queue.put(("master_reconnecting", {"error": detail}))
632         while not self.stop_event.wait(self.reconnect_interval_s):
633             try:
634                 ser = self.reopen()
635             except Exception:
636                 continue
637             self.ser = ser
638             self.event_queue.put(("master_reconnected", None))
639             return True
640         return False
```

**MBSniffer v3.2/mb_master.py — lines 641-680**

```
641
642     def transact(self, request):
643         """Send one request and wait for its response (no wait for broadcast)."""
644         try:
645             self.ser.reset_input_buffer()
646         except Exception:
647             pass
648         sent = write_discovery_request(
649             self.ser, request, self.baud, self.parity, self.stopbits,
650             self.stop_event, self.serial_module,
651         )
652         # write_discovery_request() returns once the request has left the
653         # wire, so response time and timeout count from the end of the request.
654         started = time.perf_counter()
655         result = {"tx": hex_bytes(request), "rx": "", "status": "stopped",
656                   "values": None, "exception_code": None, "response_ms": None}
657         if not sent:
658             return result
659         if request[0] == 0:
660             result["status"] = "broadcast"
661             return result
662
663         deadline = time.perf_counter() + self.response_timeout_s
664         buffer = bytearray()
665         while time.perf_counter() < deadline and not self.stop_event.is_set():
666             waiting = int(getattr(self.ser, "in_waiting", 0) or 0)
667             if waiting:
668                 buffer.extend(self.ser.read(waiting) or b"")
669                 found = find_master_response(buffer, request)
670                 if found is not None:
671                     result.update(
672                         status="exception" if found["kind"] == "EXCEPTION" else "ok",
673                         values=found["values"],
674                         exception_code=found["exception_code"],
675                         rx=hex_bytes(found["frame"]),
676                         response_ms=(time.perf_counter() - started) * 1000.0,
677                     )
678                     return result
679             elif self.stop_event.wait(0.001):
680                 break
```

**MBSniffer v3.2/mb_master.py — lines 681-686**

```
681
682         if self.stop_event.is_set():
683             return result
684         result["rx"] = hex_bytes(buffer)
685         result["status"] = "invalid" if buffer else "timeout"
686         return result
```

# MBSniffer v3.2/mb_master_ui.py

1993 source lines.

**MBSniffer v3.2/mb_master_ui.py — lines 1-40**

```
#!/usr/bin/env python3
"""Master (poll) tab: one window per Slave / data area, polled over one COM.

The protocol and scheduling live in ``mb_master.py``; this mixin owns the
widgets. Each poll window has a model (``MasterWindow``) and a pane that can
sit in the tab's grid or be popped out into its own top-level window.
"""

import json
import os
import subprocess
import sys
import time
import tkinter as tk
from tkinter import font as tkfont
from tkinter import messagebox, ttk

from mb_config import (
    DISCOVERY_BAUD_RATES,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MASTER_DEFAULT_DELAY_MS,
    MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
    MASTER_DEFAULT_SCAN_MS,
    MASTER_MAX_DELAY_MS,
    MASTER_MAX_RESPONSE_TIMEOUT_MS,
    MASTER_MAX_SCAN_MS,
    MASTER_MAX_WINDOWS,
    MASTER_MIN_DELAY_MS,
    MASTER_MIN_RESPONSE_TIMEOUT_MS,
    MASTER_WORKSPACE_FORMAT,
    MASTER_WORKSPACE_VERSION,
    MODBUS_EXCEPTION_CODES,
    get_workspaces_folder,
)
from mb_diagnostics import FUNCTION_NAMES
from mb_master import (
    BIT_FUNCTION_CODES,
    DEFAULT_FORMAT,
    FORMAT_KEYS_BY_LABEL,
    FORMAT_LABELS,
```

### MBSniffer v3.2/mb_master_ui.py — lines 41-80

```
41      FORMAT_SIZES,
42      READ_FUNCTION_CODES,
43      REGISTER_FORMATS,
44      WRITABLE_READ_FCS,
45      MasterEngine,
46      PollDefinition,
47      build_write_request,
48      encode_coil_value,
49      encode_register_value,
50      format_window_values,
51      raw_text_for_scaled,
52      register_groups,
53      validate_read_definition,
54      validate_scale,
55  )
56  from mb_theme import theme_colors
57  from mb_widgets import RefreshButton
58
59  MASTER_PARITIES = ("None", "Even", "Odd")
60  MASTER_STOP_BITS = ("1", "2")
61  MASTER_PANE_MIN_WIDTH = 600
62  MASTER_PANE_MIN_ROWS = 3
63  MASTER_PANE_MAX_ROWS = 15
64  MASTER_OK_MESSAGE_SECONDS = 3.0
65  # Background of every second table row: one step from the table background.
66  MASTER_STRIPE_LIGHT = "#f2f4f6"
67  MASTER_STRIPE_DARK = "#262d34"
68  MASTER_MAX_TITLE_LENGTH = 40
69  WORKSPACE_SUFFIX = ".mbsw"
70  # Characters Windows does not allow in file names.
71  INVALID_FILENAME_CHARS = '<>:"/\\|?*'
72  RESERVED_FILENAMES = ({"CON", "PRN", "AUX", "NUL"} | {f"COM{i}" for i in range(1, 10)}
73                        | {f"LPT{i}" for i in range(1, 10)})
74  WHEEL_GUARD_TAG = "MBSniffer.MouseWheelGuard"
75  MASTER_WHEEL_TAG = "MBSniffer.MasterWheel"
76
77
78  def fc_choice_label(fc):
79      return f"{fc:02d} {FUNCTION_NAMES[fc]}"
80
```

**MBSniffer v3.2/mb_master_ui.py — lines 81-120**

```
 81
 82 FC_CHOICES = tuple(fc_choice_label(fc) for fc in READ_FUNCTION_CODES)
 83
 84
 85 def parse_int_field(text, minimum, maximum, message):
 86     try:
 87         value = int(str(text).strip())
 88     except ValueError:
 89         raise ValueError(message) from None
 90     if not minimum <= value <= maximum:
 91         raise ValueError(message)
 92     return value
 93
 94
 95 def validate_workspace_name(name):
 96     """Return a cleaned workspace name usable as a Windows file name."""
 97     name = str(name or "").strip().rstrip(".")
 98     if not name:
 99         raise ValueError("O nome do workspace não pode ficar vazio.")
100     if len(name) > 60 or any(ch in INVALID_FILENAME_CHARS for ch in name) \
101             or any(ord(ch) < 32 for ch in name):
102         raise ValueError("O nome do workspace não pode ter os caracteres < > : \" / \\ | ? *.")
103     if name.split(".")[0].upper() in RESERVED_FILENAMES:
104         raise ValueError("Esse nome está reservado pelo Windows.")
105     return name
106
107
108 def workspace_path(name):
109     return get_workspaces_folder() / f"{validate_workspace_name(name)}{WORKSPACE_SUFFIX}"
110
111
112 def list_workspaces():
113     """Names of the saved workspaces, sorted alphabetically."""
114     try:
115         files = get_workspaces_folder().glob(f"*{WORKSPACE_SUFFIX}")
116         return sorted((f.stem for f in files if f.is_file()), key=str.casefold)
117     except OSError:
118         return []
119
120
```

### MBSniffer v3.2/mb_master_ui.py — lines 121-160

```
class MasterWindow:
    """State of one poll window (independent of where its pane is shown)."""

    def __init__(self, window_id, number, slave=1, fc=3, address=0, qty=10,
                 scan_ms=MASTER_DEFAULT_SCAN_MS, base=0, fmt=DEFAULT_FORMAT,
                 names=None, enabled=True, title="", formats=None, scales=None):
        self.window_id = window_id
        self.number = number
        # User-chosen window name; empty means the default "Janela N".
        self.title = str(title or "").strip()[:MASTER_MAX_TITLE_LENGTH]
        self.slave = slave
        self.fc = fc
        self.address = address
        self.qty = qty
        self.scan_ms = scan_ms
        self.base = base
        self.fmt = fmt
        self.names = dict(names or {})  # PDU address -> label
        # PDU address -> format of the value that starts there (per-address
        # formats chosen with the right-click menu); others use self.fmt.
        self.formats = {int(a): f for a, f in (formats or {}).items() if f in FORMAT_SIZES}
        # PDU address -> scale (raw/engineering points, decimals, unit) of the
        # value that starts there.
        self.scales = {int(a): dict(s) for a, s in (scales or {}).items()}
        self.enabled = enabled
        self.popped = False
        self.reset_counters()

    def reset_counters(self):
        self.tx = 0
        self.errors = 0
        self.timeouts = 0
        self.response_ms = None
        self.values = None
        # True when the latest read failed: the shown values are out of date.
        self.stale = False
        # (canonical PT template, values, style) shown on the status line.
        self.message = ("", {}, "normal")
        self.message_at = 0.0

```

**MBSniffer v3.2/mb_master_ui.py — lines 161-200**

```
    def definition(self):
        return PollDefinition(self.window_id, self.slave, self.fc, self.address,
                              self.qty, self.scan_ms, self.enabled)

    def to_json(self):
        return {
            "title": self.title,
            "slave": self.slave, "fc": self.fc, "address": self.address,
            "qty": self.qty, "scan_ms": self.scan_ms, "base": self.base,
            "format": self.fmt, "enabled": self.enabled,
            "names": {str(address): name for address, name in sorted(self.names.items())},
            "formats": {str(address): fmt for address, fmt in sorted(self.formats.items())},
            "scales": {str(address): scale for address, scale in sorted(self.scales.items())},
        }

    def groups(self):
        """[(row offset, size, fmt or None)] of the values shown in this window."""
        return register_groups(self.qty, self.address, self.fmt, self.formats)

    def group_of_row(self, row):
        for offset, size, fmt in self.groups():
            if offset <= row < offset + size:
                return offset, size, fmt
        return row, 1, None

    @staticmethod
    def validated_kwargs(data):
        """Validate one workspace window entry; raise ValueError if invalid."""
        if not isinstance(data, dict):
            raise ValueError("Janela inválida.")
        slave, fc = int(data.get("slave", 1)), int(data.get("fc", 3))
        address, qty = int(data.get("address", 0)), int(data.get("qty", 10))
        validate_read_definition(slave, fc, address, qty)
        scan_ms = int(data.get("scan_ms", MASTER_DEFAULT_SCAN_MS))
        if not 0 <= scan_ms <= MASTER_MAX_SCAN_MS:
            raise ValueError("Scan rate fora do intervalo.")
        fmt = data.get("format", DEFAULT_FORMAT)
        names = data.get("names", {})
        formats = data.get("formats", {})
        scales = {}
```

### MBSniffer v3.2/mb_master_ui.py — lines 201-240

```
        for key, item in (data.get("scales", {}) or {}).items() \
                if isinstance(data.get("scales", {}), dict) else ():
            try:
                if not str(key).isdigit() or int(key) > 0xFFFF or not isinstance(item, dict):
                    raise ValueError
                raw, eng = item.get("raw", []), item.get("eng", [])
                scales[int(key)] = validate_scale(raw[0], raw[1], eng[0], eng[1],
                                                  item.get("decimals", 1), item.get("unit", ""))
            except (ValueError, TypeError, IndexError):
                continue
        return {
            "title": str(data.get("title", "") or "").strip()[:MASTER_MAX_TITLE_LENGTH],
            "slave": slave, "fc": fc, "address": address, "qty": qty,
            "scan_ms": scan_ms, "base": 1 if data.get("base") == 1 else 0,
            "fmt": fmt if fmt in FORMAT_SIZES else DEFAULT_FORMAT,
            "enabled": bool(data.get("enabled", True)),
            "names": {
                int(k): str(v) for k, v in (names.items() if isinstance(names, dict) else ())
                if str(k).isdigit() and str(v).strip()
            },
            "formats": {
                int(k): v for k, v in (formats.items() if isinstance(formats, dict) else ())
                if str(k).isdigit() and int(k) <= 0xFFFF and v in FORMAT_SIZES
            },
            "scales": scales,
        }


def apply_format_to_rows(window, rows, fmt):
    """Set the per-address format of the selected rows of *window*.

    16-bit formats apply to every selected row. Multi-register formats need
    consecutive rows in a multiple of the format size and are applied in
    groups (2 registers for 32-bit, 4 for 64-bit). *fmt* None restores the
    window format. Existing per-address formats overlapping the selection are
    removed. Raises ValueError (PT canonical message) for an invalid selection.
    """
    rows = sorted(set(int(r) for r in rows))
    if not rows:
        raise ValueError("Seleciona pelo menos um endereço.")
```

### MBSniffer v3.2/mb_master_ui.py — lines 241-280

```
    size = FORMAT_SIZES[fmt] if fmt else 1
    if size > 1:
        if rows != list(range(rows[0], rows[0] + len(rows))):
            raise ValueError("Seleciona registos seguidos para este formato.")
        if len(rows) % size:
            raise ValueError(
                "Este formato ocupa {size} registos: seleciona {size} registos seguidos "
                "(ou um múltiplo de {size})."
            )
        if rows[-1] >= window.qty:
            raise ValueError("Seleciona registos seguidos para este formato.")

    covered = {window.address + r for r in rows}
    for start, existing in list(window.formats.items()):
        span = set(range(start, start + FORMAT_SIZES[existing]))
        if span & covered:
            del window.formats[start]
    if fmt is not None:
        for index in range(0, len(rows), size):
            window.formats[window.address + rows[index]] = fmt


class MasterMixin:
    """Master (poll) tab GUI."""

    # -- construction ------------------------------------------------------
    def build_master_tab(self, page):
        saved = getattr(self, "ui_settings", {}).get("master", {})
        if not isinstance(saved, dict):
            saved = {}

        self.master_windows = {}
        self._master_panes = {}
        self._master_next_id = 1
        self.master_engine = None
        self.master_serial = None
        self.master_active = False
        self._master_reflow_after_id = None
        # Workspace state: name of the open workspace, whether it changed since
        # it was opened/saved, and a guard that suppresses change tracking
```

### MBSniffer v3.2/mb_master_ui.py — lines 281-320

```
        # while a workspace is being loaded.
        self.master_workspace_dirty = False
        self._master_loading = False

        page.columnconfigure(0, weight=1)
        page.rowconfigure(2, weight=1)

        connection = ttk.LabelFrame(page, text="Ligação — o Master transmite no barramento")
        connection.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
        self._master_connection_frame = connection
        row = ttk.Frame(connection)
        row.grid(row=0, column=0, sticky="w", padx=8, pady=(8, 4))

        ttk.Label(row, text="Porta COM").grid(row=0, column=0, padx=(0, 4))
        self.master_port_var = tk.StringVar()
        self.master_port_combo = ttk.Combobox(
            row, textvariable=self.master_port_var, width=13, state="readonly"
        )
        self.master_port_combo.grid(row=0, column=1)
        self.master_refresh_btn = RefreshButton(row, command=self.refresh_ports, size=24)
        self.master_refresh_btn.grid(row=0, column=2, padx=(4, 18))

        def _sync_refresh_square(_event=None):
            height = self.master_port_combo.winfo_height()
            if height > 1:
                self.master_refresh_btn.configure(width=height, height=height)

        self.master_port_combo.bind("<Configure>", _sync_refresh_square, add="+")

        self.master_baud_var = tk.StringVar(value=str(saved.get("baud", "9600")))
        self.master_parity_var = tk.StringVar(value=saved.get("parity", "None"))
        self.master_stop_var = tk.StringVar(value=str(saved.get("stop_bits", "1")))
        self.master_timeout_var = tk.StringVar(
            value=str(saved.get("response_timeout_ms", MASTER_DEFAULT_RESPONSE_TIMEOUT_MS))
        )
        self.master_delay_var = tk.StringVar(
            value=str(saved.get("delay_ms", MASTER_DEFAULT_DELAY_MS))
        )
        fields = (
            ("Baud rate", ttk.Combobox(row, textvariable=self.master_baud_var, width=8,
```

**MBSniffer v3.2/mb_master_ui.py — lines 321-360**

```
                                    state="readonly",
                                    values=[str(b) for b in DISCOVERY_BAUD_RATES])),
            ("Parity", ttk.Combobox(row, textvariable=self.master_parity_var, width=6,
                                    state="readonly", values=MASTER_PARITIES)),
            ("Stop bits", ttk.Combobox(row, textvariable=self.master_stop_var, width=3,
                                    state="readonly", values=MASTER_STOP_BITS)),
            ("Timeout resposta (ms)", ttk.Spinbox(
                row, textvariable=self.master_timeout_var, width=6, increment=50,
                from_=MASTER_MIN_RESPONSE_TIMEOUT_MS, to=MASTER_MAX_RESPONSE_TIMEOUT_MS)),
            ("Atraso entre pedidos (ms)", ttk.Spinbox(
                row, textvariable=self.master_delay_var, width=6, increment=5,
                from_=MASTER_MIN_DELAY_MS, to=MASTER_MAX_DELAY_MS)),
        )
        column = 3
        self._master_setting_widgets = [self.master_port_combo]
        for caption, widget in fields:
            ttk.Label(row, text=caption).grid(row=0, column=column, padx=(0, 4))
            widget.grid(row=0, column=column + 1, padx=(0, 16))
            self._master_setting_widgets.append(widget)
            column += 2
        ttk.Label(row, text="Data bits: 8").grid(row=0, column=column)

        controls = ttk.Frame(connection)
        controls.grid(row=1, column=0, sticky="w", padx=8, pady=(0, 8))
        self.master_safety_var = tk.BooleanVar(value=False)
        ttk.Checkbutton(
            controls,
            text="Confirmo que não existe outro master ativo no barramento.",
            variable=self.master_safety_var,
        ).grid(row=0, column=0, padx=(0, 18))
        self.master_connect_btn = ttk.Button(
            controls, text="Ligar", command=self.master_connect, style="MBS.Primary.TButton"
        )
        self.master_connect_btn.grid(row=0, column=1, padx=(0, 6))
        self.master_disconnect_btn = ttk.Button(
            controls, text="Desligar", command=self.master_disconnect, state="disabled"
        )
        self.master_disconnect_btn.grid(row=0, column=2, padx=(0, 18))
        self.master_status_var = tk.StringVar(value=self.tr("Desligado"))
        ttk.Label(controls, textvariable=self.master_status_var).grid(row=0, column=3)
```

### MBSniffer v3.2/mb_master_ui.py — lines 361-400

```
361
362         toolbar = ttk.Frame(page)
363         toolbar.grid(row=1, column=0, sticky="ew", padx=12, pady=(0, 6))
364         toolbar.columnconfigure(3, weight=1)
365         ttk.Button(toolbar, text="Nova janela", command=self.master_add_window).grid(
366             row=0, column=0, padx=(0, 6))
367         # Workspace menu: the saved workspaces are listed directly in it.
368         self.master_workspace_btn = ttk.Menubutton(toolbar, text="Workspace")
369         self.master_workspace_btn.grid(row=0, column=1, padx=(0, 6))
370         self.master_workspace_menu = tk.Menu(
371             self.master_workspace_btn, tearoff=False,
372             postcommand=self._master_build_workspace_menu,
373         )
374         self.master_workspace_btn.configure(menu=self.master_workspace_menu)
375         self.master_workspace_var = tk.StringVar(value="")
376         ttk.Label(toolbar, textvariable=self.master_workspace_var).grid(
377             row=0, column=2, padx=(0, 6))
378         self.master_current_workspace = ""
379         self._master_update_workspace_label()
380         title_font = tkfont.nametofont("TkDefaultFont").copy()
381         title_font.configure(weight="bold")
382         self._master_title_font = title_font
383         self.ui_style.configure("MBS.Master.Title.TLabel", font=title_font)
384         ttk.Label(
385             toolbar,
386             text=("Duplo clique num valor para escrever, num nome para o editar. "
387                   "Clique direito: formato, copiar e base."),
388         ).grid(row=0, column=3, sticky="w", padx=(12, 0))
389
390         holder = ttk.Frame(page)
391         holder.grid(row=2, column=0, sticky="nsew", padx=12, pady=(0, 12))
392         holder.columnconfigure(0, weight=1)
393         holder.rowconfigure(0, weight=1)
394         self.master_canvas = tk.Canvas(holder, highlightthickness=0, borderwidth=0)
395         self.master_canvas.grid(row=0, column=0, sticky="nsew")
396         scrollbar = ttk.Scrollbar(holder, orient="vertical", command=self.master_canvas.yview)
397         scrollbar.grid(row=0, column=1, sticky="ns")
398         self.master_canvas.configure(yscrollcommand=scrollbar.set)
399         self.master_grid = ttk.Frame(self.master_canvas)
400         self._master_grid_item = self.master_canvas.create_window(
```

**MBSniffer v3.2/mb_master_ui.py — lines 401-440**

```
            (0, 0), window=self.master_grid, anchor="nw"
        )
        self.master_grid.bind(
            "<Configure>",
            lambda _e: self.master_canvas.configure(scrollregion=self.master_canvas.bbox("all")),
        )
        self.master_canvas.bind("<Configure>", self._master_schedule_reflow, add="+")
        self.master_empty_label = ttk.Label(
            self.master_grid,
            text="Sem janelas. Usa \"Nova janela\" para começar a ler um Slave.",
        )

        for sequence in ("<MouseWheel>", "<Button-4>", "<Button-5>"):
            self.bind_class(MASTER_WHEEL_TAG, sequence, self._master_wheel)

        for variable in (self.master_baud_var, self.master_parity_var, self.master_stop_var,
                         self.master_timeout_var, self.master_delay_var):
            variable.trace_add("write", lambda *_a: self._schedule_ui_settings_save())

        self._master_build_row_menu()

        # Reopen the workspace used last time; it is loaded once the window is
        # complete so the COM port list is already known. Never connects.
        last = saved.get("last_workspace", "")
        if last and last in list_workspaces():
            self.after(10, lambda: self._master_restore_workspace(last))
        else:
            self.master_add_window()
            self.master_workspace_dirty = False

    def _master_restore_workspace(self, name):
        if not self.master_open_workspace(name, quiet=True):
            self.master_add_window()
            self.master_workspace_dirty = False

    # -- helpers ---------------------------------------------------------
    def _master_install_wheel_tags(self, root_widget):
        """Give new widgets the global wheel guard; plain panels scroll the grid."""
        stack = [root_widget]
        while stack:
```

### MBSniffer v3.2/mb_master_ui.py — lines 441-480

```
            widget = stack.pop()
            try:
                stack.extend(widget.winfo_children())
                tags = tuple(t for t in widget.bindtags()
                             if t not in (WHEEL_GUARD_TAG, MASTER_WHEEL_TAG))
                if isinstance(widget, (ttk.Treeview, ttk.Scrollbar, tk.Toplevel)):
                    widget.bindtags((WHEEL_GUARD_TAG,) + tags)
                else:
                    widget.bindtags((MASTER_WHEEL_TAG, WHEEL_GUARD_TAG) + tags)
            except tk.TclError:
                continue

    def _master_wheel(self, event):
        canvas = getattr(self, "master_canvas", None)
        if canvas is None:
            return "break"
        try:
            top, bottom = canvas.yview()
            if top <= 0.0 and bottom >= 1.0:
                return "break"
            if getattr(event, "num", None) == 4:
                step = -1
            elif getattr(event, "num", None) == 5:
                step = 1
            else:
                step = -1 if event.delta > 0 else 1
            canvas.yview_scroll(step * 3, "units")
        except tk.TclError:
            pass
        return "break"

    def _master_window_name(self, window):
        return window.title or self.trf("Janela {number}", number=window.number)

    def _master_window_summary(self, window):
        return (f"Slave {window.slave} · FC{window.fc:02d} · "
                f"{window.address + window.base} · {window.qty}")

    def _master_window_title(self, window):
        return f"{self._master_window_name(window)} — {self._master_window_summary(window)}"
```

### MBSniffer v3.2/mb_master_ui.py — lines 481-520

```
481
482     def _master_counters_text(self, window):
483         response = f"{window.response_ms:.1f} ms" if window.response_ms is not None else "—"
484         return (f"Tx: {window.tx}   Err: {window.errors}   "
485                 f"Timeouts: {window.timeouts}   Resp.: {response}")
486
487     def _master_set_message(self, window, template, style="normal", **values):
488         window.message = (template, values, style)
489         window.message_at = time.monotonic()
490         self._master_render_status(window)
491
492     def _master_render_status(self, window):
493         pane = self._master_panes.get(window.window_id)
494         if pane is None:
495             return
496         template, values, style = window.message
497         if not window.enabled and style != "error":
498             template, values, style = "Pausada", {}, "normal"
499         text = self.trf(template, **values) if template else ""
500         try:
501             pane["counters_var"].set(self._master_counters_text(window))
502             pane["message_var"].set(text)
503             pane["message_label"].configure(style={
504                 "error": "MBS.Master.Error.TLabel",
505                 "ok": "MBS.Master.Ok.TLabel",
506             }.get(style, "TLabel"))
507             pane["name_var"].set(self._master_window_name(window))
508             pane["summary_var"].set(self._master_window_summary(window))
509             pane["pause_btn"].configure(text=self.tr("Retomar" if not window.enabled else "Pausar"))
510             pane["popout_btn"].configure(text=self.tr("Ancorar" if window.popped else "Destacar"))
511             if window.popped and pane.get("toplevel") is not None:
512                 pane["toplevel"].title(f"MBSniffer — {self._master_window_title(window)}")
513         except tk.TclError:
514             pass
515
516     # -- panes ---------------------------------------------------------------
517     def _master_build_pane(self, window, parent):
518         frame = ttk.LabelFrame(parent, text="")
519         frame.columnconfigure(0, weight=1)
520         frame.rowconfigure(5, weight=1)
```

**MBSniffer v3.2/mb_master_ui.py — lines 521-560**

```
        pane = {"frame": frame}
        window_id = window.window_id

        # Header: window name + summary on the left; Rename / Pop out / close on
        # the right, so closing a window never depends on the pane being wide.
        header = ttk.Frame(frame)
        header.grid(row=0, column=0, sticky="ew", padx=6, pady=(4, 2))
        header.columnconfigure(1, weight=1)
        pane["name_var"] = tk.StringVar()
        pane["summary_var"] = tk.StringVar()
        name_label = ttk.Label(header, textvariable=pane["name_var"],
                               style="MBS.Master.Title.TLabel")
        name_label.grid(row=0, column=0, sticky="w")
        name_label.bind("<Double-1>", lambda _e: self.master_rename_window(window_id))
        ttk.Label(header, textvariable=pane["summary_var"]).grid(
            row=0, column=1, sticky="w", padx=(10, 0))
        header_buttons = ttk.Frame(header)
        header_buttons.grid(row=0, column=2, sticky="e")
        ttk.Button(header_buttons, text="Renomear", width=10,
                   command=lambda: self.master_rename_window(window_id)).grid(
            row=0, column=0, padx=(0, 3))
        pane["popout_btn"] = ttk.Button(header_buttons, text="Destacar", width=10,
                                        command=lambda: self.master_toggle_popout(window_id))
        pane["popout_btn"].grid(row=0, column=1, padx=(0, 3))
        ttk.Button(header_buttons, text="×", width=3,
                   command=lambda: self.master_close_window(window_id)).grid(row=0, column=2)

        definition = ttk.Frame(frame)
        definition.grid(row=1, column=0, sticky="w", padx=6, pady=(2, 2))
        pane["slave_var"] = tk.StringVar(value=str(window.slave))
        pane["fc_var"] = tk.StringVar(value=fc_choice_label(window.fc))
        pane["address_var"] = tk.StringVar(value=str(window.address + window.base))
        pane["qty_var"] = tk.StringVar(value=str(window.qty))
        pane["scan_var"] = tk.StringVar(value=str(window.scan_ms))
        pane["format_var"] = tk.StringVar(value=FORMAT_LABELS[window.fmt])

        layout = (
            ("Slave", ttk.Spinbox(definition, from_=1, to=247, width=4,
                                  textvariable=pane["slave_var"])),
            ("FC", ttk.Combobox(definition, values=FC_CHOICES, width=24, state="readonly",
```

### MBSniffer v3.2/mb_master_ui.py — lines 561-600

```
                                textvariable=pane["fc_var"])),
            ("Endereço", ttk.Entry(definition, width=6, textvariable=pane["address_var"])),
            ("Qtd.", ttk.Entry(definition, width=5, textvariable=pane["qty_var"])),
        )
        for index, (caption, widget) in enumerate(layout):
            ttk.Label(definition, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
            widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))

        definition2 = ttk.Frame(frame)
        definition2.grid(row=2, column=0, sticky="w", padx=6, pady=2)
        format_combo = ttk.Combobox(
            definition2, values=[FORMAT_LABELS[k] for k in FORMAT_LABELS], width=20,
            state="readonly", textvariable=pane["format_var"],
        )
        layout2 = (
            ("Scan (ms)", ttk.Entry(definition2, width=7, textvariable=pane["scan_var"])),
            ("Formato", format_combo),
        )
        for index, (caption, widget) in enumerate(layout2):
            ttk.Label(definition2, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
            widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))
        pane["format_combo"] = format_combo

        buttons = ttk.Frame(frame)
        buttons.grid(row=3, column=0, sticky="w", padx=6, pady=(2, 2))
        actions = (
            ("Aplicar", lambda: self.master_apply_window(window_id)),
            ("Pausar", lambda: self.master_toggle_pause(window_id)),
            ("Ler 1x", lambda: self.master_read_once(window_id)),
            ("Duplicar", lambda: self.master_duplicate_window(window_id)),
        )
        for index, (caption, command) in enumerate(actions):
            button = ttk.Button(buttons, text=caption, command=command, width=10)
            button.grid(row=0, column=index, padx=(0, 3))
            if caption == "Pausar":
                pane["pause_btn"] = button
            elif caption == "Aplicar":
                pane["apply_btn"] = button

        status = ttk.Frame(frame)
```

### MBSniffer v3.2/mb_master_ui.py — lines 601-640

```
        status.grid(row=4, column=0, sticky="ew", padx=6, pady=(2, 2))
        status.columnconfigure(1, weight=1)
        pane["counters_var"] = tk.StringVar()
        pane["message_var"] = tk.StringVar()
        ttk.Label(status, textvariable=pane["counters_var"]).grid(row=0, column=0, sticky="w")
        pane["message_label"] = ttk.Label(status, textvariable=pane["message_var"])
        pane["message_label"].grid(row=0, column=1, sticky="w", padx=(14, 0))

        # Thin 1 px frame around the table (classic Tk highlight ring, coloured
        # from the palette in _master_configure_tree_tags()).
        table = tk.Frame(frame, borderwidth=0, highlightthickness=1)
        table.grid(row=5, column=0, sticky="nsew", padx=6, pady=(2, 6))
        table.columnconfigure(0, weight=1)
        table.rowconfigure(0, weight=1)
        pane["table_frame"] = table
        tree = ttk.Treeview(table, columns=("address", "name", "value"), show="headings",
                            height=MASTER_PANE_MAX_ROWS, selectmode="extended")
        for column, caption, width, anchor in (
            ("address", "Endereço", 90, "center"),
            ("name", "Nome", 200, "w"),
            ("value", "Valor", 160, "e"),
        ):
            tree.heading(column, text=self.tr(caption), anchor="center")
            tree.column(column, width=width, minwidth=60, anchor=anchor, stretch=column != "address")
        tree.grid(row=0, column=0, sticky="nsew")
        scrollbar = ttk.Scrollbar(table, orient="vertical", command=tree.yview)
        scrollbar.grid(row=0, column=1, sticky="ns")
        tree.configure(yscrollcommand=scrollbar.set)
        tree.bind("<Double-1>", lambda e: self._master_on_double_click(window_id, e))
        tree.bind("<Return>", lambda e: self._master_edit_value(window_id))
        tree.bind("<Button-3>", lambda e: self._master_show_row_menu(window_id, e))
        tree.bind("<ButtonPress-1>", lambda e: self._master_drag_start(window_id, e), add="+")
        tree.bind("<B1-Motion>", lambda e: self._master_drag_move(window_id, e))
        tree.bind("<ButtonRelease-1>", lambda e: self._master_drag_stop(window_id), add="+")
        for sequence in ("<Control-c>", "<Control-C>"):
            tree.bind(sequence, lambda _e: self.master_copy_rows(window_id) or "break")
        pane["tree"] = tree
        self._master_configure_tree_tags(tree)

        for widget in (definition, definition2):
```

**MBSniffer v3.2/mb_master_ui.py — lines 641-680**

```
641             for child in widget.winfo_children():
642                 if isinstance(child, (ttk.Entry, ttk.Spinbox)):
643                     child.bind("<Return>", lambda _e: self.master_apply_window(window_id))
644 
645         # Captions were written in canonical PT; show them in the current language.
646         for widget in (definition, definition2, buttons, header_buttons):
647             for child in widget.winfo_children():
648                 try:
649                     if "text" in child.keys() and child.cget("text"):
650                         child.configure(text=self.tr(child.cget("text")))
651                 except tk.TclError:
652                     pass
653 
654         self._master_panes[window_id] = pane
655         for name in ("slave_var", "fc_var", "address_var", "qty_var", "scan_var", "format_var"):
656             pane[name].trace_add("write",
657                                  lambda *_a: self._master_update_apply_state(window_id))
658         self._master_fill_rows(window)
659         self._master_update_format_state(window)
660         self._master_render_status(window)
661         self._master_update_apply_state(window_id)
662         self._master_install_wheel_tags(frame)
663         return frame
664 
665     def _master_update_format_state(self, window):
666         pane = self._master_panes.get(window.window_id)
667         if pane is not None:
668             state = "disabled" if window.fc in BIT_FUNCTION_CODES else "readonly"
669             pane["format_combo"].configure(state=state)
670 
671     def _master_fill_rows(self, window):
672         pane = self._master_panes.get(window.window_id)
673         if pane is None:
674             return
675         tree = pane["tree"]
676         tree.delete(*tree.get_children())
677         tree.configure(height=max(MASTER_PANE_MIN_ROWS, min(MASTER_PANE_MAX_ROWS, window.qty)))
678         for index in range(window.qty):
679             pdu = window.address + index
680             tree.insert("", "end", iid=f"r{index}", tags=self._master_row_tags(index),
```

**MBSniffer v3.2/mb_master_ui.py — lines 681-720**

```
                        values=(pdu + window.base, window.names.get(pdu, ""), ""))
        self._master_render_values(window)

    def _master_render_values(self, window):
        pane = self._master_panes.get(window.window_id)
        if pane is None:
            return
        tree = pane["tree"]
        stale = ("stale",) if window.stale else ()
        if window.values is None:
            texts = [""] * window.qty
        elif window.fc in BIT_FUNCTION_CODES:
            texts = [str(v) for v in window.values]
        else:
            texts = format_window_values(window.values, window.address, window.fmt,
                                         window.formats, window.scales)
        for index, text in enumerate(texts[:window.qty]):
            try:
                tree.item(f"r{index}", tags=self._master_row_tags(index) + stale)
                tree.set(f"r{index}", "value", text)
            except tk.TclError:
                pass

    def _master_schedule_reflow(self, _event=None):
        if self._master_reflow_after_id is not None:
            try:
                self.after_cancel(self._master_reflow_after_id)
            except tk.TclError:
                pass
        self._master_reflow_after_id = self.after(80, self._master_reflow)

    def _master_reflow(self):
        """Arrange docked panes in as many equal columns as the width allows."""
        self._master_reflow_after_id = None
        canvas = self.master_canvas
        width = max(1, canvas.winfo_width())
        canvas.itemconfigure(self._master_grid_item, width=width)
        docked = [w for w in sorted(self.master_windows.values(), key=lambda w: w.number)
                  if not w.popped]
        # Use the real requested pane width (it grows with Windows display
```

**MBSniffer v3.2/mb_master_ui.py — lines 721-760**

```
        # scaling) so columns never become narrower than their contents.
        pane_width = MASTER_PANE_MIN_WIDTH
        for window in docked:
            pane = self._master_panes.get(window.window_id)
            if pane is not None:
                pane_width = max(pane_width, pane["frame"].winfo_reqwidth() + 8)
        columns = max(1, width // pane_width)
        for column in range(8):
            self.master_grid.columnconfigure(column, weight=1 if column < columns else 0,
                                             uniform="master_panes" if column < columns else "")
        if not docked:
            self.master_empty_label.grid(row=0, column=0, sticky="w", padx=4, pady=8)
        else:
            self.master_empty_label.grid_remove()
        for index, window in enumerate(docked):
            pane = self._master_panes.get(window.window_id)
            if pane is not None:
                pane["frame"].grid(row=index // columns, column=index % columns,
                                   sticky="nsew", padx=(0, 8), pady=(0, 8))

    # -- window actions ------------------------------------------------------
    def master_add_window(self, kwargs=None):
        if len(self.master_windows) >= MASTER_MAX_WINDOWS:
            messagebox.showwarning(
                self.tr("Master"),
                self.trf("Máximo de {count} janelas.", count=MASTER_MAX_WINDOWS),
            )
            return None
        window_id = self._master_next_id
        self._master_next_id += 1
        numbers = {w.number for w in self.master_windows.values()}
        number = next(n for n in range(1, MASTER_MAX_WINDOWS + 2) if n not in numbers)
        window = MasterWindow(window_id, number, **(kwargs or {}))
        self.master_windows[window_id] = window
        self._master_build_pane(window, self.master_grid)
        self._master_reflow()
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        self._master_mark_changed()
        return window
```

**MBSniffer v3.2/mb_master_ui.py — lines 761-800**

```
761
762     def master_duplicate_window(self, window_id):
763         window = self.master_windows.get(window_id)
764         if window is not None:
765             data = MasterWindow.validated_kwargs(window.to_json())
766             self.master_add_window(data)
767
768     def master_close_window(self, window_id):
769         window = self.master_windows.pop(window_id, None)
770         if window is None:
771             return
772         if self.master_engine is not None:
773             self.master_engine.remove_poll(window_id)
774         pane = self._master_panes.pop(window_id, None)
775         if pane is not None:
776             top = pane.get("toplevel")
777             try:
778                 (top if top is not None else pane["frame"]).destroy()
779             except tk.TclError:
780                 pass
781         self._master_reflow()
782         if not self.master_windows and not self._master_loading:
783             # Closing every window leaves no workspace open.
784             self.master_current_workspace = ""
785             self.master_workspace_dirty = False
786             self._master_update_workspace_label()
787             self._schedule_ui_settings_save()
788         else:
789             self._master_mark_changed()
790
791     def master_apply_window(self, window_id):
792         window = self.master_windows.get(window_id)
793         pane = self._master_panes.get(window_id)
794         if window is None or pane is None:
795             return False
796         try:
797             base = window.base
798             slave = parse_int_field(pane["slave_var"].get(), 1, 247,
799                                     "O Slave ID de leitura tem de estar entre 1 e 247.")
800             fc = int(pane["fc_var"].get()[:2])
```

**MBSniffer v3.2/mb_master_ui.py — lines 801-840**

```
            address = parse_int_field(pane["address_var"].get(), base, 0xFFFF + base,
                                      "O endereço tem de estar entre 0 e 65535.") - base
            qty = parse_int_field(pane["qty_var"].get(), 1, 2000,
                                  "Quantidade fora do limite desta função.")
            scan_ms = parse_int_field(pane["scan_var"].get(), 0, MASTER_MAX_SCAN_MS,
                                      "Scan rate fora do intervalo.")
            validate_read_definition(slave, fc, address, qty)
        except ValueError as exc:
            messagebox.showerror(self.tr("Janela inválida"), self.tr(str(exc)))
            return False

        fmt = FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get(), DEFAULT_FORMAT)
        changed_area = (slave, fc, address, qty) != (window.slave, window.fc,
                                                     window.address, window.qty)
        changed = changed_area or (scan_ms, fmt) != (window.scan_ms, window.fmt)
        window.slave, window.fc, window.address, window.qty = slave, fc, address, qty
        window.scan_ms, window.fmt = scan_ms, fmt
        if changed_area:
            window.reset_counters()
        self._master_fill_rows(window)
        self._master_update_format_state(window)
        self._master_render_status(window)
        self._master_update_apply_state(window_id)
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        if changed:
            self._master_mark_changed()
        return True

    def master_toggle_pause(self, window_id):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        window.enabled = not window.enabled
        if window.enabled and window.message[0] == "Pausada":
            window.message = ("", {}, "normal")
        self._master_render_status(window)
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        self._master_mark_changed()
```

### MBSniffer v3.2/mb_master_ui.py — lines 841-880

```
841
842     def master_read_once(self, window_id):
843         if self.master_engine is None:
844             messagebox.showinfo(self.tr("Master"), self.tr("Liga primeiro o Master."))
845             return
846         self.master_engine.read_once(window_id)
847
848     def master_toggle_popout(self, window_id):
849         window = self.master_windows.get(window_id)
850         pane = self._master_panes.pop(window_id, None)
851         if window is None or pane is None:
852             return
853         old_top = pane.get("toplevel")
854         try:
855             pane["frame"].destroy()
856             if old_top is not None:
857                 old_top.destroy()
858         except tk.TclError:
859             pass
860
861         window.popped = not window.popped
862         if window.popped:
863             top = tk.Toplevel(self)
864             top.geometry("660x420")
865             top.minsize(MASTER_PANE_MIN_WIDTH, 300)
866             top.columnconfigure(0, weight=1)
867             top.rowconfigure(0, weight=1)
868             top.protocol("WM_DELETE_WINDOW", lambda: self.master_toggle_popout(window_id))
869             frame = self._master_build_pane(window, top)
870             frame.grid(row=0, column=0, sticky="nsew", padx=8, pady=8)
871             self._master_panes[window_id]["toplevel"] = top
872             self._master_apply_toplevel_palette(top)
873         else:
874             self._master_build_pane(window, self.master_grid)
875         self._master_render_status(window)
876         self._master_reflow()
877
878     # -- value / name editing ------------------------------------------------
879     def _master_on_double_click(self, window_id, event):
880         pane = self._master_panes.get(window_id)
```

### MBSniffer v3.2/mb_master_ui.py — lines 881-920

```
        if pane is None:
            return "break"
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if not row:
            return "break"
        tree.selection_set(row)
        if tree.identify_column(event.x) == "#2":
            self._master_edit_name(window_id, row)
        else:
            self._master_edit_value(window_id)
        return "break"

    def _master_dialog(self, title):
        dialog = tk.Toplevel(self)
        dialog.title(title)
        dialog.transient(self)
        dialog.resizable(False, False)
        self._master_apply_toplevel_palette(dialog)
        body = ttk.Frame(dialog, padding=12)
        body.grid(row=0, column=0, sticky="nsew")
        return dialog, body

    def _master_edit_name(self, window_id, row):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        pdu = window.address + int(row[1:])
        dialog, body = self._master_dialog(self.tr("Nome"))
        ttk.Label(body, text=self.trf("Nome do endereço {address}",
                                      address=pdu + window.base)).grid(row=0, column=0, sticky="w")
        name_var = tk.StringVar(value=window.names.get(pdu, ""))
        entry = ttk.Entry(body, textvariable=name_var, width=36)
        entry.grid(row=1, column=0, pady=(4, 8))

        def save(_event=None):
            name = name_var.get().strip()
            if name:
                window.names[pdu] = name
            else:
```

### MBSniffer v3.2/mb_master_ui.py — lines 921-960

```
                window.names.pop(pdu, None)
            pane = self._master_panes.get(window_id)
            if pane is not None:
                pane["tree"].set(row, "name", name)
            self._master_mark_changed()
            dialog.destroy()

        ttk.Button(body, text=self.tr("Guardar"), command=save).grid(row=2, column=0, sticky="e")
        entry.bind("<Return>", save)
        entry.focus_set()
        dialog.grab_set()

    def _master_edit_value(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None:
            return
        selection = pane["tree"].selection()
        if not selection:
            return
        index = int(selection[0][1:])
        write_fcs = WRITABLE_READ_FCS.get(window.fc)
        if write_fcs is None:
            messagebox.showinfo(
                self.tr("Escrita"),
                self.tr("FC02 e FC04 são só de leitura: abre uma janela FC01 ou FC03 para escrever."),
            )
            return
        bits = window.fc in BIT_FUNCTION_CODES
        if bits:
            size, fmt = 1, None
        else:
            # Multi-register values start on the first register of their group.
            index, size, fmt = window.group_of_row(index)
            if fmt is None:
                messagebox.showinfo(
                    self.tr("Escrita"),
                    self.tr("Este endereço não tem um valor completo no formato atual."),
                )
                return
```

### MBSniffer v3.2/mb_master_ui.py — lines 961-1000

```
        current = ""
        try:
            current = pane["tree"].set(f"r{index}", "value")
        except tk.TclError:
            pass
        if not bits and window.scales.get(window.address + index, {}).get("unit"):
            current = current.rsplit(" ", 1)[0]

        dialog, body = self._master_dialog(self.tr("Escrever"))
        slave_var = tk.StringVar(value=str(window.slave))
        address_var = tk.StringVar(value=str(window.address + index + window.base))
        value_var = tk.StringVar(value=current)
        single_fc, multiple_fc = write_fcs
        fc_var = tk.IntVar(value=single_fc if size == 1 else multiple_fc)

        rows = (
            ("Slave (0 = broadcast)", ttk.Spinbox(body, from_=0, to=247, width=6,
                                                  textvariable=slave_var)),
            ("Endereço", ttk.Entry(body, width=8, textvariable=address_var)),
            ("Valor(es)", ttk.Entry(body, width=30, textvariable=value_var)),
        )
        for row_index, (caption, widget) in enumerate(rows):
            ttk.Label(body, text=self.tr(caption)).grid(row=row_index, column=0, sticky="w",
                                                        pady=2, padx=(0, 8))
            widget.grid(row=row_index, column=1, sticky="w", pady=2)
        functions = ttk.Frame(body)
        functions.grid(row=3, column=0, columnspan=2, sticky="w", pady=(6, 2))
        single = ttk.Radiobutton(functions, text=fc_choice_label_write(single_fc),
                                 variable=fc_var, value=single_fc)
        single.grid(row=0, column=0, sticky="w")
        if size != 1:
            single.configure(state="disabled")
        ttk.Radiobutton(functions, text=fc_choice_label_write(multiple_fc),
                        variable=fc_var, value=multiple_fc).grid(row=1, column=0, sticky="w")
        scale = None if bits else window.scales.get(window.address + index)
        if scale is not None and fmt in ("hex16", "bin16"):
            scale = None
        format_text = "0 / 1" if bits else FORMAT_LABELS[fmt]
        if scale is not None:
            format_text += " · " + self.trf("valores em unidades da escala {unit}",
```

### MBSniffer v3.2/mb_master_ui.py — lines 1001-1040

```
                                        unit=scale.get("unit") or "")
        ttk.Label(
            body,
            text=self.trf("Formato: {fmt}. Vários valores (FC15/FC16): separar com ;",
                          fmt=format_text),
        ).grid(row=4, column=0, columnspan=2, sticky="w", pady=(6, 8))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=5, column=0, columnspan=2, sticky="w")

        def send(_event=None):
            try:
                request, description = self.master_build_write(
                    window, slave_var.get(), address_var.get(), value_var.get(),
                    fc_var.get(), fmt, scale,
                )
            except ValueError as exc:
                message_var.set(self.tr(str(exc)))
                return
            if self.master_engine is None:
                message_var.set(self.tr("Liga primeiro o Master."))
                return
            self.master_engine.queue_write(window.window_id, request, description)
            dialog.destroy()

        buttons = ttk.Frame(body)
        buttons.grid(row=6, column=0, columnspan=2, sticky="e", pady=(8, 0))
        ttk.Button(buttons, text=self.tr("Enviar"), command=send,
                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
        ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(row=0, column=1)
        rows[2][1].bind("<Return>", send)
        rows[2][1].focus_set()
        rows[2][1].select_range(0, "end")
        dialog.grab_set()

    def master_build_write(self, window, slave_text, address_text, values_text, fc,
                           fmt=None, scale=None):
        """Validate the write dialog fields and return (request, description).

        *fmt* is the format of the clicked value (per-address or window format).
```

### MBSniffer v3.2/mb_master_ui.py — lines 1041-1080

```
        With *scale*, the typed values are engineering units and are converted
        back to raw values before encoding.
        """
        fmt = fmt or window.fmt
        slave = parse_int_field(slave_text, 0, 247,
                                "O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.")
        address = parse_int_field(address_text, window.base, 0xFFFF + window.base,
                                  "O endereço tem de estar entre 0 e 65535.") - window.base
        parts = [p for p in str(values_text).split(";") if p.strip()]
        if not parts:
            raise ValueError("Valor vazio.")
        if window.fc in BIT_FUNCTION_CODES:
            values = [encode_coil_value(p) for p in parts]
        else:
            values = []
            for part in parts:
                if scale is not None:
                    part = raw_text_for_scaled(part, scale, fmt)
                values.extend(encode_register_value(part, fmt))
        if fc in (5, 6) and len(values) != 1:
            raise ValueError("FC05/FC06 escrevem um único valor; usa FC15/FC16 para vários.")
        request = build_write_request(slave, fc, address, values)
        description = f"FC{fc:02d} @ {address + window.base} × {len(values)}"
        return request, description

    # -- connection ---------------------------------------------------------
    def _master_connection_settings(self):
        timeout = parse_int_field(
            self.master_timeout_var.get(), MASTER_MIN_RESPONSE_TIMEOUT_MS,
            MASTER_MAX_RESPONSE_TIMEOUT_MS,
            "O timeout de resposta tem de estar entre 10 e 10000 ms.")
        delay = parse_int_field(
            self.master_delay_var.get(), MASTER_MIN_DELAY_MS, MASTER_MAX_DELAY_MS,
            "O atraso entre pedidos tem de estar entre 0 e 10000 ms.")
        return {
            "port": self.master_port_var.get().strip(),
            "baud": int(self.master_baud_var.get()),
            "parity": self.master_parity_var.get(),
            "stopbits": self.master_stop_var.get(),
            "response_timeout_ms": timeout,
```

**MBSniffer v3.2/mb_master_ui.py — lines 1081-1120**

```
1081            "delay_ms": delay,
1082        }
1083
1084    def master_connect(self):
1085        if self.master_active:
1086            return
1087        if self.busy:
1088            messagebox.showwarning(
1089                self.tr("Master"),
1090                self.tr("Para a captura ou a pesquisa em curso antes de ligar o Master."),
1091            )
1092            return
1093        if not self.master_safety_var.get():
1094            messagebox.showwarning(
1095                self.tr("Master"),
1096                self.tr("Confirma primeiro que não existe outro master ativo no barramento."),
1097            )
1098            return
1099        serial, _ = self.get_serial_modules()
1100        if serial is None:
1101            messagebox.showerror(self.tr("pyserial em falta"), self.tr("pyserial não está instalado."))
1102            return
1103        try:
1104            settings = self._master_connection_settings()
1105            if not settings["port"]:
1106                raise ValueError("Seleciona uma porta COM.")
1107        except ValueError as exc:
1108            messagebox.showerror(self.tr("Configuração inválida"), self.tr(str(exc)))
1109            return
1110        for window_id in list(self.master_windows):
1111            if not self.master_apply_window(window_id):
1112                return
1113        try:
1114            ser = self.open_discovery_serial_port(
1115                serial, settings["port"], settings["baud"], settings["parity"],
1116                settings["stopbits"],
1117            )
1118        except Exception as exc:
1119            messagebox.showerror(
1120                self.tr("Master"),
```

### MBSniffer v3.2/mb_master_ui.py — lines 1121-1160

```
                self.trf("Não foi possível abrir {port}: {error}",
                         port=settings["port"], error=f"{type(exc).__name__}: {exc}"),
            )
            return

        self.master_serial = ser
        self.master_engine = MasterEngine(
            ser, self.event_queue, baud=settings["baud"], parity=settings["parity"],
            stopbits=settings["stopbits"],
            response_timeout_ms=settings["response_timeout_ms"],
            delay_ms=settings["delay_ms"], serial_module=serial,
            idle_check_seconds=DISCOVERY_IDLE_CHECK_SECONDS,
            reopen=lambda: self.open_discovery_serial_port(
                serial, settings["port"], settings["baud"], settings["parity"],
                settings["stopbits"],
            ),
        )
        for window in self.master_windows.values():
            window.reset_counters()
            self._master_render_status(window)
            self.master_engine.set_poll(window.definition())
        self.master_active = True
        self.busy = True
        self._master_lock_other_features(True)
        self.master_status_var.set(self.tr("A verificar se o barramento está livre…"))
        self.master_engine.start()

    def master_disconnect(self):
        if self.master_engine is None:
            return
        self.master_status_var.set(self.tr("A desligar…"))
        self.master_disconnect_btn.configure(state="disabled")
        self.master_engine.stop()

    def _master_lock_other_features(self, locked):
        state = "disabled" if locked else "normal"
        self.master_connect_btn.configure(state=state)
        self.master_disconnect_btn.configure(state="normal" if locked else "disabled")
        for widget in self._master_setting_widgets:
            try:
```

### MBSniffer v3.2/mb_master_ui.py — lines 1161-1200

```
                widget.configure(state="disabled" if locked else "readonly"
                                 if isinstance(widget, ttk.Combobox) else "normal")
            except tk.TclError:
                pass
        self.master_refresh_btn.configure(state=state)
        self.start_btn.configure(state=state)
        self.restart_btn.configure(state=state)
        self.set_sniffer_refresh_state(state)
        self.set_simulation_button_state(state)
        self.update_discovery_start_state()

    def _master_finish_session(self, payload):
        engine = self.master_engine
        self.master_engine = None
        if engine is not None:
            engine.stop(join_timeout=0.5)
            self.master_serial = engine.ser
        if self.master_serial is not None:
            try:
                self.master_serial.close()
            except Exception:
                pass
        self.master_serial = None
        self.master_active = False
        self.busy = False
        self._master_lock_other_features(False)
        self.master_status_var.set(self.tr("Desligado"))
        for window in self.master_windows.values():
            if window.values is not None and not window.stale:
                window.stale = True
                self._master_render_values(window)

        error = (payload or {}).get("error")
        if error:
            kind, detail, count = error
            if kind == "traffic":
                text = self.trf(
                    "Foi detetado tráfego em {port} antes de transmitir "
                    "({count} bytes: {raw}). Pode existir outro master "
                    "ativo no barramento; a pesquisa não foi iniciada.",
```

### MBSniffer v3.2/mb_master_ui.py — lines 1201-1240

```
                    port=self.master_port_var.get(), count=count, raw=detail,
                )
            else:
                text = self.trf("Erro na porta série: {error}", error=detail)
            messagebox.showerror(self.tr("Master"), text)

    # -- worker events ---------------------------------------------------------
    def handle_master_event(self, event, payload):
        if event == "master_ready":
            self.master_status_var.set(self.tr("● Ligado"))
        elif event == "master_reconnecting":
            self.master_status_var.set(self.trf(
                "A religar a {port}… (porta falhou: {error})",
                port=self.master_port_var.get(), error=(payload or {}).get("error", ""),
            ))
            for window in self.master_windows.values():
                if window.values is not None and not window.stale:
                    window.stale = True
                    self._master_render_values(window)
        elif event == "master_reconnected":
            engine = self.master_engine
            if engine is not None:
                self.master_serial = engine.ser
            self.master_status_var.set(self.tr("● Ligado"))
        elif event == "master_stopped":
            self._master_finish_session(payload)
        elif event == "master_result":
            self._master_apply_result(payload)

    def _master_apply_result(self, result):
        window = self.master_windows.get(result.get("window_id"))
        if window is None:
            return
        status = result["status"]
        if status == "stopped":
            return
        window.tx += 1
        if result.get("response_ms") is not None:
            window.response_ms = result["response_ms"]

```

### MBSniffer v3.2/mb_master_ui.py — lines 1241-1280

```
        if result["kind"] == "read":
            if (result.get("fc"), result.get("address")) != (window.fc, window.address):
                return  # definition changed while the request was on the bus
            if status == "ok":
                window.values = result["values"]
                window.stale = False
                self._master_render_values(window)
                # A successful read clears errors; a write confirmation stays
                # visible for a few seconds so the user can see it.
                if (window.message[2] != "ok"
                        or time.monotonic() - window.message_at > MASTER_OK_MESSAGE_SECONDS):
                    self._master_set_message(window, "")
                return
        elif status == "ok":
            self._master_set_message(window, "Escrita OK: {what}", "ok",
                                     what=result.get("description", ""))
            if self.master_engine is not None:
                self.master_engine.read_once(window.window_id)
            return
        elif status == "broadcast":
            self._master_set_message(window, "Escrita broadcast enviada: {what}", "ok",
                                     what=result.get("description", ""))
            return

        window.errors += 1
        if result["kind"] == "read" and not window.stale:
            window.stale = True
            self._master_render_values(window)
        if status == "timeout":
            window.timeouts += 1
            self._master_set_message(window, "Timeout", "error")
        elif status == "exception":
            code = result.get("exception_code") or 0
            name = MODBUS_EXCEPTION_CODES.get(code, ("Unknown/Reserved Exception", ""))[0]
            self._master_set_message(window, "Exception 0x{code:02X} — {name}", "error",
                                     code=code, name=name)
        else:
            self._master_set_message(window, "Resposta inválida (CRC ou formato): {rx}",
                                     "error", rx=result.get("rx", "")[:47])

```

**MBSniffer v3.2/mb_master_ui.py — lines 1281-1320**

```
1281    # -- workspace ------------------------------------------------------------
1282    def master_workspace_data(self):
1283        return {
1284            "format": MASTER_WORKSPACE_FORMAT,
1285            "version": MASTER_WORKSPACE_VERSION,
1286            "connection": {
1287                "port": self.master_port_var.get(),
1288                "baud": self.master_baud_var.get(),
1289                "parity": self.master_parity_var.get(),
1290                "stop_bits": self.master_stop_var.get(),
1291                "response_timeout_ms": self.master_timeout_var.get(),
1292                "delay_ms": self.master_delay_var.get(),
1293            },
1294            "windows": [w.to_json() for w in sorted(self.master_windows.values(),
1295                                                    key=lambda w: w.number)],
1296        }
1297
1298    def _master_update_workspace_label(self):
1299        name = getattr(self, "master_current_workspace", "")
1300        if not name:
1301            text = ""
1302        elif getattr(self, "master_workspace_dirty", False):
1303            text = self.trf("Workspace atual: {name} (alterado)", name=name)
1304        else:
1305            text = self.trf("Workspace atual: {name}", name=name)
1306        self.master_workspace_var.set(text)
1307
1308    def _master_mark_changed(self):
1309        """Record a change to the open workspace (ignored while loading one)."""
1310        if getattr(self, "_master_loading", False):
1311            return
1312        if self.master_current_workspace and not self.master_workspace_dirty:
1313            self.master_workspace_dirty = True
1314            self._master_update_workspace_label()
1315
1316    def _master_build_workspace_menu(self):
1317        """Rebuild the Workspace menu from the files in the workspace folder."""
1318        menu = self.master_workspace_menu
1319        menu.delete(0, "end")
1320        for child in menu.winfo_children():
```

**MBSniffer v3.2/mb_master_ui.py — lines 1321-1360**

```
            child.destroy()
        names = list_workspaces()

        menu.add_command(label=self.tr("Guardar workspace…"),
                         command=self.master_save_workspace)
        open_menu = tk.Menu(menu, tearoff=False)
        delete_menu = tk.Menu(menu, tearoff=False)
        for name in names:
            open_menu.add_command(label=name,
                                  command=lambda n=name: self.master_open_workspace(n))
            delete_menu.add_command(label=name,
                                    command=lambda n=name: self.master_delete_workspace(n))
        if not names:
            for submenu in (open_menu, delete_menu):
                submenu.add_command(label=self.tr("(nenhum workspace guardado)"),
                                    state="disabled")
        menu.add_cascade(label=self.tr("Abrir workspace"), menu=open_menu)
        menu.add_cascade(label=self.tr("Apagar workspace"), menu=delete_menu)
        menu.add_separator()
        menu.add_command(label=self.tr("Abrir pasta dos workspaces"),
                         command=self.master_open_workspace_folder)
        self._master_apply_menu_palette()

    def master_save_workspace(self, name=None):
        """Save all windows as a named workspace in the workspace folder."""
        if name is None:
            name = self._master_ask_text(
                self.tr("Guardar workspace"), self.tr("Nome do workspace"),
                self.master_current_workspace or self.tr("Workspace 1"),
            )
            if name is None:
                return False
        try:
            name = validate_workspace_name(name)
        except ValueError as exc:
            messagebox.showerror(self.tr("Guardar workspace"), self.tr(str(exc)))
            return False
        path = workspace_path(name)
        if path.exists() and name != self.master_current_workspace:
            if not messagebox.askyesno(
```

### MBSniffer v3.2/mb_master_ui.py — lines 1361-1400

```
                self.tr("Guardar workspace"),
                self.trf("Já existe um workspace chamado \"{name}\". Substituir?", name=name),
            ):
                return False
        try:
            with open(path, "w", encoding="utf-8") as handle:
                json.dump(self.master_workspace_data(), handle, indent=2, ensure_ascii=False)
        except OSError as exc:
            messagebox.showerror(self.tr("Guardar workspace"), str(exc))
            return False
        self.master_current_workspace = name
        self.master_workspace_dirty = False
        self._master_update_workspace_label()
        self._schedule_ui_settings_save()
        return True

    def master_delete_workspace(self, name, confirm=True):
        if confirm and not messagebox.askyesno(
            self.tr("Apagar workspace"),
            self.trf("Apagar o workspace \"{name}\"?", name=name),
        ):
            return False
        try:
            workspace_path(name).unlink()
        except OSError as exc:
            messagebox.showerror(self.tr("Apagar workspace"), str(exc))
            return False
        if name == self.master_current_workspace:
            self.master_current_workspace = ""
            self.master_workspace_dirty = False
            self._master_update_workspace_label()
            self._schedule_ui_settings_save()
        return True

    def master_open_workspace_folder(self):
        folder = get_workspaces_folder()
        try:
            if os.name == "nt":
                os.startfile(str(folder))
            elif sys.platform == "darwin":
```

### MBSniffer v3.2/mb_master_ui.py — lines 1401-1440

```
1401                subprocess.Popen(["open", str(folder)])
1402            else:
1403                subprocess.Popen(["xdg-open", str(folder)])
1404        except Exception as exc:
1405            messagebox.showerror(self.tr("Abrir pasta dos workspaces"), str(exc))
1406
1407    def master_open_workspace(self, name, quiet=False):
1408        """Replace the current windows with a saved workspace (by name).
1409
1410        *quiet* (start-up restore) suppresses the error dialog when the file
1411        cannot be read.
1412        """
1413        try:
1414            with open(workspace_path(name), encoding="utf-8") as handle:
1415                data = json.load(handle)
1416            if not isinstance(data, dict) or data.get("format") != MASTER_WORKSPACE_FORMAT:
1417                raise ValueError(self.tr("O ficheiro não é um workspace do MBSniffer."))
1418        except (OSError, ValueError) as exc:
1419            if not quiet:
1420                messagebox.showerror(self.tr("Abrir workspace"), str(exc))
1421            return False
1422        self._master_loading = True
1423        try:
1424            return self._master_load_workspace_data(name, data)
1425        finally:
1426            self._master_loading = False
1427
1428    def _master_load_workspace_data(self, name, data):
1429        entries, skipped = [], 0
1430        for item in data.get("windows", [])[:MASTER_MAX_WINDOWS]:
1431            try:
1432                entries.append(MasterWindow.validated_kwargs(item))
1433            except (ValueError, TypeError):
1434                skipped += 1
1435
1436        for window_id in list(self.master_windows):
1437            self.master_close_window(window_id)
1438        connection = data.get("connection", {})
1439        if isinstance(connection, dict) and not self.master_active:
1440            if str(connection.get("baud")) in [str(b) for b in DISCOVERY_BAUD_RATES]:
```

### MBSniffer v3.2/mb_master_ui.py — lines 1441-1480

```
1441                self.master_baud_var.set(str(connection["baud"]))
1442            if connection.get("parity") in MASTER_PARITIES:
1443                self.master_parity_var.set(connection["parity"])
1444            if str(connection.get("stop_bits")) in MASTER_STOP_BITS:
1445                self.master_stop_var.set(str(connection["stop_bits"]))
1446            for key, var in (("response_timeout_ms", self.master_timeout_var),
1447                             ("delay_ms", self.master_delay_var)):
1448                if str(connection.get(key, "")).isdigit():
1449                    var.set(str(connection[key]))
1450            if connection.get("port") in tuple(self.master_port_combo["values"]):
1451                self.master_port_var.set(connection["port"])
1452        for kwargs in entries:
1453            self.master_add_window(kwargs)
1454        self.master_current_workspace = name
1455        self.master_workspace_dirty = False
1456        self._master_update_workspace_label()
1457        self._schedule_ui_settings_save()
1458        if skipped:
1459            messagebox.showwarning(
1460                self.tr("Abrir workspace"),
1461                self.trf("{count} janela(s) inválida(s) foram ignoradas.", count=skipped),
1462            )
1463        return True
1464
1465    # -- window rename ------------------------------------------------------------
1466    def _master_ask_text(self, title, caption, initial):
1467        """Small modal text prompt; returns the text or None when cancelled."""
1468        dialog, body = self._master_dialog(title)
1469        ttk.Label(body, text=caption).grid(row=0, column=0, columnspan=2, sticky="w")
1470        value = tk.StringVar(value=initial)
1471        entry = ttk.Entry(body, textvariable=value, width=36)
1472        entry.grid(row=1, column=0, columnspan=2, pady=(4, 10), sticky="ew")
1473        result = {"text": None}
1474
1475        def accept(_event=None):
1476            result["text"] = value.get()
1477            dialog.destroy()
1478
1479        ttk.Button(body, text=self.tr("OK"), command=accept,
1480                   style="MBS.Primary.TButton").grid(row=2, column=0, sticky="e", padx=(0, 6))
```

### MBSniffer v3.2/mb_master_ui.py — lines 1481-1520

```
        ttk.Button(body, text=self.tr("Cancelar"), command=dialog.destroy).grid(
            row=2, column=1, sticky="e")
        entry.bind("<Return>", accept)
        entry.bind("<Escape>", lambda _e: dialog.destroy())
        entry.focus_set()
        entry.select_range(0, "end")
        dialog.grab_set()
        self.wait_window(dialog)
        return result["text"]

    def master_rename_window(self, window_id, name=None):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        if name is None:
            name = self._master_ask_text(
                self.tr("Renomear janela"),
                self.tr("Nome da janela (vazio = nome por defeito)"),
                self._master_window_name(window),
            )
            if name is None:
                return
        name = name.strip()[:MASTER_MAX_TITLE_LENGTH]
        if name == self.trf("Janela {number}", number=window.number):
            name = ""
        window.title = name
        self._master_render_status(window)
        self._master_mark_changed()

    # -- apply highlight / table tags ------------------------------------------------
    def _master_fields_changed(self, window, pane):
        """True when the definition fields differ from what was last applied."""
        try:
            return (
                pane["slave_var"].get().strip() != str(window.slave)
                or pane["fc_var"].get() != fc_choice_label(window.fc)
                or pane["address_var"].get().strip() != str(window.address + window.base)
                or pane["qty_var"].get().strip() != str(window.qty)
                or pane["scan_var"].get().strip() != str(window.scan_ms)
                or FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get()) != window.fmt
```

### MBSniffer v3.2/mb_master_ui.py — lines 1521-1560

```
            )
        except tk.TclError:
            return False

    def _master_update_apply_state(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None or "apply_btn" not in pane:
            return
        style = ("MBS.Primary.TButton" if self._master_fields_changed(window, pane)
                 else "TButton")
        try:
            pane["apply_btn"].configure(style=style)
        except tk.TclError:
            pass

    @staticmethod
    def _master_row_tags(index):
        """Alternate rows get a slightly darker background (zebra striping)."""
        return ("odd",) if index % 2 else ()

    def _master_configure_tree_tags(self, tree):
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        colors = theme_colors(self, dark)
        try:
            tree.tag_configure("stale", foreground=colors["disabled"])
            tree.tag_configure("odd", background=MASTER_STRIPE_DARK if dark
                               else MASTER_STRIPE_LIGHT)
            frame = tree.master
            if isinstance(frame, tk.Frame):
                frame.configure(highlightbackground=colors["border"],
                                highlightcolor=colors["border"],
                                background=colors["input"])
        except tk.TclError:
            pass

    # -- right-click menu: per-address format, copy, base ---------------------------
    def _master_build_row_menu(self):
        self._master_row_menu = tk.Menu(self, tearoff=False)

```

**MBSniffer v3.2/mb_master_ui.py — lines 1561-1600**

```
1561    # -- drag selection ------------------------------------------------------------
1562    # ttk.Treeview has no rubber-band selection: pressing on a row and dragging
1563    # selects every row from that row to the one under the pointer, scrolling
1564    # when the pointer leaves the table. Ctrl+drag adds the block to the
1565    # selection that existed before the press.
1566    def _master_drag_start(self, window_id, event):
1567        pane = self._master_panes.get(window_id)
1568        if pane is None:
1569            return
1570        tree = pane["tree"]
1571        row = tree.identify_row(event.y)
1572        if not row or tree.identify_region(event.x, event.y) != "cell":
1573            pane["drag"] = None
1574            return
1575        ctrl = bool(event.state & 0x0004)
1576        before = tuple(tree.selection())
1577        pane["drag"] = {"anchor": row, "base": before if ctrl else (), "y": event.y,
1578                        "after_id": None}
1579
1580    def _master_drag_move(self, window_id, event):
1581        pane = self._master_panes.get(window_id)
1582        drag = pane.get("drag") if pane else None
1583        if not drag:
1584            return None
1585        drag["y"] = event.y
1586        self._master_drag_update(window_id)
1587        return "break"
1588
1589    def _master_drag_update(self, window_id):
1590        pane = self._master_panes.get(window_id)
1591        drag = pane.get("drag") if pane else None
1592        if not drag:
1593            return
1594        tree = pane["tree"]
1595        drag["after_id"] = None
1596        height = tree.winfo_height()
1597        y = drag["y"]
1598        if y < 0 or y > height:
1599            # Pointer outside the rows: scroll and keep extending the block.
1600            tree.yview_scroll(-1 if y < 0 else 1, "units")
```

### MBSniffer v3.2/mb_master_ui.py — lines 1601-1640

```
            y = 1 if y < 0 else height - 2
            drag["after_id"] = self.after(60, lambda: self._master_drag_update(window_id))
        row = tree.identify_row(y)
        rows = tree.get_children()
        if not row or drag["anchor"] not in rows:
            return
        first, last = sorted((rows.index(drag["anchor"]), rows.index(row)))
        block = rows[first:last + 1]
        tree.selection_set(tuple(dict.fromkeys(drag["base"] + tuple(block))))

    def _master_drag_stop(self, window_id):
        pane = self._master_panes.get(window_id)
        drag = pane.get("drag") if pane else None
        if drag and drag.get("after_id"):
            try:
                self.after_cancel(drag["after_id"])
            except tk.TclError:
                pass
        if pane is not None:
            pane["drag"] = None

    def _master_selected_rows(self, window_id):
        pane = self._master_panes.get(window_id)
        if pane is None:
            return []
        return sorted(int(iid[1:]) for iid in pane["tree"].selection() if iid.startswith("r"))

    def _master_show_row_menu(self, window_id, event):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None:
            return "break"
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if row and row not in tree.selection():
            tree.selection_set(row)

        menu = self._master_row_menu
        menu.delete(0, "end")
        for child in menu.winfo_children():
```

**MBSniffer v3.2/mb_master_ui.py — lines 1641-1680**

```
            child.destroy()

        registers = window.fc not in BIT_FUNCTION_CODES
        format_menu = tk.Menu(menu, tearoff=False)
        sub_menus = {2: tk.Menu(format_menu, tearoff=False),
                     4: tk.Menu(format_menu, tearoff=False)}
        for key, label, size in REGISTER_FORMATS:
            target = format_menu if size == 1 else sub_menus[size]
            target.add_command(label=label,
                               command=lambda k=key: self.master_set_row_format(window_id, k))
        format_menu.add_separator()
        format_menu.add_cascade(label=self.tr("32 bits (2 registos)"), menu=sub_menus[2])
        format_menu.add_cascade(label=self.tr("64 bits (4 registos)"), menu=sub_menus[4])
        menu.add_cascade(label=self.tr("Formato"), menu=format_menu,
                         state="normal" if registers else "disabled")
        menu.add_command(label=self.tr("Repor formato da janela"),
                         command=lambda: self.master_set_row_format(window_id, None),
                         state="normal" if registers else "disabled")
        menu.add_command(label=self.tr("Escala…"),
                         command=lambda: self.master_edit_scale(window_id),
                         state="normal" if registers else "disabled")
        menu.add_command(label=self.tr("Remover escala"),
                         command=lambda: self.master_remove_scale(window_id),
                         state="normal" if registers else "disabled")
        menu.add_separator()
        menu.add_command(label=self.tr("Copiar"), accelerator="Ctrl+C",
                         command=lambda: self.master_copy_rows(window_id))
        menu.add_separator()
        base_menu = tk.Menu(menu, tearoff=False)
        base_var = tk.IntVar(value=window.base)
        pane["menu_base_var"] = base_var  # keep a reference while the menu lives
        base_menu.add_radiobutton(label=self.tr("0 — PDU Address"), variable=base_var, value=0,
                                  command=lambda: self.master_set_base(window_id, 0))
        base_menu.add_radiobutton(label=self.tr("1 — numeração dos manuais"), variable=base_var,
                                  value=1, command=lambda: self.master_set_base(window_id, 1))
        menu.add_cascade(label=self.tr("Base de endereços"), menu=base_menu)
        self._master_apply_menu_palette()
        try:
            menu.tk_popup(event.x_root, event.y_root)
        finally:
```

**MBSniffer v3.2/mb_master_ui.py — lines 1681-1720**

```
            menu.grab_release()
        return "break"

    def master_set_row_format(self, window_id, fmt):
        """Apply *fmt* (None = window format) to the selected rows."""
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        try:
            apply_format_to_rows(window, self._master_selected_rows(window_id), fmt)
        except ValueError as exc:
            size = FORMAT_SIZES.get(fmt, 1)
            messagebox.showwarning(self.tr("Formato"), self.trf(str(exc), size=size))
            return False
        self._master_render_values(window)
        self._master_mark_changed()
        return True

    # -- opening windows from Sniffer / Bus Slave Finder ------------------------------
    def master_definition_from_record(self, record):
        """Read definition (slave, fc, address, qty) behind a Traffic record, or None.

        A REQUEST carries its own address/quantity; a matched RESPONSE or
        EXCEPTION carries the ones of its request. Only FC01–FC04 qualify.
        """
        try:
            slave = int(record.get("slave", ""))
            fc = int(str(record.get("fc", "")), 16) & 0x7F
        except (TypeError, ValueError):
            return None
        if fc not in READ_FUNCTION_CODES or not 1 <= slave <= 247:
            return None
        if record.get("type") == "REQUEST":
            address, qty = record.get("pdu_address"), record.get("qty")
        else:
            address, qty = record.get("request_pdu_address"), record.get("request_qty")
        if address is None or qty is None:
            return None
        try:
            validate_read_definition(slave, fc, address, qty)
```

### MBSniffer v3.2/mb_master_ui.py — lines 1721-1760

```
1721        except ValueError:
1722            return None
1723        return {"slave": slave, "fc": fc, "address": int(address), "qty": int(qty)}
1724
1725    def master_open_from_record(self, record):
1726        definition = self.master_definition_from_record(record)
1727        if definition is None:
1728            messagebox.showinfo(
1729                self.tr("Abrir no Master"),
1730                self.tr("Esta trama não tem um pedido de leitura FC01–FC04 com endereço "
1731                        "conhecido."),
1732            )
1733            return None
1734        port = ""
1735        if not getattr(self, "capture_active", False):
1736            port = self.port_a_var.get().strip()
1737        return self.master_open_definition(
1738            definition, port=port, baud=self.baud_var.get(),
1739            parity=self.parity_var.get(), stopbits=self.stop_var.get(),
1740        )
1741
1742    def master_open_definition(self, definition, port="", baud="", parity="", stopbits=""):
1743        """Add a Master window and show the Master tab.
1744
1745        The serial settings (and port) are copied to the Master connection only
1746        when it is not connected and they are valid for the Master.
1747        """
1748        if not self.master_active:
1749            if str(baud) in [str(b) for b in DISCOVERY_BAUD_RATES]:
1750                self.master_baud_var.set(str(baud))
1751            if parity in MASTER_PARITIES:
1752                self.master_parity_var.set(parity)
1753            if str(stopbits) in MASTER_STOP_BITS:
1754                self.master_stop_var.set(str(stopbits))
1755            if port and port in tuple(self.master_port_combo["values"]):
1756                self.master_port_var.set(port)
1757        window = self.master_add_window(definition)
1758        try:
1759            self.main_notebook.select(self.master_page)
1760        except tk.TclError:
```

### MBSniffer v3.2/mb_master_ui.py — lines 1761-1800

```
            pass
        return window

    def _master_selected_group_starts(self, window):
        """PDU addresses where the values of the selected rows start."""
        starts = []
        for row in self._master_selected_rows(window.window_id):
            offset, _size, fmt = window.group_of_row(row)
            address = window.address + offset
            if fmt is not None and address not in starts:
                starts.append(address)
        return starts

    def master_edit_scale(self, window_id):
        """Ask for a linear scale and apply it to the selected values."""
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        starts = self._master_selected_group_starts(window)
        if not starts:
            messagebox.showwarning(self.tr("Escala"),
                                   self.tr("Seleciona pelo menos um endereço."))
            return False
        current = window.scales.get(starts[0]) or {
            "raw": [0, 65535], "eng": [0, 100], "decimals": 1, "unit": ""}

        dialog, body = self._master_dialog(self.tr("Escala"))
        ttk.Label(body, text=self.trf("Escala de {count} valor(es) selecionado(s):",
                                      count=len(starts))).grid(
            row=0, column=0, columnspan=4, sticky="w", pady=(0, 6))

        def number_text(value):
            return f"{value:g}"

        variables = {
            "raw_min": tk.StringVar(value=number_text(current["raw"][0])),
            "raw_max": tk.StringVar(value=number_text(current["raw"][1])),
            "eng_min": tk.StringVar(value=number_text(current["eng"][0])),
            "eng_max": tk.StringVar(value=number_text(current["eng"][1])),
            "decimals": tk.StringVar(value=str(current["decimals"])),
```

### MBSniffer v3.2/mb_master_ui.py — lines 1801-1840

```
            "unit": tk.StringVar(value=current.get("unit", "")),
        }
        layout = (
            (1, 0, "Bruto mín.", "raw_min"), (1, 2, "Bruto máx.", "raw_max"),
            (2, 0, "Eng. mín.", "eng_min"), (2, 2, "Eng. máx.", "eng_max"),
            (3, 0, "Casas decimais", "decimals"), (3, 2, "Unidade", "unit"),
        )
        first_entry = None
        for row, column, caption, key in layout:
            ttk.Label(body, text=self.tr(caption)).grid(row=row, column=column, sticky="w",
                                                        padx=(0, 6), pady=2)
            entry = ttk.Entry(body, width=10, textvariable=variables[key])
            entry.grid(row=row, column=column + 1, sticky="w", padx=(0, 14), pady=2)
            first_entry = first_entry or entry
        ttk.Label(body, text=self.tr("Valor = Eng. mín. + (bruto − Bruto mín.) × "
                                     "(Eng. máx. − Eng. mín.) / (Bruto máx. − Bruto mín.)"),
                  style="TLabel").grid(row=4, column=0, columnspan=4, sticky="w", pady=(6, 4))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=5, column=0, columnspan=4, sticky="w")
        result = {"ok": False}

        def accept(_event=None):
            try:
                scale = validate_scale(
                    variables["raw_min"].get(), variables["raw_max"].get(),
                    variables["eng_min"].get(), variables["eng_max"].get(),
                    variables["decimals"].get(), variables["unit"].get(),
                )
            except ValueError as exc:
                message_var.set(self.tr(str(exc)))
                return
            for address in starts:
                window.scales[address] = dict(scale)
            self._master_render_values(window)
            self._master_mark_changed()
            result["ok"] = True
            dialog.destroy()

        buttons = ttk.Frame(body)
```

### MBSniffer v3.2/mb_master_ui.py — lines 1841-1880

```
        buttons.grid(row=6, column=0, columnspan=4, sticky="e", pady=(8, 0))
        ttk.Button(buttons, text=self.tr("Aplicar"), command=accept,
                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
        ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(
            row=0, column=1)
        dialog.bind("<Return>", accept)
        dialog.bind("<Escape>", lambda _e: dialog.destroy())
        if first_entry is not None:
            first_entry.focus_set()
        dialog.grab_set()
        self.wait_window(dialog)
        return result["ok"]

    def master_remove_scale(self, window_id):
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        removed = False
        for address in self._master_selected_group_starts(window):
            removed = window.scales.pop(address, None) is not None or removed
        if removed:
            self._master_render_values(window)
            self._master_mark_changed()
        return removed

    def master_copy_rows(self, window_id):
        """Copy the selected rows (all rows if none) as tab-separated text."""
        pane = self._master_panes.get(window_id)
        if pane is None:
            return
        tree = pane["tree"]
        rows = list(tree.selection()) or list(tree.get_children())
        lines = ["\t".join(str(v) for v in tree.item(iid, "values")) for iid in rows]
        if lines:
            self.clipboard_clear()
            self.clipboard_append("\n".join(lines))

    def master_set_base(self, window_id, base):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
```

### MBSniffer v3.2/mb_master_ui.py — lines 1881-1920

```
        if window is None or pane is None or window.base == base:
            return
        window.base = base
        pane["address_var"].set(str(window.address + window.base))
        self._master_fill_rows(window)
        self._master_render_status(window)
        self._master_update_apply_state(window_id)
        self._master_mark_changed()

    # -- closing the application ---------------------------------------------------
    def master_confirm_close(self):
        """Ask to save a changed workspace; False cancels closing the application.

        Plain windows that were never saved as a workspace close silently.
        """
        if not (getattr(self, "master_current_workspace", "")
                and getattr(self, "master_workspace_dirty", False)):
            return True
        answer = messagebox.askyesnocancel(
            self.tr("Workspace"),
            self.trf("Guardar alterações ao workspace \"{name}\"?",
                     name=self.master_current_workspace),
        )
        if answer is None:
            return False
        if answer:
            return self.master_save_workspace(self.master_current_workspace)
        return True

    # -- language / palette / shutdown ---------------------------------------------
    def master_apply_language(self):
        if not hasattr(self, "master_windows"):
            return
        self._master_update_workspace_label()
        for window in self.master_windows.values():
            pane = self._master_panes.get(window.window_id)
            if pane is None:
                continue
            for column, caption in (("address", "Endereço"), ("name", "Nome"), ("value", "Valor")):
                try:
```

### MBSniffer v3.2/mb_master_ui.py — lines 1921-1960

```
                    pane["tree"].heading(column, text=self.tr(caption))
                except tk.TclError:
                    pass
            self._master_render_status(window)
        if not self.master_active:
            self.master_status_var.set(self.tr("Desligado"))
        elif self.master_engine is not None:
            self.master_status_var.set(self.tr("● Ligado"))

    def _master_apply_toplevel_palette(self, top):
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        try:
            top.configure(background=theme_colors(self, dark)["root"])
        except tk.TclError:
            pass

    def _master_apply_menu_palette(self):
        menu = getattr(self, "master_workspace_menu", None)
        if menu is None:
            return
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        colors = theme_colors(self, dark)
        menus = [menu] + list(menu.winfo_children())
        row_menu = getattr(self, "_master_row_menu", None)
        if row_menu is not None:
            menus += [row_menu] + list(row_menu.winfo_children())
            for child in row_menu.winfo_children():
                menus += list(child.winfo_children())
        for widget in menus:
            try:
                widget.configure(
                    background=colors["panel"], foreground=colors["text"],
                    activebackground=colors["select"], activeforeground=colors["select_text"],
                    disabledforeground=colors["disabled"],
                )
            except tk.TclError:
                pass

    def master_apply_palette(self, dark):
        self._master_apply_menu_palette()
```

### MBSniffer v3.2/mb_master_ui.py — lines 1961-1993

```
        colors = theme_colors(self, dark)
        if hasattr(self, "master_canvas"):
            self.master_canvas.configure(background=colors["root"])
        if hasattr(self, "ui_style"):
            self.ui_style.configure("MBS.Master.Error.TLabel",
                                    foreground="#ff8a80" if dark else "#b42318")
            self.ui_style.configure("MBS.Master.Ok.TLabel",
                                    foreground="#a7e4b8" if dark else "#1f6b3a")
        for pane in getattr(self, "_master_panes", {}).values():
            if pane.get("toplevel") is not None:
                self._master_apply_toplevel_palette(pane["toplevel"])
            self._master_configure_tree_tags(pane["tree"])
        button = getattr(self, "master_refresh_btn", None)
        if button is not None:
            button.set_dark_mode(dark)

    def master_shutdown(self):
        engine = getattr(self, "master_engine", None)
        if engine is not None:
            engine.stop(join_timeout=0.8)
            self.master_serial = engine.ser
        ser = getattr(self, "master_serial", None)
        if ser is not None:
            try:
                ser.close()
            except Exception:
                pass
        self.master_engine = None
        self.master_serial = None


def fc_choice_label_write(fc):
    return f"FC{fc:02d} {FUNCTION_NAMES[fc]}"
```

# MBSniffer v3.2/mb_i18n.py

610 source lines.

### MBSniffer v3.2/mb_i18n.py — lines 1-40

```
#!/usr/bin/env python3
"""Runtime language support for MBSniffer.

Portuguese (Portugal) is the canonical UI language.  English translations are
kept here so switching language never mutates protocol data or application
logic.
"""

from mb_config import DISCOVERY_IDLE_CHECK_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS

LANGUAGE_PORTUGUESE = "Português"
LANGUAGE_ENGLISH = "English"
LANGUAGE_CHOICES = (LANGUAGE_PORTUGUESE, LANGUAGE_ENGLISH)

# Static UI text. Technical terms that are already English are intentionally
# left out: translate_text() returns unknown strings unchanged.
PT_TO_EN = {
    "Linguagem": "Language",
    "Modo escuro": "Dark mode",
    "Configuração": "Configuration",
    "Modo físico": "Physical mode",
    "RS232 dual RX: COM A escuta uma direção; COM B escuta a direção oposta.": "RS232 dual RX: COM A monitors one direction; COM B monitors the opposite direction.",
    "RS232 single RX: uma COM escuta apenas uma direção da ligação RS232.": "RS232 single RX: one COM port monitors only one direction of the RS232 link.",
    "RS485 2-wire: uma COM consegue observar pedidos e respostas no mesmo par diferencial.": "RS485 2-wire: one COM port can monitor requests and responses on the same differential pair.",
    "Estado": "Status",
    "Visualização": "Display",
    "Gap manual (ms)": "Manual gap (ms)",
    "Estado COM": "COM status",
    "Controlos": "Controls",
    "Iniciar Captura": "Start Capture",
    "Parar": "Stop",
    "Reiniciar COM": "Restart COM",
    "Limpar": "Clear",
    "Abrir pasta de logs": "Open logs folder",
    "Simulação": "Simulation",
    "Separar transações": "Separate transactions",
    "Realçar resultados": "Highlight results",
    "Estatísticas da sessão": "Session statistics",
    "Tráfego": "Traffic",
    "Ajuda / Ligações": "Help / Connections",
```

**MBSniffer v3.2/mb_i18n.py — lines 41-80**

```
    "Filtros": "Filters",
    "Todos": "All",
    "Pesquisa": "Search",
    "Limpar filtros": "Clear filters",
    "Exportar CSV": "Export CSV",
    "Hora ↑": "Time ↑",
    "Hora ↓": "Time ↓",
    "Tipo": "Type",
    "Detalhes": "Details",
    "Copiar Raw Hex": "Copy Raw Hex",
    "Copiar frame descodificado": "Copy decoded frame",
    "Filtrar por este Slave": "Filter by this Slave",
    "Filtrar por este FC": "Filter by this FC",
    "Mostrar apenas esta transação": "Show only this transaction",
    "Limpar filtro de transação": "Clear transaction filter",
    "Tempos de resposta": "Response times",
    "Qualidade": "Quality",
    "Pedidos": "Requests",
    "Pendentes": "Pending",
    "Slave (Filtro) ▾": "Slave (Filter) ▾",
    "Slave (Nenhum) ▾": "Slave (None) ▾",
    "✓ Sucesso": "✓ Success",
    "⚠ Resposta lenta": "⚠ Slow response",
    "● Parado": "● Stopped",
    "● A reiniciar COM": "● Restarting COM",
    "● A capturar": "● Capturing",
    "● A parar": "● Stopping",
    "● A simular": "● Simulating",
    "● Simulação concluída": "● Simulation complete",
    "● Erro na simulação": "● Simulation error",
    "Parado": "Stopped",
    "A iniciar pesquisa…": "Starting scan…",
    "A parar…": "Stopping…",
    "Erro ao iniciar pesquisa": "Error starting scan",
    "Respostas": "Responses",
    "Taxa de pedidos": "Request rate",
    "Slaves ativos": "Active Slaves",
    "Bytes observados": "Observed bytes",
    "Utilização do bus ~": "Bus utilization ~",
    "Média": "Average",
```

### MBSniffer v3.2/mb_i18n.py — lines 81-120

```
    "Mínimo": "Minimum",
    "Máximo": "Maximum",
    "Slave mais lento": "Slowest Slave",
    "Erros de CRC": "CRC errors",
    "Bytes não validados": "Unvalidated bytes",
    "Taxa de timeout": "Timeout rate",
    "Exceções": "Exceptions",
    "Exceções / Slave": "Exceptions / Slave",
    "Seleciona uma linha no Tráfego para ver a descodificação estruturada do frame.":
        "Select a row in Traffic to view the structured frame decoding.",
    "Recolher": "Collapse",
    "Expandir": "Expand",
    "Resultado": "Result",
    "Dados": "Data",
    "Sucesso": "Success",
    "Resposta lenta": "Slow response",
    "Pendente / sem resultado": "Pending / no result",
    "Sem classificação": "Unclassified",
    "Verde": "Green",
    "Amarelo": "Yellow",
    "Vermelho": "Red",
    "Laranja": "Orange",
    "Roxo": "Purple",
    "Neutro": "Neutral",
    "Desconhecida/outra": "Unknown/other",
    "Não Utilizada": "Not Used",
    "Não selecionada": "Not selected",
    "Aberta": "Open",
    "Detetada": "Detected",
    "Não detetada": "Not detected",
    "Configuração inválida": "Invalid configuration",
    "pyserial em falta": "pyserial missing",
    "Porta COM": "COM port",
    "A porta série não aceitou bytes para transmissão.": "The serial port did not accept bytes for transmission.",
    "Erro ao abrir porta": "Error opening port",
    "Erro de captura": "Capture error",
    "Erro na simulação": "Simulation error",
    "Erro interno": "Internal error",
    "Abrir pasta": "Open folder",
    "Pesquisa ativa — transmite no barramento": "Active scan — transmits on the bus",
```

### MBSniffer v3.2/mb_i18n.py — lines 121-160

```
    "Esta ferramenta atua temporariamente como master Modbus RTU e envia apenas pedidos de leitura. Não a utilizes com outro master ativo no mesmo barramento.": "This tool temporarily acts as a Modbus RTU master and sends read requests only. Do not use it while another master is active on the same bus.",
    "Confirmo que não existe outro master ativo no barramento.": "I confirm that no other master is active on the bus.",
    "FC03 é o modo rápido. Uma resposta normal OU uma Modbus Exception com CRC válido conta como slave encontrado. Ativa FC04 fallback apenas para dispositivos que possam ignorar FC03 sem devolver Exception. Device Identification envia FC43/14 apenas aos slaves encontrados e tenta ler VendorName, ProductCode e Revision. Tentativas repete cada pedido sem resposta. Antes de transmitir, a pesquisa escuta o barramento durante 1 s e não arranca se detetar tráfego.": "FC03 is the fast mode. A normal response OR a CRC-valid Modbus Exception counts as a found Slave. Enable FC04 fallback only for devices that may ignore FC03 without returning an Exception. Device Identification sends FC43/14 only to found Slaves and attempts to read VendorName, ProductCode and Revision. Attempts repeats every unanswered request. Before transmitting, the scan listens to the bus for 1 s and does not start if traffic is detected.",
    "Configuração da pesquisa": "Scan configuration",
    "Porta COM": "COM port",
    "A porta série não aceitou bytes para transmissão.": "The serial port did not accept bytes for transmission.",
    "Data bits: 8 (fixo)": "Data bits: 8 (fixed)",
    "Slave IDs / velocidade": "Slave IDs / speed",
    "De": "From",
    "até": "to",
    "Timeout mín.": "Min. timeout",
    "Iniciar pesquisa": "Start scan",
    "Limpar resultados": "Clear results",
    "Slaves encontrados": "Slaves found",
    "Encontrados: 0": "Found: 0",
    "Resultado": "Result",
    "Pesquisa ativa": "Active scan",
    "Erro ao iniciar pesquisa": "Error starting scan",
    "Erro na Bus Slave Finder": "Bus Slave Finder error",
    "Erro na pesquisa": "Scan error",
    "Nenhum": "None",
    "Nenhum slave detetado": "No Slave detected",
    "Crescente — antigo → recente": "Ascending — oldest → newest",
    "Decrescente — recente → antigo": "Descending — newest → oldest",
    "Todos os ficheiros": "All files",
    "Exportar sessão para CSV": "Export session to CSV",
    # Messages with live values (templates used with SnifferApp.trf()).
    'Seleciona a COM A.': 'Select COM A.',
    'Seleciona também a COM B.': 'Select COM B as well.',
    'COM A e COM B têm de ser diferentes.': 'COM A and COM B must be different.',
    'pyserial não está instalado.': 'pyserial is not installed.',
    'Não foi possível reiniciar a porta COM:\n\n{error}': 'Could not restart the COM port:\n\n{error}',
    'Não foi possível reabrir a porta COM. A captura foi parada.\n\n{error}': 'Could not reopen the COM port. Capture was stopped.\n\n{error}',
    'Pending timeout inválido.': 'Invalid Pending timeout.',
    'Pending timeout deve estar entre {minimum:g} e {maximum:g} segundos.': 'Pending timeout must be between {minimum:g} and {maximum:g} seconds.',
    'Frame gap manual inválido.': 'Invalid manual Frame gap.',
    'pyserial não está instalado.\n\nO build_exe.bat inclui-o automaticamente no EXE.': 'pyserial is not installed.\n\nbuild_exe.bat includes it automatically in the EXE.',
    'Ocorreu um erro ao processar eventos da interface. O processamento continuará.\n\n{error}': 'An error occurred while processing interface events. Processing will continue.\n\n{error}',
    'Slave {slave} — média {average:.1f} ms': 'Slave {slave} — average {average:.1f} ms',
    'Não existem frames na sessão atual para exportar.': 'There are no frames in the current session to export.',
```

### MBSniffer v3.2/mb_i18n.py — lines 161-200

```
    'Sessão exportada para:\n\n{path}': 'Session exported to:\n\n{path}',
    'Não foi possível guardar o CSV:\n\n{error}': 'Could not save the CSV:\n\n{error}',
    'Erro ao criar log': 'Error creating log',
    'Foi detetado tráfego, mas não foi possível criar o ficheiro de log em:\n\n{folder}\n\n{error}': 'Traffic was detected, but the log file could not be created at:\n\n{folder}\n\n{error}',
    'Os Slave IDs têm de estar entre 1 e 247.': 'Slave IDs must be between 1 and 247.',
    'O Slave ID inicial não pode ser superior ao final.': 'The starting Slave ID cannot be greater than the ending Slave ID.',
    'O timeout mínimo tem de estar entre {minimum} e {maximum} ms.': 'The minimum timeout must be between {minimum} and {maximum} ms.',
    'Tentativas tem de estar entre {minimum} e {maximum}.': 'Attempts must be between {minimum} and {maximum}.',
    'Tentativas': 'Attempts',
    'Máx. estimado: —': 'Estimated max.: —',
    ' (com {options})': ' (with {options})',
    '{configs} config. × {slaves} slaves — máx. estimado: {duration}': '{configs} config. × {slaves} slaves — estimated max.: {duration}',
    'Encontrados: {count}': 'Found: {count}',
    'pyserial não está instalado. O build_exe.bat inclui-o automaticamente no EXE.': 'pyserial is not installed. build_exe.bat includes it automatically in the EXE.',
    'Confirma primeiro que não existe outro master ativo no barramento.': 'First confirm that no other master is active on the bus.',
    'Seleciona uma porta COM.': 'Select a COM port.',
    'Seleciona pelo menos uma configuração série.': 'Select at least one serial configuration.',
    'Não foi possível iniciar a thread de pesquisa:\n\n{error}': 'Could not start the scan thread:\n\n{error}',
    'A configurar {baud} / {config}…': 'Configuring {baud} / {config}…',
    'Foi detetado tráfego em {port} antes de transmitir ({count} bytes: {raw}). Pode existir outro master ativo no barramento; a pesquisa não foi iniciada.': 'Traffic was detected on {port} before transmitting ({count} bytes: {raw}). Another master may be active on the bus; the scan was not started.',
    'A verificar se o barramento está livre…': 'Checking that the bus is idle…',
    'Não foi possível abrir {port} em {baud} / {config}: {error}': 'Could not open {port} at {baud} / {config}: {error}',
    'Falha ao preparar a receção em {context}: {error}': 'Failed to prepare reception at {context}: {error}',
    'Falha ao transmitir em {context}: {error}': 'Failed to transmit at {context}: {error}',
    'Falha ao receber em {context}: {error}': 'Failed to receive at {context}: {error}',
    'Falha em Device Identification em {context}: {error}': 'Device Identification failed at {context}: {error}',
    'Decorrido {elapsed} — restante ~{remaining}': 'Elapsed {elapsed} — remaining ~{remaining}',
    'Pesquisa parada — {found} encontrados — {elapsed}': 'Scan stopped — {found} found — {elapsed}',
    'Pesquisa concluída — {found} encontrados — {elapsed}': 'Scan complete — {found} found — {elapsed}',
    # Master (poll) tab — v3.2.
    'Ligação — o Master transmite no barramento': 'Connection — the Master transmits on the bus',
    'Timeout resposta (ms)': 'Response timeout (ms)',
    'Atraso entre pedidos (ms)': 'Delay between polls (ms)',
    'Ligar': 'Connect',
    'Desligar': 'Disconnect',
    'Desligado': 'Disconnected',
    '● Ligado': '● Connected',
    'A desligar…': 'Disconnecting…',
    'Nova janela': 'New window',
    'Abrir workspace': 'Open workspace',
```

### MBSniffer v3.2/mb_i18n.py — lines 201-240

```
    'Guardar workspace': 'Save workspace',
    'Sem janelas. Usa "Nova janela" para começar a ler um Slave.': 'No windows. Use "New window" to start reading a Slave.',
    'Endereço': 'Address',
    'Qtd.': 'Qty',
    'Formato': 'Format',
    'Aplicar': 'Apply',
    'Pausar': 'Pause',
    'Retomar': 'Resume',
    'Pausada': 'Paused',
    'Ler 1x': 'Read once',
    'Duplicar': 'Duplicate',
    'Destacar': 'Pop out',
    'Ancorar': 'Dock',
    'Fechar': 'Close',
    'Nome': 'Name',
    'Valor': 'Value',
    'Máximo de {count} janelas.': 'Maximum of {count} windows.',
    'Janela inválida': 'Invalid window',
    'Janela inválida.': 'Invalid window.',
    'Liga primeiro o Master.': 'Connect the Master first.',
    'Escrita': 'Write',
    'FC02 e FC04 são só de leitura: abre uma janela FC01 ou FC03 para escrever.': 'FC02 and FC04 are read-only: open an FC01 or FC03 window to write.',
    'Escrever': 'Write value',
    'Slave (0 = broadcast)': 'Slave (0 = broadcast)',
    'Valor(es)': 'Value(s)',
    'Formato: {fmt}. Vários valores (FC15/FC16): separar com ;': 'Format: {fmt}. Several values (FC15/FC16): separate with ;',
    'Enviar': 'Send',
    'Cancelar': 'Cancel',
    'Nome do endereço {address}': 'Name of address {address}',
    'Guardar': 'Save',
    'Para a captura ou a pesquisa em curso antes de ligar o Master.': 'Stop the running capture or scan before connecting the Master.',
    'Não foi possível abrir {port}: {error}': 'Could not open {port}: {error}',
    'Erro na porta série: {error}': 'Serial port error: {error}',
    'Escrita OK: {what}': 'Write OK: {what}',
    'Escrita broadcast enviada: {what}': 'Broadcast write sent: {what}',
    'Resposta inválida (CRC ou formato): {rx}': 'Invalid response (CRC or format): {rx}',
    'O ficheiro não é um workspace do MBSniffer.': 'The file is not an MBSniffer workspace.',
    '{count} janela(s) inválida(s) foram ignoradas.': '{count} invalid window(s) were skipped.',
    'O timeout de resposta tem de estar entre 10 e 10000 ms.': 'The response timeout must be between 10 and 10000 ms.',
    'O atraso entre pedidos tem de estar entre 0 e 10000 ms.': 'The delay between polls must be between 0 and 10000 ms.',
```

### MBSniffer v3.2/mb_i18n.py — lines 241-280

```
    'Scan rate fora do intervalo.': 'Scan rate out of range.',
    'Valor vazio.': 'Empty value.',
    'Valor fora do formato selecionado.': 'Value does not fit the selected format.',
    'Uma coil só aceita 0 ou 1.': 'A coil only accepts 0 or 1.',
    'O Slave ID de leitura tem de estar entre 1 e 247.': 'The read Slave ID must be between 1 and 247.',
    'Função de leitura não suportada.': 'Unsupported read function.',
    'O endereço tem de estar entre 0 e 65535.': 'The address must be between 0 and 65535.',
    'Quantidade fora do limite desta função.': 'Quantity outside the limit of this function.',
    'O intervalo de endereços ultrapassa 65535.': 'The address range goes beyond 65535.',
    'O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.': 'The write Slave ID must be between 0 (broadcast) and 247.',
    'FC05 escreve exatamente uma coil.': 'FC05 writes exactly one coil.',
    'FC06 escreve exatamente um registo.': 'FC06 writes exactly one register.',
    'Função de escrita não suportada.': 'Unsupported write function.',
    'FC05/FC06 escrevem um único valor; usa FC15/FC16 para vários.': 'FC05/FC06 write a single value; use FC15/FC16 for several.',
    # Master windows and workspaces — v3.2.
    'Janela {number}': 'Window {number}',
    'Renomear': 'Rename',
    'Renomear janela': 'Rename window',
    'Nome da janela (vazio = nome por defeito)': 'Window name (empty = default name)',
    'Workspace atual: {name}': 'Current workspace: {name}',
    'Guardar workspace…': 'Save workspace…',
    'Apagar workspace': 'Delete workspace',
    '(nenhum workspace guardado)': '(no saved workspace)',
    'Abrir pasta dos workspaces': 'Open workspaces folder',
    'Nome do workspace': 'Workspace name',
    'Já existe um workspace chamado "{name}". Substituir?': 'A workspace named "{name}" already exists. Replace it?',
    'Apagar o workspace "{name}"?': 'Delete workspace "{name}"?',
    'O nome do workspace não pode ficar vazio.': 'The workspace name cannot be empty.',
    'O nome do workspace não pode ter os caracteres < > : " / \\ | ? *.': 'The workspace name cannot contain the characters < > : " / \\ | ? *.',
    'Esse nome está reservado pelo Windows.': 'That name is reserved by Windows.',
    # Master per-address formats, row menu, workspace changes — v3.2.
    'Duplo clique num valor para escrever, num nome para o editar. Clique direito: formato, copiar e base.': 'Double-click a value to write it, a name to edit it. Right-click: f
ormat, copy and base.',
    'Workspace atual: {name} (alterado)': 'Current workspace: {name} (modified)',
    'Seleciona pelo menos um endereço.': 'Select at least one address.',
    'Seleciona registos seguidos para este formato.': 'Select consecutive registers for this format.',
    'Este formato ocupa {size} registos: seleciona {size} registos seguidos (ou um múltiplo de {size}).': 'This format uses {size} registers: select {size} consecutive registers
 (or a multiple of {size}).',
    'Este endereço não tem um valor completo no formato atual.': 'This address does not hold a complete value in the current format.',
    '32 bits (2 registos)': '32-bit (2 registers)',
    '64 bits (4 registos)': '64-bit (4 registers)',
    'Repor formato da janela': 'Reset to window format',
```

### MBSniffer v3.2/mb_i18n.py — lines 281-320

```
    'Copiar': 'Copy',
    '1 – numeração dos manuais': '1 – manual numbering',
    'Base de endereços': 'Address base',
    'Guardar alterações ao workspace "{name}"?': 'Save changes to workspace "{name}"?',
    # Master scale, open in Master, reconnection – v3.2.
    'Escala…': 'Scale…',
    'Remover escala': 'Remove scale',
    'Escala': 'Scale',
    'Escala de {count} valor(es) selecionado(s):': 'Scale of {count} selected value(s):',
    'Bruto mín.': 'Raw min.',
    'Bruto máx.': 'Raw max.',
    'Eng. mín.': 'Eng. min.',
    'Eng. máx.': 'Eng. max.',
    'Casas decimais': 'Decimals',
    'Unidade': 'Unit',
    'Valor = Eng. mín. + (bruto − Bruto mín.) × (Eng. máx. − Eng. mín.) / (Bruto máx. − Bruto mín.)': 'Value = Eng. min. + (raw − Raw min.) × (Eng. max. − Eng. min.) / (Raw max. − Raw min.)',
    'valores em unidades da escala {unit}': 'values in scale units {unit}',
    'A escala tem de ter números válidos.': 'The scale needs valid numbers.',
    'O bruto mínimo e o bruto máximo têm de ser diferentes.': 'Raw min. and raw max. must be different.',
    'As casas decimais têm de estar entre 0 e 6.': 'Decimals must be between 0 and 6.',
    'Esta escala não permite escrever: os valores de engenharia são iguais.': 'This scale does not allow writing: the engineering values are equal.',
    'Abrir no Master': 'Open in Master',
    'Esta trama não tem um pedido de leitura FC01–FC04 com endereço conhecido.': 'This frame has no FC01–FC04 read request with a known address.',
    'A religar a {port}… (porta falhou: {error})': 'Reconnecting to {port}… (port failed: {error})',
}

EN_TO_PT = {english: portuguese for portuguese, english in PT_TO_EN.items()}
# "None" is also a canonical technical serial-parity value in the Portuguese UI.
# Do not reverse-translate it to "Nenhum" when restoring Portuguese; menu entries
# that need "Nenhum" are rendered from their Portuguese canonical source text.
EN_TO_PT.pop("None", None)

EXCEPTION_DESCRIPTION_EN = {
    0x01: "The requested Modbus function is not supported or permitted by the device.",
    0x02: "The requested address or address range is not valid for this device.",
    0x03: "A value, quantity or request field is invalid for this function.",
    0x04: "The device encountered an internal error while attempting to execute the request.",
    0x05: "The request was accepted but requires more time to be processed.",
    0x06: "The device is busy and cannot process the request at this time.",
    0x08: "A parity error was detected while accessing the device memory.",
```

### MBSniffer v3.2/mb_i18n.py — lines 321-360

```
321     0x0A: "The gateway could not establish a path to the target device.",
322     0x0B: "The gateway did not receive a response from the target device.",
323 }
324 
325 
326 def normalize_language(value):
327     return value if value in LANGUAGE_CHOICES else LANGUAGE_PORTUGUESE
328 
329 
330 def canonical_ui_text(text):
331     """Return the canonical PT-PT form of a known static UI string."""
332     value = str(text)
333     return EN_TO_PT.get(value, value)
334 
335 
336 def translate_text(text, language):
337     """Translate a known static string; leave technical/unknown text intact."""
338     canonical = canonical_ui_text(text)
339     if normalize_language(language) == LANGUAGE_ENGLISH:
340         return PT_TO_EN.get(canonical, canonical)
341     return canonical
342 
343 
344 # Human-readable fragments embedded in protocol Details / Inspector data.
345 DETAIL_FRAGMENTS_PT_TO_EN = (
346     ("Registos (PDU Address=valor):", "Registers (PDU Address=value):"),
347     ("Bits (PDU Address=valor):", "Bits (PDU Address=value):"),
348     ("Registos=", "Registers="),
349 )
350 
351 
352 def translate_details(text, language):
353     """Translate only human-readable fragments embedded in protocol details."""
354     value = str(text or "")
355     if normalize_language(language) == LANGUAGE_ENGLISH:
356         for portuguese, english in DETAIL_FRAGMENTS_PT_TO_EN:
357             value = value.replace(portuguese, english)
358     else:
359         for portuguese, english in DETAIL_FRAGMENTS_PT_TO_EN:
360             value = value.replace(english, portuguese)
```

### MBSniffer v3.2/mb_i18n.py — lines 361-400

```
361     return value
362 
363 
364 def exception_description(code, portuguese_text, language):
365     if normalize_language(language) == LANGUAGE_ENGLISH:
366         return EXCEPTION_DESCRIPTION_EN.get(code, portuguese_text)
367     return portuguese_text
368 
369 
370 # English Help content. Tags match the tags configured by mb_gui.py.
371 def english_help_blocks(max_ui_frames_text, max_raw_lines_text,
372                         min_pending, max_pending, debug_sim=False):
373     blocks = [
374         ("title", "MBSniffer — Help and connections\n"),
375         (None, "MBSniffer is designed for Modbus RTU communications diagnostics. The Sniffer tab is passive: it opens the COM port(s) for reception and does not transmit request
    s. Bus Slave Finder is an active scan tool and must only be used when no other master is active on the same bus.\n"),
376         ("heading", "Quick start — Sniffer\n"),
377         ("bullet", "1. Select the correct physical mode: RS485 2-wire, RS232 single RX or RS232 dual RX.\n"),
378         ("bullet", "2. Select the COM port(s) and serial parameters used by the communication being observed.\n"),
379         ("bullet", "3. Keep Frame gap set to Auto unless you have a specific reason to change it.\n"),
380         ("bullet", "4. Click “Start Capture” and confirm activity through BUS ● RX.\n"),
381         ("bullet", "5. Use Traffic for quick diagnostics and Raw Hex / Log to confirm the actual bytes.\n"),
382         ("bullet", "6. Select a frame to inspect its detailed analysis in Frame Inspector.\n"),
383 
384         ("heading", "1. RS485 2-wire\n"),
385         (None, "In this mode, one COM port observes requests and responses on the same differential pair. The sniffer adapter must be connected in parallel and must not act as a
     master.\n"),
386         ("subheading", "Typical connection:\n"),
387         ("mono", "        Device A              Device B\n            D+ ---------------- D+\n            D- ---------------- D-\n              \\              /\n
       \\            /\n                 D+          D-\n                    USB-RS485 sniffer\n\n"),
388         ("bullet", "• Connect sniffer D+ in parallel with bus D+.\n"),
389         ("bullet", "• Connect sniffer D− in parallel with bus D−.\n"),
390         ("bullet", "• If a C/COM/0 V reference is available, connect it only when you know it is the correct RS485 interface reference.\n"),
391         ("warning", "• Do not add a third 120 Ω termination resistor if the bus is already terminated at both ends.\n"),
392         ("bullet", "• If the USB-RS485 adapter has selectable termination or bias, leave them disabled during passive capture unless they are intentionally required by the insta
    llation.\n"),
393         ("bullet", "• A/B naming is not universal: different manufacturers may use opposite polarities. Prefer D+/D− and the equipment documentation.\n"),
394 
395         ("heading", "2. RS232 single RX\n"),
396         (None, "Observes only one RS232 direction. RS232 is full-duplex: each direction uses an independent TX line, so one RX input cannot observe both directions simultaneousl
    y.\n"),
397         ("mono", "        Device A                      Device B\n            TX ----------------------------> RX\n             \\\n              \\----> USB-RS232 sniffer R
    X (COM A)\n\n            GND --------------------------- GND\n             \\------------------------> sniffer GND\n\n       Sniffer TX: DO NOT CONNECT\n\n"),
398         ("bullet", "• Connect sniffer RX to the TX line you want to observe.\n"),
399         ("bullet", "• Connect the common GND as well.\n"),
400         ("warning", "• Leave the TX pin of the adapter used as a sniffer physically disconnected.\n"),
```

### MBSniffer v3.2/mb_i18n.py — lines 401-440

```
401
402         ("heading", "3. RS232 dual RX\n"),
403         (None, "Allows both RS232 directions to be observed at the same time. Two independent RX channels are required, normally two USB-RS232 adapters.\n"),
404         ("mono", "        Device A                      Device B\n             TX ----------------------------> RX\n              \\----> sniffer RX COM A   (A → B)\n\n       RX <---------------------------- TX\n                                      \\----> sniffer RX COM B   (B → A)\n\n              GND ---------------------------- GND\n          \\---------------------------- COM A GND\n                \\---------------------------- COM B GND\n\n        TX on both sniffers: DO NOT CONNECT\n\n"),
405         ("bullet", "• COM A is shown as A→B and COM B as B→A.\n"),
406         ("bullet", "• Both COM ports must use the same serial parameters as the observed link.\n"),
407
408         ("heading", "RS232 is not TTL\n"),
409         ("warning", "Do not connect RS232 signals directly to 3.3 V or 5 V USB-TTL/UART interfaces. Use a USB-RS232 adapter with a transceiver suitable for RS232 voltage levels.\n"),
410
411         ("heading", "Serial communication parameters\n"),
412         (None, "Baud rate, Data bits, Parity and Stop bits must match the actual communication. Incorrect parameters may produce apparently random bytes, frames with invalid CRC, or no recognizable frames at all.\n"),
413         ("bullet", "• Baud rate — speed in bit/s, for example 9600 or 19200.\n"),
414         ("bullet", "• Data bits — normally 8 for Modbus RTU.\n"),
415         ("bullet", "• Parity — None, Even, Odd, Mark or Space, according to the installation.\n"),
416         ("bullet", "• Stop bits — 1, 1.5 or 2, according to the observed configuration.\n"),
417
418         ("heading", "Frame gap\n"),
419         (None, "In Auto mode, MBSniffer uses 3.5 character times up to and including 19200 bit/s. Above 19200 bit/s it uses the fixed 1.750 ms t3.5 recommended for Modbus RTU. Manual mode allows another millisecond value to be used when diagnosing equipment or drivers with particular behavior.\n"),
420         (None, "USB adapters and Windows may group bytes and introduce latency. MBSniffer timestamps are therefore useful for sequence diagnostics and relative timing, but they do not replace a logic analyzer when high-precision timing measurements are required.\n"),
421         (None, f"When a burst ends with the start of a known frame that is still incomplete, MBSniffer waits up to {SNIFFER_SPLIT_FRAME_HOLD_MS:g} ms more before closing it. A frame split in two by the USB adapter therefore does not appear as RAW. Frames merged into one burst keep the arrival time of their own block of bytes.\n"),
422
423         ("heading", "Pending timeout\n"),
424         (None, "Defines how long a CRC-valid REQUEST may remain waiting for a matchable RESPONSE/EXCEPTION. When it expires, the original REQUEST is marked as timeout and the Timeout/Timeout rate counters increase.\n"),
425         (None, f"Allowed range: {min_pending:g} to {max_pending:g} s. The value should be greater than the normal maximum response time of the installation to avoid false timeouts.\n"),
426         (None, "Broadcast requests (Slave 0) never have a response by definition: they are not kept pending, never count as a timeout and are excluded from the Timeout rate.\n"),
427
428         ("heading", "How to read the Traffic table\n"),
429         ("bullet", "• REQUEST — Modbus RTU request recognized by the parser.\n"),
430         ("bullet", "• RESPONSE — recognized Modbus RTU response that is not an EXCEPTION.\n"),
431         ("bullet", "• FC05/FC06/FC08/FC22 — the request and the response have the same format. The frame is a RESPONSE when a pending request from the same Slave/FC exists (for FC05/06/22 the bytes must be identical; in RS232 dual RX it must come from the opposite channel). Otherwise it is a REQUEST.\n"),
432         ("bullet", "• FC01/FC02 responses show the bits in address order, grouped per byte. In Frame Inspector, matched read responses (FC01–FC04) show the PDU Address of every value.\n"),
433         ("bullet", "• EXCEPTION — Modbus response with an exception Function Code (bit 7 set). Details shows the code and normalized description.\n"),
434         ("bullet", "• CRC OK — received CRC matches the calculated CRC.\n"),
435         ("bullet", "• CRC ERROR — the structure resembles a supported frame, but the received CRC does not match the calculated CRC.\n"),
436         ("bullet", "• RAW/UNSYNC — bytes discarded during resynchronization before a structurally recognized, CRC-valid frame is found later in the same burst.\n"),
437         ("bullet", "• RAW/UNPARSED — remaining bytes that could not be interpreted or resynchronized as a valid Modbus frame. This is the state normally shown for a noise block with no later valid frame in the same burst.\n"),
438         (None, "RAW/UNSYNC and RAW/UNPARSED are different conditions and both may appear in the Type column. Neither represents a valid Modbus response.\n"),
439
440         ("heading", "Traffic table columns\n"),
```

**MBSniffer v3.2/mb_i18n.py — lines 441-480**

```
441         ("bullet", "• Time — timestamp for the start of the processed burst/frame.\n"),
442         ("bullet", "• Δt — interval since the previous observed frame.\n"),
443         ("bullet", "• Resp. — time between a REQUEST and its matched RESPONSE/EXCEPTION. Blank when no valid pair exists.\n"),
444         ("bullet", "• COM — capture channel: BUS in RS485, A→B/B→A in RS232 dual RX and SIM in development mode. It does not necessarily represent the Windows COM number.\n"),
445         ("bullet", "• Type — REQUEST, RESPONSE, EXCEPTION or a RAW state.\n"),
446         ("bullet", "• Slave — Modbus Unit/Slave ID when identifiable.\n"),
447         ("bullet", "• FC — hexadecimal Function Code observed in the frame.\n"),
448         ("bullet", "• Details — address, quantity, data or operation description.\n"),
449         ("bullet", "• CRC — OK, ERROR or ? when a complete Modbus frame is not available.\n"),
450 
451         ("heading", "REQUEST/RESPONSE pairing\n"),
452         (None, "Modbus RTU has no Transaction ID. MBSniffer pairs frames by (Slave ID, Function Code) in FIFO order. This is appropriate for normal RTU traffic, but can become ambiguous when requests using the same Slave and FC overlap before their corresponding responses arrive.\n"),
453         (None, "An EXCEPTION uses its base Function Code for pairing. For example, response 0x83 is associated with a pending FC03 REQUEST when a compatible request exists.\n"),
454 
455         ("heading", "PDU Address, 1-based and Qty\n"),
456         (None, "PDU Address is the address encoded in the Modbus request. 1-based is simply PDU Address + 1 and helps compare with manuals that number the first register/coil as 1. Qty is the requested quantity.\n"),
457         (None, "These fields belong to the REQUEST. A RESPONSE does not automatically repeat Address or Qty in the protocol, so MBSniffer does not invent those fields on the response row.\n"),
458 
459         ("heading", "Modbus exception codes\n"),
460         ("bullet", "• 0x01 — Illegal Function: function not supported or not permitted.\n"),
461         ("bullet", "• 0x02 — Illegal Data Address: invalid address/address range.\n"),
462         ("bullet", "• 0x03 — Illegal Data Value: invalid value, quantity or field.\n"),
463         ("bullet", "• 0x04 — Server Device Failure: device-side execution failure.\n"),
464         ("bullet", "• 0x05 — Acknowledge.\n"),
465         ("bullet", "• 0x06 — Server Device Busy.\n"),
466         ("bullet", "• 0x08 — Memory Parity Error.\n"),
467         ("bullet", "• 0x0A — Gateway Path Unavailable.\n"),
468         ("bullet", "• 0x0B — Gateway Target Device Failed to Respond.\n"),
469 
470         ("heading", "Advanced filters\n"),
471         (None, "The bar at the top of Traffic can combine several filters without changing the original session data:\n"),
472         ("bullet", "• Type — All, Requests, Responses, Exceptions, CRC errors, Timeouts or RAW.\n"),
473         ("bullet", "• FC (hex) — Function Code, with or without the 0x prefix.\n"),
474         ("bullet", "• Resp. > — only completed responses above the specified time in ms.\n"),
475         ("bullet", "• Search — searches Details, Raw Hex, Slave, FC, Type and COM/channel.\n"),
476         (None, "The Slave filter remains available in the column header and combines with the advanced filters. “Clear filters” resets all display filters. Filters do not change statistics or the capture log.\n"),
477 
478         ("heading", "Highlight results\n"),
479         (None, "This option is disabled by default to keep the table neutral. When enabled, results are differentiated by color: a completed normal transaction with a CRC-valid RESPONSE is green; CRC error, timeout, Exception, RAW and slow response retain their respective diagnostic colors.\n"),
480         (None, "In a successfully completed normal transaction, both the REQUEST and its RESPONSE are green. A still-pending REQUEST remains neutral. A slow response keeps the slow-response color even if valid so that the timing result is not hidden.\n"),
```

**MBSniffer v3.2/mb_i18n.py — lines 481-520**

```
481
482         ("heading", "Frame Inspector\n"),
483         (None, "Select a Traffic row to analyze the frame without manually interpreting every byte. Frame Inspector shows Slave, Type, Function, response time, Result, PDU Addre
    ss, 1-based address, Qty, ByteCount, CRC, decoded Data and Raw Hex.\n"),
484         (None, "The CRC field shows both received and calculated values for a complete frame. Result uses the same diagnostic classification as Traffic, independently of whether
     Highlight results is enabled.\n"),
485
486         ("heading", "Traffic context menu\n"),
487         (None, "Right-click a Traffic row to:\n"),
488         ("bullet", "• Copy Raw Hex.\n"),
489         ("bullet", "• Copy the decoded frame.\n"),
490         ("bullet", "• Filter by that frame's Slave.\n"),
491         ("bullet", "• Filter by that frame's Function Code.\n"),
492         ("bullet", "• Show only the matching REQUEST/RESPONSE transaction.\n"),
493         ("bullet", "• Clear the temporary transaction filter.\n"),
494
495         ("heading", "Bus Health\n"),
496         (None, "Bus Health summarizes session quality and performance without replacing the frame table.\n"),
497         ("bullet", "• Requests / Responses — session counters.\n"),
498         ("bullet", "• Request rate — average number of observed requests per second.\n"),
499         ("bullet", "• Active Slaves — valid Modbus IDs observed in the session.\n"),
500         ("bullet", "• Observed bytes — sum of bytes delivered to the parser.\n"),
501         ("bullet", "• Bus utilization ~ — estimate based on observed bytes, baud rate, data bits, parity and stop bits. It is approximate and depends on the capture point.\n"),
502         ("bullet", "• Average / Minimum / Maximum — paired response times.\n"),
503         ("bullet", "• P95 — 95% of measured responses were at or below this value.\n"),
504         ("bullet", "• Slowest Slave — Slave with the highest average response time among observed samples.\n"),
505         ("bullet", "• Unvalidated bytes — percentage of observed bytes belonging to invalid-CRC frames or RAW blocks. By itself this does not prove electrical noise; incorrect s
    erial parameters, incomplete capture or parser-unsupported functions can also produce RAW.\n"),
506         ("bullet", "• Timeout rate — percentage of REQUESTs that expired without a matched response.\n"),
507         ("bullet", "• Exceptions / Slave — distribution of Exception responses by Slave ID.\n"),
508
509         ("heading", "BUS RX indicator\n"),
510         (None, "BUS ● RX pulses briefly when a batch of frames reaches the interface. At rest, BUS ○ is displayed. It is only a visual confirmation of received activity; it does
     not indicate electrical quality or guarantee that frames are valid.\n"),
511
512         ("heading", "Raw Hex / Log\n"),
513         (None, "Shows captured bytes in hexadecimal with context information. Long lines wrap to the available width. The view follows the same display filters used in Traffic;
    the on-disk TXT file, when present, keeps the complete capture and is not truncated by filters.\n"),
514
515         ("heading", "Export CSV\n"),
516         (None, "Export CSV saves the frames currently retained in the UI history using Excel-compatible UTF-8. It includes Time, Δt, Response time, Channel, Type, Slave, FC, Det
    ails, CRC, Raw, PDU Address, 1-based, Qty, ByteCount, Timeout and anomaly class.\n"),
517         (None, f"The interface retains approximately the latest {max_ui_frames_text} frames. In longer sessions, the CSV therefore represents the history still retained by the G
    UI, while the TXT file from a real capture remains the complete on-disk record.\n"),
518
519         ("heading", "Capture logs\n"),
520         (None, "The TXT log is created only after real traffic arrives. Starting and stopping without receiving frames does not create an empty file.\n"),
```

**MBSniffer v3.2/mb_i18n.py — lines 521-560**

```
521         (None, "Logs folder:\n"),
522         ("mono", "        <application folder>\\MBSniffer Logs\n"),
523         (None, "Filters, sorting, Highlight results and Frame Inspector are display-only features and do not change frames written to the log.\n"),
524 
525         ("heading", "Visible history limits\n"),
526         (None, f"To keep the application responsive, the interface retains approximately {max_ui_frames_text} frames and {max_raw_lines_text} Raw Hex lines. Pruning is performed
     in blocks. The capture TXT log, when active, is independent of this limit.\n"),
527 
528         ("heading", "COM status and ports\n"),
529         (None, "COM status distinguishes a port opened by MBSniffer, a port merely detected by Windows and a port that is no longer detected. Status is refreshed periodically.\n
    "),
530         (None, "Restart COM closes and reopens the port(s) used by Sniffer. This restarts the serial-port handle inside MBSniffer; it does not restart the Windows USB/COM driver
    .\n"),
531         (None, "The ↻ buttons beside the COM selectors refresh the available port enumeration in Sniffer and Bus Slave Finder.\n"),
532 
533         ("heading", "Bus Slave Finder — active scan\n"),
534         (None, "Bus Slave Finder transmits Modbus RTU requests, so it is not passive. Before starting, confirm that no other master is active on the same bus.\n"),
535         (None, "Select COM, baud rates, parities, stop bits and the Slave ID range. Data bits is fixed at 8. The scan tries FC03 Address 0 Qty 1; either a normal response or a C
    RC-valid Modbus Exception confirms that the Slave exists.\n"),
536         (None, "FC04 fallback is optional and increases the maximum number of attempts. The minimum timeout is adapted to baud rate. A write timeout means transmission through t
    he port/driver failed and must not be interpreted simply as “slave did not respond”.\n"),
537         (None, f"Attempts (1 to 3) repeats every unanswered request before moving on to the next Slave. Before the first transmission, the scan listens to the bus for {DISCOVERY
    _IDLE_CHECK_SECONDS:g} s; if any byte is received, it does not transmit and reports that another master may be active.\n"),
538         (None, "Device Identification (FC43/14) is optional. When enabled, it is sent only after a Slave has been found and attempts to read Basic Device Identification: VendorN
    ame, ProductCode and MajorMinorRevision. An Exception or no response to FC43/14 does not invalidate Slave discovery.\n"),
539         (None, "Finder and Sniffer share the global activity lock so the same application cannot actively scan and capture at the same time.\n"),
540 
541         ("heading", "Master — read and write as a Modbus RTU master\n"),
542         (None, f"The Master tab transmits requests: use it only when no other master is active on the bus. When connecting, the application listens to the bus for {DISCOVERY_IDL
    E_CHECK_SECONDS:g} s and does not transmit if traffic is detected. While the Master is connected, Sniffer and Bus Slave Finder are locked.\n"),
543         (None, "Each window reads one data area of a Slave: Slave ID, function (FC01 Coils, FC02 Discrete Inputs, FC03 Holding Registers or FC04 Input Registers), address, quant
    ity and scan rate. Changes take effect only after “Apply”, which is highlighted in blue while some fields are not applied yet. The table shows up to 15 rows; beyond that it scro
    lls.\n"),
544         (None, "All windows share the same COM port: Modbus RTU allows only one request at a time, so windows are served in turn according to their scan rate. Writes and “Read o
    nce” go ahead of cyclic reads. The “Delay between polls” (at least t3.5) is always left between two requests.\n"),
545         (None, "Formats: signed and unsigned 16-bit, hex and binary; signed and unsigned 32-bit and float, in byte orders ABCD (big-endian), CDAB (word swap), BADC (byte swap) a
    nd DCBA (little-endian); signed and unsigned 64-bit and double, in orders ABCDEFGH, GHEFCDAB, BADCFEHG and HGFEDCBA. 32 and 64-bit values use 2 and 4 registers and appear on the
     row of the first one.\n"),
546         (None, "The “Format” field is the window's default format. Right-clicking one or more addresses (drag with the mouse, or Ctrl/Shift, to select several) sets the format o
    f those addresses only: 32-bit formats need 2 consecutive registers and 64-bit formats 4 (or a multiple, applied in groups). “Reset to window format” returns to the default. The
     same menu has “Copy” (also Ctrl+C), which copies addresses, names and values to paste into Excel, and “Address base”: 0 shows the PDU Address; 1 numbers from 1, as in many manu
    als.\n"),
547         (None, "When a read fails (Timeout, Exception or invalid response), the values turn grey until a correct read arrives; they also turn grey when disconnecting.\n"),
548         (None, "Scale: in the right-click menu, “Scale…” converts the raw value into engineering units through two points (Raw min./max. → Eng. min./max.), with decimals and a u
    nit (for example 2000–10000 → −180–180 °). When writing, the value is typed in units and converted back to raw. “Remove scale” returns to the raw value. Scales are saved in the
    workspace.\n"),
549         (None, "“Open in Master”: in the Sniffer Traffic table (right-click an FC01–FC04 request or its matched response) and in the Bus Slave Finder results (right-click or dou
    ble-click) it creates a Master window with the same Slave, function, address and quantity, and copies the serial parameters to the Master connection (when it is not connected).\
    n"),
550         (None, "If the COM port fails while connected (for example, the USB adapter was unplugged), the Master does not stop: it tries to reopen the port every 2 s and resumes r
    eading when the port is back. To stop, click “Disconnect”.\n"),
551         (None, "Writing: double-click a value in an FC01 or FC03 window. Coils with FC05 or FC15, registers with FC06 or FC16; several values separated by “;” are written to con
    secutive addresses with FC15/FC16. Slave 0 sends a broadcast, with no response. After an accepted write, the window is read again.\n"),
552         (None, "Each window shows Tx, errors, timeouts, the last response time and the last error (Timeout, Exception or invalid response). In each window header, “Rename” chang
    es the window name (also by double-clicking the name), “Pop out” opens it outside the tab (“Dock” or closing that window returns it) and “×” closes the window.\n"),
553         (None, "The “Workspace” menu saves the connection, every window, names and formats under a name of your choice, and lists the saved workspaces so they can be opened or d
    eleted directly. The files are kept in %APPDATA%\\MBSniffer\\Workspaces, next to the preferences.\n")
```

```
        eleted directly. The files are kept in %APPDATA%\\MBSniffer\\Workspaces, next to the preferences.\n"),
554         (None, "The last workspace opened or saved is reopened when the program starts, without connecting to the bus. While there are unsaved changes, “(modified)” appears besi
    de its name; closing the program in that state asks whether to save. Windows never saved in a workspace close without asking, and closing every window leaves no workspace open.\
    n"),
555
556         ("heading", "Saved preferences\n"),
557         (None, "User preferences are stored in %APPDATA%\\MBSniffer\\settings.json. They include Light/Dark, Language, last tab, serial parameters, Frame gap, Pending timeout, A
    uto-scroll, Separate transactions, Highlight results, Bus Slave Finder options and Master connection parameters. Master windows are saved in workspaces, in the %APPDATA%\\MBSnif
    fer\\Workspaces folder.\n"),
558         (None, "COM ports are not saved because they are specific to each computer. The Bus Slave Finder safety confirmation is not saved either.\n"),
559
560         ("heading", "Diagnostic best practices\n"),
```

MBSniffer v3.2/mb_i18n.py - lines 561-600

### MBSniffer v3.2/mb_i18n.py — lines 561-600

```
561         ("bullet", "• Confirm wiring, polarity and serial parameters first.\n"),
562         ("bullet", "• On RS485, keep the sniffer stub short and avoid changing the existing termination.\n"),
563         ("bullet", "• Make sure no other program has the same COM port open.\n"),
564         ("bullet", "• Treat CRC ERROR and RAW as indicators of framing, noise, incorrect parameters or incomplete capture.\n"),
565         ("bullet", "• Treat Exceptions as valid protocol responses: they are not equivalent to CRC errors.\n"),
566         ("bullet", "• Compare response times with normal equipment behavior, not only with a generic threshold.\n"),
567         ("bullet", "• Keep the TXT log when you need later analysis of the complete capture.\n"),
568     ]
569     if debug_sim:
570         blocks.extend([
571             ("heading", "Simulation mode\n"),
572             (None, "This development mode appears only when debug_sim = 1 in MBSniffer.py. It does not open serial ports and never creates TXT logs.\n"),
573             (None, "Each run uses the same diagnostic scenario types: one normal response, one slow response, one Modbus Exception, one CRC error, one timeout and one RAW/UNPARSED block. This makes it possible to compare the same information directly with and without visual highlighting.\n"),
574             (None, "Highlight results does not change Simulation data. When disabled, all rows use normal table colors; when enabled, the completed normal transaction is green and Exception, CRC error, timeout, slow response and RAW rows use their respective diagnostic colors.\n"),
575             (None, "Frames in this scenario use the SIM channel so they are not confused with real physical capture.\n"),
576         ])
577     blocks.extend([
578         ("heading", "Important limitation\n"),
579         (None, "MBSniffer interprets bytes, CRC, Function Code, addresses, quantities, pairing and timing observed by the operating system. It does not directly measure electrical levels, ringing, reflections, bias, common-mode, rise/fall times or analog signal integrity. Those problems still require an oscilloscope or an analyzer suitable for the physical interface.\n"),
580     ])
581     return blocks
582 
583 
584 def translate_runtime_text(text, language):
585     """Translate status strings that contain live values."""
586     import re
587 
588     value = str(text or "")
589     target_en = normalize_language(language) == LANGUAGE_ENGLISH
590 
591     # Normalize either language to canonical PT where practical, then render.
592     patterns = [
593         (r"^Encontrados: (\d+)$", r"Found: \1", target_en),
594         (r"^Found: (\d+)$", r"Encontrados: \1", not target_en),
595         (r"^Decorrido (.+) — restante ~(.+)$", r"Elapsed \1 — remaining ~\2", target_en),
596         (r"^Elapsed (.+) — remaining ~(.+)$", r"Decorrido \1 — restante ~\2", not target_en),
597         (r"^A configurar (.+)…$", r"Configuring \1…", target_en),
598         (r"^Configuring (.+)…$", r"A configurar \1…", not target_en),
599         (r"^Pesquisa parada — (\d+) encontrados — (.+)$", r"Scan stopped — \1 found — \2", target_en),
600         (r"^Scan stopped — (\d+) found — (.+)$", r"Pesquisa parada — \1 encontrados — \2", not target_en),
```

**MBSniffer v3.2/mb_i18n.py — lines 601-610**

```
601         (r"^Pesquisa concluída — (\d+) encontrados — (.+)$", r"Scan complete — \1 found — \2", target_en),
602         (r"^Scan complete — (\d+) found — (.+)$", r"Pesquisa concluída — \1 encontrados — \2", not target_en),
603         (r"^(\d+) config\. × (\d+) slaves — máx\. estimado: (.+)$", r"\1 config. × \2 slaves — estimated max.: \3", target_en),
604         (r"^(\d+) config\. × (\d+) slaves — estimated max\.: (.+)$", r"\1 config. × \2 slaves — máx. estimado: \3", not target_en),
605     ]
606     for pattern, replacement, enabled in patterns:
607         if enabled and re.match(pattern, value):
608             return re.sub(pattern, replacement, value)
609 
610     return translate_text(value, language)
```

# MBSniffer v3.2/mb_theme.py

692 source lines.

**MBSniffer v3.2/mb_theme.py — lines 1-40**

```python
#!/usr/bin/env python3
"""MBSniffer fixed-geometry Light/Dark theme.

The application uses one ttk construction for its entire lifetime. Light/Dark
switching changes colours only; it never changes ttk theme engine, layouts,
padding, borders, margins, widget metrics or row heights.
"""

import tkinter as tk
from tkinter import ttk


BOOTSTRAP_ACTIVE = False
THEME_ENGINE = "clam"

LIGHT = {
    "root": "#f3f4f6",
    "panel": "#f3f4f6",
    "panel_alt": "#ffffff",
    "input": "#ffffff",
    "input_readonly": "#f5f6f7",
    "text": "#1f2328",
    "muted": "#59636e",
    "disabled": "#8b949e",
    "border": "#c3c9d0",
    "divider": "#d6dbe0",
    "button": "#ffffff",
    "button_hover": "#f0f5fa",
    "button_pressed": "#e3edf6",
    "accent": "#2f80d1",
    "accent_hover": "#3b8edb",
    "accent_pressed": "#246eB8",
    "select": "#2f80d1",
    "select_text": "#ffffff",
    "scroll_trough": "#e8ebee",
    "scroll_thumb": "#b9c2ca",
    "scroll_thumb_hover": "#929faa",
}

DARK = {
```

**MBSniffer v3.2/mb_theme.py — lines 41-80**

```
    "root": "#171a1f",
    "panel": "#171a1f",
    "panel_alt": "#1f242a",
    "input": "#20262c",
    "input_readonly": "#252c33",
    "text": "#e8eaed",
    "muted": "#aab2ba",
    "disabled": "#707982",
    "border": "#46525d",
    "divider": "#46525d",
    "button": "#252c33",
    "button_hover": "#303942",
    "button_pressed": "#1f252b",
    "accent": "#2f8fd8",
    "accent_hover": "#43a2e8",
    "accent_pressed": "#287bb8",
    "select": "#285f80",
    "select_text": "#ffffff",
    "scroll_trough": "#1d2329",
    "scroll_thumb": "#65717c",
    "scroll_thumb_hover": "#7d8994",
}

_CHECK_IMAGES = None
_CHECK_ELEMENT = "MBS.Check.indicator"
_FIXED_GEOMETRY_READY = False
_INTERP_ID = None


def theme_colors(root=None, dark=False):
    return dict(DARK if dark else LIGHT)


def _ensure_engine(style):
    """Select/configure the fixed engine once per Tk interpreter."""
    global _FIXED_GEOMETRY_READY, _CHECK_IMAGES, _INTERP_ID

    # A normal MBSniffer process owns one Tk root. Handling a replaced Tk
    # interpreter as well makes tests/relaunches in the same Python process
    # safe and prevents stale PhotoImage command references.
```

**MBSniffer v3.2/mb_theme.py — lines 81-120**

```
    current_interp = id(style.master.tk)
    if _INTERP_ID != current_interp:
        _INTERP_ID = current_interp
        _FIXED_GEOMETRY_READY = False
        _CHECK_IMAGES = None

    try:
        if THEME_ENGINE in style.theme_names() and style.theme_use() != THEME_ENGINE:
            style.theme_use(THEME_ENGINE)
    except Exception:
        pass

    if not _FIXED_GEOMETRY_READY:
        _configure_fixed_geometry(style)
        _FIXED_GEOMETRY_READY = True


def _configure_fixed_geometry(style):
    """
    Define every geometry/state rule once.

    Nothing in this function is called as part of a Light/Dark toggle after
    startup. This is what guarantees that the page construction cannot move.
    """

    # ------------------------------------------------------------------
    # LabelFrames: titles above the rectangular border in both modes.
    # ------------------------------------------------------------------
    style.configure(
        "TLabelframe",
        borderwidth=1,
        relief="solid",
        padding=0,
        labeloutside=True,
        labelmargins=(0, 0, 0, 4),
    )
    style.configure(
        "TLabelframe.Label",
        borderwidth=0,
        padding=0,
```

**MBSniffer v3.2/mb_theme.py — lines 121-160**

```
        font=("Segoe UI", 9, "bold"),
    )

    # ------------------------------------------------------------------
    # Notebook/tabs: fixed Vista-like top geometry.
    # ------------------------------------------------------------------
    style.configure(
        "TNotebook",
        borderwidth=1,
        padding=0,
        tabmargins=(2, 2, 2, 0),
    )
    style.configure(
        "TNotebook.Tab",
        borderwidth=1,
        padding=(12, 6),
        font=("Segoe UI", 9),
    )
    style.map(
        "TNotebook.Tab",
        expand=[("selected", (2, 2, 2, 2))],
        padding=[("selected", (12, 6))],
        relief=[
            ("selected", "flat"),
            ("active", "flat"),
            ("pressed", "flat"),
        ],
    )

    # ------------------------------------------------------------------
    # Buttons.
    # ------------------------------------------------------------------
    style.configure(
        "TButton",
        borderwidth=1,
        relief="solid",
        padding=(7, 4),
        font=("Segoe UI", 9),
    )
    style.configure(
```

### MBSniffer v3.2/mb_theme.py — lines 161-200

```
        "MBS.Toolbar.TButton",
        borderwidth=1,
        relief="solid",
        padding=(9, 5),
        font=("Segoe UI", 9),
    )
    style.configure(
        "MBS.Primary.TButton",
        borderwidth=1,
        relief="solid",
        padding=(9, 5),
        font=("Segoe UI", 9, "bold"),
    )
    style.configure(
        "MBS.Compact.TButton",
        borderwidth=1,
        relief="solid",
        padding=(7, 4),
        font=("Segoe UI", 9),
    )

    # ------------------------------------------------------------------
    # Input controls.
    # ------------------------------------------------------------------
    style.configure("TEntry", borderwidth=1, relief="solid", padding=1)
    style.configure("TCombobox", borderwidth=1, relief="solid", padding=1, arrowsize=10)
    style.configure("TSpinbox", borderwidth=1, relief="solid", padding=1, arrowsize=7)

    # ------------------------------------------------------------------
    # Checkboxes: fixed 15x15 image indicator. The image pixels are recoloured
    # on toggle; the element/layout never changes.
    # ------------------------------------------------------------------
    style.configure(
        "TCheckbutton",
        borderwidth=0,
        relief="flat",
        padding=(1, 1),
        font=("Segoe UI", 9),
    )

```

**MBSniffer v3.2/mb_theme.py — lines 201-240**

```
    # Create the indicator element from the active clam layout once.
    _install_fixed_checkbutton_element(style)

    # ------------------------------------------------------------------
    # Treeviews.
    # ------------------------------------------------------------------
    style.configure(
        "Treeview",
        borderwidth=1,
        relief="solid",
        font=("Segoe UI", 9),
    )
    style.configure(
        "MBSniffer.Treeview",
        borderwidth=1,
        relief="solid",
        font=("Segoe UI", 9),
    )

    # Important: right + bottom border only. This reproduces Light mode's
    # vertical column divider lines without making every heading look like a
    # separate raised rectangular button.
    for heading in ("Treeview.Heading", "MBSniffer.Treeview.Heading"):
        style.configure(
            heading,
            borderwidth=(0, 0, 1, 1),
            relief="solid",
            padding=(5, 4),
            font=("Segoe UI", 9),
        )
        style.map(
            heading,
            relief=[
                ("active", "solid"),
                ("pressed", "solid"),
            ],
        )

    # Keep MBSniffer.Treeview on the same heading layout as Treeview.
    try:
```

### MBSniffer v3.2/mb_theme.py — lines 241-280

```
        heading_layout = style.layout("Treeview.Heading")
        if heading_layout:
            style.layout("MBSniffer.Treeview.Heading", heading_layout)
    except Exception:
        pass

    # ------------------------------------------------------------------
    # Scrollbars and named labels.
    # ------------------------------------------------------------------
    # Minimal scrollbars: slim track + thumb only, without arrow buttons or extra controls.
    style.layout(
        "Vertical.TScrollbar",
        [(
            "Vertical.Scrollbar.trough",
            {
                "sticky": "ns",
                "children": [
                    ("Vertical.Scrollbar.thumb", {"sticky": "nswe"})
                ],
            },
        )],
    )
    style.layout(
        "Horizontal.TScrollbar",
        [(
            "Horizontal.Scrollbar.trough",
            {
                "sticky": "we",
                "children": [
                    ("Horizontal.Scrollbar.thumb", {"sticky": "nswe"})
                ],
            },
        )],
    )
    style.configure(
        "Vertical.TScrollbar",
        width=6,
        borderwidth=0,
        relief="flat",
    )
```

**MBSniffer v3.2/mb_theme.py — lines 281-320**

```
    style.configure(
        "Horizontal.TScrollbar",
        width=6,
        borderwidth=0,
        relief="flat",
    )

    style.configure(
        "Horizontal.TProgressbar",
        borderwidth=0,
        thickness=8,
        relief="flat",
    )

    style.configure("MBS.Stat.TLabel", font=("Segoe UI", 9))
    style.configure("MBS.Status.TLabel", font=("Segoe UI", 9, "bold"))


def _paint_checkbox_image(image, *, fill, border, check=None):
    """Repaint one fixed-size checkbox image without changing its dimensions."""
    size = 15
    image.put(border, to=(0, 0, size, size))
    image.put(fill, to=(1, 1, size - 1, size - 1))

    if check:
        # White/black check mark. Coordinates remain identical in both modes.
        points = (
            (3, 7), (4, 8), (5, 9), (6, 10),
            (7, 9), (8, 8), (9, 7), (10, 6), (11, 5), (12, 4),
        )
        for x, y in points:
            image.put(check, to=(x, y, x + 1, y + 2))
            if x + 1 < size - 1:
                image.put(check, to=(x + 1, y, x + 2, y + 1))


def _install_fixed_checkbutton_element(style):
    global _CHECK_IMAGES

    root = style.master
```

### MBSniffer v3.2/mb_theme.py — lines 321-360

```
    if _CHECK_IMAGES is None:
        _CHECK_IMAGES = {
            "off": tk.PhotoImage(master=root, width=15, height=15),
            "on": tk.PhotoImage(master=root, width=15, height=15),
            "disabled_off": tk.PhotoImage(master=root, width=15, height=15),
            "disabled_on": tk.PhotoImage(master=root, width=15, height=15),
        }

    try:
        if _CHECK_ELEMENT not in style.element_names():
            style.element_create(
                _CHECK_ELEMENT,
                "image",
                _CHECK_IMAGES["off"],
                ("disabled selected", _CHECK_IMAGES["disabled_on"]),
                ("disabled", _CHECK_IMAGES["disabled_off"]),
                ("selected", _CHECK_IMAGES["on"]),
                sticky="",
            )
    except tk.TclError:
        pass

    try:
        layout = style.layout("TCheckbutton")
    except Exception:
        layout = None

    if not layout:
        return

    def replace_indicator(nodes):
        result = []
        for name, options in nodes:
            options = dict(options)
            if "children" in options:
                options["children"] = replace_indicator(options["children"])
            if name == "Checkbutton.indicator":
                name = _CHECK_ELEMENT
            result.append((name, options))
        return result
```

### MBSniffer v3.2/mb_theme.py — lines 361-400

```
361
362     # Avoid repeatedly replacing an already-custom element.
363     if _CHECK_ELEMENT not in repr(layout):
364         try:
365             style.layout("TCheckbutton", replace_indicator(layout))
366         except tk.TclError:
367             pass
368
369
370 def _repaint_checkboxes(colors, dark):
371     if not _CHECK_IMAGES:
372         return
373
374     if dark:
375         face = colors["input_readonly"]
376         border = colors["disabled"]
377         check = colors["select_text"]
378         disabled_face = colors["panel_alt"]
379         disabled_check = colors["disabled"]
380     else:
381         face = colors["input"]
382         border = "#7a7a7a"
383         check = "#111111"
384         disabled_face = "#e4e4e4"
385         disabled_check = "#999999"
386
387     _paint_checkbox_image(
388         _CHECK_IMAGES["off"],
389         fill=face,
390         border=border,
391     )
392     _paint_checkbox_image(
393         _CHECK_IMAGES["on"],
394         fill=face,
395         border=border,
396         check=check,
397     )
398     _paint_checkbox_image(
399         _CHECK_IMAGES["disabled_off"],
400         fill=disabled_face,
```

### MBSniffer v3.2/mb_theme.py — lines 401-440

```
401         border=colors["disabled"],
402     )
403     _paint_checkbox_image(
404         _CHECK_IMAGES["disabled_on"],
405         fill=disabled_face,
406         border=colors["disabled"],
407         check=disabled_check,
408     )
409
410
411 def _apply_palette(root, style, colors, dark):
412     """
413     Apply colours only.
414
415     No call in this function changes:
416     - theme engine
417     - layout
418     - padding
419     - border width
420     - relief
421     - margins
422     - font
423     - widget dimensions
424     """
425     c = colors
426
427     style.configure(".", background=c["root"], foreground=c["text"])
428     style.configure("TFrame", background=c["root"])
429     style.configure("TLabel", background=c["root"], foreground=c["text"])
430     style.configure(
431         "TLabelframe",
432         background=c["root"],
433         foreground=c["text"],
434         bordercolor=c["border"],
435         lightcolor=c["border"],
436         darkcolor=c["border"],
437     )
438     style.configure(
439         "TLabelframe.Label",
440         background=c["root"],
```

**MBSniffer v3.2/mb_theme.py — lines 441-480**

```
        foreground=c["text"],
    )

    # Notebook.
    style.configure(
        "TNotebook",
        background=c["root"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.configure(
        "TNotebook.Tab",
        background=c["panel"],
        foreground=c["text"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.map(
        "TNotebook.Tab",
        background=[
            ("selected", c["panel_alt"]),
            ("active", c["panel_alt"]),
        ],
        foreground=[
            ("selected", c["text"]),
            ("active", c["text"]),
        ],
    )

    # Buttons: neutral controls have clear contrast; the primary action uses
    # the same restrained accent in Light and Dark.
    for name in (
        "TButton",
        "MBS.Toolbar.TButton",
        "MBS.Compact.TButton",
    ):
        style.configure(
            name,
```

### MBSniffer v3.2/mb_theme.py — lines 481-520

```
            background=c["button"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
            focuscolor=c["accent"],
        )
        style.map(
            name,
            background=[
                ("pressed", c["button_pressed"]),
                ("active", c["button_hover"]),
                ("disabled", c["panel"]),
            ],
            foreground=[("disabled", c["disabled"])],
        )

    style.configure(
        "MBS.Primary.TButton",
        background=c["accent"],
        foreground=c["select_text"],
        bordercolor=c["accent"],
        lightcolor=c["accent"],
        darkcolor=c["accent"],
        focuscolor=c["accent"],
    )
    style.map(
        "MBS.Primary.TButton",
        background=[
            ("pressed", c["accent_pressed"]),
            ("active", c["accent_hover"]),
            ("disabled", c["panel"]),
        ],
        foreground=[
            ("disabled", c["disabled"]),
        ],
        bordercolor=[
            ("pressed", c["accent_pressed"]),
            ("active", c["accent_hover"]),
            ("disabled", c["border"]),
```

**MBSniffer v3.2/mb_theme.py — lines 521-560**

```
        ],
    )

    # Entry / Spinbox.
    for name in ("TEntry", "TSpinbox"):
        style.configure(
            name,
            fieldbackground=c["input"],
            background=c["input"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
            insertcolor=c["text"],
        )
        style.map(
            name,
            fieldbackground=[
                ("readonly", c["input_readonly"]),
                ("disabled", c["panel"]),
            ],
            foreground=[("disabled", c["disabled"])],
        )

    # Combobox.
    style.configure(
        "TCombobox",
        fieldbackground=c["input"],
        background=c["button"],
        foreground=c["text"],
        arrowcolor=c["text"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.map(
        "TCombobox",
        fieldbackground=[
            ("readonly", c["input_readonly"]),
            ("disabled", c["panel"]),
```

### MBSniffer v3.2/mb_theme.py — lines 561-600

```
        ],
        foreground=[
            ("readonly", c["text"]),
            ("disabled", c["disabled"]),
        ],
        background=[
            ("active", c["button_hover"]),
            ("disabled", c["panel"]),
        ],
        arrowcolor=[("disabled", c["disabled"])],
    )

    # Checkbox label/background only; indicator pixels are repainted separately.
    style.configure(
        "TCheckbutton",
        background=c["root"],
        foreground=c["text"],
    )
    style.map(
        "TCheckbutton",
        background=[("active", c["root"])],
        foreground=[("disabled", c["disabled"])],
    )
    _repaint_checkboxes(c, dark)

    # Treeview body and flat heading.
    for name in ("Treeview", "MBSniffer.Treeview"):
        style.configure(
            name,
            background=c["input"],
            fieldbackground=c["input"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
        )
        style.map(
            name,
            background=[("selected", c["select"])],
            foreground=[("selected", c["select_text"])],
```

**MBSniffer v3.2/mb_theme.py — lines 601-640**

```
        )

    for name in ("Treeview.Heading", "MBSniffer.Treeview.Heading"):
        style.configure(
            name,
            background=c["input"],
            foreground=c["text"],
            bordercolor=c["divider"],
            lightcolor=c["divider"],
            darkcolor=c["divider"],
        )
        style.map(
            name,
            background=[
                ("active", c["input_readonly"]),
                ("pressed", c["input_readonly"]),
            ],
            foreground=[("active", c["text"])],
        )

    # Scrollbars: only the slim thumb should be visually apparent.
    # The required trough element remains for ttk mechanics, but is painted
    # exactly like the content background so it disappears visually.
    for name in ("Vertical.TScrollbar", "Horizontal.TScrollbar", "TScrollbar"):
        style.configure(
            name,
            background=c["scroll_thumb"],
            troughcolor=c["input"],
            bordercolor=c["input"],
            darkcolor=c["scroll_thumb"],
            lightcolor=c["scroll_thumb"],
        )
        style.map(
            name,
            background=[
                ("active", c["scroll_thumb_hover"]),
                ("pressed", c["scroll_thumb_hover"]),
            ],
            darkcolor=[
                ("active", c["scroll_thumb_hover"]),
```

### MBSniffer v3.2/mb_theme.py — lines 641-680

```
641                 ("pressed", c["scroll_thumb_hover"]),
642             ],
643             lightcolor=[
644                 ("active", c["scroll_thumb_hover"]),
645                 ("pressed", c["scroll_thumb_hover"]),
646             ],
647         )
648 
649     style.configure(
650         "Horizontal.TProgressbar",
651         background=c["accent"],
652         troughcolor=c["scroll_trough"],
653         bordercolor=c["scroll_trough"],
654         lightcolor=c["accent"],
655         darkcolor=c["accent"],
656     )
657 
658     style.configure("MBS.Stat.TLabel", background=c["root"], foreground=c["text"])
659     style.configure("MBS.Status.TLabel", background=c["root"], foreground=c["text"])
660 
661     try:
662         root.configure(background=c["root"])
663     except Exception:
664         pass
665 
666     return style
667 
668 
669 def apply_theme(root, dark=False, style=None):
670     """
671     Change palette only.
672 
673     The ttk engine and all geometry were fixed before the UI was built.
674     """
675     style = style or ttk.Style(root)
676     _ensure_engine(style)
677     return _apply_palette(root, style, DARK if dark else LIGHT, bool(dark))
678 
679 
680 def apply_light_theme(root, style=None):
```

### MBSniffer v3.2/mb_theme.py — lines 681-692

```
    return apply_theme(root, dark=False, style=style)


def apply_dark_theme(root, style=None):
    return apply_theme(root, dark=True, style=style)


def apply_modern_theme(root, dark=False):
    """Initial application setup with the same engine used for both modes."""
    style = ttk.Style(root)
    _ensure_engine(style)
    return _apply_palette(root, style, DARK if dark else LIGHT, bool(dark))
```

# MBSniffer v3.2/mb_widgets.py

418 source lines.

**MBSniffer v3.2/mb_widgets.py — lines 1-40**

```
#!/usr/bin/env python3
"""Small reusable Tk widgets for MBSniffer."""

import tkinter as tk


class RefreshButton(tk.Canvas):
    # The fixed clam UI gives the refresh control stable dimensions, so the ↻
    # can now use its measured rendered-bbox centre without manual offsets.
    OPTICAL_OFFSET_X = 0
    OPTICAL_OFFSET_Y = 0

    """
    Square refresh button using the Unicode ↻ glyph.

    The glyph is centered from its actual rendered Tk bounding box. No manual
    X/Y optical offset is applied.
    """

    def __init__(
        self,
        master,
        command,
        size=26,
        cursor="hand2",
        **kwargs,
    ):
        self._command = command
        self._hover = False
        self._pressed = False
        self._state = "normal"
        self._dark_mode = False
        self._final_redraw_after_id = None

        super().__init__(
            master,
            width=size,
            height=size,
            bd=0,
            highlightthickness=0,
```

### MBSniffer v3.2/mb_widgets.py — lines 41-80

```
41            relief="flat",
42            cursor=cursor,
43            takefocus=0,
44            **kwargs,
45        )
46
47        self.bind("<Configure>", self._redraw, add="+")
48        self.bind("<Enter>", self._on_enter, add="+")
49        self.bind("<Leave>", self._on_leave, add="+")
50        self.bind("<ButtonPress-1>", self._on_press, add="+")
51        self.bind("<ButtonRelease-1>", self._on_release, add="+")
52        self._redraw()
53        self._schedule_final_redraw()
54
55    @staticmethod
56    def _system_color(widget, name, fallback):
57        try:
58            widget.winfo_rgb(name)
59            return name
60        except Exception:
61            return fallback
62
63    def _schedule_final_redraw(self):
64        """
65        Redraw after Tk has settled requested widget dimensions.
66
67        Some themed comboboxes report their final height only after the first
68        layout pass. A short deferred redraw prevents the Unicode glyph from
69        staying centered for an earlier 24 px square after the control becomes
70        26 px high.
71        """
72        try:
73            if self._final_redraw_after_id is not None:
74                self.after_cancel(self._final_redraw_after_id)
75            self._final_redraw_after_id = self.after(20, self._final_redraw)
76        except tk.TclError:
77            self._final_redraw_after_id = None
78
79    def _final_redraw(self):
80        self._final_redraw_after_id = None
```

### MBSniffer v3.2/mb_widgets.py — lines 81-120

```
        try:
            self._redraw()
        except tk.TclError:
            pass

    def configure(self, cnf=None, **kwargs):
        state = kwargs.pop("state", None)
        if state is not None:
            self._state = str(state)

        result = super().configure(cnf, **kwargs)
        try:
            self._redraw()
            self._schedule_final_redraw()
        except tk.TclError:
            pass
        return result

    config = configure

    def cget(self, key):
        if key == "state":
            return self._state
        return super().cget(key)

    def invoke(self):
        if self._state != "disabled" and callable(self._command):
            return self._command()
        return None

    def set_dark_mode(self, enabled):
        self._dark_mode = bool(enabled)
        self._redraw()
        self._schedule_final_redraw()

    def _palette(self):
        if self._dark_mode:
            face = "#2a3138"
            hover = "#36404a"
            pressed = "#20262c"
```

**MBSniffer v3.2/mb_widgets.py — lines 121-160**

```
            text = "#e8eaed"
            gray = "#707982"
            border = "#4a5560"
            highlight = "#2f8fd8"

            if self._state == "disabled":
                return face, gray, border
            if self._pressed:
                return pressed, text, highlight
            if self._hover:
                return hover, text, highlight
            return face, text, border

        face = self._system_color(self, "SystemButtonFace", "#f0f0f0")
        text = self._system_color(self, "SystemButtonText", "#202020")
        gray = self._system_color(self, "SystemGrayText", "#8a8a8a")
        shadow = self._system_color(self, "System3DShadow", "#adadad")
        highlight = self._system_color(self, "SystemHighlight", "#7a9fc2")

        if self._state == "disabled":
            return face, gray, shadow
        if self._pressed:
            return face, text, highlight
        if self._hover:
            return face, text, highlight
        return face, text, shadow

    def _center_item_bbox(self, item, width, height):
        """Center the visible Tk bbox of one Canvas item on the square."""
        target_x = width / 2.0
        target_y = height / 2.0

        # Tk text bboxes land on integer pixels. Repeat because a fractional
        # first move may be rounded by the Canvas implementation.
        for _ in range(4):
            bbox = self.bbox(item)
            if not bbox:
                return
            current_x = (bbox[0] + bbox[2]) / 2.0
            current_y = (bbox[1] + bbox[3]) / 2.0
```

### MBSniffer v3.2/mb_widgets.py — lines 161-200

```
            dx = target_x - current_x
            dy = target_y - current_y

            if abs(dx) <= 0.5 and abs(dy) <= 0.5:
                break

            # Move by integral pixels when a full-pixel correction is needed.
            move_x = round(dx) if abs(dx) > 0.5 else 0
            move_y = round(dy) if abs(dy) > 0.5 else 0
            if move_x == 0 and move_y == 0:
                break
            self.move(item, move_x, move_y)

        # Keep the explicit offset hook at zero so all three refresh buttons
        # use the same measured centre.
        if self.OPTICAL_OFFSET_X or self.OPTICAL_OFFSET_Y:
            self.move(
                item,
                self.OPTICAL_OFFSET_X,
                self.OPTICAL_OFFSET_Y,
            )

    def _redraw(self, _event=None):
        if not self.winfo_exists():
            return

        # During a <Configure> callback winfo_width()/height() may still
        # report the previous geometry on some Tk builds. Prefer the event's
        # new dimensions so centering is calculated for the final square.
        if _event is not None and hasattr(_event, "width") and hasattr(_event, "height"):
            width = max(2, int(_event.width))
            height = max(2, int(_event.height))
        else:
            width = max(2, int(self.winfo_width()))
            height = max(2, int(self.winfo_height()))

        background, foreground, border = self._palette()

        self.delete("all")
        self.create_rectangle(
```

### MBSniffer v3.2/mb_widgets.py — lines 201-240

```
            0,
            0,
            width - 1,
            height - 1,
            fill=background,
            outline=border,
            width=1,
            tags=("background",),
        )

        glyph = self.create_text(
            width / 2.0,
            height / 2.0,
            text="↻",
            fill=foreground,
            font=("Segoe UI Symbol", 13),
            anchor="center",
            tags=("glyph",),
        )
        self._center_item_bbox(glyph, width, height)

    def _on_enter(self, _event):
        if self._state != "disabled":
            self._hover = True
            self._redraw()

    def _on_leave(self, _event):
        self._hover = False
        self._pressed = False
        self._redraw()

    def _on_press(self, _event):
        if self._state != "disabled":
            self._pressed = True
            self._redraw()

    def _on_release(self, event):
        if self._state == "disabled":
            return

```

### MBSniffer v3.2/mb_widgets.py — lines 241-280

```
241         was_pressed = self._pressed
242         self._pressed = False
243         self._redraw()
244 
245         inside = (
246             0 <= event.x < self.winfo_width()
247             and 0 <= event.y < self.winfo_height()
248         )
249         if was_pressed and inside and callable(self._command):
250             self._command()
251 
252 
253 
254 class ToggleSwitch(tk.Canvas):
255     """Compact on/off slider used for the Light/Dark mode control."""
256 
257     def __init__(
258         self,
259         master,
260         variable,
261         command=None,
262         width=42,
263         height=22,
264         cursor="hand2",
265         **kwargs,
266     ):
267         self.variable = variable
268         self.command = command
269         self._dark_mode = False
270         self._hover = False
271         self._final_redraw_after_id = None
272 
273         super().__init__(
274             master,
275             width=width,
276             height=height,
277             bd=0,
278             highlightthickness=0,
279             relief="flat",
280             cursor=cursor,
```

### MBSniffer v3.2/mb_widgets.py — lines 281-320

```
            takefocus=0,
            **kwargs,
        )

        self.bind("<Configure>", self._redraw, add="+")
        self.bind("<ButtonPress-1>", self._press, add="+")
        self.bind("<ButtonRelease-1>", self._toggle, add="+")
        self.bind("<Enter>", self._enter, add="+")
        self.bind("<Leave>", self._leave, add="+")
        self.variable.trace_add("write", lambda *_a: self._redraw())
        self._redraw()
        self._schedule_final_redraw()

    def _schedule_final_redraw(self):
        try:
            if self._final_redraw_after_id is not None:
                self.after_cancel(self._final_redraw_after_id)
            self._final_redraw_after_id = self.after(20, self._final_redraw)
        except tk.TclError:
            self._final_redraw_after_id = None

    def _final_redraw(self):
        self._final_redraw_after_id = None
        try:
            self._redraw()
        except tk.TclError:
            pass

    def set_dark_mode(self, enabled):
        self._dark_mode = bool(enabled)
        self._redraw()
        self._schedule_final_redraw()

    def _parent_bg(self):
        if self._dark_mode:
            return "#171a1f"
        try:
            self.winfo_rgb("SystemButtonFace")
            return "SystemButtonFace"
        except Exception:
```

### MBSniffer v3.2/mb_widgets.py — lines 321-360

```
321             return "#f0f0f0"
322
323     def _press(self, _event=None):
324         # Deliberately do not focus the Canvas. This removes the white focus
325         # rectangle that appeared around the switch after mouse clicks.
326         return "break"
327
328     def _toggle(self, _event=None):
329         self.variable.set(not bool(self.variable.get()))
330         if callable(self.command):
331             self.command()
332         return "break"
333
334     def _enter(self, _event=None):
335         self._hover = True
336         self._redraw()
337
338     def _leave(self, _event=None):
339         self._hover = False
340         self._redraw()
341
342     def _redraw(self, _event=None):
343         if not self.winfo_exists():
344             return
345
346         # Use the final dimensions delivered by <Configure>. Without this, the
347         # first draw can happen while Tk still reports the initial 1x1 Canvas,
348         # which made the switch look small until the first mouse hover.
349         if _event is not None and hasattr(_event, "width") and hasattr(_event, "height"):
350             width = max(22, int(_event.width))
351             height = max(14, int(_event.height))
352         else:
353             width = max(22, int(self.winfo_width()))
354             height = max(14, int(self.winfo_height()))
355
356         on = bool(self.variable.get())
357
358         self.configure(background=self._parent_bg())
359         self.delete("all")
360
```

### MBSniffer v3.2/mb_widgets.py — lines 361-400

```
361         pad = 2
362         radius = (height - 2 * pad) / 2.0
363         x0, y0 = pad, pad
364         x1, y1 = width - pad, height - pad
365 
366         if self._dark_mode:
367             off_track = "#5a646e"
368             on_track = "#2f8fd8"
369             hover_track = "#43a2e8"
370             knob = "#f4f6f8"
371             outline = "#68737d"
372         else:
373             off_track = "#a8a8a8"
374             on_track = "#2689d9"
375             hover_track = "#3b9be5"
376             knob = "#ffffff"
377             outline = "#8f8f8f"
378 
379         track = hover_track if self._hover and on else (on_track if on else off_track)
380 
381         # Rounded track: rectangle + two circles.
382         self.create_rectangle(
383             x0 + radius,
384             y0,
385             x1 - radius,
386             y1,
387             fill=track,
388             outline=track,
389         )
390         self.create_oval(
391             x0,
392             y0,
393             x0 + 2 * radius,
394             y1,
395             fill=track,
396             outline=track,
397         )
398         self.create_oval(
399             x1 - 2 * radius,
400             y0,
```

**MBSniffer v3.2/mb_widgets.py — lines 401-418**

```
            x1,
            y1,
            fill=track,
            outline=track,
        )

        knob_radius = max(4.0, radius - 2)
        knob_cx = (x1 - radius) if on else (x0 + radius)
        knob_cy = height / 2.0
        self.create_oval(
            knob_cx - knob_radius,
            knob_cy - knob_radius,
            knob_cx + knob_radius,
            knob_cy + knob_radius,
            fill=knob,
            outline=outline,
            width=1,
        )
```

# MBSniffer v3.2/mb_gui.py

2584 source lines.

**MBSniffer v3.2/mb_gui.py — lines 1-40**

```python
#!/usr/bin/env python3
"""Main Tkinter window composition for MBSniffer."""

import queue
import threading
from collections import deque

import tkinter as tk
from tkinter import ttk
import tkinter.font as tkfont

from mb_config import (
    APP_NAME,
    APP_VERSION,
    DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
    MAX_RAW_TEXT_LINES,
    MAX_UI_FRAMES,
    MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
    SLOW_RESPONSE_THRESHOLD_MS,
    SNIFFER_SPLIT_FRAME_HOLD_MS,
    debug_sim,
    load_ui_settings,
    resource_path,
    save_ui_settings,
)
from mb_protocol import SessionMetrics
from mb_slave_finder import SlaveFinderMixin
from mb_master_ui import MasterMixin
from mb_capture import CaptureMixin
from mb_view import TRAFFIC_MENU_OPEN_IN_MASTER, ViewMixin
from mb_widgets import RefreshButton, ToggleSwitch
from mb_theme import apply_modern_theme, apply_theme, theme_colors
from mb_i18n import (
    LANGUAGE_CHOICES,
    LANGUAGE_ENGLISH,
    LANGUAGE_PORTUGUESE,
    canonical_ui_text,
    english_help_blocks,
```

### MBSniffer v3.2/mb_gui.py — lines 41-80

```
41      normalize_language,
42      translate_text,
43      translate_runtime_text,
44  )
45
46
47  class SnifferApp(MasterMixin, SlaveFinderMixin, CaptureMixin, ViewMixin, tk.Tk):
48      def __init__(self):
49          super().__init__()
50          self.title(f"{APP_NAME} v{APP_VERSION}")
51
52          # Load the remembered mode before any ttk widgets are constructed.
53          # Light and Dark use one fixed ttk construction; only colours differ.
54          self.ui_settings = load_ui_settings()
55          self.current_language = normalize_language(
56              self.ui_settings.get("language", LANGUAGE_PORTUGUESE)
57          )
58          self.initial_dark_mode = bool(self.ui_settings.get("dark_mode", False))
59          self.ui_style = apply_modern_theme(
60              self,
61              dark=self.initial_dark_mode,
62          )
63
64          # Explicit application/window icon. This works when launched through
65          # MBSniffer.bat (Python) and when bundled as a PyInstaller EXE.
66          try:
67              icon_file = resource_path("MBSniffer.ico")
68              if icon_file.exists():
69                  self.iconbitmap(default=str(icon_file))
70          except Exception:
71              pass
72
73          self.geometry("1280x780")
74          self.minsize(1050, 650)
75
76          self.serial_objects = []
77          self.serial_by_port = {}
78          self.reader_threads = []
79          self.reader_generation = 0
80          self.stop_event = threading.Event()
```

**MBSniffer v3.2/mb_gui.py — lines 81-120**

```
        self.event_queue = queue.Queue()
        self.busy = False
        self.capture_active = False
        self.restart_in_progress = False

        # Active slave-discovery state. This is deliberately independent from
        # passive capture/simulation state, but shares the global busy lock so
        # the application can never act as scanner and sniffer at the same time.
        self.discovery_active = False
        self.discovery_stop_event = threading.Event()
        self.discovery_thread = None
        self.discovery_serial = None
        self.discovery_found_count = 0
        self._closing = False
        self._process_queue_error_shown = False

        self.log_file = None
        self.log_path = None

        # Real-capture logs are created lazily: starting/stopping a capture with
        # zero received traffic must not create an empty .txt file.
        self.pending_log_filename = None
        self.pending_log_header = ""
        self.pending_log_notes = []
        self.log_start_failed = False
        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._transaction_filter_seqs = None
        self._successful_request_seqs = set()
        self._advanced_filter_after_id = None
        self._bus_activity_after_id = None

        # Bounded GUI-history bookkeeping.
        self._tree_history = deque()   # current visible Treeview rows
        self._ui_frame_count = 0
        self._raw_text_line_count = 0

        # Source-of-truth for the bounded GUI history. Filtering/sorting only
        # changes the view; it never changes statistics or the disk log.
        self._frame_history = deque()
```

**MBSniffer v3.2/mb_gui.py — lines 121-160**

```
121         self._next_frame_seq = 1
122
123         # Visible Treeview bookkeeping by logical frame sequence.
124         self._tree_iids_by_seq = {}
125         self._tree_separator_iid_by_seq = {}
126
127         # Raw Hex separator width/reflow bookkeeping.
128         self._raw_separator_width_chars = 1
129         self._raw_reflow_after_id = None
130
131         # Slave filter state. "All mode" also controls whether newly discovered
132         # slaves are automatically selected.
133         self.slave_filter_vars = {}
134         self.slave_filter_all_mode = True
135
136         self.metrics = SessionMetrics()
137
138         self._build_ui()
139         self._freeze_language_layout_geometry()
140         self._capture_portuguese_help()
141         self.apply_language(save=False)
142         self._install_mouse_wheel_guard()
143         self.rebuild_slave_filter_menu()
144         self.refresh_ports()
145         self.update_mode_ui()
146         self.update_gap_state()
147         self.update_com_status()
148         self._restore_saved_main_tab()
149         self._install_ui_settings_persistence()
150
151         self.protocol("WM_DELETE_WINDOW", self.on_close)
152         self.after(50, self.process_queue)
153         self.after(1000, self.poll_com_status)
154
155
156     def _freeze_language_layout_geometry(self):
157         """Freeze language-independent container geometry using the PT layout.
158
159         The UI is constructed from the canonical Portuguese captions. Capture
160         the natural geometry before any runtime translation and use those
```

**MBSniffer v3.2/mb_gui.py — lines 161-200**

```
        dimensions as layout constraints. Language changes may replace text,
        but must never move or resize the surrounding controls/panels.
        """
        try:
            self.update_idletasks()
        except tk.TclError:
            return

        # The global language/theme overlay is anchored to the top-right.
        # Freeze the canonical PT grid-cell widths. English captions are shorter
        # here, so this preserves the exact existing PT positions and prevents
        # the Dark Mode switch / language combobox from sliding left or right.
        overlay = getattr(self, "theme_overlay", None)
        if overlay is not None:
            try:
                self.update_idletasks()
                canonical_column_widths = [
                    max(1, int(overlay.grid_bbox(column, 0)[2]))
                    for column in range(4)
                ]
                for column, width in enumerate(canonical_column_widths):
                    overlay.columnconfigure(column, minsize=width)
                self.update_idletasks()
                self._language_overlay_width = max(1, overlay.winfo_reqwidth())
                self._language_overlay_height = max(1, overlay.winfo_reqheight())
                overlay.place_configure(
                    width=self._language_overlay_width,
                    height=self._language_overlay_height,
                )
            except (tk.TclError, IndexError, TypeError):
                pass

        # The Finder configuration includes wrapped explanatory text. It must
        # be measured while the Finder page is actually laid out; measuring a
        # hidden Notebook page can under-estimate the PT height by one line.
        page = getattr(self, "discovery_page", None)
        options = getattr(self, "_discovery_options_frame", None)
        if page is not None and options is not None:
            try:
                selected_tab = self.main_notebook.select()
```

### MBSniffer v3.2/mb_gui.py — lines 201-240

```
                self.main_notebook.select(page)
                self.update_idletasks()
                height = max(options.winfo_height(), options.winfo_reqheight(), 1)
                # grid row height also includes the widget's external pady.
                # Preserve the complete PT cell, not only the LabelFrame body.
                try:
                    cell_height = max(height, int(page.grid_bbox(0, 1)[3]))
                except (tk.TclError, TypeError, IndexError):
                    cell_height = height + 12
                self._language_discovery_options_height = height
                page.rowconfigure(1, minsize=cell_height)
                options.grid_configure(sticky="nsew")
                self.main_notebook.select(selected_tab)
                self.update_idletasks()
            except tk.TclError:
                pass

    def _guard_mouse_wheel(self, event):
        """
        Keep wheel input local to genuinely scrollable content widgets.

        Treeviews and Text widgets retain their normal vertical wheel scrolling,
        as do the scrollbar widgets themselves.  Everywhere else the event is
        consumed before ttk class bindings see it; in particular, this prevents
        Windows ttk Notebook tabs from changing when the user turns the wheel
        over a tab or over non-scrollable page content.
        """
        widget = getattr(event, "widget", None)
        if isinstance(
            widget,
            (ttk.Treeview, tk.Text, tk.Listbox, ttk.Scrollbar, tk.Scrollbar),
        ):
            return None
        return "break"

    def _install_mouse_wheel_guard(self):
        """
        Prepend one application bindtag to every existing widget.

        The UI is statically constructed before this method is called, so this
```

### MBSniffer v3.2/mb_gui.py — lines 241-280

```
        covers the Sniffer, Bus Health, Help and Bus Slave Finder pages without
        changing individual widget behavior or notebook keyboard navigation.
        """
        tag = "MBSniffer.MouseWheelGuard"

        for sequence in (
            "<MouseWheel>",
            "<Shift-MouseWheel>",
            "<Button-4>",
            "<Button-5>",
        ):
            try:
                self.bind_class(
                    tag,
                    sequence,
                    self._guard_mouse_wheel,
                )
            except tk.TclError:
                pass

        stack = [self]
        while stack:
            widget = stack.pop()
            try:
                tags = tuple(widget.bindtags())
                if tag not in tags:
                    widget.bindtags((tag,) + tags)
                stack.extend(widget.winfo_children())
            except tk.TclError:
                continue

    def _apply_classic_widget_palette(self, dark):
        """Apply palette to the small set of classic Tk widgets."""
        colors = theme_colors(self, dark)

        if hasattr(self, "raw_text"):
            self.raw_text.configure(
                background=colors["input"],
                foreground=colors["text"],
                insertbackground=colors["text"],
```

### MBSniffer v3.2/mb_gui.py — lines 281-320

```
                selectbackground=colors["select"],
                selectforeground=colors["select_text"],
            )

        if hasattr(self, "help_text"):
            self.help_text.configure(
                background=colors["input"],
                foreground=colors["text"],
                insertbackground=colors["text"],
                selectbackground=colors["select"],
                selectforeground=colors["select_text"],
            )

        for menu_name in ("slave_filter_menu", "traffic_context_menu", "discovery_context_menu"):
            menu = getattr(self, menu_name, None)
            if menu is None:
                continue
            try:
                menu.configure(
                    background=colors["panel"],
                    foreground=colors["text"],
                    activebackground=colors["select"],
                    activeforeground=colors["select_text"],
                    selectcolor=colors["text"],
                )
            except tk.TclError:
                pass

        if hasattr(self, "tree"):
            self.configure_anomaly_tags(dark)
            self.configure_inspector_result_styles()

        for name in (
            "refresh_a_btn",
            "refresh_b_btn",
            "discovery_refresh_btn",
            "dark_mode_switch",
        ):
            widget = getattr(self, name, None)
            setter = getattr(widget, "set_dark_mode", None)
```

**MBSniffer v3.2/mb_gui.py — lines 321-360**

```
            if callable(setter):
                setter(dark)

        if hasattr(self, "master_canvas"):
            self.master_apply_palette(dark)

    def configure_inspector_result_styles(self):
        """
        Match the Inspector Result foreground to the Traffic result palette.

        Only colour values are configured here; no padding/font/layout metrics
        are changed, so Light/Dark construction remains identical.
        """
        if not hasattr(self, "ui_style"):
            return

        colors = theme_colors(
            self,
            bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get()),
        )

        tag_map = {
            "success": "result_success",
            "slow": "anomaly_slow",
            "exception": "anomaly_exception",
            "crc": "anomaly_crc",
            "timeout": "anomaly_timeout",
            "raw": "anomaly_raw",
        }

        self.ui_style.configure(
            "MBS.Result.neutral.TLabel",
            background=colors["root"],
            foreground=colors["text"],
        )
        self.ui_style.configure(
            "MBS.Result.pending.TLabel",
            background=colors["root"],
            foreground=colors["text"],
        )
```

### MBSniffer v3.2/mb_gui.py — lines 361-400

```
361
362         if hasattr(self, "tree"):
363             for key, tag in tag_map.items():
364                 try:
365                     foreground = self.tree.tag_configure(tag)["foreground"]
366                 except Exception:
367                     foreground = colors["text"]
368
369                 self.ui_style.configure(
370                     f"MBS.Result.{key}.TLabel",
371                     background=colors["root"],
372                     foreground=foreground,
373                 )
374
375     def tr(self, text):
376         """Translate one canonical UI string to the currently selected language."""
377         return translate_text(text, self.current_language)
378
379     def trf(self, text, **values):
380         """Translate a format template and substitute values afterwards."""
381         return self.tr(text).format(**values)
382
383     def _capture_portuguese_help(self):
384         """Store the original PT-PT Help text and tag ranges exactly as built."""
385         if not hasattr(self, "help_text"):
386             return
387         try:
388             self._help_pt_text = self.help_text.get("1.0", "end-1c")
389             ranges = {}
390             for tag in self.help_text.tag_names():
391                 if tag == "sel":
392                     continue
393                 tag_ranges = list(self.help_text.tag_ranges(tag))
394                 ranges[tag] = [
395                     (str(tag_ranges[i]), str(tag_ranges[i + 1]))
396                     for i in range(0, len(tag_ranges), 2)
397                 ]
398             self._help_pt_tag_ranges = ranges
399         except tk.TclError:
400             self._help_pt_text = ""
```

### MBSniffer v3.2/mb_gui.py — lines 401-440

```
401             self._help_pt_tag_ranges = {}
402 
403     def _render_help_language(self):
404         """Render Help in English or restore the original PT-PT text byte-for-byte."""
405         if not hasattr(self, "help_text"):
406             return
407         try:
408             self.help_text.configure(state="normal")
409             self.help_text.delete("1.0", "end")
410 
411             if self.current_language == LANGUAGE_ENGLISH:
412                 max_ui_frames_text = f"{MAX_UI_FRAMES:_}".replace("_", " ")
413                 max_raw_lines_text = f"{MAX_RAW_TEXT_LINES:_}".replace("_", " ")
414                 for tag, text in english_help_blocks(
415                     max_ui_frames_text,
416                     max_raw_lines_text,
417                     MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
418                     MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
419                     debug_sim=(debug_sim == 1),
420                 ):
421                     self.help_text.insert("end", text, tag)
422             else:
423                 self.help_text.insert("1.0", getattr(self, "_help_pt_text", ""))
424                 for tag, ranges in getattr(self, "_help_pt_tag_ranges", {}).items():
425                     for start, end in ranges:
426                         self.help_text.tag_add(tag, start, end)
427 
428             self.help_text.configure(state="disabled")
429             self.help_text.see("1.0")
430         except tk.TclError:
431             pass
432 
433     def _translate_widget_tree(self):
434         """Translate static widget captions without changing widget geometry."""
435         stack = [self]
436         while stack:
437             widget = stack.pop()
438             try:
439                 stack.extend(widget.winfo_children())
440             except tk.TclError:
```

### MBSniffer v3.2/mb_gui.py — lines 441-480

```
441                 continue
442
443             # Text widgets contain documents, not captions.
444             if isinstance(widget, tk.Text):
445                 continue
446
447             try:
448                 keys = widget.keys()
449             except Exception:
450                 keys = ()
451             if "text" in keys:
452                 try:
453                     current = widget.cget("text")
454                     if current:
455                         widget.configure(text=self.tr(current))
456                 except (tk.TclError, TypeError):
457                     pass
458
459             if isinstance(widget, ttk.Notebook):
460                 try:
461                     for tab_id in widget.tabs():
462                         current = widget.tab(tab_id, "text")
463                         widget.tab(tab_id, text=self.tr(current))
464                 except tk.TclError:
465                     pass
466
467     def _translate_traffic_context_menu(self):
468         menu = getattr(self, "traffic_context_menu", None)
469         if menu is None:
470             return
471         labels = (
472             (0, "Copiar Raw Hex"),
473             (1, "Copiar frame descodificado"),
474             (3, "Filtrar por este Slave"),
475             (4, "Filtrar por este FC"),
476             (5, "Mostrar apenas esta transação"),
477             (6, "Limpar filtro de transação"),
478             (TRAFFIC_MENU_OPEN_IN_MASTER, "Abrir no Master"),
479         )
480         for index, canonical in labels:
```

### MBSniffer v3.2/mb_gui.py — lines 481-520

```
481             try:
482                 menu.entryconfigure(index, label=self.tr(canonical))
483             except tk.TclError:
484                 pass
485 
486     def _apply_table_language(self):
487         """Refresh headings and filter values that are not normal widget captions."""
488         if hasattr(self, "advanced_filter_type_combo"):
489             current = canonical_ui_text(self.advanced_filter_type_var.get())
490             canonical_values = (
491                 "Todos", "Requests", "Responses", "Exceptions",
492                 "CRC errors", "Timeouts", "RAW",
493             )
494             self.advanced_filter_type_combo.configure(
495                 values=tuple(self.tr(v) for v in canonical_values)
496             )
497             self.advanced_filter_type_var.set(self.tr(current))
498 
499         if hasattr(self, "time_order_var"):
500             canonical_order = canonical_ui_text(self.time_order_var.get())
501             self.time_order_var.set(self.tr(canonical_order))
502 
503         if hasattr(self, "tree") and hasattr(self, "tree_headings"):
504             canonical_headings = {
505                 "time": "Hora ↓" if self.time_order_descending() else "Hora ↑",
506                 "delta": "Δt (ms)",
507                 "response": "Resp. (ms)",
508                 "channel": "COM",
509                 "type": "Tipo",
510                 "slave": canonical_ui_text(self.tree_headings.get("slave", "Slave ▾")),
511                 "fc": "FC",
512                 "details": "Detalhes",
513                 "crc": "CRC",
514             }
515             for col, canonical in canonical_headings.items():
516                 translated = self.tr(canonical)
517                 self.tree_headings[col] = translated
518                 try:
519                     self.tree.heading(col, text=translated)
520                 except tk.TclError:
```

**MBSniffer v3.2/mb_gui.py — lines 521-560**

```
                    pass

        if hasattr(self, "discovery_tree"):
            canonical = {
                "slave": "Slave",
                "baud": "Baud",
                "config": "Config.",
                "fc": "FC",
                "result": "Resultado",
                "resp": "Resp. (ms)",
                "device_id": "Device Identification",
                "raw": "Raw Hex",
            }
            for col, value in canonical.items():
                try:
                    self.discovery_tree.heading(col, text=self.tr(value))
                except tk.TclError:
                    pass

    def apply_language(self, save=True):
        """Apply the selected language immediately to the complete UI."""
        selected = getattr(self, "language_var", None)
        if selected is not None:
            self.current_language = normalize_language(selected.get())
            if selected.get() != self.current_language:
                selected.set(self.current_language)

        self._translate_widget_tree()
        self._apply_table_language()
        self._translate_traffic_context_menu()
        self._render_help_language()

        # Rebuild dynamic captions/values in the selected language.
        try:
            self.update_mode_ui()
            self.update_com_status()
            self.update_stats_labels()
            self.rebuild_slave_filter_menu()
            self.update_slave_heading()
            self.update_frame_inspector(self.selected_record())
```

**MBSniffer v3.2/mb_gui.py — lines 561-600**

```
            self.update_discovery_estimate()
            self.master_apply_language()
        except (AttributeError, tk.TclError):
            pass

        # Translate idle/current status variables without altering state.
        for name in ("status_var", "discovery_status_var", "discovery_found_var"):
            var = getattr(self, name, None)
            if var is not None:
                try:
                    var.set(translate_runtime_text(var.get(), self.current_language))
                except tk.TclError:
                    pass

        # Refresh translated row contents without allowing the language change
        # itself to alter table/card geometry.  Column widths are restored after
        # rendering; normal viewport resizing and new traffic may still use the
        # existing responsive/autofit behaviour.
        traffic_widths = {}
        discovery_widths = {}
        try:
            if hasattr(self, "tree"):
                traffic_widths = {
                    col: int(self.tree.column(col, "width"))
                    for col in self.tree["columns"]
                }
            if hasattr(self, "discovery_tree"):
                discovery_widths = {
                    col: int(self.discovery_tree.column(col, "width"))
                    for col in self.discovery_tree["columns"]
                }
            self.render_filtered_view()
            for col, width in traffic_widths.items():
                self.tree.column(col, width=width)
            for col, width in discovery_widths.items():
                self.discovery_tree.column(col, width=width)
        except (AttributeError, tk.TclError):
            pass

        if save:
```

### MBSniffer v3.2/mb_gui.py — lines 601-640

```
601             self.save_current_ui_settings()
602 
603     def on_language_changed(self, _event=None):
604         self.apply_language(save=True)
605 
606     def toggle_dark_mode(self):
607         """
608         Switch Light/Dark colours only.
609 
610         No ttk theme replacement, layout change, padding change or row-height
611         change occurs here. The page construction is immutable after startup.
612         """
613         dark = bool(self.dark_mode_var.get())
614 
615         self.ui_style = apply_theme(
616             self,
617             dark=dark,
618             style=self.ui_style,
619         )
620         self._apply_classic_widget_palette(dark)
621 
622         # Remember Dark mode together with the rest of the user's preferences.
623         self.save_current_ui_settings()
624 
625     def _restore_saved_main_tab(self):
626         saved = self.ui_settings.get("last_main_tab", "Sniffer")
627         pages = {"Bus Slave Finder": self.discovery_page, "Master": self.master_page}
628         try:
629             self.main_notebook.select(pages.get(saved, self.sniffer_page))
630         except tk.TclError:
631             pass
632 
633     def _current_main_tab_name(self):
634         try:
635             selected = self.main_notebook.select()
636             if selected == str(self.discovery_page):
637                 return "Bus Slave Finder"
638             if selected == str(self.master_page):
639                 return "Master"
640             return "Sniffer"
```

**MBSniffer v3.2/mb_gui.py — lines 641-680**

```
641         except Exception:
642             return "Sniffer"
643 
644     def _schedule_ui_settings_save(self, *_args):
645         if getattr(self, "_closing", False):
646             return
647 
648         previous = getattr(self, "_settings_save_after_id", None)
649         if previous is not None:
650             try:
651                 self.after_cancel(previous)
652             except tk.TclError:
653                 pass
654 
655         try:
656             self._settings_save_after_id = self.after(
657                 300,
658                 self.save_current_ui_settings,
659             )
660         except tk.TclError:
661             self._settings_save_after_id = None
662 
663     def save_current_ui_settings(self):
664         """Persist all roaming-safe user preferences in one settings.json."""
665         pending = getattr(self, "_settings_save_after_id", None)
666         if pending is not None:
667             try:
668                 self.after_cancel(pending)
669             except tk.TclError:
670                 pass
671         self._settings_save_after_id = None
672 
673         try:
674             finder_bauds = [
675                 int(baud)
676                 for baud, var in self.discovery_baud_vars.items()
677                 if bool(var.get())
678             ]
679             finder_parities = [
680                 parity
```

**MBSniffer v3.2/mb_gui.py — lines 681-720**

```
                for parity, var in self.discovery_parity_vars.items()
                if bool(var.get())
            ]
            finder_stops = [
                stop
                for stop, var in self.discovery_stop_vars.items()
                if bool(var.get())
            ]

            payload = {
                "dark_mode": bool(self.dark_mode_var.get()),
                "language": normalize_language(self.current_language),
                "last_main_tab": self._current_main_tab_name(),
                "sniffer": {
                    "physical_mode": self.mode_var.get(),
                    "baud": self.baud_var.get(),
                    "data_bits": self.data_var.get(),
                    "parity": self.parity_var.get(),
                    "stop_bits": self.stop_var.get(),
                    "frame_gap_mode": self.gap_mode_var.get(),
                    "frame_gap_ms": self.gap_ms_var.get(),
                    "pending_timeout_s": self.pending_timeout_var.get(),
                    "auto_scroll": bool(self.auto_scroll_var.get()),
                    "separate_transactions": bool(
                        self.separate_transactions_var.get()
                    ),
                    "highlight_results": bool(
                        self.highlight_anomalies_var.get()
                    ),
                },
                "bus_slave_finder": {
                    "baud_rates": finder_bauds,
                    "parities": finder_parities,
                    "stop_bits": finder_stops,
                    "slave_start": self.discovery_slave_start_var.get(),
                    "slave_end": self.discovery_slave_end_var.get(),
                    "min_timeout_ms": self.discovery_min_timeout_var.get(),
                    "attempts": self.discovery_attempts_var.get(),
                    "fc04_fallback": bool(
                        self.discovery_fc04_fallback_var.get()
```

**MBSniffer v3.2/mb_gui.py — lines 721-760**

```
                    ),
                    "device_identification": bool(
                        self.discovery_device_identification_var.get()
                    ),
                },
                "master": {
                    "baud": self.master_baud_var.get(),
                    "parity": self.master_parity_var.get(),
                    "stop_bits": self.master_stop_var.get(),
                    "response_timeout_ms": self.master_timeout_var.get(),
                    "delay_ms": self.master_delay_var.get(),
                    "last_workspace": self.master_current_workspace,
                },
            }
            save_ui_settings(settings=payload)
        except Exception:
            # Persistence is convenience state only.
            pass

    def _install_ui_settings_persistence(self):
        """Debounced persistence for all roaming-safe user preferences."""
        self._settings_save_after_id = None

        variables = (
            self.mode_var,
            self.baud_var,
            self.data_var,
            self.parity_var,
            self.stop_var,
            self.gap_mode_var,
            self.gap_ms_var,
            self.pending_timeout_var,
            self.auto_scroll_var,
            self.separate_transactions_var,
            self.highlight_anomalies_var,
            self.discovery_slave_start_var,
            self.discovery_slave_end_var,
            self.discovery_min_timeout_var,
            self.discovery_fc04_fallback_var,
            self.discovery_device_identification_var,
```

### MBSniffer v3.2/mb_gui.py — lines 761-800

```
        )

        for variable in variables:
            variable.trace_add(
                "write",
                lambda *_a: self._schedule_ui_settings_save(),
            )

        for mapping in (
            self.discovery_baud_vars,
            self.discovery_parity_vars,
            self.discovery_stop_vars,
        ):
            for variable in mapping.values():
                variable.trace_add(
                    "write",
                    lambda *_a: self._schedule_ui_settings_save(),
                )

        self.main_notebook.bind(
            "<<NotebookTabChanged>>",
            lambda _e: self._schedule_ui_settings_save(),
            add="+",
        )

    def _schedule_bus_health_layout_sync(self, _event=None):
        """Debounce Bus Health reflow after notebook/window/Inspector resizing."""
        previous = getattr(self, "_bus_health_layout_after_id", None)
        if previous is not None:
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass

        try:
            self._bus_health_layout_after_id = self.after_idle(
                self._sync_bus_health_layout
            )
        except tk.TclError:
            self._bus_health_layout_after_id = None
```

### MBSniffer v3.2/mb_gui.py — lines 801-840

```
801
802     def _initialize_bus_health_layout(self):
803         """Measure the natural full Bus Health height before adaptive reflow."""
804         self._bus_health_layout_after_id = None
805
806         cards = getattr(self, "_health_cards", ())
807         if not cards:
808             return
809
810         try:
811             self.update_idletasks()
812         except tk.TclError:
813             return
814
815         if getattr(self, "_health_full_required_card_height", None) is None:
816             requested = [
817                 card.winfo_reqheight()
818                 for card in cards
819                 if card.winfo_exists()
820             ]
821             if requested:
822                 self._health_full_required_card_height = max(requested)
823
824         self._sync_bus_health_layout()
825
826     def _apply_bus_health_layout(self, compact):
827         """
828         Re-grid Bus Health metrics inside equal vertical bands.
829
830         Full mode uses one metric/value pair per row.  Compact mode uses two
831         pairs per row.  In both modes every used row receives the same weight,
832         so the metrics are distributed evenly over the card height instead of
833         accumulating at the top.  Compact columns use proportional uniform
834         widths (label/value + label/value), which keeps both halves aligned and
835         prevents one metric group from consuming the neighbouring group.
836         """
837         groups = getattr(self, "_health_metric_groups", ())
838         if not groups:
839             return
840
```

### MBSniffer v3.2/mb_gui.py — lines 841-880

```
841         for card, pairs in groups:
842             for label_widget, value_widget in pairs:
843                 label_widget.grid_forget()
844                 value_widget.grid_forget()
845 
846             # Clear weights left by the other layout before applying the new
847             # one.  Uniform groups are local to each LabelFrame.
848             for column in range(4):
849                 card.columnconfigure(column, weight=0, minsize=0, uniform="")
850             for row in range(max(6, len(pairs))):
851                 card.rowconfigure(row, weight=0, minsize=0, uniform="")
852 
853             if compact:
854                 # Size the four compact columns from their real content.
855                 # The two label columns absorb any spare width equally; value
856                 # columns keep their natural size.  This avoids the clipping
857                 # caused by forcing every card into one fixed percentage split.
858                 for column in (0, 2):
859                     card.columnconfigure(column, weight=1, minsize=0, uniform="")
860                 for column in (1, 3):
861                     card.columnconfigure(column, weight=0, minsize=0, uniform="")
862 
863                 used_rows = max(1, (len(pairs) + 1) // 2)
864                 for row in range(used_rows):
865                     card.rowconfigure(
866                         row,
867                         weight=1,
868                         minsize=0,
869                         uniform="health_metric_rows",
870                     )
871 
872                 pair_count = len(pairs)
873                 card_width = max(320, int(card.winfo_width()))
874                 # At normal widths the technical labels/values remain on one
875                 # line.  Near the application's minimum width labels may wrap
876                 # so the pair stays inside the LabelFrame instead of colliding
877                 # with the neighbouring pair.
878                 compact_label_wrap = (
879                     0 if card_width >= 390 else max(80, int(card_width * 0.29))
880                 )
```

**MBSniffer v3.2/mb_gui.py — lines 881-920**

```
881
882                 for index, (label_widget, value_widget) in enumerate(pairs):
883                     row = index // 2
884                     group = index % 2
885
886                     try:
887                         label_widget.configure(wraplength=compact_label_wrap)
888                         value_widget.configure(wraplength=0)
889                     except tk.TclError:
890                         pass
891
892                     # Odd final metrics use the complete last row, leaving the
893                     # value the whole right-hand side of the card.
894                     if index == pair_count - 1 and pair_count % 2:
895                         label_widget.grid(
896                             row=row,
897                             column=0,
898                             sticky="w",
899                             padx=(6, 3),
900                             pady=1,
901                         )
902                         value_widget.grid(
903                             row=row,
904                             column=1,
905                             columnspan=3,
906                             sticky="e",
907                             padx=(3, 6),
908                             pady=1,
909                         )
910                         continue
911
912                     base_column = group * 2
913                     label_widget.grid(
914                         row=row,
915                         column=base_column,
916                         sticky="w",
917                         padx=((6 if base_column == 0 else 5), 3),
918                         pady=1,
919                     )
920                     value_widget.grid(
```

**MBSniffer v3.2/mb_gui.py — lines 921-960**

```
                        row=row,
                        column=base_column + 1,
                        sticky="e",
                        padx=(3, (5 if base_column == 0 else 6)),
                        pady=1,
                    )
            else:
                # One pair per row, with every row sharing the card height
                # equally.  Proportional columns keep labels and values aligned
                # while still allowing long values to use the right-hand side.
                card.columnconfigure(0, weight=1, minsize=0, uniform="")
                card.columnconfigure(1, weight=0, minsize=0, uniform="")

                used_rows = max(1, len(pairs))
                for row in range(used_rows):
                    card.rowconfigure(
                        row,
                        weight=1,
                        minsize=0,
                        uniform="health_metric_rows",
                    )

                card_width = max(320, int(card.winfo_width()))
                full_label_wrap = (
                    0 if card_width >= 330 else max(100, int(card_width * 0.55))
                )

                for row, (label_widget, value_widget) in enumerate(pairs):
                    try:
                        label_widget.configure(wraplength=full_label_wrap)
                        value_widget.configure(wraplength=0)
                    except tk.TclError:
                        pass
                    label_widget.grid(
                        row=row,
                        column=0,
                        sticky="w",
                        padx=(10, 6),
                        pady=1,
                    )
```

### MBSniffer v3.2/mb_gui.py — lines 961-1000

```
                    value_widget.grid(
                        row=row,
                        column=1,
                        sticky="e",
                        padx=(6, 10),
                        pady=1,
                    )

        self._health_layout_compact = bool(compact)

    def _sync_bus_health_layout(self):
        """
        Keep Bus Health metrics inside their rectangles at every available height.

        The natural/full card height is measured from the real widgets. If the
        Frame Inspector expands and the notebook becomes too short, the metrics
        reflow automatically. When enough vertical room returns, the original
        layout is restored. Hysteresis prevents resize oscillation.
        """
        self._bus_health_layout_after_id = None

        health_frame = getattr(self, "health_frame", None)
        cards = getattr(self, "_health_cards", ())
        full_required = getattr(
            self,
            "_health_full_required_card_height",
            None,
        )
        if health_frame is None or not cards:
            return

        try:
            if not health_frame.winfo_exists():
                return

            available_frame_height = health_frame.winfo_height()
            if available_frame_height <= 1:
                return

            # Each card has 6 px external top and bottom grid padding.
```

### MBSniffer v3.2/mb_gui.py — lines 1001-1040

```
            available_card_height = max(0, available_frame_height - 12)

            if full_required is None:
                self.update_idletasks()
                full_required = max(card.winfo_reqheight() for card in cards)
                self._health_full_required_card_height = full_required

            compact = bool(getattr(self, "_health_layout_compact", False))
            if compact:
                should_compact = available_card_height < (full_required + 8)
            else:
                should_compact = available_card_height < (full_required + 2)

            if should_compact != compact:
                self._apply_bus_health_layout(should_compact)
                self.update_idletasks()
            else:
                # Width can change without crossing the vertical compact/full
                # threshold. Reapply the current layout so wrap limits and the
                # equal-width grid continue to match the real card size.
                self._apply_bus_health_layout(compact)

        except tk.TclError:
            return

    def _build_ui(self):
        sniffer_saved = self.ui_settings.get("sniffer", {})
        if not isinstance(sniffer_saved, dict):
            sniffer_saved = {}

        self.columnconfigure(0, weight=1)
        self.rowconfigure(0, weight=1)

        # Refresh controls use a custom Canvas widget whose arrow geometry is
        # centered explicitly; this avoids ttk theme/font/image alignment drift.

        self.main_notebook = ttk.Notebook(self)
        self.main_notebook.grid(row=0, column=0, sticky="nsew")

        self.sniffer_page = ttk.Frame(self.main_notebook)
```

### MBSniffer v3.2/mb_gui.py — lines 1041-1080

```
        self.sniffer_page.columnconfigure(0, weight=1)
        self.sniffer_page.rowconfigure(1, weight=1)
        self.main_notebook.add(self.sniffer_page, text="Sniffer")

        self.discovery_page = ttk.Frame(self.main_notebook)
        self.discovery_page.columnconfigure(0, weight=1)
        self.discovery_page.rowconfigure(3, weight=1)
        self.main_notebook.add(self.discovery_page, text="Bus Slave Finder")

        self.master_page = ttk.Frame(self.main_notebook)
        self.main_notebook.add(self.master_page, text="Master")

        # Global Light/Dark control: root-level overlay in the top-right corner.
        # It is outside both tabs and therefore applies equally to Sniffer and
        # Bus Slave Finder. `place()` means it consumes no layout height.
        self.theme_overlay = ttk.Frame(self)
        self.theme_overlay.place(
            relx=1.0,
            x=-12,
            y=4,
            anchor="ne",
        )

        ttk.Label(
            self.theme_overlay,
            text="Modo escuro",
        ).grid(row=0, column=0, sticky="e", padx=(0, 6))

        self.dark_mode_var = tk.BooleanVar(value=self.initial_dark_mode)
        self.dark_mode_switch = ToggleSwitch(
            self.theme_overlay,
            variable=self.dark_mode_var,
            command=self.toggle_dark_mode,
            width=42,
            height=22,
        )
        self.dark_mode_switch.grid(row=0, column=1, sticky="e")

        self.language_label = ttk.Label(
            self.theme_overlay,
```

**MBSniffer v3.2/mb_gui.py — lines 1081-1120**

```
            text="Linguagem",
        )
        self.language_label.grid(
            row=0, column=2, sticky="e", padx=(18, 6)
        )
        self.language_var = tk.StringVar(value=self.current_language)
        self.language_combo = ttk.Combobox(
            self.theme_overlay,
            textvariable=self.language_var,
            values=LANGUAGE_CHOICES,
            state="readonly",
            width=10,
        )
        self.language_combo.grid(row=0, column=3, sticky="e")
        self.language_combo.bind(
            "<<ComboboxSelected>>", self.on_language_changed
        )
        self.theme_overlay.lift()

        # ================================================================
        # Compact top workspace: configuration on the left, controls/stats
        # on the right.  This reclaims vertical space for the traffic view.
        # ================================================================
        top_area = ttk.Frame(self.sniffer_page)
        top_area.grid(row=0, column=0, sticky="ew", padx=10, pady=(10, 5))
        # Keep the 3:2 workspace split independent of translated text widths.
        # Without a shared uniform group Tk first honours each child widget's
        # requested width, so changing language moves the two main rectangles.
        top_area.columnconfigure(0, weight=3, uniform="top_workspace")
        top_area.columnconfigure(1, weight=2, uniform="top_workspace")
        # The left Configuração card defines the natural top-workspace height.
        # The right stack stretches to exactly the same top/bottom limits.
        top_area.rowconfigure(0, weight=1)

        cfg = ttk.LabelFrame(top_area, text="Configuração")
        cfg.grid(row=0, column=0, sticky="nsew", padx=(0, 6))
        cfg.columnconfigure(0, weight=1)

        # Four compact equal columns.  Widgets keep the same fixed widths as
        # v2.6; only the card itself is narrower, eliminating wasted whitespace.
```

**MBSniffer v3.2/mb_gui.py — lines 1121-1160**

```
1121        config_grid = ttk.Frame(cfg)
1122        config_grid.grid(row=0, column=0, sticky="ew", padx=10, pady=(8, 4))
1123        for col in range(4):
1124            config_grid.columnconfigure(col, weight=1, uniform="sniffer_cfg")
1125
1126        def config_cell(row, column):
1127            cell = ttk.Frame(config_grid)
1128            cell.grid(
1129                row=row, column=column, sticky="w",
1130                padx=(0 if column == 0 else 8, 8 if column < 3 else 0),
1131                pady=(0, 7),
1132            )
1133            return cell
1134
1135        # Row 1 — physical mode / COM A / COM B / capture status
1136        cell = config_cell(0, 0)
1137        ttk.Label(cell, text="Modo físico").grid(row=0, column=0, sticky="w", pady=(0, 2))
1138        self.mode_var = tk.StringVar(value=sniffer_saved.get("physical_mode", "RS485 2-wire"))
1139        self.mode_combo = ttk.Combobox(
1140            cell, textvariable=self.mode_var, width=13, state="readonly",
1141            values=["RS485 2-wire", "RS232 single RX", "RS232 dual RX"],
1142        )
1143        self.mode_combo.grid(row=1, column=0, sticky="w")
1144        self.mode_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_mode_ui())
1145
1146        cell = config_cell(0, 1)
1147        ttk.Label(cell, text="COM A").grid(row=0, column=0, sticky="w", pady=(0, 2))
1148        com_a_controls = ttk.Frame(cell)
1149        com_a_controls.grid(row=1, column=0, sticky="w")
1150        self.port_a_var = tk.StringVar()
1151        self.port_a_combo = ttk.Combobox(
1152            com_a_controls, textvariable=self.port_a_var, width=13, state="readonly"
1153        )
1154        self.port_a_combo.grid(row=0, column=0, sticky="w")
1155        self.port_a_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_com_status())
1156        self.refresh_a_btn = RefreshButton(com_a_controls, command=self.refresh_ports, size=24)
1157        self.refresh_a_btn.grid(row=0, column=1, sticky="w", padx=(4, 0))
1158
1159        def _sync_sniffer_refresh_a_square(_event=None):
1160            height = self.port_a_combo.winfo_height()
```

### MBSniffer v3.2/mb_gui.py — lines 1161-1200

```
            if height > 1:
                self.refresh_a_btn.configure(width=height, height=height)

        self.port_a_combo.bind("<Configure>", _sync_sniffer_refresh_a_square, add="+")

        cell = config_cell(0, 2)
        self.port_b_label = ttk.Label(cell, text="COM B")
        self.port_b_label.grid(row=0, column=0, sticky="w", pady=(0, 2))
        com_b_controls = ttk.Frame(cell)
        com_b_controls.grid(row=1, column=0, sticky="w")
        self.port_b_var = tk.StringVar()
        self.port_b_combo = ttk.Combobox(
            com_b_controls, textvariable=self.port_b_var, width=13, state="readonly"
        )
        self.port_b_combo.grid(row=0, column=0, sticky="w")
        self.port_b_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_com_status())
        self.refresh_b_btn = RefreshButton(com_b_controls, command=self.refresh_ports, size=24)
        self.refresh_b_btn.grid(row=0, column=1, sticky="w", padx=(4, 0))

        def _sync_sniffer_refresh_b_square(_event=None):
            height = self.port_b_combo.winfo_height()
            if height > 1:
                self.refresh_b_btn.configure(width=height, height=height)

        self.port_b_combo.bind("<Configure>", _sync_sniffer_refresh_b_square, add="+")

        cell = config_cell(0, 3)
        ttk.Label(cell, text="Estado").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.status_var = tk.StringVar(value="● Parado")
        ttk.Label(
            cell,
            textvariable=self.status_var,
            style="MBS.Status.TLabel",
            width=22,
            anchor="w",
        ).grid(
            row=1, column=0, sticky="w"
        )

        # Row 2 — serial format
```

**MBSniffer v3.2/mb_gui.py — lines 1201-1240**

```
        cell = config_cell(1, 0)
        ttk.Label(cell, text="Baud rate").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.baud_var = tk.StringVar(value=sniffer_saved.get("baud", "9600"))
        ttk.Combobox(
            cell, textvariable=self.baud_var, width=13,
            values=["1200", "2400", "4800", "9600", "19200", "38400", "57600", "115200"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 1)
        ttk.Label(cell, text="Data bits").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.data_var = tk.StringVar(value=sniffer_saved.get("data_bits", "8"))
        ttk.Combobox(
            cell, textvariable=self.data_var, width=13, state="readonly",
            values=["5", "6", "7", "8"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 2)
        ttk.Label(cell, text="Parity").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.parity_var = tk.StringVar(value=sniffer_saved.get("parity", "None"))
        ttk.Combobox(
            cell, textvariable=self.parity_var, width=13, state="readonly",
            values=["None", "Even", "Odd", "Mark", "Space"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 3)
        ttk.Label(cell, text="Stop bits").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.stop_var = tk.StringVar(value=sniffer_saved.get("stop_bits", "1"))
        ttk.Combobox(
            cell, textvariable=self.stop_var, width=13, state="readonly",
            values=["1", "1.5", "2"],
        ).grid(row=1, column=0, sticky="w")

        # Row 3 — timing and display options
        cell = config_cell(2, 0)
        ttk.Label(cell, text="Frame gap").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.gap_mode_var = tk.StringVar(value=sniffer_saved.get("frame_gap_mode", "Auto"))
        self.gap_combo = ttk.Combobox(
            cell, textvariable=self.gap_mode_var, width=13, state="readonly",
            values=["Auto", "Manual"],
        )
```

### MBSniffer v3.2/mb_gui.py — lines 1241-1280

```
        self.gap_combo.grid(row=1, column=0, sticky="w")
        self.gap_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_gap_state())

        cell = config_cell(2, 1)
        ttk.Label(cell, text="Gap manual (ms)").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.gap_ms_var = tk.StringVar(value=sniffer_saved.get("frame_gap_ms", ""))
        self.gap_entry = ttk.Entry(cell, textvariable=self.gap_ms_var, width=15, state="disabled")
        self.gap_entry.grid(row=1, column=0, sticky="w")

        cell = config_cell(2, 2)
        ttk.Label(cell, text="Visualização").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.auto_scroll_var = tk.BooleanVar(value=sniffer_saved.get("auto_scroll", True))
        ttk.Checkbutton(cell, text="Auto-scroll", variable=self.auto_scroll_var).grid(
            row=1, column=0, sticky="w"
        )

        cell = config_cell(2, 3)
        ttk.Label(cell, text="Pending timeout (s)").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.pending_timeout_var = tk.StringVar(value=sniffer_saved.get("pending_timeout_s", f"{DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS:g}"))
        # Direct numeric entry is cleaner here than tiny spin arrows. Range
        # validation still happens in get_pending_timeout_seconds().
        self.pending_timeout_entry = ttk.Entry(
            cell,
            textvariable=self.pending_timeout_var,
            width=13,
        )
        self.pending_timeout_entry.grid(row=1, column=0, sticky="w")
        ttk.Label(
            cell,
            text=(
                f"min {MIN_PENDING_REQUEST_TIMEOUT_SECONDS:g} / "
                f"default {DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS:g} / "
                f"max {MAX_PENDING_REQUEST_TIMEOUT_SECONDS:g}"
            ),
        ).grid(row=2, column=0, sticky="w", pady=(2, 0))

        # COM status + mode help: compact footer inside the configuration card.
        com_status = ttk.Frame(cfg)
        com_status.grid(row=1, column=0, sticky="w", padx=10, pady=(0, 3))
        ttk.Label(com_status, text="Estado COM").grid(
```

### MBSniffer v3.2/mb_gui.py — lines 1281-1320

```
            row=0, column=0, rowspan=2, sticky="nw", padx=(0, 14)
        )
        self.port_a_status_var = tk.StringVar(value="COM A - —")
        self.port_b_status_var = tk.StringVar(value="COM B - —")
        ttk.Label(com_status, textvariable=self.port_a_status_var).grid(row=0, column=1, sticky="w")
        ttk.Label(com_status, textvariable=self.port_b_status_var).grid(
            row=1, column=1, sticky="w", pady=(1, 0)
        )

        self.mode_help_var = tk.StringVar()
        ttk.Label(cfg, textvariable=self.mode_help_var).grid(
            row=2, column=0, sticky="w", padx=10, pady=(2, 7)
        )

        # ------------------------------------------------------------------
        # Right-hand command/status panel.
        # ------------------------------------------------------------------
        side_panel = ttk.Frame(top_area)
        side_panel.grid(row=0, column=1, sticky="nsew", padx=(6, 0))
        side_panel.columnconfigure(0, weight=1)
        side_panel.rowconfigure(0, weight=0)
        side_panel.rowconfigure(1, weight=1)

        actions = ttk.LabelFrame(side_panel, text="Controlos")
        # Same top coordinate as Configuração; only the gap below Controlos
        # separates it from Estatísticas.
        actions.grid(row=0, column=0, sticky="ew", pady=(0, 6))
        actions.columnconfigure(0, weight=1, uniform="action_columns")
        actions.columnconfigure(1, weight=1, uniform="action_columns")

        self.start_btn = ttk.Button(
            actions, text="Iniciar Captura", command=self.start_capture,
            style="MBS.Primary.TButton"
        )
        self.start_btn.grid(row=0, column=0, sticky="ew", padx=(8, 4), pady=(7, 3))

        self.stop_btn = ttk.Button(
            actions, text="Parar", command=self.stop_capture, state="disabled",
            style="MBS.Toolbar.TButton"
        )
```

### MBSniffer v3.2/mb_gui.py — lines 1321-1360

```
1321        self.stop_btn.grid(row=0, column=1, sticky="ew", padx=(4, 8), pady=(7, 3))
1322
1323        self.restart_btn = ttk.Button(
1324            actions, text="Reiniciar COM", command=self.restart_com_port,
1325            style="MBS.Toolbar.TButton"
1326        )
1327        self.restart_btn.grid(row=1, column=0, sticky="ew", padx=(8, 4), pady=3)
1328
1329        ttk.Button(
1330            actions, text="Limpar", command=self.clear_view,
1331            style="MBS.Toolbar.TButton"
1332        ).grid(row=1, column=1, sticky="ew", padx=(4, 8), pady=3)
1333
1334        ttk.Button(
1335            actions, text="Abrir pasta de logs", command=self.open_log_folder,
1336            style="MBS.Toolbar.TButton"
1337        ).grid(row=2, column=0, columnspan=2, sticky="ew", padx=8, pady=3)
1338
1339        self.sim_btn = None
1340        if debug_sim == 1:
1341            self.sim_btn = ttk.Button(
1342                actions, text="Simulação", command=self.run_simulation,
1343                style="MBS.Toolbar.TButton"
1344            )
1345            self.sim_btn.grid(row=3, column=0, columnspan=2, sticky="ew", padx=8, pady=3)
1346            options_row = 4
1347        else:
1348            options_row = 3
1349
1350        options_bar = ttk.Frame(actions)
1351        options_bar.grid(
1352            row=options_row,
1353            column=0,
1354            columnspan=2,
1355            sticky="ew",
1356            padx=8,
1357            pady=(3, 7),
1358        )
1359        options_bar.columnconfigure(2, weight=1)
1360
```

**MBSniffer v3.2/mb_gui.py — lines 1361-1400**

```
        self.separate_transactions_var = tk.BooleanVar(
            value=sniffer_saved.get("separate_transactions", True)
        )
        ttk.Checkbutton(
            options_bar,
            text="Separar transações",
            variable=self.separate_transactions_var,
            command=self.render_filtered_view,
        ).grid(row=0, column=0, sticky="w", padx=(0, 12))

        self.highlight_anomalies_var = tk.BooleanVar(
            value=sniffer_saved.get("highlight_results", False)
        )
        ttk.Checkbutton(
            options_bar,
            text="Realçar resultados",
            variable=self.highlight_anomalies_var,
            command=self.on_highlight_anomalies_changed,
        ).grid(row=0, column=1, sticky="w")

        self.bus_activity_var = tk.StringVar(value="BUS ○")
        ttk.Label(
            options_bar,
            textvariable=self.bus_activity_var,
            style="MBS.Status.TLabel",
            anchor="e",
            width=9,
        ).grid(row=0, column=2, sticky="e")

        stats = ttk.LabelFrame(side_panel, text="Estatísticas da sessão")
        # Expand vertically into all remaining space so the lower border always
        # lines up with the lower border of Configuração, even if Configuração
        # changes height in the future.
        stats.grid(row=1, column=0, sticky="nsew")
        for col in range(4):
            stats.columnconfigure(col, weight=1, uniform="stats")
        # Keep the eight counters centred in a fixed 4 x 2 matrix.  The
        # labels must not hug the LabelFrame border or shift with translation.
        for row in range(2):
            stats.rowconfigure(row, weight=1, uniform="stats_rows")
```

### MBSniffer v3.2/mb_gui.py — lines 1401-1440

```
1401
1402        self.stat_requests_var = tk.StringVar(value="Pedidos: 0")
1403        self.stat_responses_var = tk.StringVar(value="Respostas: 0")
1404        self.stat_pending_var = tk.StringVar(value="Pendentes: 0")
1405        self.stat_timeouts_var = tk.StringVar(value="Timeouts: 0")
1406        self.stat_crc_var = tk.StringVar(value="Erros de CRC: 0")
1407        self.stat_exceptions_var = tk.StringVar(value="Exceções: 0")
1408        self.stat_raw_var = tk.StringVar(value="RAW: 0")
1409        self.stat_slaves_var = tk.StringVar(value="Slaves: 0")
1410
1411        stat_vars = [
1412            self.stat_requests_var,
1413            self.stat_responses_var,
1414            self.stat_pending_var,
1415            self.stat_timeouts_var,
1416            self.stat_crc_var,
1417            self.stat_exceptions_var,
1418            self.stat_raw_var,
1419            self.stat_slaves_var,
1420        ]
1421        for idx, var in enumerate(stat_vars):
1422            ttk.Label(
1423                stats,
1424                textvariable=var,
1425                style="MBS.Stat.TLabel",
1426                anchor="center",
1427            ).grid(
1428                row=idx // 4,
1429                column=idx % 4,
1430                sticky="",
1431                padx=6,
1432                pady=3,
1433            )
1434
1435        # Notebook
1436        notebook = ttk.Notebook(self.sniffer_page)
1437        notebook.grid(row=1, column=0, sticky="nsew", padx=10, pady=(5, 8))
1438
1439        parsed = ttk.Frame(notebook)
1440        parsed.columnconfigure(0, weight=1)
```

**MBSniffer v3.2/mb_gui.py — lines 1441-1480**

```
        parsed.rowconfigure(1, weight=1)
        notebook.add(parsed, text="Tráfego")

        filter_bar = ttk.Frame(parsed)
        filter_bar.grid(
            row=0,
            column=0,
            columnspan=2,
            sticky="ew",
            padx=4,
            pady=(4, 3),
        )
        filter_bar.columnconfigure(8, weight=1)
        # "Pesquisa" is wider than "Search". Keep its canonical PT cell width
        # reserved so the search Entry never moves when language changes.
        filter_bar.columnconfigure(7, minsize=65)

        ttk.Label(filter_bar, text="Filtros").grid(
            row=0, column=0, sticky="w", padx=(2, 6)
        )

        self.advanced_filter_type_var = tk.StringVar(value="Todos")
        type_filter = ttk.Combobox(
            filter_bar,
            textvariable=self.advanced_filter_type_var,
            state="readonly",
            width=11,
            values=(
                "Todos",
                "Requests",
                "Responses",
                "Exceptions",
                "CRC errors",
                "Timeouts",
                "RAW",
            ),
        )
        type_filter.grid(row=0, column=1, sticky="w", padx=(0, 8))
        self.advanced_filter_type_combo = type_filter
        type_filter.bind(
```

### MBSniffer v3.2/mb_gui.py — lines 1481-1520

```
            "<<ComboboxSelected>>",
            lambda _e: self.render_filtered_view(),
        )

        ttk.Label(filter_bar, text="FC (hex)").grid(
            row=0, column=2, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_fc_var = tk.StringVar(value="")
        fc_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_fc_var,
            width=7,
        )
        fc_filter.grid(row=0, column=3, sticky="w", padx=(0, 8))

        ttk.Label(filter_bar, text="Resp. >").grid(
            row=0, column=4, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_response_ms_var = tk.StringVar(value="")
        response_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_response_ms_var,
            width=7,
        )
        response_filter.grid(row=0, column=5, sticky="w")
        ttk.Label(filter_bar, text="ms").grid(
            row=0, column=6, sticky="w", padx=(3, 8)
        )

        ttk.Label(filter_bar, text="Pesquisa").grid(
            row=0, column=7, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_text_var = tk.StringVar(value="")
        search_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_text_var,
            width=22,
        )
        search_filter.grid(row=0, column=8, sticky="ew")

```

**MBSniffer v3.2/mb_gui.py — lines 1521-1560**

```
        filter_actions = ttk.Frame(filter_bar)
        filter_actions.grid(row=0, column=9, sticky="e")
        ttk.Button(
            filter_actions,
            text="Limpar filtros",
            command=self.clear_advanced_filters,
            style="MBS.Compact.TButton",
        ).grid(row=0, column=0, padx=(6, 4))
        ttk.Button(
            filter_actions,
            text="Exportar CSV",
            command=self.export_session_csv,
            style="MBS.Compact.TButton",
        ).grid(row=0, column=1)

        for variable in (
            self.advanced_filter_fc_var,
            self.advanced_filter_response_ms_var,
            self.advanced_filter_text_var,
        ):
            variable.trace_add(
                "write",
                self.schedule_advanced_filter_refresh,
            )

        cols = ("time", "delta", "response", "channel", "type", "slave", "fc", "details", "crc")
        # Dedicated style so wrapped multi-line rows do not affect unrelated
        # widgets. Treeview has one row height for the whole table, therefore
        # the row height is adjusted to the largest wrapped Details cell that
        # is currently visible.
        self.tree_style_name = "MBSniffer.Treeview"
        self.tree_style = ttk.Style(self)
        self.tree = ttk.Treeview(
            parsed,
            columns=cols,
            show="headings",
            selectmode="browse",
            style=self.tree_style_name
        )

```

**MBSniffer v3.2/mb_gui.py — lines 1561-1600**

```
        self._tree_reflow_after_id = None
        self.tree_detail_min_width_px = 220
        self.tree_detail_border_reserve_px = 18

        # Internal sort state. The user changes it only by clicking Hora.
        self.time_order_var = tk.StringVar(
            value="Crescente — antigo → recente"
        )

        self.tree_headings = {
            "time": "Hora ↑",
            "delta": "Δt (ms)",
            "response": "Resp. (ms)",
            "channel": "COM",
            "type": "Tipo",
            "slave": "Slave ▾",
            "fc": "FC",
            "details": "Detalhes",
            "crc": "CRC",
        }

        # Keep columns as compact as possible while leaving breathing room
        # between the longest text and the column borders.
        self.tree_column_padding_px = 28  # total horizontal padding
        self.tree_cell_font = tkfont.nametofont("TkDefaultFont")
        try:
            heading_font_name = ttk.Style(self).lookup("Treeview.Heading", "font")
            self.tree_heading_font = (
                tkfont.nametofont(heading_font_name)
                if heading_font_name
                else self.tree_cell_font
            )
        except Exception:
            self.tree_heading_font = self.tree_cell_font

        self.tree_font_linespace = max(
            1,
            int(self.tree_cell_font.metrics("linespace"))
        )
        self.tree_single_line_rowheight = max(
```

### MBSniffer v3.2/mb_gui.py — lines 1601-1640

```
            20,
            self.tree_font_linespace + 5
        )
        self.tree_style.configure(
            self.tree_style_name,
            rowheight=self.tree_single_line_rowheight
        )

        for col in cols:
            align = "w" if col == "details" else "center"
            self.tree.heading(
                col,
                text=self.tree_headings[col],
                anchor=align
            )
            initial_width = self.measure_tree_column_width(
                col,
                [],
                heading_text=self.tree_headings[col]
            )
            self.tree.column(
                col,
                width=initial_width,
                minwidth=initial_width,
                anchor=align,
                stretch=(col == "details")
            )

        self.tree.heading(
            "time",
            text=self.tree_headings["time"],
            anchor="center",
            command=self.toggle_time_order
        )
        self.tree.heading(
            "slave",
            text=self.tree_headings["slave"],
            anchor="center",
            command=self.show_slave_filter_menu
        )
```

**MBSniffer v3.2/mb_gui.py — lines 1641-1680**

```
1641
1642        # Popup menu opened directly from the Slave column heading.
1643        self.slave_filter_menu = tk.Menu(self, tearoff=False)
1644
1645        # Right-click actions for the selected traffic frame.
1646        self.traffic_context_menu = tk.Menu(self, tearoff=False)
1647        self.traffic_context_menu.add_command(
1648            label="Copiar Raw Hex",
1649            command=self.copy_selected_raw,
1650        )
1651        self.traffic_context_menu.add_command(
1652            label="Copiar frame descodificado",
1653            command=self.copy_selected_decoded,
1654        )
1655        self.traffic_context_menu.add_separator()
1656        self.traffic_context_menu.add_command(
1657            label="Filtrar por este Slave",
1658            command=self.context_filter_selected_slave,
1659        )
1660        self.traffic_context_menu.add_command(
1661            label="Filtrar por este FC",
1662            command=self.context_filter_selected_fc,
1663        )
1664        self.traffic_context_menu.add_command(
1665            label="Mostrar apenas esta transação",
1666            command=self.context_show_selected_transaction,
1667        )
1668        self.traffic_context_menu.add_command(
1669            label="Limpar filtro de transação",
1670            command=self.clear_transaction_filter,
1671        )
1672        self.traffic_context_menu.add_separator()
1673        self.traffic_context_menu.add_command(
1674            label="Abrir no Master",
1675            command=self.context_open_selected_in_master,
1676        )
1677
1678        ys = ttk.Scrollbar(parsed, orient="vertical", command=self.tree.yview)
1679        self.tree.configure(yscrollcommand=ys.set)
1680        self.tree.grid(row=1, column=0, sticky="nsew")
```

### MBSniffer v3.2/mb_gui.py — lines 1681-1720

```
        ys.grid(row=1, column=1, sticky="ns")
        self.tree.bind("<<TreeviewSelect>>", self.on_row_select)
        self.tree.bind("<Button-3>", self.show_traffic_context_menu, add="+")
        self.tree.bind("<Configure>", self.schedule_tree_reflow)
        self.configure_anomaly_tags(self.initial_dark_mode)

        raw = ttk.Frame(notebook)
        raw.columnconfigure(0, weight=1)
        raw.rowconfigure(0, weight=1)
        notebook.add(raw, text="Raw Hex / Log")

        self.raw_text = tk.Text(
            raw,
            wrap="word",
            font=("Consolas", 10)
        )
        self.raw_text_font = tkfont.Font(font=self.raw_text.cget("font"))
        self.raw_text.bind("<Configure>", self.schedule_raw_separator_reflow)

        raw_y = ttk.Scrollbar(raw, orient="vertical", command=self.raw_text.yview)
        self.raw_text.configure(yscrollcommand=raw_y.set)
        self.raw_text.grid(row=0, column=0, sticky="nsew")
        raw_y.grid(row=0, column=1, sticky="ns")

        # Bus Health — session-level diagnostic metrics.
        self.health_frame = ttk.Frame(notebook)
        health_frame = self.health_frame
        health_frame.columnconfigure(0, weight=1, uniform="health")
        health_frame.columnconfigure(1, weight=1, uniform="health")
        health_frame.columnconfigure(2, weight=1, uniform="health")
        health_frame.rowconfigure(0, weight=1)
        notebook.add(health_frame, text="Bus Health")

        traffic_health = ttk.LabelFrame(
            health_frame,
            text="Tráfego",
        )
        traffic_health.grid(
            row=0, column=0, sticky="nsew", padx=(6, 3), pady=6
        )
```

**MBSniffer v3.2/mb_gui.py — lines 1721-1760**

```
        traffic_health.columnconfigure(1, weight=1)

        response_health = ttk.LabelFrame(
            health_frame,
            text="Tempos de resposta",
        )
        response_health.grid(
            row=0, column=1, sticky="nsew", padx=3, pady=6
        )
        response_health.columnconfigure(1, weight=1)

        quality_health = ttk.LabelFrame(
            health_frame,
            text="Qualidade",
        )
        quality_health.grid(
            row=0, column=2, sticky="nsew", padx=(3, 6), pady=6
        )
        quality_health.columnconfigure(1, weight=1)

        self.health_requests_var = tk.StringVar(value="0")
        self.health_responses_var = tk.StringVar(value="0")
        self.health_crc_count_var = tk.StringVar(value="0")
        self.health_timeout_count_var = tk.StringVar(value="0")
        self.health_exception_count_var = tk.StringVar(value="0")
        self.health_req_rate_var = tk.StringVar(value="0.00 req/s")
        self.health_resp_avg_var = tk.StringVar(value="—")
        self.health_resp_min_var = tk.StringVar(value="—")
        self.health_resp_max_var = tk.StringVar(value="—")
        self.health_p95_var = tk.StringVar(value="—")
        self.health_nonvalidated_rate_var = tk.StringVar(value="0.00 %")
        self.health_timeout_rate_var = tk.StringVar(value="0.00 %")
        self.health_bus_load_var = tk.StringVar(value="0.00 %")
        self.health_slaves_var = tk.StringVar(value="0")
        self.health_total_bytes_var = tk.StringVar(value="0")
        self.health_slowest_var = tk.StringVar(value="—")
        self.health_exception_slaves_var = tk.StringVar(value="—")

        self._health_metric_groups = []

```

### MBSniffer v3.2/mb_gui.py — lines 1761-1800

```
        def health_item(parent, label, variable, *, wraplength=0):
            return (
                ttk.Label(parent, text=label),
                ttk.Label(
                    parent,
                    textvariable=variable,
                    anchor="e",
                    wraplength=wraplength,
                    justify="right",
                ),
            )

        traffic_pairs = [
            health_item(traffic_health, "Pedidos", self.health_requests_var),
            health_item(traffic_health, "Respostas", self.health_responses_var),
            health_item(traffic_health, "Taxa de pedidos", self.health_req_rate_var),
            health_item(traffic_health, "Slaves ativos", self.health_slaves_var),
            health_item(traffic_health, "Bytes observados", self.health_total_bytes_var),
            health_item(traffic_health, "Utilização do bus ~", self.health_bus_load_var),
        ]
        response_pairs = [
            health_item(response_health, "Média", self.health_resp_avg_var),
            health_item(response_health, "Mínimo", self.health_resp_min_var),
            health_item(response_health, "Máximo", self.health_resp_max_var),
            health_item(response_health, "P95", self.health_p95_var),
            health_item(response_health, "Slave mais lento", self.health_slowest_var),
        ]
        quality_pairs = [
            health_item(quality_health, "Erros de CRC", self.health_crc_count_var),
            health_item(quality_health, "Bytes não validados", self.health_nonvalidated_rate_var),
            health_item(quality_health, "Timeouts", self.health_timeout_count_var),
            health_item(quality_health, "Taxa de timeout", self.health_timeout_rate_var),
            health_item(quality_health, "Exceções", self.health_exception_count_var),
            health_item(
                quality_health,
                "Exceções / Slave",
                self.health_exception_slaves_var,
                wraplength=190,
            ),
        ]
```

### MBSniffer v3.2/mb_gui.py — lines 1801-1840

```
1801
1802        self._health_metric_groups = [
1803            (traffic_health, traffic_pairs),
1804            (response_health, response_pairs),
1805            (quality_health, quality_pairs),
1806        ]
1807        self._health_cards = (
1808            traffic_health,
1809            response_health,
1810            quality_health,
1811        )
1812        self._health_layout_compact = False
1813        self._health_full_required_card_height = None
1814        self._bus_health_layout_after_id = None
1815
1816        # Start with the normal layout, then adapt only if the real available
1817        # height is insufficient.
1818        self._apply_bus_health_layout(False)
1819
1820        health_frame.bind(
1821            "<Configure>",
1822            self._schedule_bus_health_layout_sync,
1823            add="+",
1824        )
1825        notebook.bind(
1826            "<<NotebookTabChanged>>",
1827            self._schedule_bus_health_layout_sync,
1828            add="+",
1829        )
1830        self.after_idle(self._initialize_bus_health_layout)
1831
1832        # Help tab
1833        help_frame = ttk.Frame(notebook)
1834        help_frame.columnconfigure(0, weight=1)
1835        help_frame.rowconfigure(0, weight=1)
1836        notebook.add(help_frame, text="Ajuda / Ligações")
1837
1838        self.help_text = tk.Text(
1839            help_frame,
1840            wrap="word",
```

### MBSniffer v3.2/mb_gui.py — lines 1841-1880

```
            font=("Segoe UI", 10),
            padx=14,
            pady=12,
            spacing1=2,
            spacing3=5
        )
        help_scroll = ttk.Scrollbar(help_frame, orient="vertical", command=self.help_text.yview)
        self.help_text.configure(yscrollcommand=help_scroll.set)
        self.help_text.grid(row=0, column=0, sticky="nsew")
        help_scroll.grid(row=0, column=1, sticky="ns")

        self.help_text.tag_configure("title", font=("Segoe UI", 14, "bold"), spacing1=8, spacing3=8)
        self.help_text.tag_configure("heading", font=("Segoe UI", 11, "bold"), spacing1=10, spacing3=4)
        self.help_text.tag_configure("subheading", font=("Segoe UI", 10, "bold"), spacing1=6, spacing3=2)
        self.help_text.tag_configure("mono", font=("Consolas", 10), lmargin1=18, lmargin2=18)
        self.help_text.tag_configure("warning", font=("Segoe UI", 10, "bold"), lmargin1=12, lmargin2=12)
        self.help_text.tag_configure("bullet", lmargin1=12, lmargin2=28)

        def h(text_value, tag=None):
            self.help_text.insert("end", text_value, tag)

        max_ui_frames_text = f"{MAX_UI_FRAMES:_}".replace("_", " ")
        max_raw_lines_text = f"{MAX_RAW_TEXT_LINES:_}".replace("_", " ")

        h("MBSniffer — Ajuda e ligações\n", "title")
        h(
            "O MBSniffer foi pensado para diagnóstico de comunicações Modbus RTU. "
            "O separador Sniffer é passivo: abre a(s) porta(s) COM para receção e "
            "não transmite pedidos. O separador Bus Slave Finder é uma ferramenta "
            "ativa de pesquisa e deve ser utilizado apenas quando não existe outro "
            "master ativo no mesmo barramento.\n"
        )

        h("Início rápido — Sniffer\n", "heading")
        h("1. Escolhe o modo físico correto: RS485 2-wire, RS232 single RX ou RS232 dual RX.\n", "bullet")
        h("2. Seleciona a(s) porta(s) COM e os parâmetros série da comunicação observada.\n", "bullet")
        h("3. Mantém Frame gap em Auto, salvo quando tens uma razão concreta para o ajustar.\n", "bullet")
        h("4. Clica em “Iniciar Captura” e confirma atividade através de BUS ● RX.\n", "bullet")
        h("5. Usa Tráfego para diagnóstico rápido e Raw Hex / Log para confirmar os bytes reais.\n", "bullet")
        h("6. Seleciona um frame para obter a análise detalhada no Frame Inspector.\n", "bullet")
```

**MBSniffer v3.2/mb_gui.py — lines 1881-1920**

```
1881
1882        h("1. RS485 2-wire\n", "heading")
1883        h(
1884            "Neste modo uma única COM observa pedidos e respostas no mesmo par diferencial. "
1885            "O adaptador do sniffer deve ser ligado em paralelo e não deve atuar como master.\n"
1886        )
1887        h("Ligação típica:\n", "subheading")
1888        h(
1889            "        Equipamento A          Equipamento B\n"
1890            "             D+ --------------- D+\n"
1891            "             D- --------------- D-\n"
1892            "               \\               /\n"
1893            "                \\             /\n"
1894            "                 D+           D-\n"
1895            "                  USB-RS485 sniffer\n\n",
1896            "mono"
1897        )
1898        h("• Liga D+ do sniffer em paralelo com D+ do barramento.\n", "bullet")
1899        h("• Liga D− do sniffer em paralelo com D− do barramento.\n", "bullet")
1900        h(
1901            "• Se existir C/COM/0 V de referência, liga-o apenas quando souberes que "
1902            "corresponde à referência correta da interface RS485.\n",
1903            "bullet"
1904        )
1905        h(
1906            "• Não acrescentes uma terceira resistência de 120 Ω se o barramento já "
1907            "estiver terminado nas duas extremidades.\n",
1908            "warning"
1909        )
1910        h(
1911            "• Se o adaptador USB-RS485 tiver terminação ou bias selecionáveis, deixa-os "
1912            "desativados durante uma captura passiva, salvo se forem intencionalmente "
1913            "necessários na instalação.\n",
1914            "bullet"
1915        )
1916        h(
1917            "• A/B não é uma nomenclatura universal: fabricantes diferentes podem usar "
1918            "polaridades opostas. Dá prioridade a D+/D− e à documentação do equipamento.\n",
1919            "bullet"
1920        )
```

**MBSniffer v3.2/mb_gui.py — lines 1921-1960**

```
1921
1922        h("2. RS232 single RX\n", "heading")
1923        h(
1924            "Observa apenas uma direção RS232. O RS232 é full-duplex: cada direção utiliza "
1925            "uma linha TX independente, pelo que uma única entrada RX não consegue observar "
1926            "simultaneamente os dois sentidos.\n"
1927        )
1928        h(
1929            "        Equipamento A                    Equipamento B\n"
1930            "             TX ----------------------------> RX\n"
1931            "              \\\n"
1932            "               \\----> RX do USB-RS232 sniffer (COM A)\n"
1933            "\n"
1934            "             GND --------------------------- GND\n"
1935            "               \\--------------------------> GND sniffer\n\n"
1936            "        TX do sniffer: NÃO LIGAR\n\n",
1937            "mono"
1938        )
1939        h("• Liga RX do sniffer ao TX da direção que pretendes observar.\n", "bullet")
1940        h("• Liga também o GND comum.\n", "bullet")
1941        h("• Deixa fisicamente desligado o TX do adaptador usado como sniffer.\n", "warning")
1942
1943        h("3. RS232 dual RX\n", "heading")
1944        h(
1945            "Permite observar as duas direções RS232 ao mesmo tempo. São necessários "
1946            "dois canais RX independentes, normalmente dois adaptadores USB-RS232.\n"
1947        )
1948        h(
1949            "        Equipamento A                    Equipamento B\n"
1950            "             TX ----------------------------> RX\n"
1951            "              \\----> RX sniffer COM A   (A → B)\n"
1952            "\n"
1953            "             RX <---------------------------- TX\n"
1954            "                                  \\----> RX sniffer COM B   (B → A)\n"
1955            "\n"
1956            "             GND --------------------------- GND\n"
1957            "               \\--------------------------- GND COM A\n"
1958            "                \\--------------------------- GND COM B\n\n"
1959            "        TX de ambos os sniffers: NÃO LIGAR\n\n",
1960            "mono"
```

### MBSniffer v3.2/mb_gui.py — lines 1961-2000

```
        )
        h("• COM A é apresentada como A→B e COM B como B→A.\n", "bullet")
        h("• As duas COM devem usar os mesmos parâmetros série da ligação observada.\n", "bullet")

        h("RS232 não é TTL\n", "heading")
        h(
            "Não ligues sinais RS232 diretamente a interfaces USB-TTL/UART de 3,3 V ou "
            "5 V. Usa um adaptador USB-RS232 com transceiver apropriado aos níveis RS232.\n",
            "warning"
        )

        h("Parâmetros da comunicação série\n", "heading")
        h(
            "Baud, Data bits, Parity e Stop bits têm de coincidir com a comunicação real. "
            "Parâmetros errados podem produzir bytes aparentemente aleatórios, frames com "
            "CRC inválido ou ausência total de frames reconhecíveis.\n"
        )
        h("• Baud rate — velocidade em bit/s, por exemplo 9600 ou 19200.\n", "bullet")
        h("• Data bits — normalmente 8 em Modbus RTU.\n", "bullet")
        h("• Parity — None, Even, Odd, Mark ou Space, conforme a instalação.\n", "bullet")
        h("• Stop bits — 1, 1.5 ou 2, conforme a configuração observada.\n", "bullet")

        h("Frame gap\n", "heading")
        h(
            "Em Auto, o programa usa 3,5 tempos de carácter até 19200 bit/s, inclusive. "
            "Acima de 19200 bit/s usa o t3.5 fixo de 1,750 ms recomendado para Modbus RTU. "
            "O modo Manual permite definir outro valor em milissegundos para diagnóstico "
            "de equipamentos ou drivers com comportamento particular.\n"
        )
        h(
            "Os adaptadores USB e o Windows podem agrupar bytes e introduzir latência. "
            "Por isso, os timestamps do MBSniffer são úteis para diagnóstico de sequência "
            "e tempos relativos, mas não substituem um analisador lógico quando são "
            "necessárias medições temporais de elevada precisão.\n"
        )
        h(
            "Quando um burst termina com o início de um frame conhecido ainda incompleto, "
            f"o MBSniffer espera até {SNIFFER_SPLIT_FRAME_HOLD_MS:g} ms adicionais antes de o "
            "fechar. Assim, um frame partido em dois pelo adaptador USB não aparece como RAW. "
            "Frames juntos no mesmo burst mantêm a hora de chegada do seu próprio bloco de bytes.\n"
```

**MBSniffer v3.2/mb_gui.py — lines 2001-2040**

```
        )

        h("Pending timeout\n", "heading")
        h(
            "Define durante quanto tempo um REQUEST CRC-válido pode permanecer à espera de "
            "uma RESPONSE/EXCEPTION emparelhável. Ao expirar, o REQUEST original é marcado "
            "como timeout e aumenta os contadores de Timeout/Timeout rate.\n"
        )
        h(
            f"Intervalo permitido: {MIN_PENDING_REQUEST_TIMEOUT_SECONDS:g} a "
            f"{MAX_PENDING_REQUEST_TIMEOUT_SECONDS:g} s. O valor deve ser superior ao "
            "tempo de resposta máximo normal da instalação para evitar falsos timeouts.\n"
        )
        h(
            "Pedidos broadcast (Slave 0) não têm resposta por definição: não ficam pendentes, "
            "nunca contam como timeout e não entram no cálculo da Timeout rate.\n"
        )

        h("Como interpretar a tabela Tráfego\n", "heading")
        h("• REQUEST — pedido Modbus RTU reconhecido pelo parser.\n", "bullet")
        h("• RESPONSE — resposta Modbus RTU reconhecida que não é uma EXCEPTION.\n", "bullet")
        h(
            "• FC05/FC06/FC08/FC22 — o pedido e a resposta têm o mesmo formato. A trama é "
            "RESPONSE quando existe um pedido pendente do mesmo Slave/FC (em FC05/06/22 os "
            "bytes têm de ser iguais; em RS232 dual RX tem de vir do canal oposto). Caso "
            "contrário é REQUEST.\n",
            "bullet"
        )
        h(
            "• Respostas FC01/FC02 mostram os bits por ordem de endereço, agrupados por byte. "
            "No Frame Inspector, as respostas de leitura emparelhadas (FC01–FC04) mostram o "
            "PDU Address de cada valor.\n",
            "bullet"
        )
        h(
            "• EXCEPTION — resposta Modbus com Function Code de exceção (bit 7 ativo). "
            "Detalhes mostra o código e a descrição normalizada.\n",
            "bullet"
        )
        h("• CRC OK — CRC recebido coincide com o CRC calculado.\n", "bullet")
```

**MBSniffer v3.2/mb_gui.py — lines 2041-2080**

```
2041        h(
2042            "• CRC ERROR — a estrutura parece um frame suportado, mas o CRC recebido não "
2043            "coincide com o CRC calculado.\n",
2044            "bullet"
2045        )
2046        h(
2047            "• RAW/UNSYNC — bytes descartados durante uma recuperação de sincronismo antes "
2048            "de ser encontrado, mais à frente no mesmo burst, um frame de estrutura "
2049            "reconhecida com CRC válido.\n",
2050            "bullet"
2051        )
2052        h(
2053            "• RAW/UNPARSED — restante conjunto de bytes que não pôde ser interpretado nem "
2054            "ressincronizado como um frame Modbus válido. É este estado que aparece, por "
2055            "exemplo, num bloco de ruído sem um frame válido posterior no mesmo burst.\n",
2056            "bullet"
2057        )
2058        h(
2059            "RAW/UNSYNC e RAW/UNPARSED são portanto situações diferentes e ambas podem "
2060            "aparecer na coluna Tipo. Nenhuma delas representa uma resposta Modbus válida.\n"
2061        )
2062
2063        h("Colunas da tabela Tráfego\n", "heading")
2064        h("• Hora — timestamp do início do burst/frame processado.\n", "bullet")
2065        h("• Δt — intervalo desde o frame anterior observada.\n", "bullet")
2066        h(
2067            "• Resp. — tempo entre um REQUEST e a RESPONSE/EXCEPTION emparelhada. "
2068            "Fica vazio quando não existe um par válido.\n",
2069            "bullet"
2070        )
2071        h("• COM — identifica o canal de captura: BUS em RS485, A→B/B→A em RS232 dual RX e SIM no modo de desenvolvimento. Não representa necessariamente o número COM do Windows
    .\n", "bullet")
2072        h("• Tipo — REQUEST, RESPONSE, EXCEPTION ou um estado RAW.\n", "bullet")
2073        h("• Slave — Unit/Slave ID Modbus quando identificável.\n", "bullet")
2074        h("• FC — Function Code hexadecimal observado no frame.\n", "bullet")
2075        h("• Detalhes — endereço, quantidade, dados ou descrição da operação.\n", "bullet")
2076        h("• CRC — OK, ERROR ou ? quando não existe um frame Modbus completo.\n", "bullet")
2077
2078        h("Emparelhamento REQUEST / RESPONSE\n", "heading")
2079        h(
2080            "Modbus RTU não possui Transaction ID. O MBSniffer emparelha frames por "
```

**MBSniffer v3.2/mb_gui.py — lines 2081-2120**

```
            "(Slave ID, Function Code) em ordem FIFO. Isto é apropriado para tráfego RTU "
            "normal, mas pode tornar-se ambíguo se existirem pedidos sobrepostos com o mesmo "
            "Slave e FC antes das respostas correspondentes.\n"
        )
        h(
            "Uma EXCEPTION usa internamente o Function Code base para emparelhamento. "
            "Exemplo: uma resposta 0x83 é associada a um REQUEST FC03 quando existe um "
            "pedido pendente compatível.\n"
        )

        h("PDU Address, 1-based e Qty\n", "heading")
        h(
            "PDU Address é o endereço codificado no pedido Modbus. 1-based é simplesmente "
            "PDU Address + 1 e serve para comparação com manuais que numeram o primeiro "
            "registo/coil como 1. Qty é a quantidade pedida.\n"
        )
        h(
            "Estes campos pertencem ao REQUEST. Uma RESPONSE não repete automaticamente "
            "Address ou Qty no protocolo, por isso o MBSniffer não inventa esses campos na "
            "linha da resposta.\n"
        )

        h("Códigos de exceção Modbus\n", "heading")
        h("• 0x01 — Illegal Function: função não suportada ou não permitida.\n", "bullet")
        h("• 0x02 — Illegal Data Address: endereço/gama de endereços inválida.\n", "bullet")
        h("• 0x03 — Illegal Data Value: valor, quantidade ou campo inválido.\n", "bullet")
        h("• 0x04 — Server Device Failure: falha interna do dispositivo.\n", "bullet")
        h("• 0x05 — Acknowledge: pedido aceite; processamento ainda em curso.\n", "bullet")
        h("• 0x06 — Server Device Busy: dispositivo temporariamente ocupado.\n", "bullet")
        h("• 0x08 — Memory Parity Error.\n", "bullet")
        h("• 0x0A — Gateway Path Unavailable.\n", "bullet")
        h("• 0x0B — Gateway Target Device Failed to Respond.\n", "bullet")

        h("Filtros avançados\n", "heading")
        h(
            "A barra no topo de Tráfego permite combinar vários filtros sem alterar os "
            "dados originais da sessão:\n"
        )
        h("• Tipo — Todos, Requests, Responses, Exceptions, CRC errors, Timeouts ou RAW.\n", "bullet")
        h("• FC (hex) — por exemplo 03, 04, 10 ou 0x04.\n", "bullet")
```

**MBSniffer v3.2/mb_gui.py — lines 2121-2160**

```
2121        h("• Resp. > — mostra apenas respostas acima do limite indicado em ms.\n", "bullet")
2122        h(
2123            "• Pesquisa — procura texto em Detalhes, Raw Hex, Slave, FC, Tipo e COM/canal.\n",
2124            "bullet"
2125        )
2126        h(
2127            "O filtro Slave continua disponível no cabeçalho da coluna e combina-se com "
2128            "os filtros avançados. “Limpar filtros” repõe todos os filtros de visualização. "
2129            "Os filtros não alteram as estatísticas nem o conteúdo do log de captura.\n"
2130        )
2131
2132        h("Realçar resultados\n", "heading")
2133        h(
2134            "Esta opção vem desligada por defeito para manter a tabela neutra. Quando está "
2135            "ativa, os resultados passam a ser diferenciados por cor: uma transação normal "
2136            "concluída com RESPONSE CRC-válida fica verde; CRC error, timeout, Exception, "
2137            "RAW e resposta lenta mantêm as respetivas cores de diagnóstico.\n"
2138        )
2139        h(
2140            "Numa transação normal concluída com sucesso, tanto o REQUEST como a RESPONSE "
2141            "correspondente ficam verdes. Um REQUEST ainda pendente permanece neutro. "
2142            "Uma resposta lenta continua com a cor de resposta lenta, mesmo sendo válida, "
2143            "para não esconder o resultado de diagnóstico.\n"
2144        )
2145        h(
2146            f"Uma resposta é considerada lenta a partir de "
2147            f"{SLOW_RESPONSE_THRESHOLD_MS:g} ms. Este limite é uma referência visual do "
2148            "MBSniffer; deve ser interpretado no contexto do equipamento e do ciclo de poll.\n"
2149        )
2150
2151        h("Código de cores dos resultados\n", "heading")
2152        h("• Verde — Sucesso: REQUEST/RESPONSE normal, emparelhada e com CRC válido.\n", "bullet")
2153        h(
2154            f"• Amarelo — Resposta lenta: tempo de resposta >= "
2155            f"{SLOW_RESPONSE_THRESHOLD_MS:g} ms.\n",
2156            "bullet"
2157        )
2158        h("• Vermelho — CRC Error ou resposta Modbus Exception.\n", "bullet")
2159        h("• Laranja — Timeout: REQUEST expirou sem resposta emparelhada.\n", "bullet")
2160        h("• Roxo — RAW/UNSYNC ou RAW/UNPARSED: bytes sem frame Modbus válido.\n", "bullet")
```

**MBSniffer v3.2/mb_gui.py — lines 2161-2200**

```
        h(
            "Com “Realçar resultados” desligado, a tabela permanece neutra. O Frame "
            "Inspector continua a indicar a classificação do resultado em texto; quando "
            "o destaque está ligado, o campo Resultado também usa a respetiva cor.\n"
        )

        h("Frame Inspector\n", "heading")
        h(
            "Seleciona uma linha de Tráfego para analisar o frame sem ter de interpretar "
            "manualmente todos os bytes. O Inspector mostra Slave, Tipo, Function Code + "
            "nome, tempo de resposta, Resultado, PDU Address, 1-based, "
            "Qty, ByteCount, CRC recebido, CRC calculado, dados/registos/Exception e Raw Hex.\n"
        )
        h(
            "“Recolher” esconde o conteúdo do Inspector e devolve espaço vertical à tabela. "
            "“Expandir” volta a apresentar a análise.\n"
        )

        h("Menu de contexto do Tráfego\n", "heading")
        h("Clica com o botão direito numa frame real para:\n")
        h("• Copiar Raw Hex.\n", "bullet")
        h("• Copiar o frame descodificado.\n", "bullet")
        h("• Filtrar pelo Slave dessa frame.\n", "bullet")
        h("• Filtrar pelo Function Code dessa frame.\n", "bullet")
        h("• Mostrar apenas a transação REQUEST/RESPONSE correspondente.\n", "bullet")
        h("• Limpar o filtro temporário de transação.\n", "bullet")

        h("Bus Health\n", "heading")
        h(
            "O separador Bus Health resume a qualidade e o desempenho da sessão sem "
            "substituir a tabela de frames.\n"
        )
        h("• Pedidos / Respostas — contadores da sessão.\n", "bullet")
        h("• Taxa de pedidos — número médio de pedidos observados por segundo.\n", "bullet")
        h("• Slaves ativos — IDs Modbus válidos observados na sessão.\n", "bullet")
        h("• Bytes observados — soma dos bytes entregues ao parser.\n", "bullet")
        h(
            "• Utilização bus ~ — estimativa com base em bytes observados, baud, data bits, "
            "parity e stop bits. É aproximada e depende do ponto de captura.\n",
            "bullet"
```

**MBSniffer v3.2/mb_gui.py — lines 2201-2240**

```
        )
        h("• Média / Mínimo / Máximo — tempos de resposta emparelhados.\n", "bullet")
        h("• P95 — 95% das respostas medidas ficaram abaixo ou iguais a este valor.\n", "bullet")
        h(
            "• Slave mais lento — Slave com maior tempo médio de resposta entre as amostras "
            "observadas.\n",
            "bullet"
        )
        h("• Bytes não validados — percentagem dos bytes observados que pertencem a frames com CRC inválido ou a blocos RAW. Não prova, por si só, a existência de ruído elétrico
    ; parâmetros série errados, captura incompleta ou funções não suportadas pelo parser também podem produzir RAW.\n", "bullet")
        h("• Taxa de timeout — percentagem de REQUEST que expiraram sem resposta emparelhada.\n", "bullet")
        h("• Exceções / Slave — distribuição das respostas Exception por Slave ID.\n", "bullet")

        h("Indicador BUS RX\n", "heading")
        h(
            "BUS ● RX pulsa brevemente quando chega um lote de frames à interface. Em repouso "
            "é apresentado BUS ○. Serve apenas como confirmação visual de atividade recebida; "
            "não representa qualidade elétrica nem garante que os frames sejam válidos.\n"
        )

        h("Raw Hex / Log\n", "heading")
        h(
            "Mostra os bytes capturados em hexadecimal com informação de contexto. As linhas "
            "longas fazem wrap à largura disponível. A vista acompanha os mesmos filtros "
            "visuais usados em Tráfego; o ficheiro TXT em disco, quando existe, mantém a "
            "captura completa e não é truncado pelos filtros.\n"
        )

        h("Exportar CSV\n", "heading")
        h(
            "Exportar CSV grava os frames atualmente retidos no histórico da interface em "
            "UTF-8 compatível com Excel. Inclui Time, Δt, Response time, Channel, Type, "
            "Slave, FC, Details, CRC, Raw, PDU Address, 1-based, Qty, ByteCount, Timeout "
            "e classe de anomalia.\n"
        )
        h(
            f"A interface mantém aproximadamente os últimos {max_ui_frames_text} frames. "
            "Por isso, em sessões maiores o CSV representa o histórico ainda retido pela GUI, "
            "enquanto o TXT de uma captura real continua a ser o registo completo em disco.\n"
        )

```

**MBSniffer v3.2/mb_gui.py — lines 2241-2280**

```
        h("Logs de captura\n", "heading")
        h(
            "O log TXT só é criado depois de chegar tráfego real. Iniciar e parar sem receber "
            "frames não cria um ficheiro vazio.\n"
        )
        h("Pasta dos logs:\n")
        h("        <pasta da aplicação>\\MBSniffer Logs\n", "mono")
        h(
            "Os filtros, ordenação, Realçar resultados e o Frame Inspector são apenas funções "
            "de visualização e não alteram os frames gravados no log.\n"
        )

        h("Limites do histórico visível\n", "heading")
        h(
            f"Para manter a aplicação responsiva, a interface conserva aproximadamente "
            f"{max_ui_frames_text} frames e {max_raw_lines_text} linhas de Raw Hex. "
            "A remoção é feita por blocos. O log TXT de captura, quando ativo, é independente "
            "deste limite.\n"
        )

        h("Estado e portas COM\n", "heading")
        h(
            "Estado COM distingue uma porta aberta pelo MBSniffer, uma porta apenas detetada "
            "pelo Windows e uma porta que deixou de ser detetada. O estado é atualizado "
            "periodicamente.\n"
        )
        h(
            "Reiniciar COM fecha e volta a abrir a(s) porta(s) usada(s) pelo Sniffer. "
            "Isto reinicia o handle da porta série dentro do MBSniffer; não reinicia o "
            "driver USB/COM do Windows.\n"
        )
        h(
            "Os botões ↻ ao lado das COM atualizam a enumeração de portas disponível no "
            "Sniffer e no Bus Slave Finder.\n"
        )

        h("Bus Slave Finder — pesquisa ativa\n", "heading")
        h(
            "O Bus Slave Finder transmite pedidos Modbus RTU, por isso não é uma função "
            "passiva. Antes de iniciar, confirma que não existe outro master ativo no mesmo "
```

**MBSniffer v3.2/mb_gui.py — lines 2281-2320**

```
            "barramento.\n"
        )
        h(
            "Seleciona COM, baud rates, paridades, stop bits e intervalo de Slave IDs. "
            "Data bits é fixo em 8. A pesquisa tenta FC03 Address 0 Qty 1; uma resposta "
            "normal ou uma Exception Modbus com CRC válido confirma que o Slave existe.\n"
        )
        h(
            "FC04 fallback é opcional e aumenta o número máximo de tentativas. O timeout "
            "mínimo é adaptado ao baud rate. Um write timeout significa falha ao transmitir "
            "pela porta/driver e não deve ser interpretado simplesmente como “slave sem resposta”.\n"
        )
        h(
            "Tentativas (1 a 3) repete cada pedido sem resposta antes de passar ao Slave "
            "seguinte. Antes da primeira transmissão, a pesquisa escuta o barramento durante "
            f"{DISCOVERY_IDLE_CHECK_SECONDS:g} s; se receber qualquer byte, não transmite e "
            "indica que pode existir outro master ativo.\n"
        )
        h(
            "Device Identification (FC43/14) é opcional. Quando ativo, só é enviado depois "
            "de um Slave ter sido encontrado e tenta ler Basic Device Identification: "
            "VendorName, ProductCode e MajorMinorRevision. Uma Exception ou ausência de "
            "resposta a FC43/14 não invalida a descoberta do Slave.\n"
        )
        h(
            "O Finder e o Sniffer partilham o bloqueio global de atividade para impedir que "
            "a mesma aplicação faça pesquisa ativa e captura ao mesmo tempo.\n"
        )

        h("Master — leitura e escrita como master Modbus RTU\n", "heading")
        h(
            "O separador Master transmite pedidos: só deve ser usado quando não existe "
            "outro master ativo no barramento. Ao ligar, a aplicação escuta o barramento "
            f"durante {DISCOVERY_IDLE_CHECK_SECONDS:g} s e não transmite se detetar tráfego. "
            "Enquanto o Master está ligado, o Sniffer e o Bus Slave Finder ficam bloqueados.\n"
        )
        h(
            "Cada janela lê uma zona de memória de um Slave: Slave ID, função (FC01 Coils, "
            "FC02 Discrete Inputs, FC03 Holding Registers ou FC04 Input Registers), endereço, "
            "quantidade e scan rate. As alterações só valem depois de “Aplicar”, que fica "
```

**MBSniffer v3.2/mb_gui.py — lines 2321-2360**

```
            "destacado a azul enquanto houver campos por aplicar. A tabela mostra até 15 "
            "linhas; acima disso tem scroll.\n"
        )
        h(
            "Todas as janelas partilham a mesma porta COM: o Modbus RTU só admite um pedido "
            "de cada vez, por isso as janelas são servidas à vez conforme o scan rate. "
            "Escritas e “Ler 1x” passam à frente das leituras cíclicas. Entre dois pedidos "
            "fica sempre o “Atraso entre pedidos” (no mínimo t3.5).\n"
        )
        h(
            "Formatos: 16 bits com e sem sinal, hex e binário; 32 bits com e sem sinal e "
            "float, nas ordens de bytes ABCD (big-endian), CDAB (palavras trocadas), BADC "
            "(bytes trocados) e DCBA (little-endian); 64 bits com e sem sinal e double, nas "
            "ordens ABCDEFGH, GHEFCDAB, BADCFEHG e HGFEDCBA. Os valores de 32 e 64 bits "
            "ocupam 2 e 4 registos e aparecem na linha do primeiro.\n"
        )
        h(
            "O campo “Formato” é o formato por defeito da janela. Com clique direito num ou "
            "mais endereços (arrastar com o rato, ou Ctrl/Shift, para selecionar vários) escolhe-se o formato só "
            "desses endereços: os de 32 bits exigem 2 registos seguidos e os de 64 bits 4 "
            "(ou um múltiplo, aplicado em grupos). “Repor formato da janela” volta ao "
            "formato por defeito. O mesmo menu tem “Copiar” (também Ctrl+C), que copia "
            "endereços, nomes e valores para colar no Excel, e “Base de endereços”: 0 mostra "
            "o PDU Address; 1 numera a partir de 1, como em muitos manuais.\n"
        )
        h(
            "Quando uma leitura falha (Timeout, Exception ou resposta inválida), os valores "
            "ficam a cinzento até voltar a haver uma leitura correta; ao desligar também.\n"
        )
        h(
            "Escala: no clique direito, “Escala…” converte o valor bruto em unidades de "
            "engenharia com dois pontos (Bruto mín./máx. → Eng. mín./máx.), com casas "
            "decimais e unidade (por exemplo 2000–10000 → −180–180 °). Na escrita, o valor "
            "é escrito em unidades e convertido para bruto. “Remover escala” volta ao valor "
            "bruto. As escalas ficam guardadas no workspace.\n"
        )
        h(
            "“Abrir no Master”: no Tráfego do Sniffer (clique direito num pedido FC01–FC04 "
            "ou na resposta correspondente) e nos resultados do Bus Slave Finder (clique "
            "direito ou duplo clique) cria uma janela do Master com o mesmo Slave, função, "
```

**MBSniffer v3.2/mb_gui.py — lines 2361-2400**

```
            "endereço e quantidade, e copia os parâmetros série para a ligação do Master "
            "(se não estiver ligado).\n"
        )
        h(
            "Se a porta COM falhar durante a ligação (por exemplo, o adaptador USB foi "
            "desligado), o Master não para: tenta voltar a abrir a porta a cada 2 s e "
            "retoma as leituras quando a porta volta. Para parar, carregar em “Desligar”.\n"
        )
        h(
            "Escrita: duplo clique num valor de uma janela FC01 ou FC03. Coils com FC05 ou "
            "FC15, registos com FC06 ou FC16; vários valores separados por “;” são escritos "
            "em endereços consecutivos com FC15/FC16. Slave 0 envia em broadcast, sem "
            "resposta. Depois de uma escrita aceite, a janela é lida de novo.\n"
        )
        h(
            "Cada janela mostra Tx, erros, timeouts, o último tempo de resposta e o último "
            "erro (Timeout, Exception ou resposta inválida). No cabeçalho de cada janela, "
            "“Renomear” muda o nome da janela (também com duplo clique no nome), "
            "“Destacar” abre-a à parte do separador (“Ancorar” ou fechar essa janela "
            "devolve-a) e “x” fecha a janela.\n"
        )
        h(
            "O menu “Workspace” guarda a ligação, todas as janelas, os nomes e os formatos "
            "com um nome à escolha, e lista os workspaces guardados para os abrir ou apagar "
            "diretamente. Os ficheiros ficam em %APPDATA%\\MBSniffer\\Workspaces, junto das "
            "preferências.\n"
        )
        h(
            "O último workspace aberto ou guardado é reaberto ao arrancar o programa, sem "
            "ligar ao barramento. Enquanto houver alterações por guardar, aparece "
            "“(alterado)” ao lado do nome; ao fechar o programa nessa situação, pergunta se "
            "quer guardar. Janelas que nunca foram guardadas num workspace fecham sem "
            "perguntar, e fechar todas as janelas deixa de ter workspace aberto.\n"
        )

        h("Preferências guardadas\n", "heading")
        h(
            "As preferências do utilizador são guardadas em "
            "%APPDATA%\\MBSniffer\\settings.json. Incluem Light/Dark, último separador, "
            "parâmetros série, Frame gap, Pending timeout, Auto-scroll, Separar transações, "
```

**MBSniffer v3.2/mb_gui.py — lines 2401-2440**

```
            "Realçar resultados, opções do Bus Slave Finder e parâmetros de ligação do "
            "Master. As janelas do Master guardam-se em workspaces, na pasta "
            "%APPDATA%\\MBSniffer\\Workspaces.\n"
        )
        h(
            "As portas COM não são guardadas porque são específicas de cada computador. "
            "A confirmação de segurança do Bus Slave Finder também não é persistida.\n"
        )

        h("Boas práticas de diagnóstico\n", "heading")
        h("• Confirma primeiro cablagem, polaridade e parâmetros série.\n", "bullet")
        h("• Em RS485, mantém a derivação do sniffer curta e evita alterar a terminação existente.\n", "bullet")
        h("• Garante que nenhum outro programa tem a mesma COM aberta.\n", "bullet")
        h("• Usa CRC ERROR e RAW como indícios de framing, ruído, parâmetros errados ou captura incompleta.\n", "bullet")
        h("• Usa Exceptions como respostas válidas do protocolo: não são equivalentes a CRC error.\n", "bullet")
        h("• Compara tempos de resposta com o comportamento normal do equipamento, não apenas com um limite genérico.\n", "bullet")
        h("• Guarda o TXT quando precisares de análise posterior da captura completa.\n", "bullet")

        if debug_sim == 1:
            h("Modo Simulação\n", "heading")
            h(
                "Este modo de desenvolvimento aparece apenas quando debug_sim = 1 em "
                "MBSniffer.py. Não abre portas série e nunca cria logs TXT.\n"
            )
            h(
                "Cada execução usa sempre o mesmo tipo de cenário de diagnóstico: uma "
                "resposta normal, uma resposta lenta, uma Modbus Exception, um CRC error, "
                "um timeout e um bloco RAW/UNPARSED. Assim é possível comparar diretamente "
                "a mesma informação com e sem destaque visual.\n"
            )
            h(
                "“Realçar resultados” não altera os dados gerados pela Simulação. Desligado, "
                "todas as linhas usam as cores normais da tabela; ligado, a transação normal "
                "concluída fica verde e as linhas de Exception, CRC error, timeout, resposta "
                "lenta e RAW recebem as respetivas cores de diagnóstico.\n"
            )
            h(
                "As frames deste cenário usam o canal SIM para não serem confundidas com uma "
                "captura física real.\n"
            )
```

**MBSniffer v3.2/mb_gui.py — lines 2441-2480**

```
2441
2442        h("Limitação importante\n", "heading")
2443        h(
2444            "O MBSniffer interpreta bytes, CRC, Function Code, endereços, quantidades, "
2445            "emparelhamento e tempos observados pelo sistema operativo. Não mede diretamente "
2446            "níveis elétricos, ringing, reflexões, bias, common-mode, tempos de subida/descida "
2447            "ou integridade analógica do sinal. Para esses problemas continua a ser necessário "
2448            "um osciloscópio ou analisador adequado à interface física.\n"
2449        )
2450
2451        self.help_text.configure(state="disabled")
2452
2453        detail = ttk.LabelFrame(self.sniffer_page, text="Frame Inspector")
2454        detail.grid(row=2, column=0, sticky="ew", padx=10, pady=(0, 10))
2455        detail.columnconfigure(0, weight=1)
2456
2457        inspector_header = ttk.Frame(detail)
2458        inspector_header.grid(
2459            row=0, column=0, sticky="ew", padx=8, pady=(4, 2)
2460        )
2461        inspector_header.columnconfigure(0, weight=1)
2462        ttk.Label(
2463            inspector_header,
2464            text=(
2465                "Seleciona uma linha no Tráfego para ver a descodificação "
2466                "estruturada do frame."
2467            ),
2468        ).grid(row=0, column=0, sticky="w")
2469
2470        self.inspector_collapsed = False
2471        self.inspector_toggle_btn = ttk.Button(
2472            inspector_header,
2473            text="Recolher",
2474            command=self.toggle_frame_inspector,
2475            style="MBS.Compact.TButton",
2476        )
2477        self.inspector_toggle_btn.grid(row=0, column=1, sticky="e")
2478
2479        self.inspector_body = ttk.Frame(detail)
2480        self.inspector_body.grid(
```

### MBSniffer v3.2/mb_gui.py — lines 2481-2520

```
            row=1, column=0, sticky="ew", padx=8, pady=(2, 7)
        )
        for col in range(4):
            self.inspector_body.columnconfigure(
                col,
                weight=1,
                uniform="inspector",
            )

        self.inspector_vars = {
            "slave": tk.StringVar(value="Slave: –"),
            "type": tk.StringVar(value="Tipo: –"),
            "function": tk.StringVar(value="Function: –"),
            "response": tk.StringVar(value="Resp.: –"),
            "result": tk.StringVar(value="Resultado: –"),
            "pdu": tk.StringVar(value="PDU Address: –"),
            "one_based": tk.StringVar(value="1-based: –"),
            "qty": tk.StringVar(value="Qty: –"),
            "byte_count": tk.StringVar(value="ByteCount: –"),
            "crc": tk.StringVar(value="CRC: –"),
            "data": tk.StringVar(value="Dados: –"),
        }

        inspector_grid = (
            ("slave", 0, 0),
            ("type", 0, 1),
            ("function", 0, 2),
            ("response", 0, 3),
            ("pdu", 1, 0),
            ("one_based", 1, 1),
            ("qty", 1, 2),
            ("byte_count", 1, 3),
        )
        for key, row, col in inspector_grid:
            ttk.Label(
                self.inspector_body,
                textvariable=self.inspector_vars[key],
                anchor="w",
            ).grid(
                row=row,
```

**MBSniffer v3.2/mb_gui.py — lines 2521-2560**

```
                column=col,
                sticky="ew",
                padx=(0 if col == 0 else 8, 8 if col < 3 else 0),
                pady=2,
            )

        self.inspector_result_label = ttk.Label(
            self.inspector_body,
            textvariable=self.inspector_vars["result"],
            anchor="w",
            style="MBS.Result.neutral.TLabel",
        )
        self.inspector_result_label.grid(
            row=2,
            column=0,
            columnspan=4,
            sticky="ew",
            pady=(3, 2),
        )

        ttk.Label(
            self.inspector_body,
            textvariable=self.inspector_vars["crc"],
            anchor="w",
        ).grid(
            row=3, column=0, columnspan=2, sticky="ew", pady=2
        )
        # Data/Register payloads can be very long. A readonly Entry keeps the
        # inspector compact instead of letting content change requested width.
        ttk.Entry(
            self.inspector_body,
            textvariable=self.inspector_vars["data"],
            state="readonly",
        ).grid(
            row=3,
            column=2,
            columnspan=2,
            sticky="ew",
            padx=(8, 0),
            pady=2,
```

**MBSniffer v3.2/mb_gui.py — lines 2561-2584**

```
        )

        self.selected_raw_var = tk.StringVar()
        ttk.Entry(
            self.inspector_body,
            textvariable=self.selected_raw_var,
            state="readonly",
        ).grid(
            row=4,
            column=0,
            columnspan=4,
            sticky="ew",
            pady=(4, 0),
        )

        self.build_discovery_tab(self.discovery_page)
        self.build_master_tab(self.master_page)
        self.update_stats_labels()
        self.update_frame_inspector(None)

        # Classic Tk/Canvas widgets are outside ttk.Style; apply the remembered
        # startup palette after every such widget has been created.
        self._apply_classic_widget_palette(self.initial_dark_mode)

```

# MBSniffer v3.2/tests/test_regression.py

787 source lines.

**MBSniffer v3.2/tests/test_regression.py — lines 1-40**

```
import json
import queue
import random
import tempfile
import threading
import time
import unittest
from collections import deque
from pathlib import Path

import mb_config
import mb_slave_finder
from mb_capture import CaptureMixin
from mb_protocol import (
    SessionMetrics,
    append_crc,
    burst_may_continue,
    crc16_modbus,
    crc_ok,
    modbus_rtu_interframe_gap_seconds,
    parse_burst,
    read_bits_details,
    split_capture_buffer,
)
from mb_slave_finder import SlaveFinderMixin
from mb_view import ViewMixin
from mb_diagnostics import inspector_values
from mb_i18n import (
    LANGUAGE_ENGLISH,
    LANGUAGE_PORTUGUESE,
    english_help_blocks,
    normalize_language,
    translate_details,
    translate_runtime_text,
    translate_text,
)

from mb_slave_finder import (
    build_device_identification_probe,
    build_discovery_probe,
```

### MBSniffer v3.2/tests/test_regression.py — lines 41-80

```
    device_identification_timeout_seconds,
    find_device_identification_response,
    find_discovery_response,
    format_device_identification,
    read_basic_device_identification,
)


def with_bad_crc(frame: bytes) -> bytes:
    damaged = bytearray(frame)
    damaged[-1] ^= 0xFF
    return bytes(damaged)


def device_id_response(
    slave=1,
    objects=None,
    *,
    more_follows=False,
    next_object_id=0,
    conformity=0x01,
):
    if objects is None:
        objects = {
            0x00: b"Schneider Electric",
            0x01: b"TEST-PRODUCT",
            0x02: b"1.2.3",
        }
    payload = bytearray((
        slave,
        0x2B,
        0x0E,
        0x01,
        conformity,
        0xFF if more_follows else 0x00,
        next_object_id,
        len(objects),
    ))
    for object_id, value in objects.items():
        value = bytes(value)
```

### MBSniffer v3.2/tests/test_regression.py — lines 81-120

```
81          payload.extend((int(object_id) & 0xFF, len(value)))
82          payload.extend(value)
83      return append_crc(bytes(payload))
84
85
86  class FakeSerial:
87      """Minimal non-blocking serial fake for active Device ID regression tests."""
88
89      def __init__(self, response_by_object):
90          self.response_by_object = dict(response_by_object)
91          self.rx = bytearray()
92          self.writes = []
93
94      @property
95      def in_waiting(self):
96          return len(self.rx)
97
98      def reset_input_buffer(self):
99          self.rx.clear()
100
101     def reset_output_buffer(self):
102         pass
103
104     def write(self, data):
105         data = bytes(data)
106         self.writes.append(data)
107         object_id = data[4] if len(data) >= 5 else 0
108         response = self.response_by_object.get(object_id, b"")
109         self.rx.extend(response)
110         return len(data)
111
112     def read(self, size):
113         size = min(int(size), len(self.rx))
114         chunk = bytes(self.rx[:size])
115         del self.rx[:size]
116         return chunk
117
118
119 class FakeSerialModule:
120     class SerialTimeoutException(TimeoutError):
```

**MBSniffer v3.2/tests/test_regression.py — lines 121-160**

```
121         pass
122
123     class SerialException(OSError):
124         pass
125
126
127 class StaticVar:
128     def __init__(self, value):
129         self.value = value
130
131     def get(self):
132         return self.value
133
134
135 class DummyCapture:
136     calculated_gap_ms = CaptureMixin.calculated_gap_ms
137
138     def __init__(self, *, mode="Auto", baud="9600", data="8", parity="None", stop="1", manual="2.0"):
139         self.gap_mode_var = StaticVar(mode)
140         self.baud_var = StaticVar(baud)
141         self.data_var = StaticVar(data)
142         self.parity_var = StaticVar(parity)
143         self.stop_var = StaticVar(stop)
144         self.gap_ms_var = StaticVar(manual)
145
146
147 class TimingRegressionTests(unittest.TestCase):
148     def test_t35_uses_character_time_at_or_below_19200(self):
149         self.assertAlmostEqual(
150             modbus_rtu_interframe_gap_seconds(9600, 11),
151             3.5 * 11 / 9600,
152             places=9,
153         )
154         self.assertAlmostEqual(
155             modbus_rtu_interframe_gap_seconds(19200, 11),
156             3.5 * 11 / 19200,
157             places=9,
158         )
159
160     def test_t35_is_fixed_above_19200(self):
```

### MBSniffer v3.2/tests/test_regression.py — lines 161-200

```
161         for baud in (38400, 57600, 115200):
162             with self.subTest(baud=baud):
163                 self.assertAlmostEqual(
164                     modbus_rtu_interframe_gap_seconds(baud, 10),
165                     0.001750,
166                     places=9,
167                 )
168 
169     def test_sniffer_auto_gap_uses_fixed_1750_us_above_19200(self):
170         capture = DummyCapture(baud="115200", parity="None", stop="1")
171         self.assertAlmostEqual(capture.calculated_gap_ms(), 1.750, places=6)
172 
173     def test_sniffer_manual_gap_is_unchanged(self):
174         capture = DummyCapture(mode="Manual", manual="2,75")
175         self.assertAlmostEqual(capture.calculated_gap_ms(), 2.75, places=6)
176 
177     def test_device_id_timeout_covers_a_full_rtu_frame(self):
178         timeout = device_identification_timeout_seconds(9600, "None", "1", 40)
179         self.assertGreaterEqual(timeout, 256 * 10 / 9600 + 0.001750)
180 
181 
182 class ParserRegressionTests(unittest.TestCase):
183     def test_crc_known_request(self):
184         frame = build_discovery_probe(1, 3)
185         self.assertTrue(crc_ok(frame))
186         self.assertEqual(frame[:6], bytes.fromhex("01 03 00 00 00 01"))
187 
188     def test_invalid_frame_before_valid_frame_stays_crc_error(self):
189         first = with_bad_crc(build_discovery_probe(1, 3))
190         second = build_discovery_probe(2, 4)
191         parts = split_capture_buffer(first + second)
192         self.assertEqual(parts[0], (first, "REQUEST", "ERROR"))
193         self.assertEqual(parts[1], (second, "REQUEST", "OK"))
194 
195     def test_long_unsynchronised_noise_recovers_next_valid_frame(self):
196         noise = bytes([0xFF]) * 4000
197         frame = build_discovery_probe(7, 3)
198         parts = split_capture_buffer(noise + frame)
199         self.assertEqual(parts[-1], (frame, "REQUEST", "OK"))
200         self.assertTrue(parts[0][1].startswith("RAW/"))
```

### MBSniffer v3.2/tests/test_regression.py — lines 201-240

```
201         self.assertEqual(sum(len(part[0]) for part in parts), len(noise) + len(frame))
202
203     def test_supported_function_code_shapes(self):
204         frames = {
205             "fc01_req": append_crc(bytes.fromhex("01 01 00 00 00 08")),
206             "fc01_resp": append_crc(bytes.fromhex("01 01 01 5A")),
207             "fc02_req": append_crc(bytes.fromhex("01 02 00 00 00 08")),
208             "fc02_resp": append_crc(bytes.fromhex("01 02 01 A5")),
209             "fc03_req": append_crc(bytes.fromhex("01 03 00 00 00 01")),
210             "fc03_resp": append_crc(bytes.fromhex("01 03 02 00 2A")),
211             "fc04_req": append_crc(bytes.fromhex("01 04 00 00 00 01")),
212             "fc04_resp": append_crc(bytes.fromhex("01 04 02 00 2A")),
213             "fc05": append_crc(bytes.fromhex("01 05 00 01 FF 00")),
214             "fc06": append_crc(bytes.fromhex("01 06 00 01 00 2A")),
215             "fc07_req": append_crc(bytes.fromhex("01 07")),
216             "fc07_resp": append_crc(bytes.fromhex("01 07 AA")),
217             "fc08": append_crc(bytes.fromhex("01 08 00 00 12 34")),
218             "fc15_req": append_crc(bytes.fromhex("01 0F 00 00 00 08 01 5A")),
219             "fc15_resp": append_crc(bytes.fromhex("01 0F 00 00 00 08")),
220             "fc16_req": append_crc(bytes.fromhex("01 10 00 00 00 01 02 00 2A")),
221             "fc16_resp": append_crc(bytes.fromhex("01 10 00 00 00 01")),
222             "fc22": append_crc(bytes.fromhex("01 16 00 01 FF 00 00 2A")),
223             "fc23_req": append_crc(bytes.fromhex(
224                 "01 17 00 00 00 01 00 10 00 01 02 00 2A"
225             )),
226             "fc23_resp": append_crc(bytes.fromhex("01 17 02 00 2A")),
227             "exception": append_crc(bytes.fromhex("01 83 02")),
228         }
229         for name, frame in frames.items():
230             with self.subTest(name=name):
231                 parts = split_capture_buffer(frame)
232                 self.assertEqual(len(parts), 1)
233                 self.assertEqual(parts[0][0], frame)
234                 self.assertEqual(parts[0][2], "OK")
235                 self.assertFalse(parts[0][1].startswith("RAW"))
236
237     def test_random_noise_never_crashes_and_preserves_all_bytes(self):
238         rng = random.Random(20260910)
239         for _ in range(250):
240             length = rng.randint(1, 512)
```

### MBSniffer v3.2/tests/test_regression.py — lines 241-280

```
241             data = bytes(rng.randrange(256) for _ in range(length))
242             parts = split_capture_buffer(data)
243             self.assertEqual(sum(len(part[0]) for part in parts), len(data))
244             self.assertTrue(all(len(part[0]) > 0 for part in parts))
245
246     def test_request_response_pairing_and_response_time(self):
247         req = parse_burst(
248             append_crc(bytes.fromhex("01 03 00 00 00 01")),
249             "12:00:00.000",
250             "BUS",
251             100.000,
252         )[0]
253         resp = parse_burst(
254             append_crc(bytes.fromhex("01 03 02 00 2A")),
255             "12:00:00.125",
256             "BUS",
257             100.125,
258         )[0]
259         req["_view_seq"] = 10
260         resp["_view_seq"] = 11
261
262         metrics = SessionMetrics(pending_timeout_seconds=1.0)
263         first = metrics.process(req)
264         second = metrics.process(resp)
265
266         self.assertEqual(first["pending"], 1)
267         self.assertEqual(second["pending"], 0)
268         self.assertEqual(second["matched_request"]["seq"], 10)
269         self.assertAlmostEqual(second["response_ms"], 125.0, places=6)
270
271     def test_timeout_regression(self):
272         req = parse_burst(
273             append_crc(bytes.fromhex("05 04 00 00 00 01")),
274             "12:00:00.000",
275             "BUS",
276             10.0,
277         )[0]
278         req["_view_seq"] = 77
279         metrics = SessionMetrics(pending_timeout_seconds=1.0)
280         metrics.process(req)
```

### MBSniffer v3.2/tests/test_regression.py — lines 281-320

```
281         self.assertEqual(metrics.expire_pending(11.1), 1)
282         expired = metrics.pop_expired_requests()
283         self.assertEqual(expired[0]["seq"], 77)
284         self.assertEqual(metrics.timeouts, 1)
285
286
287 class SlaveFinderRegressionTests(unittest.TestCase):
288     def test_normal_discovery_response(self):
289         response = append_crc(bytes.fromhex("04 03 02 12 34"))
290         found = find_discovery_response(response, 4, 3)
291         self.assertIsNotNone(found)
292         self.assertEqual(found["kind"], "RESPONSE")
293
294     def test_discovery_exception_counts_as_response(self):
295         response = append_crc(bytes.fromhex("04 83 02"))
296         found = find_discovery_response(response, 4, 3)
297         self.assertIsNotNone(found)
298         self.assertEqual(found["kind"], "EXCEPTION")
299         self.assertEqual(found["exception_code"], 2)
300
301     def test_device_identification_request(self):
302         request = build_device_identification_probe(4)
303         self.assertEqual(request[:5], bytes.fromhex("04 2B 0E 01 00"))
304         self.assertEqual(len(request), 7)
305         self.assertTrue(crc_ok(request))
306
307     def test_device_identification_response_parser(self):
308         response = device_id_response(slave=4)
309         found = find_device_identification_response(response, 4)
310         self.assertIsNotNone(found)
311         self.assertEqual(found["kind"], "DEVICE_ID")
312         self.assertEqual(found["objects"][0x00], "Schneider Electric")
313         self.assertEqual(found["objects"][0x01], "TEST-PRODUCT")
314         self.assertEqual(found["objects"][0x02], "1.2.3")
315         self.assertEqual(
316             format_device_identification(found),
317             "Schneider Electric | TEST-PRODUCT | 1.2.3",
318         )
319
320     def test_device_identification_parser_ignores_request_echo(self):
```

**MBSniffer v3.2/tests/test_regression.py — lines 321-360**

```
        request = build_device_identification_probe(4)
        response = device_id_response(slave=4)
        found = find_device_identification_response(request + response, 4)
        self.assertIsNotNone(found)
        self.assertEqual(found["objects"][0x01], "TEST-PRODUCT")

    def test_device_identification_exception(self):
        response = append_crc(bytes.fromhex("04 AB 01"))
        found = find_device_identification_response(response, 4)
        self.assertIsNotNone(found)
        self.assertEqual(found["kind"], "EXCEPTION")
        self.assertEqual(found["exception_code"], 1)
        self.assertIn("Illegal Function", format_device_identification(found))

    def test_device_identification_rejects_bad_crc(self):
        response = with_bad_crc(device_id_response(slave=4))
        self.assertIsNone(find_device_identification_response(response, 4))

    def test_device_identification_follows_more_follows(self):
        page0 = device_id_response(
            slave=4,
            objects={0x00: b"Vendor"},
            more_follows=True,
            next_object_id=0x01,
        )
        page1 = device_id_response(
            slave=4,
            objects={0x01: b"Product", 0x02: b"2.0"},
            more_follows=False,
            next_object_id=0x00,
        )
        serial = FakeSerial({0x00: page0, 0x01: page1})
        result = read_basic_device_identification(
            serial,
            4,
            115200,
            "None",
            "1",
            10,
            threading.Event(),
```

### MBSniffer v3.2/tests/test_regression.py — lines 361-400

```
361             FakeSerialModule,
362         )
363         self.assertIsNotNone(result)
364         self.assertEqual(result["objects"][0x00], "Vendor")
365         self.assertEqual(result["objects"][0x01], "Product")
366         self.assertEqual(result["objects"][0x02], "2.0")
367         self.assertEqual(len(serial.writes), 2)
368         self.assertEqual(serial.writes[0][4], 0x00)
369         self.assertEqual(serial.writes[1][4], 0x01)
370
371
372 class InternationalizationRegressionTests(unittest.TestCase):
373     def test_language_normalization_defaults_to_portuguese(self):
374         self.assertEqual(normalize_language("invalid"), LANGUAGE_PORTUGUESE)
375         self.assertEqual(normalize_language(LANGUAGE_ENGLISH), LANGUAGE_ENGLISH)
376
377     def test_static_ui_translation_is_reversible(self):
378         self.assertEqual(translate_text("Configuração", LANGUAGE_ENGLISH), "Configuration")
379         self.assertEqual(translate_text("Configuration", LANGUAGE_PORTUGUESE), "Configuração")
380         self.assertEqual(translate_text("Frame gap", LANGUAGE_ENGLISH), "Frame gap")
381         # "None" is a technical parity value and must remain English in the PT UI.
382         self.assertEqual(translate_text("None", LANGUAGE_PORTUGUESE), "None")
383         self.assertEqual(translate_text("Nenhum", LANGUAGE_ENGLISH), "None")
384
385     def test_dynamic_status_translation_is_reversible(self):
386         pt = "Pesquisa concluída — 7 encontrados — 00:12"
387         en = "Scan complete — 7 found — 00:12"
388         self.assertEqual(translate_runtime_text(pt, LANGUAGE_ENGLISH), en)
389         self.assertEqual(translate_runtime_text(en, LANGUAGE_PORTUGUESE), pt)
390
391     def test_register_details_follow_language(self):
392         self.assertEqual(
393             translate_details("ByteCount=2  Registos=[42]", LANGUAGE_ENGLISH),
394             "ByteCount=2  Registers=[42]",
395         )
396         self.assertEqual(
397             translate_details("ByteCount=2  Registers=[42]", LANGUAGE_PORTUGUESE),
398             "ByteCount=2  Registos=[42]",
399         )
400
```

**MBSniffer v3.2/tests/test_regression.py — lines 401-440**

```
401     def test_inspector_english_labels_and_crc(self):
402         frame = append_crc(bytes.fromhex("01 03 02 00 2A"))
403         row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
404         row.update({"response_text": "12.3", "matched": True})
405         values = inspector_values(row, language=LANGUAGE_ENGLISH)
406         self.assertTrue(values["type"].startswith("Type:"))
407         self.assertTrue(values["result"].startswith("Result:"))
408         self.assertTrue(values["data"].startswith("Data:"))
409         self.assertIn("Registers=[42]", values["data"])
410         self.assertIn("received", values["crc"])
411         self.assertIn("calculated", values["crc"])
412 
413     def test_english_help_contains_no_portuguese_diacritics(self):
414         text = "".join(
415             block for _tag, block in english_help_blocks(
416                 "20 000", "20 000", 1.0, 300.0, debug_sim=True
417             )
418         )
419         for character in "ãõçáéíóúâê":
420             self.assertNotIn(character, text.casefold())
421         self.assertIn("Bus Slave Finder — active scan", text)
422         self.assertIn("Device Identification (FC43/14)", text)
423 
424 
425 def bitwise_crc16(data: bytes) -> int:
426     """Reference CRC used up to v3.0, kept to verify the table-driven CRC."""
427     crc = 0xFFFF
428     for b in data:
429         crc ^= b
430         for _ in range(8):
431             crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
432     return crc
433 
434 
435 def process_frames(metrics, frames):
436     """Run (bytes, channel, epoch) frames through SessionMetrics."""
437     out = []
438     for index, (frame, channel, epoch) in enumerate(frames):
439         for row in parse_burst(frame, "12:00:00.000", channel, epoch):
440             row["_view_seq"] = index
```

### MBSniffer v3.2/tests/test_regression.py — lines 441-480

```
441             out.append((row, metrics.process(row)))
442     return out
443
444
445 class ProtocolV31RegressionTests(unittest.TestCase):
446     def test_crc_table_matches_bitwise_reference(self):
447         rng = random.Random(31)
448         for length in (0, 1, 2, 7, 64, 256):
449             data = bytes(rng.randrange(256) for _ in range(length))
450             self.assertEqual(crc16_modbus(data), bitwise_crc16(data))
451
452     def test_broadcast_request_never_times_out(self):
453         request = append_crc(bytes.fromhex("00 10 00 00 00 01 02 00 05"))
454         metrics = SessionMetrics(pending_timeout_seconds=1.0)
455         process_frames(metrics, [(request, "BUS", 10.0)])
456         self.assertEqual(metrics.requests, 1)
457         self.assertEqual(metrics.broadcast_requests, 1)
458         self.assertEqual(metrics.pending_count(), 0)
459         metrics.expire_pending(20.0)
460         self.assertEqual(metrics.timeouts, 0)
461         self.assertEqual(metrics.health_summary(now=20.0)["timeout_rate_pct"], 0.0)
462
463     def test_fc06_echo_pair_is_request_then_response(self):
464         write = append_crc(bytes.fromhex("01 06 00 01 00 09"))
465         metrics = SessionMetrics(pending_timeout_seconds=1.0)
466         results = process_frames(
467             metrics, [(write, "BUS", 100.0), (write, "BUS", 100.040)]
468         )
469         self.assertEqual(
470             [row["type"] for row, _timing in results], ["REQUEST", "RESPONSE"]
471         )
472         self.assertEqual((metrics.requests, metrics.responses), (1, 1))
473         self.assertAlmostEqual(results[1][1]["response_ms"], 40.0, places=6)
474
475     def test_unanswered_fc05_write_times_out(self):
476         write = append_crc(bytes.fromhex("02 05 00 10 FF 00"))
477         metrics = SessionMetrics(pending_timeout_seconds=1.0)
478         results = process_frames(metrics, [(write, "BUS", 5.0)])
479         self.assertEqual(results[0][0]["type"], "REQUEST")
480         self.assertEqual(metrics.expire_pending(6.5), 1)
```

**MBSniffer v3.2/tests/test_regression.py — lines 481-520**

```
481
482     def test_different_fc06_writes_are_both_requests(self):
483         first = append_crc(bytes.fromhex("01 06 00 01 00 09"))
484         second = append_crc(bytes.fromhex("01 06 00 02 00 0A"))
485         metrics = SessionMetrics()
486         results = process_frames(
487             metrics, [(first, "BUS", 1.0), (second, "BUS", 1.1)]
488         )
489         self.assertEqual(
490             [row["type"] for row, _timing in results], ["REQUEST", "REQUEST"]
491         )
492
493     def test_dual_rx_echo_must_come_from_opposite_channel(self):
494         write = append_crc(bytes.fromhex("01 06 00 01 00 09"))
495         metrics = SessionMetrics()
496         results = process_frames(metrics, [
497             (write, "A→B", 1.0),
498             (write, "A→B", 1.5),
499             (write, "B→A", 1.52),
500         ])
501         self.assertEqual(
502             [row["type"] for row, _timing in results],
503             ["REQUEST", "REQUEST", "RESPONSE"],
504         )
505
506     def test_split_frame_is_held_until_complete(self):
507         response = append_crc(bytes.fromhex("01 03 08") + bytes(8))
508         self.assertTrue(burst_may_continue(response[:7]))
509         self.assertTrue(burst_may_continue(response[:1]))
510         self.assertFalse(burst_may_continue(response))
511         self.assertFalse(burst_may_continue(bytes((0xFF,) * 6)))
512
513     def test_merged_frames_keep_their_chunk_arrival_times(self):
514         request = append_crc(bytes.fromhex("01 03 00 00 00 01"))
515         response = append_crc(bytes.fromhex("01 03 02 00 2A"))
516         rows = parse_burst(
517             request + response,
518             "12:00:00.000",
519             "BUS",
520             100.0,
```

### MBSniffer v3.2/tests/test_regression.py — lines 521-560

```
521             chunk_times=[(0, 100.0), (len(request), 100.030)],
522         )
523         self.assertEqual([row["type"] for row in rows], ["REQUEST", "RESPONSE"])
524         self.assertAlmostEqual(rows[1]["_epoch"], 100.030, places=6)
525         metrics = SessionMetrics()
526         metrics.process(rows[0])
527         self.assertAlmostEqual(metrics.process(rows[1])["response_ms"], 30.0, places=6)
528 
529     def test_read_bits_details_use_request_qty(self):
530         response = append_crc(bytes.fromhex("01 01 02 CD 01"))
531         self.assertEqual(
532             read_bits_details(response),
533             "ByteCount=2  Bits=10110011 10000000",
534         )
535         self.assertEqual(
536             read_bits_details(response, qty=10),
537             "ByteCount=2  Bits=10110011 10",
538         )
539 
540 
541 class _ChunkedSerial:
542     """Delivers byte chunks at fixed delays, like a USB latency timer."""
543 
544     def __init__(self, chunks, stop_event):
545         self.started = time.perf_counter()
546         self.chunks = list(chunks)  # [(delay_s, bytes)]
547         self.stop_event = stop_event
548 
549     def _due(self):
550         elapsed = time.perf_counter() - self.started
551         if self.chunks and elapsed >= self.chunks[0][0]:
552             return self.chunks[0][1]
553         if not self.chunks and elapsed > 0.3:
554             self.stop_event.set()
555         return b""
556 
557     @property
558     def in_waiting(self):
559         return len(self._due())
560 
```

### MBSniffer v3.2/tests/test_regression.py — lines 561-600

```
561     def read(self, size):
562         data = self._due()
563         if data:
564             self.chunks.pop(0)
565         return data
566 
567     def close(self):
568         pass
569 
570 
571 class _ReaderStub(CaptureMixin):
572     def __init__(self):
573         self.stop_event = threading.Event()
574         self.event_queue = queue.Queue()
575 
576 
577 class ReaderV31RegressionTests(unittest.TestCase):
578     def test_frame_split_by_usb_latency_is_reassembled(self):
579         response = append_crc(bytes.fromhex("01 03 08") + bytes(8))
580         reader = _ReaderStub()
581         serial = _ChunkedSerial(
582             [(0.0, response[:7]), (0.016, response[7:])], reader.stop_event
583         )
584         # 115200 bit/s: Auto t3.5 is 1.75 ms, far shorter than the 16 ms split.
585         reader.reader_loop(serial, 1.75, "BUS", 1)
586         rows = []
587         while not reader.event_queue.empty():
588             name, payload = reader.event_queue.get()
589             if name == "frame":
590                 rows.append(payload)
591         self.assertEqual([(row["type"], row["crc"]) for row in rows], [("RESPONSE", "OK")])
592 
593 
594 class _FakeVar:
595     def __init__(self, value):
596         self.value = value
597 
598     def get(self):
599         return self.value
600 
```

**MBSniffer v3.2/tests/test_regression.py — lines 601-640**

```
601
602 class _FakeTree:
603     def __init__(self):
604         self.tags = {}
605
606     def item(self, iid, tags=()):
607         self.tags[iid] = tags
608
609
610 class _TimeoutView(ViewMixin):
611     def __init__(self, type_filter):
612         self.tree = _FakeTree()
613         self.advanced_filter_type_var = _FakeVar(type_filter)
614         self.highlight_anomalies_var = _FakeVar(True)
615         self._successful_request_seqs = set()
616         self._frame_history = deque([{
617             "seq": 5, "type": "REQUEST", "crc": "OK", "timed_out": False,
618             "response_text": "",
619         }])
620         self._tree_iids_by_seq = {5: ["I001", "I002"]}
621         self.full_renders = 0
622
623     def render_filtered_view(self):
624         self.full_renders += 1
625
626     def selected_record(self):
627         return None
628
629
630 class ViewV31RegressionTests(unittest.TestCase):
631     def test_timeout_retags_rows_without_full_rebuild(self):
632         view = _TimeoutView("Todos")
633         self.assertEqual(view.mark_timed_out_records([{"seq": 5}]), 1)
634         self.assertEqual(view.full_renders, 0)
635         self.assertEqual(view.tree.tags["I001"], ("anomaly_timeout",))
636         self.assertEqual(view.tree.tags["I002"], ("anomaly_timeout",))
637
638     def test_timeouts_filter_still_rebuilds_view(self):
639         view = _TimeoutView("Timeouts")
640         view.mark_timed_out_records([{"seq": 5}])
```

### MBSniffer v3.2/tests/test_regression.py — lines 641-680

```
641         self.assertEqual(view.full_renders, 1)
642
643     def test_inspector_shows_address_of_each_register(self):
644         frame = append_crc(bytes.fromhex("01 03 04 00 2A 00 2B"))
645         row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
646         row["request_pdu_address"] = 100
647         pt = inspector_values(row, language=LANGUAGE_PORTUGUESE)
648         en = inspector_values(row, language=LANGUAGE_ENGLISH)
649         self.assertIn("Registos (PDU Address=valor): 100=42, 101=43", pt["data"])
650         self.assertIn("Registers (PDU Address=value): 100=42, 101=43", en["data"])
651
652     def test_inspector_shows_address_of_each_bit(self):
653         frame = append_crc(bytes.fromhex("01 02 01 05"))
654         row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
655         row.update({"request_pdu_address": 8, "request_qty": 3})
656         values = inspector_values(row, language=LANGUAGE_PORTUGUESE)
657         self.assertIn("Bits (PDU Address=valor): 8=1, 9=0, 10=1", values["data"])
658
659
660 class _FakeFinderSerial:
661     def __init__(self, incoming=b""):
662         self.incoming = bytearray(incoming)
663         self.writes = []
664
665     @property
666     def in_waiting(self):
667         return len(self.incoming)
668
669     def read(self, size):
670         data = bytes(self.incoming[:size])
671         del self.incoming[:size]
672         return data
673
674     def write(self, data):
675         self.writes.append(bytes(data))
676         return len(data)
677
678     def reset_input_buffer(self):
679         pass
680
```

### MBSniffer v3.2/tests/test_regression.py — lines 681-720

```
681     def reset_output_buffer(self):
682         pass
683 
684     def close(self):
685         pass
686 
687 
688 class _FinderStub(SlaveFinderMixin):
689     current_language = LANGUAGE_PORTUGUESE
690 
691     def __init__(self, fake_serial):
692         self.fake_serial = fake_serial
693         self.event_queue = queue.Queue()
694         self.discovery_stop_event = threading.Event()
695         self.discovery_serial = None
696 
697     def tr(self, text):
698         return translate_text(text, self.current_language)
699 
700     def trf(self, text, **values):
701         return self.tr(text).format(**values)
702 
703     def open_discovery_serial_port(self, serial, port, baud, parity, stopbits):
704         return self.fake_serial
705 
706     def run_worker(self, slaves, attempts):
707         self.discovery_worker(
708             None, "COM9", [(115200, "None", "1")], slaves, 10, False, False,
709             attempts,
710         )
711         events = []
712         while not self.event_queue.empty():
713             events.append(self.event_queue.get())
714         return events
715 
716 
717 class FinderV31RegressionTests(unittest.TestCase):
718     def setUp(self):
719         self._idle_check = mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS
720         mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS = 0.05
```

**MBSniffer v3.2/tests/test_regression.py — lines 721-760**

```
721
722     def tearDown(self):
723         mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS = self._idle_check
724
725     def test_each_silent_slave_gets_all_attempts(self):
726         stub = _FinderStub(_FakeFinderSerial())
727         events = stub.run_worker([1, 2], attempts=3)
728         done = [payload for name, payload in events if name == "discovery_done"][0]
729         self.assertIsNone(done["error"])
730         self.assertEqual(len(stub.fake_serial.writes), 6)
731
732     def test_traffic_before_scan_blocks_transmission(self):
733         stub = _FinderStub(_FakeFinderSerial(b"\x01\x03\x00"))
734         events = stub.run_worker([1], attempts=1)
735         done = [payload for name, payload in events if name == "discovery_done"][0]
736         self.assertIn("Foi detetado tráfego em COM9", done["error"])
737         self.assertEqual(stub.fake_serial.writes, [])
738
739     def test_legacy_default_timeout_migrates(self):
740         with tempfile.TemporaryDirectory() as folder:
741             path = Path(folder) / "settings.json"
742             original = mb_config.get_settings_file
743             mb_config.get_settings_file = lambda: path
744             try:
745                 path.write_text(
746                     json.dumps({"bus_slave_finder": {"min_timeout_ms": 40}}),
747                     encoding="utf-8",
748                 )
749                 finder = mb_config.load_ui_settings()["bus_slave_finder"]
750                 self.assertEqual(finder["min_timeout_ms"], "100")
751                 self.assertEqual(finder["attempts"], "1")
752
753                 path.write_text(
754                     json.dumps({"bus_slave_finder": {
755                         "min_timeout_ms": 40, "attempts": 2,
756                     }}),
757                     encoding="utf-8",
758                 )
759                 finder = mb_config.load_ui_settings()["bus_slave_finder"]
760                 self.assertEqual(finder["min_timeout_ms"], "40")
```

### MBSniffer v3.2/tests/test_regression.py — lines 761-787

```
                self.assertEqual(finder["attempts"], "2")
            finally:
                mb_config.get_settings_file = original


class SourceHygieneV31Tests(unittest.TestCase):
    def test_no_inline_language_branches(self):
        package = Path(__file__).resolve().parents[1]
        for source in package.glob("*.py"):
            text = source.read_text(encoding="utf-8")
            self.assertNotIn(
                'current_language == "English"', text, source.name
            )

    def test_message_templates_translate(self):
        self.assertEqual(
            translate_text("Encontrados: {count}", LANGUAGE_ENGLISH).format(count=3),
            "Found: 3",
        )
        self.assertEqual(
            translate_text("A verificar se o barramento está livre…", LANGUAGE_ENGLISH),
            "Checking that the bus is idle…",
        )


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

# MBSniffer v3.2/tests/test_master.py

563 source lines.

**MBSniffer v3.2/tests/test_master.py — lines 1-40**

```python
import json
import os
import queue
import struct
import tempfile
import time
import unittest
from pathlib import Path

import mb_config
import mb_master_ui
from mb_master import (
    BYTE_ORDERS,
    BYTE_ORDERS_64,
    format_value,
    format_window_values,
    raw_text_for_scaled,
    register_groups,
    validate_scale,
    MasterEngine,
    PollDefinition,
    build_read_request,
    build_write_request,
    encode_coil_value,
    encode_register_value,
    find_master_response,
    format_register_values,
)
from mb_master_ui import MasterMixin, MasterWindow, apply_format_to_rows
from mb_protocol import append_crc, crc_ok


class FakeSlaveSerial:
    """In-memory Modbus RTU slave behind a fake serial port.

    Answers FC01/03 reads and FC05/06/15/16 writes for ``slave_id`` from its
    coil/register banks, can echo every request like some RS485 adapters, and
    can hold bytes that are already on the bus before the master transmits.
    """

```

**MBSniffer v3.2/tests/test_master.py — lines 41-80**

```
41      def __init__(self, slave_id=1, *, echo=False, bus_traffic=b"", exception_for=None):
42          self.slave_id = slave_id
43          self.echo = echo
44          self.exception_for = exception_for or {}
45          self.registers = {i: i * 10 for i in range(200)}
46          self.coils = {i: i % 2 for i in range(200)}
47          self.rx = bytearray(bus_traffic)
48          self.requests = []
49
50      @property
51      def in_waiting(self):
52          return len(self.rx)
53
54      def read(self, size):
55          data = bytes(self.rx[:size])
56          del self.rx[:size]
57          return data
58
59      def reset_input_buffer(self):
60          self.rx.clear()
61
62      def reset_output_buffer(self):
63          pass
64
65      def cancel_read(self):
66          pass
67
68      def cancel_write(self):
69          pass
70
71      def close(self):
72          pass
73
74      def write(self, data):
75          data = bytes(data)
76          self.requests.append(data)
77          if self.echo:
78              self.rx.extend(data)
79          if crc_ok(data) and data[0] == self.slave_id:
80              self.rx.extend(self._answer(data))
```

**MBSniffer v3.2/tests/test_master.py — lines 81-120**

```
        elif crc_ok(data) and data[0] == 0:
            self._answer(data)  # broadcast: execute, never answer
        return len(data)

    def _answer(self, req):
        fc = req[1]
        if fc in self.exception_for:
            return append_crc(bytes((req[0], fc | 0x80, self.exception_for[fc])))
        address = (req[2] << 8) | req[3]
        qty = (req[4] << 8) | req[5]
        if fc == 3:
            payload = b"".join(struct.pack(">H", self.registers.get(address + i, 0))
                               for i in range(qty))
            return append_crc(bytes((req[0], 3, len(payload))) + payload)
        if fc == 1:
            data = bytearray((qty + 7) // 8)
            for i in range(qty):
                if self.coils.get(address + i):
                    data[i // 8] |= 1 << (i % 8)
            return append_crc(bytes((req[0], 1, len(data))) + bytes(data))
        if fc == 6:
            self.registers[address] = qty
            return req
        if fc == 5:
            self.coils[address] = 1 if qty == 0xFF00 else 0
            return req
        if fc == 16:
            for i in range(qty):
                self.registers[address + i] = (req[7 + 2 * i] << 8) | req[8 + 2 * i]
            return append_crc(req[:6])
        if fc == 15:
            for i in range(qty):
                self.coils[address + i] = (req[7 + i // 8] >> (i % 8)) & 1
            return append_crc(req[:6])
        return append_crc(bytes((req[0], fc | 0x80, 1)))


def drain(event_queue):
    events = []
    while not event_queue.empty():
```

**MBSniffer v3.2/tests/test_master.py — lines 121-160**

```
121         events.append(event_queue.get())
122     return events
123
124
125 def make_engine(serial, **overrides):
126     options = dict(baud=115200, parity="None", stopbits="1",
127                    response_timeout_ms=30, delay_ms=0)
128     options.update(overrides)
129     return MasterEngine(serial, queue.Queue(), **options)
130
131
132 class MasterFormatTests(unittest.TestCase):
133     def test_32_bit_formats_round_trip_in_every_byte_order(self):
134         for order in BYTE_ORDERS:
135             for kind, text in (("int32", "-123456"), ("uint32", "4000000000"),
136                                ("float32", "123.456")):
137                 fmt = f"{kind}_{order}"
138                 registers = encode_register_value(text, fmt)
139                 self.assertEqual(len(registers), 2)
140                 self.assertEqual(format_register_values(registers, fmt)[0], text)
141
142     def test_float_byte_orders_match_wire_layout(self):
143         big_endian = struct.pack(">f", 1.5)  # 3F C0 00 00 = A B C D
144         self.assertEqual(encode_register_value("1.5", "float32_ABCD"), [0x3FC0, 0x0000])
145         self.assertEqual(encode_register_value("1.5", "float32_CDAB"), [0x0000, 0x3FC0])
146         self.assertEqual(encode_register_value("1.5", "float32_BADC"), [0xC03F, 0x0000])
147         self.assertEqual(encode_register_value("1.5", "float32_DCBA"), [0x0000, 0xC03F])
148         self.assertEqual(big_endian, bytes.fromhex("3FC00000"))
149
150     def test_16_bit_formats(self):
151         self.assertEqual(format_register_values([0xFFFE], "int16"), ["-2"])
152         self.assertEqual(format_register_values([0xFFFE], "uint16"), ["65534"])
153         self.assertEqual(format_register_values([0x00AB], "hex16"), ["0x00AB"])
154         self.assertEqual(format_register_values([0x0105], "bin16"), ["00000001 00000101"])
155         self.assertEqual(encode_register_value("-2", "int16"), [0xFFFE])
156         self.assertEqual(encode_register_value("0x1A2B", "hex16"), [0x1A2B])
157         self.assertEqual(encode_register_value("0000 0001 0000 0101", "bin16"), [0x0105])
158
159     def test_values_outside_the_format_are_rejected(self):
160         for text, fmt in (("40000", "int16"), ("-1", "uint16"), ("abc", "float32_ABCD"),
```

**MBSniffer v3.2/tests/test_master.py — lines 161-200**

```
                          ("1e40", "float32_ABCD"), ("", "int16")):
            with self.assertRaises(ValueError):
                encode_register_value(text, fmt)
        with self.assertRaises(ValueError):
            encode_coil_value("2")

    def test_unpaired_register_in_32_bit_format(self):
        self.assertEqual(format_register_values([1, 2, 3], "uint32_ABCD"), ["65538", "", "—"])


class MasterFrameTests(unittest.TestCase):
    def test_write_frames_match_modbus_specification_examples(self):
        # Modbus Application Protocol V1.1b3 examples (CRC appended for RTU).
        self.assertEqual(build_write_request(0x11, 16, 1, [0x000A, 0x0102])[:-2],
                         bytes.fromhex("11 10 0001 0002 04 000A 0102"))
        coils = [1, 0, 1, 1, 0, 0, 1, 1, 1, 0]
        self.assertEqual(build_write_request(0x11, 15, 0x13, coils)[:-2],
                         bytes.fromhex("11 0F 0013 000A 02 CD 01"))
        self.assertEqual(build_write_request(0x11, 5, 0xAC, [1])[:-2],
                         bytes.fromhex("11 05 00AC FF00"))
        self.assertEqual(build_write_request(0x11, 6, 1, [3])[:-2],
                         bytes.fromhex("11 06 0001 0003"))

    def test_invalid_definitions_are_rejected(self):
        for args in ((0, 3, 0, 1), (1, 7, 0, 1), (1, 3, 0, 126), (1, 1, 0, 2001),
                     (1, 3, 65535, 2)):
            with self.assertRaises(ValueError):
                build_read_request(*args)
        with self.assertRaises(ValueError):
            build_write_request(1, 6, 0, [1, 2])

    def test_read_response_found_after_adapter_echo(self):
        request = build_read_request(1, 3, 0, 2)
        response = append_crc(bytes.fromhex("01 03 04 0001 0002"))
        found = find_master_response(request + response, request)
        self.assertEqual(found["kind"], "RESPONSE")
        self.assertEqual(found["values"], [1, 2])

    def test_coil_response_is_trimmed_to_quantity(self):
        request = build_read_request(1, 1, 0, 3)
```

### MBSniffer v3.2/tests/test_master.py — lines 201-240

```
201         found = find_master_response(append_crc(bytes.fromhex("01 01 01 FD")), request)
202         self.assertEqual(found["values"], [1, 0, 1])
203 
204     def test_exception_and_bad_crc(self):
205         request = build_read_request(1, 3, 0, 1)
206         exception = append_crc(bytes.fromhex("01 83 02"))
207         self.assertEqual(find_master_response(exception, request)["exception_code"], 2)
208         damaged = bytearray(append_crc(bytes.fromhex("01 03 02 0001")))
209         damaged[-1] ^= 0xFF
210         self.assertIsNone(find_master_response(bytes(damaged), request))
211 
212     def test_write_response_must_match_request(self):
213         request = build_write_request(1, 16, 5, [1, 2])
214         self.assertIsNotNone(find_master_response(append_crc(request[:6]), request))
215         other = append_crc(bytes.fromhex("01 10 0006 0002"))
216         self.assertIsNone(find_master_response(other, request))
217 
218 
219 class MasterEngineTests(unittest.TestCase):
220     def run_engine(self, engine, seconds):
221         engine.start()
222         time.sleep(seconds)
223         engine.stop()
224         return drain(engine.event_queue)
225 
226     def test_scheduler_serves_writes_then_read_once_then_due_polls(self):
227         engine = make_engine(FakeSlaveSerial())
228         engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 1000))
229         engine.set_poll(PollDefinition(2, 1, 3, 0, 1, 1000))
230         engine.read_once(2)
231         engine.queue_write(1, build_write_request(1, 6, 0, [5]))
232         self.assertEqual(engine.next_job(now=10.0)[0], "write")
233         self.assertEqual(engine.next_job(now=10.0)[1:], (engine._polls[2][0], True))
234         first = engine.next_job(now=10.0)
235         second = engine.next_job(now=10.0)
236         self.assertEqual({first[1].window_id, second[1].window_id}, {1, 2})
237         self.assertEqual(engine.next_job(now=10.5)[0], "wait")
238         self.assertEqual(engine.next_job(now=11.01)[0], "read")
239 
240     def test_fast_and_slow_windows_share_one_bus(self):
```

### MBSniffer v3.2/tests/test_master.py — lines 241-280

```
241         serial = FakeSlaveSerial()
242         engine = make_engine(serial)
243         engine.set_poll(PollDefinition(1, 1, 3, 0, 4, 0))
244         engine.set_poll(PollDefinition(2, 1, 1, 0, 8, 10000))
245         events = self.run_engine(engine, 0.4)
246         results = [p for name, p in events if name == "master_result"]
247         fast = [r for r in results if r["window_id"] == 1]
248         slow = [r for r in results if r["window_id"] == 2]
249         self.assertGreater(len(fast), 3)
250         self.assertEqual(len(slow), 1)
251         self.assertEqual(fast[0]["values"], [0, 10, 20, 30])
252         self.assertEqual(slow[0]["values"], [0, 1, 0, 1, 0, 1, 0, 1])
253         self.assertEqual(events[0][0], "master_ready")
254         self.assertEqual(events[-1], ("master_stopped", {"error": None}))
255 
256     def test_write_is_executed_and_reported(self):
257         serial = FakeSlaveSerial(echo=True)
258         engine = make_engine(serial)
259         engine.queue_write(1, build_write_request(1, 16, 10, [0xABCD, 0x1234]), "FC16")
260         results = [p for n, p in self.run_engine(engine, 0.15) if n == "master_result"]
261         self.assertEqual(results[0]["status"], "ok")
262         self.assertEqual((serial.registers[10], serial.registers[11]), (0xABCD, 0x1234))
263 
264     def test_broadcast_write_does_not_wait_for_a_response(self):
265         serial = FakeSlaveSerial()
266         engine = make_engine(serial, response_timeout_ms=5000)
267         engine.queue_write(1, build_write_request(0, 6, 3, [77]))
268         started = time.perf_counter()
269         result = engine.transact(engine._writes.popleft()[1])
270         self.assertLess(time.perf_counter() - started, 1.0)
271         self.assertEqual(result["status"], "broadcast")
272         self.assertEqual(serial.registers[3], 77)
273 
274     def test_timeout_exception_and_invalid_results(self):
275         engine = make_engine(FakeSlaveSerial(slave_id=9))
276         self.assertEqual(engine.transact(build_read_request(1, 3, 0, 1))["status"], "timeout")
277         engine = make_engine(FakeSlaveSerial(exception_for={3: 2}))
278         result = engine.transact(build_read_request(1, 3, 0, 1))
279         self.assertEqual((result["status"], result["exception_code"]), ("exception", 2))
280 
```

### MBSniffer v3.2/tests/test_master.py — lines 281-320

```
281     def test_traffic_on_the_bus_blocks_the_master(self):
282         serial = FakeSlaveSerial(bus_traffic=b"\x01\x03\x00")
283         engine = make_engine(serial, idle_check_seconds=0.05)
284         engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
285         events = self.run_engine(engine, 0.2)
286         self.assertEqual(serial.requests, [])
287         self.assertEqual(events[-1][0], "master_stopped")
288         self.assertEqual(events[-1][1]["error"][0], "traffic")
289 
290 
291 class MasterWindowTests(unittest.TestCase):
292     def test_workspace_entry_round_trip_and_validation(self):
293         window = MasterWindow(1, 1, slave=5, fc=3, address=100, qty=4, scan_ms=250, base=1,
294                               fmt="float32_CDAB", names={100: "Temperatura"})
295         data = MasterWindow.validated_kwargs(window.to_json())
296         self.assertEqual(data["names"], {100: "Temperatura"})
297         self.assertEqual((data["slave"], data["fmt"], data["base"]), (5, "float32_CDAB", 1))
298         for bad in ({"slave": 0}, {"fc": 6}, {"qty": 200}, {"scan_ms": -1}, "x"):
299             with self.assertRaises((ValueError, TypeError)):
300                 MasterWindow.validated_kwargs(bad)
301 
302     def test_write_dialog_builds_fc16_for_float_values(self):
303         window = MasterWindow(1, 1, slave=2, fc=3, address=0, qty=10, base=1,
304                               fmt="float32_ABCD")
305         request, _ = MasterMixin.master_build_write(None, window, "2", "11", "1.5;2", 16)
306         self.assertEqual(request[:-2], bytes.fromhex("02 10 000A 0004 08 3FC0 0000 4000 0000"))
307         with self.assertRaises(ValueError):
308             MasterMixin.master_build_write(None, window, "2", "11", "1.5", 6)
309 
310 
311 class MasterWorkspaceFolderTests(unittest.TestCase):
312     def setUp(self):
313         self._appdata = os.environ.get("APPDATA")
314         self._folder = tempfile.TemporaryDirectory()
315         os.environ["APPDATA"] = self._folder.name
316 
317     def tearDown(self):
318         if self._appdata is None:
319             os.environ.pop("APPDATA", None)
320         else:
```

### MBSniffer v3.2/tests/test_master.py — lines 321-360

```
            os.environ["APPDATA"] = self._appdata
        self._folder.cleanup()

    def test_workspaces_live_beside_the_settings(self):
        folder = mb_config.get_workspaces_folder()
        self.assertEqual(folder.name, "Workspaces")
        self.assertEqual(folder.parent, mb_config.get_settings_file().parent)
        self.assertTrue(folder.is_dir())

    def test_saved_workspaces_are_listed_by_name(self):
        for name in ("Subestação B", "armazém", "Linha 1"):
            mb_master_ui.workspace_path(name).write_text("{}", encoding="utf-8")
        (mb_config.get_workspaces_folder() / "notas.txt").write_text("x", encoding="utf-8")
        self.assertEqual(mb_master_ui.list_workspaces(),
                         ["armazém", "Linha 1", "Subestação B"])

    def test_workspace_names_must_be_valid_file_names(self):
        self.assertEqual(mb_master_ui.validate_workspace_name("  Linha 1. "), "Linha 1")
        for bad in ("", "   ", "a/b", "x:y", "CON", "com3", "a" * 61, 'q"'):
            with self.assertRaises(ValueError):
                mb_master_ui.validate_workspace_name(bad)

    def test_window_title_is_saved_in_the_workspace(self):
        window = MasterWindow(1, 2, title="  Inversor 3  ")
        self.assertEqual(window.title, "Inversor 3")
        self.assertEqual(MasterWindow.validated_kwargs(window.to_json())["title"], "Inversor 3")
        self.assertEqual(MasterWindow.validated_kwargs({"slave": 1})["title"], "")


class MasterPerAddressFormatTests(unittest.TestCase):
    def test_64_bit_formats_round_trip_in_every_byte_order(self):
        for order in BYTE_ORDERS_64:
            for kind, text in (("int64", "-1234567890123"), ("uint64", "18000000000000000000"),
                               ("float64", "123456.789012345")):
                fmt = f"{kind}_{order}"
                registers = encode_register_value(text, fmt)
                self.assertEqual(len(registers), 4)
                self.assertEqual(format_value(registers, fmt), text)

    def test_double_byte_orders_match_wire_layout(self):
```

### MBSniffer v3.2/tests/test_master.py — lines 361-400

```
361         # 1.0 = 3FF0 0000 0000 0000 in IEEE 754 big-endian.
362         self.assertEqual(encode_register_value("1", "float64_ABCDEFGH"), [0x3FF0, 0, 0, 0])
363         self.assertEqual(encode_register_value("1", "float64_GHEFCDAB"), [0, 0, 0, 0x3FF0])
364         self.assertEqual(encode_register_value("1", "float64_BADCFEHG"), [0xF03F, 0, 0, 0])
365         self.assertEqual(encode_register_value("1", "float64_HGFEDCBA"), [0, 0, 0, 0xF03F])
366 
367     def test_per_address_formats_override_the_window_format(self):
368         registers = [0x42F6, 0xE979, 0xFFFF, 0x0001, 0x0002, 7]
369         texts = format_window_values(registers, 100, "int16", {100: "float32_ABCD"})
370         self.assertEqual(texts, ["123.456", "", "-1", "1", "2", "7"])
371         self.assertEqual(register_groups(6, 100, "int16", {102: "int64_ABCDEFGH"}),
372                          [(0, 1, "int16"), (1, 1, "int16"), (2, 4, "int64_ABCDEFGH")])
373 
374     def test_default_multi_register_values_cut_by_an_override_show_a_dash(self):
375         texts = format_window_values([1, 2, 3, 4, 5], 0, "uint32_ABCD", {1: "int16"})
376         self.assertEqual(texts, ["—", "2", "196612", "", "—"])
377 
378     def test_selection_rules_for_multi_register_formats(self):
379         window = MasterWindow(1, 1, address=100, qty=10)
380         apply_format_to_rows(window, [2, 3, 4, 5], "float32_CDAB")
381         self.assertEqual(window.formats, {102: "float32_CDAB", 104: "float32_CDAB"})
382         apply_format_to_rows(window, [4, 5, 6, 7], "float64_ABCDEFGH")
383         self.assertEqual(window.formats, {102: "float32_CDAB", 104: "float64_ABCDEFGH"})
384         apply_format_to_rows(window, [0, 1], "hex16")
385         self.assertEqual(window.formats[100], "hex16")
386         self.assertEqual(window.formats[101], "hex16")
387         apply_format_to_rows(window, [4], None)
388         self.assertNotIn(104, window.formats)
389         for rows, fmt in (([1], "float32_ABCD"), ([1, 3], "float32_ABCD"),
390                           ([0, 1, 2], "float32_ABCD"), ([0, 1], "int64_ABCDEFGH"),
391                           ([8, 9, 10, 11], "int64_ABCDEFGH"), ([], "int16")):
392             with self.assertRaises(ValueError):
393                 apply_format_to_rows(window, rows, fmt)
394 
395     def test_formats_are_saved_and_invalid_ones_dropped(self):
396         window = MasterWindow(1, 1, address=0, qty=4, formats={0: "float32_ABCD"})
397         data = MasterWindow.validated_kwargs(window.to_json())
398         self.assertEqual(data["formats"], {0: "float32_ABCD"})
399         data = MasterWindow.validated_kwargs(
400             {"formats": {"0": "nonsense", "x": "int16", "70000": "int16", "2": "hex16"}})
```

### MBSniffer v3.2/tests/test_master.py — lines 401-440

```
401         self.assertEqual(data["formats"], {2: "hex16"})
402
403     def test_write_uses_the_format_of_the_clicked_value(self):
404         window = MasterWindow(1, 1, slave=3, address=0, qty=8, fmt="int16")
405         request, _ = MasterMixin.master_build_write(None, window, "3", "4", "1", 16,
406                                                     "float64_ABCDEFGH")
407         self.assertEqual(request[:-2], bytes.fromhex("03 10 0004 0004 08 3FF0 0000 0000 0000"))
408
409
410 class MasterScaleTests(unittest.TestCase):
411     SCALE = validate_scale("2000", "10000", "-180", "180", 1, "°")
412
413     def test_scaled_values_are_shown_in_engineering_units(self):
414         texts = format_window_values([2000, 6000, 10000, 0x00AB], 0, "int16", {3: "hex16"},
415                                      {0: self.SCALE, 1: self.SCALE, 2: self.SCALE,
416                                       3: self.SCALE})
417         self.assertEqual(texts, ["-180.0 °", "0.0 °", "180.0 °", "0x00AB"])
418
419     def test_scale_applies_to_multi_register_values(self):
420         registers = encode_register_value("50", "float32_ABCD")
421         scale = validate_scale(0, 100, 0, 10, 2, "bar")
422         self.assertEqual(format_window_values(registers, 7, "float32_ABCD", None, {7: scale}),
423                          ["5.00 bar", ""])
424
425     def test_written_engineering_values_are_converted_to_raw(self):
426         self.assertEqual(raw_text_for_scaled("90", self.SCALE, "int16"), "8000")
427         self.assertEqual(raw_text_for_scaled("-180", self.SCALE, "uint16"), "2000")
428         window = MasterWindow(1, 1, slave=1, address=0, qty=4,
429                               scales={0: self.SCALE})
430         request, _ = MasterMixin.master_build_write(None, window, "1", "0", "45", 6,
431                                                     "int16", self.SCALE)
432         self.assertEqual(request[:-2], bytes.fromhex("01 06 0000 1B58"))  # 7000
433
434     def test_invalid_scales_are_rejected(self):
435         for args in (("1", "1", "0", "10"), ("a", "1", "0", "1"), ("0", "1", "0", "1", "9"),
436                      ("0", "nan", "0", "1")):
437             with self.assertRaises(ValueError):
438                 validate_scale(*args)
439         with self.assertRaises(ValueError):
440             raw_text_for_scaled("5", validate_scale(0, 10, 3, 3), "int16")
```

**MBSniffer v3.2/tests/test_master.py — lines 441-480**

```
441
442     def test_scales_are_saved_and_invalid_ones_dropped(self):
443         window = MasterWindow(1, 1, scales={4: self.SCALE})
444         data = MasterWindow.validated_kwargs(window.to_json())
445         self.assertEqual(data["scales"], {4: self.SCALE})
446         data = MasterWindow.validated_kwargs({"scales": {
447             "1": {"raw": [0, 0], "eng": [0, 1]}, "x": self.SCALE, "2": "bad",
448             "3": {"raw": [0, 10], "eng": [0, 1], "decimals": 2, "unit": "A"}}})
449         self.assertEqual(list(data["scales"]), [3])
450
451
452 class MasterOpenFromTrafficTests(unittest.TestCase):
453     def definition(self, record):
454         return MasterMixin.master_definition_from_record(None, record)
455
456     def test_request_and_matched_response_give_the_read_definition(self):
457         request = {"type": "REQUEST", "slave": "5", "fc": "03", "pdu_address": 100, "qty": 4}
458         self.assertEqual(self.definition(request),
459                          {"slave": 5, "fc": 3, "address": 100, "qty": 4})
460         response = {"type": "RESPONSE", "slave": "5", "fc": "04",
461                     "request_pdu_address": 7, "request_qty": 2}
462         self.assertEqual(self.definition(response)["address"], 7)
463         exception = {"type": "EXCEPTION", "slave": "5", "fc": "83",
464                      "request_pdu_address": 1, "request_qty": 1}
465         self.assertEqual(self.definition(exception)["fc"], 3)
466
467     def test_frames_without_a_read_definition_are_refused(self):
468         for record in (
469             {"type": "REQUEST", "slave": "5", "fc": "06", "pdu_address": 1, "qty": None},
470             {"type": "RESPONSE", "slave": "5", "fc": "03"},
471             {"type": "RAW/UNPARSED", "slave": "", "fc": ""},
472             {"type": "REQUEST", "slave": "0", "fc": "03", "pdu_address": 1, "qty": 1},
473         ):
474             self.assertIsNone(self.definition(record))
475
476
477 class FlakySlaveSerial(FakeSlaveSerial):
478     """Fake slave whose port fails (like an unplugged USB adapter)."""
479
480     def __init__(self, failures, **kwargs):
```

### MBSniffer v3.2/tests/test_master.py — lines 481-520

```
        super().__init__(**kwargs)
        self.failures = failures
        self.closed = False

    def write(self, data):
        if self.failures:
            self.failures -= 1
            raise OSError("device disconnected")
        return super().write(data)

    def close(self):
        self.closed = True


class MasterReconnectTests(unittest.TestCase):
    def test_engine_reopens_the_port_after_a_failure(self):
        broken = FlakySlaveSerial(failures=1)
        fresh = FakeSlaveSerial()
        reopen_calls = []
        engine = make_engine(broken, reconnect_interval_s=0.05,
                             reopen=lambda: reopen_calls.append(1) or fresh)
        engine.set_poll(PollDefinition(1, 1, 3, 0, 2, 0))
        engine.start()
        time.sleep(0.4)
        engine.stop()
        events = drain(engine.event_queue)
        names = [name for name, _payload in events]
        self.assertIn("master_reconnecting", names)
        self.assertIn("master_reconnected", names)
        self.assertTrue(broken.closed)
        self.assertIs(engine.ser, fresh)
        after = names[names.index("master_reconnected"):]
        self.assertIn("master_result", after)
        self.assertEqual(events[-1], ("master_stopped", {"error": None}))

    def test_engine_keeps_retrying_until_stopped(self):
        attempts = []

        def reopen():
            attempts.append(1)
```

### MBSniffer v3.2/tests/test_master.py — lines 521-560

```
            raise OSError("still unplugged")

        engine = make_engine(FlakySlaveSerial(failures=5), reconnect_interval_s=0.02,
                             reopen=reopen)
        engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
        engine.start()
        time.sleep(0.3)
        engine.stop()
        events = drain(engine.event_queue)
        self.assertGreater(len(attempts), 3)
        self.assertNotIn("master_reconnected", [n for n, _p in events])
        self.assertEqual(events[-1][0], "master_stopped")

    def test_without_reopen_a_failure_ends_the_session(self):
        engine = make_engine(FlakySlaveSerial(failures=1))
        engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
        engine.start()
        time.sleep(0.2)
        engine.stop()
        stopped = drain(engine.event_queue)[-1]
        self.assertEqual(stopped[0], "master_stopped")
        self.assertEqual(stopped[1]["error"][0], "serial")


class MasterSettingsTests(unittest.TestCase):
    def test_last_workspace_is_validated(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "settings.json"
            original = mb_config.get_settings_file
            mb_config.get_settings_file = lambda: path
            try:
                for value, expected in (("Linha 1", "Linha 1"), ("a/b", ""), (5, ""),
                                        ("x" * 61, ""), ("  Subestação  ", "Subestação")):
                    path.write_text(json.dumps({"master": {"last_workspace": value}}),
                                    encoding="utf-8")
                    self.assertEqual(
                        mb_config.load_ui_settings()["master"]["last_workspace"], expected)
            finally:
                mb_config.get_settings_file = original

```

### MBSniffer v3.2/tests/test_master.py — lines 561-563

```
561
562 if __name__ == "__main__":
563     unittest.main(verbosity=2)
```

# MBSniffer.bat

210 source lines.

### MBSniffer.bat — lines 1-40

```
@echo off
setlocal EnableExtensions
title MBSniffer 3.2
set "ROOT=%~dp0"
set "APPDIR=%~dp0MBSniffer v3.2"
set "SCRIPT=%APPDIR%\MBSniffer.py"
cd /d "%APPDIR%"
set "PYRUN="
set "PYEXE="
set "PYWEXE="

if not exist "%SCRIPT%" (
    echo.
    echo ERRO: os ficheiros do MBSniffer nao foram encontrados em:
    echo   %SCRIPT%
    echo.
    pause
    exit /b 1
)

rem ============================================================
rem 1. Procurar uma instalacao funcional de Python 3
rem ============================================================
call :DETECT_PYTHON

if defined PYRUN goto PYTHON_OK

echo.
echo Python 3 nao foi encontrado.
echo A tentar instalar automaticamente Python 3.13 com WinGet...
echo.

where winget >nul 2>&1
if errorlevel 1 goto NO_WINGET

winget install --id Python.Python.3.13 --exact --scope user --silent ^
    --accept-package-agreements --accept-source-agreements

if errorlevel 1 (
    echo.
```

### MBSniffer.bat — lines 41-80

```
    echo ERRO: A instalacao automatica do Python falhou.
    echo O WinGet pode estar bloqueado pela politica da empresa ou sem acesso a Internet.
    echo.
    pause
    exit /b 1
)

rem O instalador pode alterar o PATH sem atualizar esta janela CMD.
rem Por isso voltamos a procurar tambem nos caminhos standard.
call :DETECT_PYTHON

if not defined PYRUN (
    echo.
    echo ERRO: Python foi instalado, mas nao consegui localizar python.exe.
    echo Fecha esta janela e volta a executar MBSniffer.bat.
    echo.
    pause
    exit /b 1
)

:PYTHON_OK
rem ============================================================
rem 2. Descobrir o python.exe real e o respetivo pythonw.exe
rem ============================================================
for /f "usebackq delims=" %%P in (`%PYRUN% -c "import sys; print(sys.executable)" 2^>nul`) do (
    set "PYEXE=%%P"
)

if not defined PYEXE (
    echo.
    echo ERRO: Nao foi possivel determinar a instalacao do Python.
    echo.
    pause
    exit /b 1
)

for %%P in ("%PYEXE%") do set "PYWEXE=%%~dpPpythonw.exe"

rem ============================================================
rem 3. Verificar tkinter
```

### MBSniffer.bat — lines 81-120

```
rem ============================================================
%PYRUN% -c "import tkinter" >nul 2>&1
if not errorlevel 1 goto TK_OK

echo.
echo tkinter nao esta disponivel nesta instalacao do Python.
echo A tentar instalar/reparar Python 3.13 com WinGet...
echo.

where winget >nul 2>&1
if errorlevel 1 goto TK_ERROR

winget install --id Python.Python.3.13 --exact --scope user --silent --force ^
    --accept-package-agreements --accept-source-agreements

call :DETECT_PYTHON
if not defined PYRUN goto TK_ERROR

%PYRUN% -c "import tkinter" >nul 2>&1
if errorlevel 1 goto TK_ERROR

for /f "usebackq delims=" %%P in (`%PYRUN% -c "import sys; print(sys.executable)" 2^>nul`) do (
    set "PYEXE=%%P"
)
for %%P in ("%PYEXE%") do set "PYWEXE=%%~dpPpythonw.exe"

:TK_OK
rem ============================================================
rem 4. Verificar pyserial (obrigatorio)
rem ============================================================
%PYRUN% -c "import serial" >nul 2>&1
if not errorlevel 1 goto PYSERIAL_OK

echo.
echo pyserial nao esta instalado.
echo A instalar pyserial...
echo.

%PYRUN% -m pip --version >nul 2>&1
if errorlevel 1 (
```

### MBSniffer.bat — lines 121-160

```
121     %PYRUN% -m ensurepip --upgrade
122     if errorlevel 1 goto DEPENDENCIES_ERROR
123 )
124
125 %PYRUN% -m pip install --user pyserial
126 if errorlevel 1 goto DEPENDENCIES_ERROR
127
128 %PYRUN% -c "import serial" >nul 2>&1
129 if errorlevel 1 goto DEPENDENCIES_ERROR
130
131 :PYSERIAL_OK
132 rem ============================================================
133 rem 5. Dependencias verificadas
134 rem ============================================================
135 :DEPENDENCIES_OK
136 rem ============================================================
137 rem 6. Executar o MBSniffer
138 rem ============================================================
139 if exist "%PYWEXE%" (
140     start "" "%PYWEXE%" "%SCRIPT%"
141     exit /b 0
142 )
143
144 rem Fallback caso pythonw.exe nao exista.
145 start "" %PYRUN% "%SCRIPT%"
146 exit /b 0
147
148
149 :DETECT_PYTHON
150 set "PYRUN="
151
152 rem Python Launcher for Windows
153 py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
154 if not errorlevel 1 (
155     set "PYRUN=py -3"
156     exit /b 0
157 )
158
159 rem Python no PATH
160 python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
```

### MBSniffer.bat — lines 161-200

```
161 if not errorlevel 1 (
162     set "PYRUN=python"
163     exit /b 0
164 )
165
166 rem Caminhos standard para Python 3.13 instalado por utilizador/WinGet.
167 if exist "%LocalAppData%\Programs\Python\Python313\python.exe" (
168     set PYRUN="%LocalAppData%\Programs\Python\Python313\python.exe"
169     exit /b 0
170 )
171
172 if exist "%ProgramFiles%\Python313\python.exe" (
173     set PYRUN="%ProgramFiles%\Python313\python.exe"
174     exit /b 0
175 )
176
177 if exist "%ProgramFiles(x86)%\Python313\python.exe" (
178     set PYRUN="%ProgramFiles(x86)%\Python313\python.exe"
179     exit /b 0
180 )
181
182 exit /b 0
183
184
185 :NO_WINGET
186 echo.
187 echo ERRO: Python 3 nao esta instalado e o WinGet nao esta disponivel.
188 echo Nao e possivel fazer a instalacao automatica neste PC.
189 echo Instala Python 3 manualmente e volta a executar MBSniffer.bat.
190 echo.
191 pause
192 exit /b 1
193
194
195 :TK_ERROR
196 echo.
197 echo ERRO: tkinter continua indisponivel.
198 echo E necessario usar uma instalacao normal do Python para Windows com Tcl/Tk.
199 echo.
200 pause
```

### MBSniffer.bat — lines 201-210

```
exit /b 1


:DEPENDENCIES_ERROR
echo.
echo ERRO: Nao foi possivel instalar pyserial.
echo Verifica acesso a Internet, proxy/firewall e permissoes do Python.
echo.
pause
exit /b 1
```

# MBSniffer v3.2/build_exe.bat

151 source lines.

**MBSniffer v3.2/build_exe.bat — lines 1-40**

```
@echo off
setlocal EnableExtensions
title Build MBSniffer 3.2

rem Normalizar caminhos sem barra final. Isto evita problemas de parsing
rem do Windows/Python quando um argumento quoted termina em "\".
for %%I in ("%~dp0.") do set "APPDIR=%%~fI"
for %%I in ("%APPDIR%\..") do set "DISTDIR=%%~fI"

set "SCRIPT=%APPDIR%\MBSniffer.py"
set "ICON=%APPDIR%\MBSniffer.ico"
set "WORKDIR=%APPDIR%\build"
set "SPECFILE=%APPDIR%\MBSniffer.spec"
set "PYRUN="

cd /d "%APPDIR%"

if not exist "%SCRIPT%" (
    echo.
    echo ERRO: MBSniffer.py nao foi encontrado.
    echo.
    pause
    exit /b 1
)

rem ============================================================
rem 1. Procurar Python 3
rem ============================================================
py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 (
    set "PYRUN=py -3"
    goto PYTHON_OK
)

python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 (
    set "PYRUN=python"
    goto PYTHON_OK
)

```

### MBSniffer v3.2/build_exe.bat — lines 41-80

```
41 if exist "%LocalAppData%\Programs\Python\Python313\python.exe" (
42     set PYRUN="%LocalAppData%\Programs\Python\Python313\python.exe"
43     goto PYTHON_OK
44 )
45
46 echo.
47 echo ERRO: Python 3 nao foi encontrado.
48 echo Executa primeiro MBSniffer.bat para instalar/verificar as dependencias.
49 echo.
50 pause
51 exit /b 1
52
53
54 :PYTHON_OK
55 rem ============================================================
56 rem 2. Instalar/atualizar dependencias de build
57 rem ============================================================
58 echo.
59 echo [1/4] A verificar pyserial e PyInstaller...
60 %PYRUN% -m pip install --upgrade pyserial pyinstaller
61 if errorlevel 1 (
62     echo.
63     echo ERRO ao instalar pyserial/PyInstaller.
64     echo.
65     pause
66     exit /b 1
67 )
68
69 rem ============================================================
70 rem 3. Executar testes de regressao antes do build
71 rem ============================================================
72 echo.
73 echo [2/4] A executar testes de regressao...
74 %PYRUN% -m unittest discover -s tests -p "test_*.py" -v
75 if errorlevel 1 (
76     echo.
77     echo ERRO: os testes de regressao falharam. Build cancelado.
78     echo.
79     pause
80     exit /b 1
```

### MBSniffer v3.2/build_exe.bat — lines 81-120

```
)

rem ============================================================
rem 4. Limpar builds anteriores
rem ============================================================
echo.
echo [3/4] A limpar build anterior...
if exist "%WORKDIR%" rmdir /s /q "%WORKDIR%"
if exist "%SPECFILE%" del /q "%SPECFILE%"
if exist "%DISTDIR%\MBSniffer.exe" del /q "%DISTDIR%\MBSniffer.exe"

rem ============================================================
rem 5. Criar EXE
rem ============================================================
echo.
echo [4/4] A criar MBSniffer.exe...

if exist "%ICON%" (
    %PYRUN% -m PyInstaller ^
      --noconfirm ^
      --clean ^
      --onefile ^
      --windowed ^
      --name MBSniffer ^
      --distpath "%DISTDIR%" ^
      --workpath "%WORKDIR%" ^
      --icon "%ICON%" ^
      --add-data "%ICON%;." ^
      "%SCRIPT%"
) else (
    echo AVISO: MBSniffer.ico nao foi encontrado.
    %PYRUN% -m PyInstaller ^
      --noconfirm ^
      --clean ^
      --onefile ^
      --windowed ^
      --name MBSniffer ^
      --distpath "%DISTDIR%" ^
      --workpath "%WORKDIR%" ^
      "%SCRIPT%"
```

**MBSniffer v3.2/build_exe.bat — lines 121-151**

```
)

if errorlevel 1 (
    echo.
    echo ERRO: o build falhou.
    echo.
    pause
    exit /b 1
)

if not exist "%DISTDIR%\MBSniffer.exe" (
    echo.
    echo ERRO: o PyInstaller terminou sem criar:
    echo   %DISTDIR%\MBSniffer.exe
    echo.
    pause
    exit /b 1
)

echo.
echo ============================================================
echo BUILD CONCLUIDO
echo ============================================================
echo.
echo Executavel criado em:
echo   %DISTDIR%\MBSniffer.exe
echo.
echo O icone MBSniffer.ico foi aplicado ao EXE e incluido
echo como recurso para a janela/barra de tarefas.
echo.
pause
```

# MBSniffer v3.2/run_tests.bat

45 source lines.

**MBSniffer v3.2/run_tests.bat — lines 1-40**

```
@echo off
setlocal EnableExtensions
title MBSniffer regression tests

for %%I in ("%~dp0.") do set "APPDIR=%%~fI"
cd /d "%APPDIR%"
set "PYRUN="

py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 set "PYRUN=py -3"

if not defined PYRUN (
    python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
    if not errorlevel 1 set "PYRUN=python"
)

if not defined PYRUN (
    echo.
    echo ERRO: Python 3 nao foi encontrado.
    echo.
    pause
    exit /b 1
)

echo.
echo A executar testes de regressao do MBSniffer...
echo.
%PYRUN% -m unittest discover -s tests -p "test_*.py" -v
if errorlevel 1 (
    echo.
    echo ============================================================
    echo TESTES FALHARAM - build/release nao deve prosseguir.
    echo ============================================================
    echo.
    pause
    exit /b 1
)

echo.
echo ============================================================
```

### MBSniffer v3.2/run_tests.bat — lines 41-45

```
41 echo TODOS OS TESTES PASSARAM
42 echo ============================================================
43 echo.
44 pause
45 exit /b 0
```